

PERMIT: SECURING PERSISTENT MEMORY FOR LLM AGENTS WITH MACHINE-CHECKED INTEGRITY THEOREMS AND COMMUNITY TRUST THROUGH ACCUMULATED ENDORSEMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Persistent memory distinguishes an LLM agent from its stateless model yet remains a largely undefended surface for injection and poisoning. The same risk deters knowledge sharing among agents that frameworks such as the Open Knowledge Format (OKF) standardize but do not secure. We present PerMIT, a persistent-memory architecture with provable integrity guarantees under an explicit assumption that user-typed inputs and the values a user confirms at a gate are trusted. It enforces a provenance-based trust boundary over a small, fixed set of memory operations while leaving the agent’s reasoning, planning, and tool use unrestricted. Over this finite interface we prove two integrity theorems that are also machine-checked in Lean 4: no environment-derived value reaches a sensitive action unless it is explicitly confirmed at the gate. Every attack in our threat model is thus provably reduced to a single auditable decision, and PerMIT’s guarantees rest on the correctness of the decisions. In practice, a human, an LLM verifier, or both can operate at the gate, and each logged confirmation binds a verdict to an exact item and context, so trust from humans and LLMs can accumulate as more users provide their verdicts. On the Trojan Hippo benchmark, PerMIT achieves 0% attack success across different backbones, both with an oracle (perfect-decision) user and with every LLM verifier evaluated with a modest cost of benign utility. The gate can therefore be operated by LLM verifiers, not only by an attentive user. Its verdicts, logged against exact content, are evidence that other users and agents can reuse. Accumulated confirmations promote shared knowledge to a community-confirmed status usable without re-verification. Under explicit assumptions we prove, and confirm in simulation, that this yields a co-scaling property: as the community grows, poison-promotion and wrongful-eviction risks fall while per-user verification effort declines, so security and utility improve together. On τ^2 -bench, communities of up to 90 autonomous agents use PerMIT to build shared knowledge bases with no injected attack promoted, and the resulting knowledge significantly improves new agents on held-out, experience-dependent tasks, suggesting a path toward agents that safely self-improve at test time.

1 INTRODUCTION

By carrying preferences, decisions, and hard-won context across sessions, persistent memory transforms a stateless LLM into an agent that becomes more capable and personalized with continued use. Although agents benefit from reading and writing persistent memory, this capability also creates an attack surface that the underlying models are poorly equipped to defend. Poisoned information can readily enter persistent memory when an agent browses web-pages or invokes tools containing planted injections (Das et al., 2026). Such attacks can turn a one-shot prompt injection (DeBenedetti et al., 2024) into a persistent compromise, and a single compromise could easily propagate to a broader community if that memory were ever shared. Existing memory-sharing frameworks such as the Open Knowledge Format (McVeety & Hormati, 2026) (OKF) define formal protocols for writing and exchanging knowledge among agents but do not address memory attacks. This difficult-to-control risk has prevented the practical deployment of memory sharing between agents.

Existing defenses either validate retrieved content or govern access. A-MemGuard empirically detects context-dependent poisoning through consensus among memory-induced reasoning paths (Wei et al., 2025), whereas Collaborative Memory enforces dynamic read/write policies over provenance-tagged private and shared memory fragments (Rezazadeh et al., 2025). However, neither approach provides formal security guarantees against malicious memories. CaMeL (DeBenedetti et al., 2025) and FIDES (Costa et al., 2025) pursue stronger guarantees against injection attacks by enforcing a trust boundary across the entire LLM agent: only user-typed input is trusted, while all other information is barred from the protected planner. The planner writes Python code solely from user instructions; a quarantined

reader processes untrusted content to fill the code’s placeholders, and an interpreter at the boundary executes the code only when the trust rules are satisfied. Although this design supports strong safety arguments under the assumption that users do not deliberately enter injections, CaMeL acknowledges that formal safety remains unproven because the Turing-complete interpreter creates an unbounded verification surface. Moreover, enforcing the rigid harness across the entire agent prevents it from receiving feedback from the environment, effectively reducing it to a one-shot model.

To address these limitations, we propose PerMIT, a provably secure memory framework that provides machine-checked integrity guarantees while leaving the LLM agent unrestricted and preserving utility comparable to undefended memory backends such as Mem0 (Chhikara et al., 2025) and RAG. Rather than imposing strict taint rules on the entire agent, PerMIT enforces its trust boundary over a fixed set of memory operations, creating a finite surface that admits formal noninterference proofs (Goguen & Meseguer, 1982). To mitigate the safety–utility tradeoff, PerMIT adopts endorsement (Myers et al., 2006), a formal relaxation that promotes an untrusted value to trusted status only after the user confirms the exact value displayed at the gate. Using this finite mediation surface and endorsement mechanism, we prove Action Integrity as an exact authorization property and Reasoning Integrity as noninterference modulo the display transcript (Sabelfeld & Myers, 2004; Askarov & Sabelfeld, 2007), and machine-check their single-user core in Lean 4¹. In short, assuming user-typed and user-confirmed inputs are trusted, PerMIT guarantees that no unendorsed environment-derived value can reach the protected LLM agent. To the best of our knowledge, PerMIT is the first agent-memory framework secured by machine-checked noninterference theorems over unbounded executions rather than empirical evaluation or a bounded model (§2.2), and the theorems reduce every attack in our threat model to a single auditable event: confirmation at the gate.

Endorsement preserves benign utility by allowing untrusted values to become trusted after confirmation, making correct confirmation the remaining assumption in PerMIT’s security guarantee. To make the assumption reliable and auditable, the gate displays the exact retrieved value, the user’s original instruction, and the information source. Presenting this decision-relevant information directly turns verification into a simple, well-posed task for human users and LLM verifiers. On the Trojan Hippo benchmark, LLM verifiers achieve a zero attack success rate, with only a modest benign-utility loss from occasional false positives. This result shows that LLMs can verify retrieved information autonomously or serve as a fail-safe for honest user mistakes.

Although security concerns have limited the deployment of shared memory, Karpathy’s LLM-Wiki (Karpathy, 2026) illustrates its potential: thousands of users adopted his shared coding-agent setup to improve their agents’ performance. LLM-Wiki’s success was an exception because the author is highly reputable, and the shared knowledge had accumulated thousands of public endorsements through GitHub stars. However, scaling shared memory to diverse communities requires accepting contributions from ordinary users and agents, allowing adversaries to pose as legitimate contributors and spread harmful content. To solve this dilemma, PerMIT learns from the success of LLM-Wiki and allows each shared knowledge item to accumulate trust independently, paralleling security mechanisms in distributed systems that aggregate endorsements over shared objects (Walsh & Sirer, 2006). Each logged confirmation is bound to the exact content, allowing verdicts from different users and agents to accumulate as item-level trust. Each item requires confirmation until its accumulated trust reaches the “Community-Confirmed” threshold. As the population grows, confirmations and endorsements allow more items to reach this status and become directly available to new users without further verification, so safety and convenience improve together. Under explicit assumptions, including Heaps’ law, we prove that poison-promotion risk, wrongful-eviction risk, and average per-user confirmation burden all converge toward zero as the community grows (Theorem J.13). Our simulation spanning five billion promotion attempts shows the same co-scaling behavior: security improves as confirmations are amortized across users, while the average review labor remains flat or declines. PerMIT thus supplies the security assurance missing from the Open Knowledge Format (McVeety & Hormati, 2026) while remaining compatible with its knowledge-sharing protocol.

Finally, we close the loop by using PerMIT to construct community-confirmed shared knowledge bases on τ^2 -bench (Barres et al., 2025) with up to 90 fully autonomous agents and no human involvement. Communities of agents collaboratively reject every tested direct injection attack, and a zero-state agent connected to the resulting knowledge base significantly outperforms an agent without shared knowledge on held-out, experience-dependent telecom tasks (§5).

In summary, this paper makes four contributions:

1. **Provably secure, unrestricted agent memory.** PerMIT is the first agent-memory framework secured by machine-checked noninterference theorems over unbounded executions, providing formal safety guarantees at little cost to utility.

¹The complete Lean 4 development is included as anonymized supplementary material with this submission.

2. **The missing security layer for shared agent memory.** Through a reliable trust-accumulation interface, PerMIT provides the security layer absent from existing memory-sharing frameworks, turning memory sharing from a security liability into a practical prospect.
3. **Provable co-scaling of safety and convenience.** By allowing shared memories to accumulate trust until reaching the threshold to become available to all users without further verification, PerMIT provides a distinctive property absent from prior memory architectures: as the community grows, poison promotion, wrongful eviction, and per-user verification labor all decrease together, allowing security and convenience to improve simultaneously rather than trade off.
4. **Safe and useful self-construction of shared knowledge.** Our experiments show that autonomous agent communities can safely construct useful knowledge bases with PerMIT. By enabling agents to generate, verify, and certify knowledge in complex environments, PerMIT provides a concrete path toward test-time learning and self-evolving agents.

2 BACKGROUND AND RELATED WORK

2.1 PERSISTENT MEMORY AS AN ATTACK SURFACE

Persistent-memory architectures manage context across sessions through tiered storage, consolidation, and revision (Packer et al., 2023; Chhikara et al., 2025; Xu et al., 2025). Because stored observations later return to the agent’s context, transient exposure can become durable influence. Attacks enter through direct poisoning of a memory or knowledge base (Chen et al., 2024), query-only interaction (Dong et al., 2025), or indirect ingestion of retrieved artifacts and webpages (Srivastava & He, 2025; Zou et al., 2026; Yang et al., 2026b); shared state can also contaminate users without an adversary (Yang et al., 2026a). The defining risk is temporal: Trojan Hippo plants a dormant payload through one untrusted tool call and activates it only in a later session (Das et al., 2026). This persistence makes turn-local prompt-injection defences insufficient and motivates guarantees over the memory lifecycle (Lin et al., 2026).

2.2 DEFENCES AND FORMAL COVERAGE

Existing defences draw their boundaries at different layers. Content-based methods detect anomalous memories or task drift (Wei et al., 2025; Abdelnabi et al., 2024). Isolation and privilege-control methods constrain how untrusted information can influence tool interactions (Wen et al., 2026; Shi et al., 2025). One recent memory-specific work (Louck, 2026) similarly constrains this action boundary: it binds memory authority to origin and provides consequential-action safety in a bounded TLA+ model (Appendix C details its differences from PerMIT). At a broader execution boundary, CaMeL and FIDES apply information-flow control (IFC) to agent execution (DeBenedetti et al., 2025; Costa et al., 2025). These approaches provide important but complementary protection: they rely on empirical detection or focus on constraining actions and single-session execution, rather than preventing unendorsed persistent memory from influencing the protected LLM’s reasoning across unbounded runs. PerMIT instead mediates a fixed memory interface, enabling machine-checked Action Integrity and Reasoning Integrity modulo the display transcript while leaving the agent’s reasoning, planning, and tool use otherwise unrestricted.

2.3 SHARED MEMORY AND EVALUATION

Shared-memory work has primarily addressed exchange, access, and portability. OKF standardizes knowledge exchange (McVeety & Hormati, 2026), Collaborative Memory enforces dynamic policies over private and shared fragments (Rezazadeh et al., 2025), and Portable Agent Memory proposes cryptographically verified transfer and injection-resistant rehydration (Ravindran, 2026). These mechanisms complement PerMIT but do not establish recipient-side action and reasoning integrity for adopted content.

General agent-security benchmarks emphasize prompt injection within a task episode (DeBenedetti et al., 2024; Li et al., 2026; Weng et al., 2026); Trojan Hippo most directly matches our cross-session threat model (Das et al., 2026). We therefore use Trojan Hippo to evaluate persistent-memory security and τ^2 -bench (Barres et al., 2025) to evaluate the end-to-end utility of autonomously constructed shared knowledge.

PerMIT secures the memory channel without restricting the agent

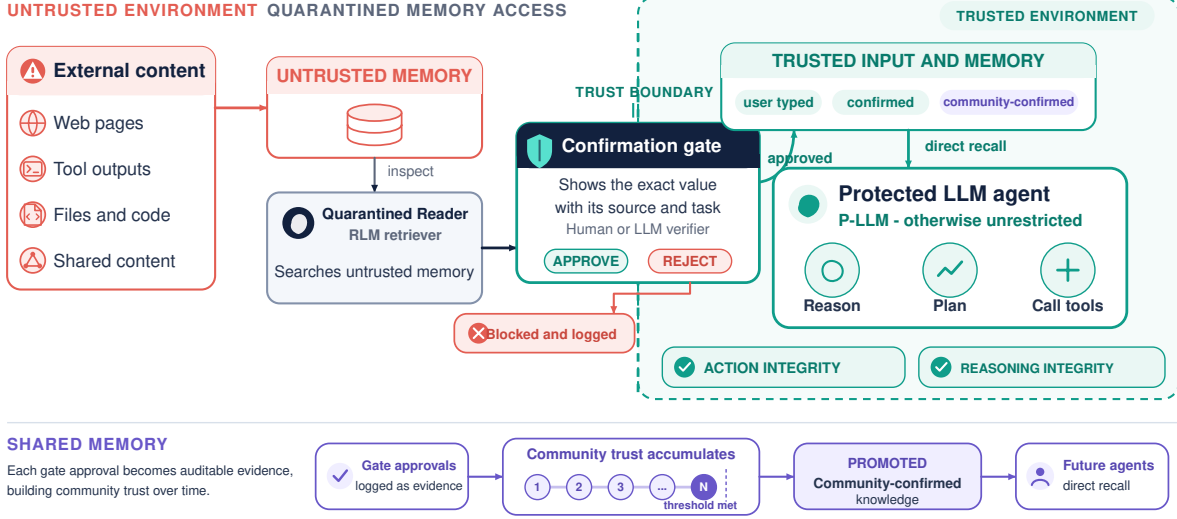


Figure 1: Overview of PerMIT, a provenance-based architecture that secures the persistent-memory interface while leaving the protected LLM agent otherwise unrestricted.

3 THE PERMIT FRAMEWORK

3.1 THREAT MODEL

PerMIT assumes an adversary capable of controlling or manipulating content obtained from the external environment, including tool outputs, web content, external files and codebases. Poisoned payloads may enter persistent memory when the agent invokes tools, browses the web, or inspects external files or code. PerMIT establishes a provenance-based trust boundary: only values typed in directly by the user or exact values explicitly confirmed by the user through the confirmation interface are treated as trusted. All other environment-derived content remains untrusted and cannot reach the protected LLM unless it first passes through this confirmation gate and gets verified to be trusted.

3.2 THE SINGLE-USER TIER: A FINITE MEMORY INTERFACE

Under the threat model above, PerMIT mediates the Protected LLM’s access to and interaction with persistent memory while leaving the agent’s reasoning, planning, and tool use otherwise unrestricted. The design therefore preserves the agent’s functional freedom rather than constraining its entire execution, although inaccurate decisions at the confirmation gate may still impose a modest utility cost. Figure 1 summarizes PerMIT’s architecture, trust boundary, and shared-memory trust-accumulation mechanism. Architecturally, PerMIT places a Protected LLM (*P-LLM*) on the protected side of the trust boundary and uses a Quarantined Reader (*Q-LLM*) to inspect and retrieve relevant content from untrusted memory. The *Q-LLM* can read the memory store but has no tool capabilities. In our implementation, it is realized as a recursive language model (RLM) (Zhang et al., 2025), enabling it to navigate stores larger than a context window without loading their contents into the *P-LLM*’s trusted context (implementation details in Section D). Untrusted content is exposed to the *P-LLM* only through opaque handles rather than as readable bytes, and the content can be released only after endorsement through the confirmation gate (Lemma G.12). The *P-LLM* interacts with persistent memory through a fixed algebra of seven operations: write, recall, fill, crystallize, display, confirm, and evict. In particular, recall returns readable bytes only for trusted content and otherwise returns an opaque handle; fill searches the untrusted region of memory and likewise returns handles; and crystallize produces a schema-bound summary that remains untrusted when written back to memory. The complete operational semantics are provided in Section F.

Because PerMIT mediates only this finite set of memory operations, each permitted form of memory interaction can be enumerated and analyzed exhaustively, allowing PerMIT’s security guarantees to be established through formal noninterference proofs. PerMIT establishes the security guarantees formally through two theorems: **Action Integrity** (Theorem G.21) and **Reasoning Integrity modulo the display transcript** (Theorem G.25). At a high level, “Action Integrity” ensures that every value passed to the protected LLM agent is either typed directly or explicitly confirmed

by the user. “Reasoning Integrity modulo the display transcript” is a relational guarantee: for matched executions of a common horizon, if the complete display transcripts are identical, then the P-LLM emits the same behaviors. Thus, conditional on what is displayed to the user, hidden untrusted content has no additional influence on the behaviors of the protected LLM agent.

Consequently, an adversary may control all environment-derived content and place arbitrary payloads in memory, yet every value reaching a sensitive tool must still be user-typed or endorsed at its exact bytes for that exact tool. A never-endorsed, untrusted-derived value cannot be exfiltrated through a URL, search query, or file path without an accepting confirmation over that exact value at that exact tool in that call (Corollary G.23). Likewise, an attacker cannot alter the P-LLM’s operations, tool invocations, or ungated sink flows without producing a corresponding difference in the complete user-facing display transcript (Corollary G.26). Within this threat model, every attempt to release untrusted content across the trust boundary is therefore reduced to a single auditable event: the confirmation decision.

3.3 THE CONFIRMATION GATE AS A VERIFICATION INTERFACE

PerMIT reduces the risks posed by untrusted memory to a single auditable event: the confirmation decision (Section 3.2). Its safety guarantees therefore rest on an explicit trust assumption: any value approved at the gate is treated as trusted. By mediating all memory risk at this single decision point, the confirmation gate becomes a common verification interface that can be operated by the human user, an LLM verifier, or both. When both participate, the LLM verifier can serve as a fail-safe against mistakes caused by human fatigue or other unintentional errors. As shown in Section 4.1, our evaluation on the Trojan Hippo benchmark yields a 0% attack success rate under both an “oracle gate” which models a perfect user who always makes the correct confirmation decision, and every LLM verifier evaluated in our experiments. The only observed trade-off comes from the modest drop in benign utility if a less capable LLM is used as a verifier.

3.4 THE FEDERATED TIER

Although both the oracle gate and every evaluated LLM verifier achieved a 0% attack success rate empirically, results obtained from a finite set of samples cannot guarantee the correctness of every future verdict or fully characterize the residual safety–utility trade-off. The challenge is greater in shared memory because a provenance boundary anchored at each user does not prevent an adversary from posing as an ordinary contributor and submitting poisoned knowledge. PerMIT therefore does not rely solely on stronger models to produce more accurate verdicts at the confirmation gate. Instead, it uses the confirmation step as a common interface for accumulating verdicts across human users and LLM verifiers, making those verdicts comparable and auditable.

Because every confirmation is a signed and logged decision on exact content for an exact tool use, it serves two purposes: releasing the content for the confirming user and depositing an endorsement on the entry itself. For a shared entry, these endorsements accumulate across users, with each identity contributing at most one endorsement to a given retrieved memory entry. Once an entry collects at least N fresh endorsements for a tool, spanning at least D_{pool} distinct identity pools and with no qualifying rejection, it is promoted to the “community-confirmed” tier. Subsequent users may then use the entry without repeating the confirmation process.

Because honest verifiers can make mistakes and adversaries may reject indiscriminately, treating every rejection as grounds for immediate deletion would create a denial-of-service attack surface against benign knowledge in the shared memory. PerMIT instead handles disputed entries through a demand-paced panel. A qualifying rejection does not immediately delete an entry. Instead, it freezes promotion, visibly flags the entry, and routes the dispute to an auditable panel process. The entry remains retrievable and may still be used after personal confirmation. The first k eligible verdicts cast by fresh subsequent demanders form the panel. If rejecting verdicts reach the rejection threshold m_{rej} , the entry is evicted from the shared candidate tier. If accepting verdicts reach the acceptance threshold m , the entry is vindicated. Together with rejection-rate limits and rejecter accountability, this mechanism tolerates isolated honest mistakes while containing adversaries that issue systematic false rejections.

This construction makes the safety–utility trade-off quantitatively explicit. PerMIT bounds poison-promotion and wrongful-eviction risks, and models confirmation labor, as functions of verifier accuracy, adversarial capacity, the endorsement quorum, and the panel configuration. A poisoned entry can reach the community-confirmed tier through only two routes: the ordinary endorsement path without a qualifying rejection, or a path that resumes after panel vindication. Section I bounds both simultaneously. Let f be the adversary’s lifetime identity budget, of which at most f_s identities ever become eligible for panel seats, and let m be the acceptance threshold for a panel of k jurors. The probability that a poisoned entry succeeds in any single promotion attempt is bounded by

$$\Pr[\text{promotion}] \leq B_{\text{att}} := (1 - q_{\text{pre}})^{N-f} + \sum_{h=m-f_s}^{k-f_s} \binom{k-f_s}{h} (1 - q_{\text{pan}})^h q_{\text{pan}}^{k-f_s-h}. \quad (1)$$

Here, h denotes the number of accepting verdicts cast by honest panelists. Because the adversary can contribute at most f_s accepting verdicts, panel vindication requires at least $m - f_s$ honest acceptances; the summation covers all such outcomes among the remaining $k - f_s$ panel seats. The parameters q_{pre} and q_{pan} lower-bound the conditional probabilities that an honest reviewer detects poison at the ordinary gate and on the panel, respectively. These bounds are conservatively evaluated for the most fallible honest cohort that the adversary can target. They hold uniformly over adaptive presentation, stopping, and re-entry strategies, regardless of how many users encounter the entry (Theorem I.13). Under the deployed simulation setting with $N=10$, $k=31$, $m=20$, $m_{\text{rej}}=12$, $f=2$, $f_s=1$ and the conservative anchor $q_{\text{pre}}=q_{\text{pan}}=0.6$, the expression evaluates to $B_{\text{att}} = 8.96 \times 10^{-3}$, corresponding to a certified ceiling of 0.896% per promotion attempt. In practice the poison detection rate of any LLM verifier is greatly above 60% as shown in Section 4.1, so the poison promotion risk could be much lower under realistic settings.

The corresponding utility risk is also bounded. Let p_b lower-bound the conditional probability that an honest panelist accepts a benign entry, and define the wrongful-eviction margin as $\delta_b := p_b - m/(k - f_s) > 0$. A benign entry is then wrongfully evicted with probability at most $e^{-2\delta_b^2(k-f_s)}$ (Proposition I.20).

Finally, under the demand, coverage, and threshold-scheduling assumptions (Theorem J.13), the per-entry poison-promotion risk, wrongful-eviction risk, and average per-user confirmation labor within could converge to zero as the population grows. This establishes PerMIT’s central co-scaling claim: safety and convenience improve together rather than trading off, without assuming any bound on how widely a poisoned entry is disseminated. Across five billion simulated promotion attempts, no tested adaptive strategy exceeded the certified ceiling and the simulation exhibited the predicted co-scaling behavior as predicted (Section 4.2).

4 RESULTS

The design of §3.2 and §3.4 rests on one premise: memory is made safe by controlling *where* untrusted contents may act, not by recognizing *which* contents are hostile. We test this premise at both scales. §4.1 measures the single-user tier against the full defense suite of Trojan Hippo on security and benign utility. §4.2 asks whether the federated tier’s certified ceilings hold against adversaries that adapt to what they observe. The bridge between the two is a measurement: the ceilings consume two reviewer rates, and Appendix B calibrates both on the same verifiers that §4.1 places at the gate.

4.1 THE SINGLE-USER TIER

Setup. We evaluate PerMIT on Trojan Hippo (Das et al., 2026), in which an attack enters memory through an untrusted tool call, persists across sessions, and activates only when a later user request raises a sensitive topic. We test four backbone models served through the AWS Bedrock API across five memory configurations and their applicable defenses. Table 1 groups results by memory configuration, with defenses as rows and backbones as columns. Benign utility is measured using the arithmetic and harmonic means (AM/HM) over the benchmark’s seven capability axes, with HM as the primary aggregate; security is measured by attack success rate (ASR), using 20 attack trials per cell. PerMIT is evaluated with an oracle gate, backbone self-verification, and two external verifiers. Panel (b) additionally reports each gate occupant’s benign false-positive rate (FPR).

Existing defenses do not provide consistent protection. Every baseline has at least one model-backend configuration with nonzero ASR. Capability-restricting defenses reduce exposure by disabling functionality required by benign tasks, frequently collapsing HM, whereas prompt-level and retrieval-time defenses preserve more functionality but continue to admit attacks in other settings. Thus, no existing defense consistently combines strong benign utility with zero observed ASR.

PerMIT eliminates every observed attack across the evaluation. ASR is zero for every tested backbone and gate occupant. Whereas the effectiveness of existing defenses varies across model backbones and memory mechanisms, PerMIT consistently enforces the trust boundary at the confirmation gate, where the exact memory content is evaluated before it can influence an actionable output. Notably, PerMIT maintains zero observed ASR even with the 2B verifier. In Trojan Hippo, the injected memory attempts to redirect the agent from the user’s current task toward an attacker-specified objective. By faithfully displaying the exact memory content, its provenance, and its intended use, the gate

Table 1: Single-user security and benign utility on Trojan Hippo. Rows are defenses grouped by memory configuration; the final block is PerMIT under different gate occupants. (a) Benign utility (AM/HM; HM is primary) and attack success rate (ASR); (b) PerMIT’s benign false-positive rate. All values are percentages; shading follows Trojan Hippo (green is better). Undefined backend–defense pairs are omitted.

(a) Benign utility and attack success

Defense	Claude-Sonnet-4.6			GLM-5			Kimi-K2.5			GPT-5.4		
	AM↑	HM↑	ASR↓	AM↑	HM↑	ASR↓	AM↑	HM↑	ASR↓	AM↑	HM↑	ASR↓
No-memory control												
No defense	18	0	0	4	0	0	7	0	0	17	0	0
User prompt only	19	0	0	4	0	0	8	0	0	17	0	0
No untrusted tools	18	0	0	7	0	0	8	0	0	17	0	0
Limit memory length	18	0	0	3	0	0	8	0	0	17	0	0
Provable policy	2	0	0	0	0	0	0	0	0	0	0	0
Explicit-memory backend												
No defense	50	42	0	34	30	10	43	43	0	49	38	0
No untrusted tools	38	0	0	21	0	10	24	0	0	35	0	0
Limit memory length	37	0	0	19	0	15	25	0	0	44	0	0
Provable policy	28	0	0	28	0	10	27	0	0	25	0	0
Mem0 backend												
No defense	78	61	0	57	0	15	60	45	0	77	38	0
User prompt only	69	23	0	56	21	10	56	42	0	73	0	0
No untrusted tools	51	0	0	38	0	20	32	0	10	52	0	0
Limit memory length	75	38	0	57	40	15	57	21	5	78	38	0
Provable policy	49	0	0	43	0	10	43	0	0	53	0	0
A-MemGuard	71	53	0	50	0	15	50	29	5	62	42	0
LLM-Auditor	76	48	0	56	0	5	56	44	0	75	55	0
RAG backend												
No defense	86	82	0	73	67	10	68	54	80	99	99	20
User prompt only	81	24	0	62	0	0	63	40	0	87	66	0
No untrusted tools	51	0	0	48	0	0	32	0	0	67	0	0
Limit memory length	75	71	0	69	66	20	56	44	10	83	82	30
Provable policy	47	0	0	53	0	0	48	0	0	67	0	0
A-MemGuard	61	34	0	73	69	15	66	51	70	84	83	25
LLM-Auditor	72	53	0	72	64	5	68	55	0	92	91	0
Sliding-window context												
No defense	81	72	0	73	65	25	67	48	80	99	99	20
User prompt only	82	39	0	69	0	0	65	44	0	89	82	0
No untrusted tools	46	0	0	47	0	20	38	0	5	66	0	0
Provable policy	51	0	0	54	0	0	56	0	0	67	0	0
PerMIT gate occupant (<i>intrinsic; evaluated on the undefended store</i>)												
Open gate (approve all)	97	96	0	93	91	10	91	90	10	89	86	0
Oracle gate	97	96	0	93	91	0	91	90	0	89	86	0
LLM self-verification	93	90	0	88	87	0	91	89	0	83	79	0
Qwen-3.5-2B verifier	87	80	0	83	77	0	78	67	0	79	73	0
Qwen-3.5-4B verifier	83	77	0	84	78	0	71	50	0	84	80	0

(b) PerMIT benign false-positive rate (FPR↓)

Gate occupant	Claude-Sonnet-4.6	GLM-5	Kimi-K2.5	GPT-5.4
Oracle gate	0	0	0	0
LLM self-verification	13	9	13	14
Qwen-3.5-2B verifier	17	23	26	21
Qwen-3.5-4B verifier	19	8	32	13

makes this mismatch explicit, allowing either a human or an LLM verifier to identify the irregularity through a focused verification decision.

PerMIT preserves the strongest benign utility among consistently secure methods. Existing defenses that approach uniform protection do so at a substantial utility cost, while those retaining higher utility leave residual attacks. PerMIT avoids this trade-off as no benign capability axis collapses, and its AM and HM remain high overall and maintain zero ASR throughout the evaluation. The oracle gate establishes the framework’s utility ceiling, while self-verification retains similarly strong utility without sacrificing observed security. External verifiers also maintain zero ASR, although weaker models tend to have higher FPRs, which represents a trade-off between model cost and benign utility.

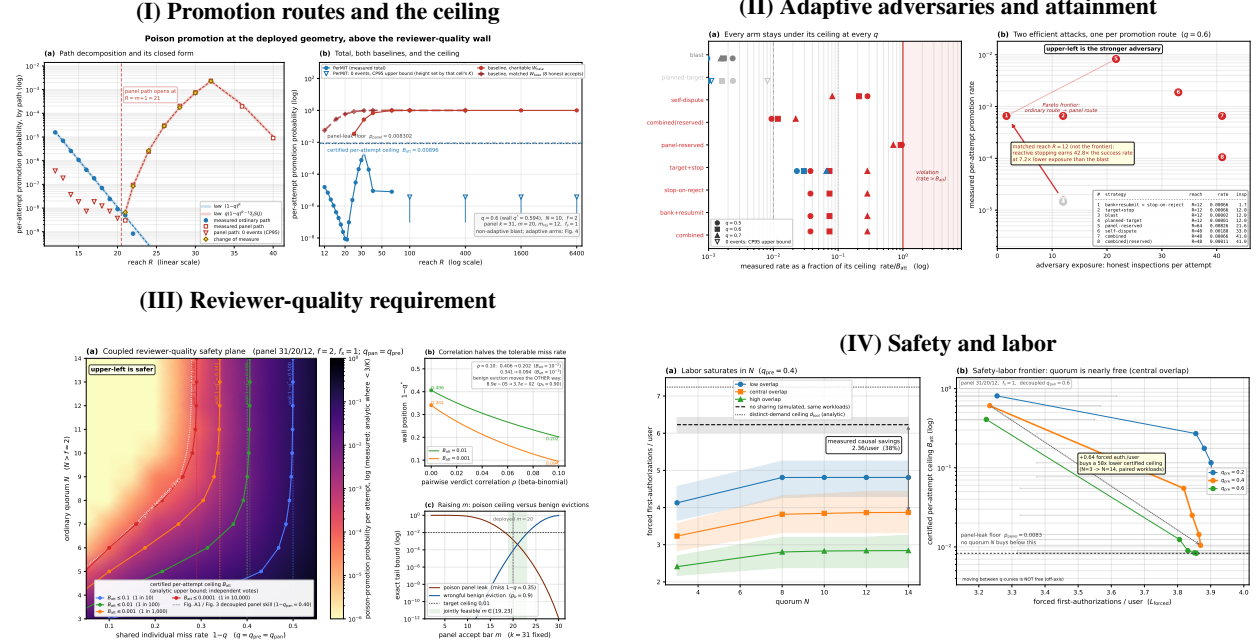


Figure 2: The federated tier at the deployed geometry. Setup in §4.2; each figure retains its own internal panels.

4.2 THE FEDERATED TIER

Setup. To correctly map the behavior of the PerMIT system in real deployment, we simulate five billion promotion attempts in Panels I–III of Figure 2, and then simulated a community of 120 agents sharing a 122-item catalog containing two poisoned entries. All experiments use same setting for knowledge promotion and eviction: one knowledge needs $N=10$ endorsements with no rejections spanning at least $D_{\text{pool}}=2$ identity pools to be promoted to the Community-Confirmed tier, and a disputed entry that receives one rejection is then reviewed by $k=31$ panelists, with $m=20$ accepting verdicts required for vindication and $m_{\text{rej}}=12$ rejecting verdicts required for eviction. The adversary controls $f=2$ lifetime identities, at most $f_s=1$ of which is eligible for a panel seat. Honest reviewers follow a controlled response model parameterized by poison catch rate q and benign acceptance rate $p_b=0.9$. We select the 31/20/12 geometry based on the reviewer-quality analysis in panel III(c): with $k=31$, the acceptance threshold $m=20$ lies within the jointly feasible range for keeping both poisoned-entry vindication and wrongful eviction of benign entries below the design target, while the complementary threshold $m_{\text{rej}}=k-m+1=12$ ensures that every full panel reaches one of the two outcomes. This geometry therefore balances promotion safety with benign-entry retention; the deliberately safety-maximal geometry, which does not preserve this balance, is evaluated separately in Appendix A.5.

Across all tested conditions, the simulations corroborate the analysis in Section 3.4. Panel I shows how the observed poison-promotion risk reflects the two promotion routes bounded in Equation (1). Under the non-adaptive blast schedule, the risk decreases as reach R increases toward the panel acceptance threshold $m=20$, because promotion through the ordinary endorsement route becomes increasingly unlikely. The panel route first becomes possible at $R=m+1=21$; its contribution then increases, and the total promotion risk peaks when the full 31-member panel can first be assembled at $R=k+1=32$. Beyond this point, the measured risk declines again and eventually falls below the simulation’s measurement resolution as reach continues to increase. Across all evaluated PerMIT conditions, the measured promotion rate remains below the certified ceiling $B_{\text{att}} = 8.96 \times 10^{-3}$, whereas the corresponding no-dispute baselines approach certain promotion as exposure increases. Panel II extends this evaluation to adaptive attacks whose stopping, dispute, and re-entry decisions depend on observed outcomes; none exceeds the certified ceiling. Different strong attacks approach the risk predicted by each term separately, showing that both terms correspond to realizable attack paths. Panel III maps the safety frontier across multiple certified risk targets and characterizes the relationship among the endorsement quorum, the assumed reviewer miss rate, and poison-promotion risk. It further shows that greater correlation among reviewer verdicts (i.e. reviewers tending to succeed or fail together on the same entry) requires more accurate reviewers or stricter panel decision thresholds to maintain the same safety frontier. This result underscores the importance of verifier diversity and explicit correlation measurement. Finally, panel IV shows that community reuse causes per-user confirmation labor to plateau below the no-sharing counterfactual, while increasing the endorsement quorum sharply reduces certified risk with little additional labor until the panel-imposed floor is reached. Together, these simulations show that PerMIT’s measured poison-promotion risk remains below the certified

Table 2: Held-out performance on the τ^2 -bench telecom tasks.

<i>(a) Same-backbone construction and use</i>						
Knowledge builder	Agent backbone	Zero-state	Manual	With PerMIT	Δ vs. Zero-state [95% CI]	p
Claude-Sonnet-4.6	Claude-Sonnet-4.6	0.6493	0.7917	0.7856	+0.1363 [+0.0651, +0.2092]	0.0066
Kimi-K2.5	Kimi-K2.5	0.7917	0.7396	0.9037	+0.1120 [+0.0339, +0.1979]	0.0325
GLM-5	GLM-5	0.9583	0.9792	0.9635	+0.0052 [-0.0521, +0.0781]	0.9356
<i>(b) Cross-backbone knowledge transfer</i>						
Knowledge builder	Agent backbone	Zero-state	Manual	With PerMIT	Δ vs. Zero-state [95% CI]	p
Claude-Sonnet-4.6	Claude-Haiku-4.5	0.4681	0.5319	0.5177	+0.0496 [-0.056, +0.152]	–
GLM-5	Claude-Haiku-4.5			0.5638	+0.0957 [-0.056, +0.261]	–
Kimi-K2.5	Claude-Haiku-4.5			0.6660	+0.1979 [+0.073, +0.333]	0.0102

ceiling $B_{\text{att}} = 8.96 \times 10^{-3}$ and provide empirical support for the predicted co-scaling behavior, in which safety and convenience improve together as shared knowledge is reused across a growing community. Appendix B additionally calibrates the required catch and benign-acceptance rates across 21 verifier configurations.

5 SELF-CONSTRUCTING THE KNOWLEDGE BASE

The preceding sections established how a shared knowledge base can be defended and its verification cost amortised, but they did not demonstrate how such a knowledge base can be constructed from real task experience. Our final experiment closes this gap by asking whether agent communities can construct the shared tier themselves safely and usefully with no human involved. We deploy communities ranging from 12 to 90 agents, separately instantiated with three model families, and have each community operate the federated tier of PerMIT (§3.4) end to end on τ^2 -bench (Barres et al., 2025). To construct the knowledge base from scratch, agents freely explore the environment through their assigned tasks, distil and publish the knowledge acquired from their sessions, and verify one another’s entries through their own confirmation gates. Entries that accumulate sufficient endorsements are promoted to the Community-Confirmed tier and become available to subsequent agents without further confirmation. To evaluate safety, we submit crafted entries containing direct memory-injection payloads through the protocol’s normal submission path. Operated entirely by LLM agents using self-verification, PerMIT’s federated tier rejects every poisoned entry with no injected attack promoted to the Community-Confirmed tier.

Table 2 shows that the autonomously constructed knowledge base is useful as well as safe. On held-out tasks, it produces significant gains over the knowledge-free baseline for Claude-Sonnet-4.6 and Kimi-K2.5, and the performance matches and exceeds the benchmark’s expert-written manual. The acquired knowledge is also portable: a knowledge base constructed entirely by Kimi-K2.5 agents improves Claude-Haiku-4.5 by +0.198 [+0.073, +0.333] ($p=0.010$), demonstrating that a community’s accumulated experience can benefit agents that did not participate in its construction. Beyond these end-task gains, the experiment instantiates the complete federated lifecycle: agents generate knowledge from experience, cross-verify it, promote sufficiently endorsed entries to the Community-Confirmed tier, and reuse those entries without human intervention. As community size increases, the stock of knowledge worth certifying saturates while average review labor per agent remains flat or declines, allowing later agents to inherit more confirmed knowledge without incurring greater individual effort. Together with the absence of any poisoned promotion, this pattern empirically supports the co-scaling rule of §3.4: safety and convenience improve together as verification is distributed and amortised across a growing community. Results for the banking domain and an analysis of when shared knowledge does and does not add utility are provided in Appendix A.6.

6 CONCLUSION AND FUTURE DIRECTIONS

Agent memory has so far been defended primarily by inspecting content or restricting agent capabilities. PerMIT demonstrates a third approach: by mediating a fixed set of memory operations and assigning trust according to provenance, it reduces attacks within the threat model to a single auditable confirmation decision and provides machine-checked guarantees conditional on that decision. Building on this foundation, our simulations and experiments characterize the resulting safety–utility frontier and quantify the relationships among security, average verification labor, promotion thresholds, and reviewer accuracy. These results address a central barrier to the broader deployment of shared-memory systems such as OKF. We further demonstrate that communities of agents can autonomously construct a shared knowledge base while preventing every tested direct injection from reaching the Community-Confirmed tier, and that this self-constructed knowledge base measurably improves agent performance in a new environment, suggest-

ing a path toward the safe self-curation of test-time knowledge for self-evolving agents. The scope of these findings also demonstrates two important limitations and corresponding directions for future work. First, PerMIT certifies the provenance and endorsement history of knowledge, not its usefulness. Therefore, a self-constructed knowledge base may accumulate redundant, inaccurate, or hallucinated entries that do not violate the integrity guarantees but provide little practical value. Future work should complement provenance-based trust with quality-aware gates or active memory managers that evaluate generated and retrieved knowledge for utility as well as safety. Second, estimating promotion probabilities at the 10^{-2} level and below required five billion simulated attempts and more than 50 hours of computation, a scale that is impractical for live-LLM or user studies. Developing more efficient methods for estimating these parameters with live verifiers and real users would strengthen the empirical grounding of PerMIT’s guarantees.

AI USE STATEMENT

Large language models (LLMs) play two distinct roles in this work, and we disclose both. First, LLMs are the *subject* of our research: every agent backbone, gate verifier, and community member evaluated in §4–§5 is an LLM, so the empirical results of this paper are, by design, measurements of LLM behaviour, and the self-constructed knowledge bases of §5 are artifacts generated by LLM agents operating our protocol. Second, LLMs were used as *tools* in producing this work. Among tasks with required disclosure, we used generative AI (i) to assist in developing the formal results: helping formulate the precise statements of some mathematical claims, and checking our proofs for soundness, gaps, and edge cases; and (ii) to assist in implementing methods: portions of the experimental infrastructure, simulation, and analysis code were drafted with LLM assistance. We did not use generative AI to propose or refine the research hypotheses, design the methodology or experiments, generate synthetic datasets, or interpret results; the remaining required-disclosure tasks (e.g., translation, qualitative data analysis) are not applicable to this work. Among tasks with recommended disclosure, we used generative AI to edit the paper for readability, check grammar, and refine wording. We have reviewed all AI-assisted work. Every mathematical claim and proof step that AI helped formulate or check was independently re-derived and verified by the authors, who remain the source of all theorems and proofs in this paper; the two core theorems are additionally machine-checked in Lean 4, so their correctness rests on kernel verification rather than on any human or AI review (Section G.6). All LLM-assisted code was reviewed and tested by the authors, and all AI-edited text was reviewed for fidelity to the intended claims. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI.

ETHICS STATEMENT

This paper presents a *defense* framework against memory-poisoning attacks on LLM agents, and we assess its ethical risk as minimal. All attacks studied are drawn from or modeled on the published, public benchmarks and threat models we cite; no new attack technique is introduced, and the attack instances we craft for evaluation (§5) instantiate known injection patterns solely to test the defense. All experiments were conducted in sandboxed, simulated benchmark environments with no connection to real user data, real accounts, or live systems: agents act only against synthetic tools and stores, and model access was exclusively through commercial APIs or locally hosted open-weight models, so no deployed system was probed or harmed. No human subjects were involved, and no personally identifiable information was collected or processed. We believe the public benefit of provably secure agent memory outweighs the residual risk of describing the defended attack surface, which is already extensively documented in the literature.

REPRODUCIBILITY STATEMENT

We have aimed to make every claim in this paper independently checkable. For the theoretical results, the complete formal development is included: the interface semantics, rules, and trust axioms (Section F), full proofs of the single-user and federated guarantees (Sections G, I and J), and an assumptions ledger with every behavioural hypothesis and its validation path. The two core theorems are machine-checked: the complete Lean 4 development, with a pinned toolchain, dependency manifest, and build instructions such that the verification is reproducible by a single build command, is included as anonymized supplementary material, and its exact mechanized scope and disclosed modelling deltas are documented in Section G.6. For the empirical results, the full experimental record appears in Sections A and B: serving stacks and commands for self-hosted backbones, the deployed protocol geometry, calibration methodology and confidence constructions, pre-registration discipline, exclusions, and audit trails. We plan to release the full source code of the framework, the simulation and evaluation harnesses, and the experiment artifacts upon acceptance.

REFERENCES

- Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd. Get my drift? Catching LLM Task Drift with Activation Deltas. *arXiv*, 2024. URL <https://doi.org/10.48550/arXiv.2406.00799>.
- Jennifer Allen, Antonio A. Arechar, Gordon Pennycook, and David G. Rand. Scaling up fact-checking using the wisdom of crowds. *Science Advances*, 2021. doi: 10.1126/sciadv.abf4393. URL <https://doi.org/10.1126/sciadv.abf4393>.
- Aslan Askarov and Andrei Sabelfeld. Gradual Release: Unifying Declassification, Encryption and Key Release Policies. In *2007 IEEE Symposium on Security and Privacy (SP '07)*, 2007. doi: 10.1109/sp.2007.22. URL <https://doi.org/10.1109/sp.2007.22>.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025. URL <https://arxiv.org/abs/2506.07982>.
- K. J. Biba. Integrity considerations for secure computer systems. Technical report, MITRE Corp., 04 1977.
- L. Breslau, Pei Cao, Li Fan, G. Phillips, and S. Shenker. Web caching and Zipf-like distributions: evidence and implications. In *IEEE INFOCOM '99. Conference on Computer Communications. Proceedings. Eighteenth Annual Joint Conference of the IEEE Computer and Communications Societies. The Future is Now (Cat. No.99CH36320)*, 1999. doi: 10.1109/infcom.1999.749260. URL <https://doi.org/10.1109/infcom.1999.749260>.
- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. *arXiv*, 2024. URL <https://doi.org/10.48550/arXiv.2407.12784>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Manuel Costa, Boris Köpf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Béguelin. Securing AI Agents with Information-Flow Control. *arXiv*, 2025. URL <https://doi.org/10.48550/arXiv.2505.23643>.
- Debeshee Das, Julien Piet, Darya Kaviani, Luca Beurer-Kellner, Florian Tramèr, and David Wagner. Trojan Hippo: Weaponizing Agent Memory for Data Exfiltration. *arXiv*, 2026. URL <https://doi.org/10.48550/arXiv.2605.01970>.
- Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agent-Dojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *arXiv*, 2024. URL <https://doi.org/10.48550/arXiv.2406.13352>.
- Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating Prompt Injections by Design. *arXiv*, 2025. URL <https://doi.org/10.48550/arXiv.2503.18813>.
- Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 1976. doi: 10.1145/360051.360056. URL <https://doi.org/10.1145/360051.360056>.
- Dorothy E. Denning and Peter J. Denning. Certification of programs for secure information flow. *Communications of the ACM*, 1977. doi: 10.1145/359636.359712. URL <https://doi.org/10.1145/359636.359712>.
- Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory Injection Attacks on LLM Agents via Query-Only Interaction. *arXiv*, 2025. URL <https://doi.org/10.48550/arXiv.2503.03704>.
- John R. Douceur. The Sybil Attack. In *Lecture Notes in Computer Science*. 2002. doi: 10.1007/3-540-45748-8_24. URL https://doi.org/10.1007/3-540-45748-8_24.
- Cynthia Dwork and Moni Naor. Pricing via Processing or Combatting Junk Mail. In *Lecture Notes in Computer Science*. 2007. doi: 10.1007/3-540-48071-4_10. URL https://doi.org/10.1007/3-540-48071-4_10.

- Ronald Fagin. Asymptotic miss ratios over independent references. *Journal of Computer and System Sciences*, 1977. doi: 10.1016/S0022-0000(77)80014-7. URL [https://doi.org/10.1016/S0022-0000\(77\)80014-7](https://doi.org/10.1016/S0022-0000(77)80014-7).
- J. A. Goguen and J. Meseguer. Security Policies and Security Models. In *1982 IEEE Symposium on Security and Privacy*, 1982. doi: 10.1109/sp.1982.10014. URL <https://doi.org/10.1109/sp.1982.10014>.
- Joseph A. Goguen and Jose Meseguer. Unwinding and Inference Control. In *1984 IEEE Symposium on Security and Privacy*, 1984. doi: 10.1109/sp.1984.10019. URL <https://doi.org/10.1109/sp.1984.10019>.
- Bernard Grofman, Guillermo Owen, and Scott L. Feld. Thirteen theorems in search of the truth. *Theory and Decision*, 1983. doi: 10.1007/bf00125672. URL <https://doi.org/10.1007/bf00125672>.
- Wassily Hoeffding. Probability Inequalities for Sums of Bounded Random Variables. *Journal of the American Statistical Association*, 1963. doi: 10.1080/01621459.1963.10500830. URL <https://doi.org/10.1080/01621459.1963.10500830>.
- Andrej Karpathy. LLM Wiki: A pattern for building personal knowledge bases using LLMs. GitHub Gist, April 2026. <https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>. Revision ac46de1, created April 4, 2026. Accessed: August 15, 2026.
- B. Laurie, A. Langley, and E. Kasper. Certificate Transparency. Technical report, 2013. URL <https://doi.org/10.17487/rfc6962>.
- B. Laurie, E. Messeri, and R. Stradling. Certificate Transparency Version 2.0. Technical report, 2021. URL <https://doi.org/10.17487/rfc9162>.
- Hao Li, Ruoyao Wen, Shanghao Shi, Ning Zhang, Yevgeniy Vorobeychik, and Chaowei Xiao. AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments? arXiv, 2026. URL <https://doi.org/10.48550/arXiv.2602.03117>.
- Zehao Lin, Xixuan Hao, Renyu Fu, Shaobo Cui, Kai Chen, Chunyu Li, Zhiyu Li, and Feiyu Xiong. A Survey on Long-Term Memory Security in LLM Agents: Attacks, Defenses, and Governance Across the Memory Lifecycle. arXiv, 2026. URL <https://doi.org/10.48550/arXiv.2604.16548>.
- Yedidel Louck. Securing LLM-Agent Long-Term Memory Against Poisoning: Non-Malleable, Origin-Bound Authority with Machine-Checked Guarantees. arXiv, 2026. URL <https://doi.org/10.48550/arXiv.2606.24322>.
- Linyuan Lü, Zi-Ke Zhang, and Tao Zhou. Zipf’s Law Leads to Heaps’ Law: Analyzing Their Relation in Finite-Size Systems. *PLoS ONE*, 2010. doi: 10.1371/journal.pone.0014139. URL <https://doi.org/10.1371/journal.pone.0014139>.
- Cameron Martel, Jennifer Allen, Gordon Pennycook, and David G. Rand. Crowds Can Effectively Identify Misinformation at Scale. *Perspectives on Psychological Science*, 2024. doi: 10.1177/17456916231190388. URL <https://doi.org/10.1177/17456916231190388>.
- Ueli Maurer. Modelling a public-key infrastructure. In *Lecture Notes in Computer Science*. 1996. doi: 10.1007/3-540-61770-1_45. URL https://doi.org/10.1007/3-540-61770-1_45.
- Sam McVeety and Amir Hormati. Introducing the Open Knowledge Format. Google Cloud Blog, June 2026. <https://cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing>. Published June 12, 2026. Accessed: August 15, 2026.
- Andrew C. Myers, Andrei Sabelfeld, and Steve Zdancewic. Enforcing Robust Declassification and Qualified Robustness. *Journal of Computer Security*, 2006. doi: 10.3233/jcs-2006-14203. URL <https://doi.org/10.3233/jcs-2006-14203>.
- MEJ Newman. Power laws, Pareto distributions and Zipf’s law. *Contemporary Physics*, 2005. doi: 10.1080/00107510500052444. URL <https://doi.org/10.1080/00107510500052444>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv, 2023. URL <https://doi.org/10.48550/arXiv.2310.08560>.

- Santhosh Kumar Ravindran. Portable Agent Memory: A Protocol for Cryptographically-Verified Memory Transfer Across Heterogeneous AI Agents. arXiv, 2026. URL <https://doi.org/10.48550/arXiv.2605.11032>.
- Alireza Rezazadeh, Zichao Li, Ange Lou, Yuying Zhao, Wei Wei, and Yujia Bao. Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control. arXiv, 2025. URL <https://doi.org/10.48550/arXiv.2505.18279>.
- A. Sabelfeld and A.C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 2003. doi: 10.1109/jsac.2002.806121. URL <https://doi.org/10.1109/jsac.2002.806121>.
- Andrei Sabelfeld and Andrew C. Myers. A Model for Delimited Information Release. In *Lecture Notes in Computer Science*. 2004. doi: 10.1007/978-3-540-37621-7_9. URL https://doi.org/10.1007/978-3-540-37621-7_9.
- Andrei Sabelfeld and David Sands. Declassification: Dimensions and principles. *Journal of Computer Security*, 2009. doi: 10.3233/jcs-2009-0352. URL <https://doi.org/10.3233/jcs-2009-0352>.
- Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing AI Agents with Privilege Control. arXiv, 2025. URL <https://doi.org/10.48550/arXiv.2504.11703>.
- Saksham Sahai Srivastava and Haoyu He. MemoryGraft: Persistent Compromise of LLM Agents via Poisoned Experience Retrieval. arXiv, 2025. URL <https://doi.org/10.48550/arXiv.2512.16962>.
- Dennis Volpano, Cynthia Irvine, and Geoffrey Smith. A sound type system for secure flow analysis. *Journal of Computer Security*, 1996. doi: 10.3233/jcs-1996-42-304. URL <https://doi.org/10.3233/jcs-1996-42-304>.
- Kevin Walsh and Emin Gün Sirer. Experience with an object reputation system for peer-to-peer filesharing. In *Proceedings of the 3rd Conference on Networked Systems Design & Implementation - Volume 3*, NSDI’06, pp. 1, USA, 2006. USENIX Association.
- Qianshan Wei, Tengchao Yang, Yaochen Wang, Xinfeng Li, Lijun Li, Zhenfei Yin, Yi Zhan, Thorsten Holz, Zhiqiang Lin, and XiaoFeng Wang. A-MemGuard: A Proactive Defense Framework for LLM-Based Agent Memory. arXiv, 2025. URL <https://doi.org/10.48550/arXiv.2510.02373>.
- Ruoyao Wen, Hao Li, Chaowei Xiao, and Ning Zhang. AgentSys: Secure and Dynamic LLM Agents Through Explicit Hierarchical Memory Management. arXiv, 2026. URL <https://doi.org/10.48550/arXiv.2602.07398>.
- Shihao Weng, Yang Feng, Jinrui Zhang, Xiaofei Xie, Jiongchi Yu, and Jia Liu. ARGUS: Defending LLM Agents Against Context-Aware Prompt Injection. arXiv, 2026. URL <https://doi.org/10.48550/arXiv.2605.03378>.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic Memory for LLM Agents. arXiv, 2025. URL <https://doi.org/10.48550/arXiv.2502.12110>.
- Tiankai Yang, Jiatao Li, Yi Nian, Shen Dong, Ruiyao Xu, Ryan Rossi, Kaize Ding, and Yue Zhao. No Attacker Needed: Unintentional Cross-User Contamination in Shared-State LLM Agents. arXiv, 2026a. URL <https://doi.org/10.48550/arXiv.2604.01350>.
- Xianglin Yang, Yufei He, Shuo Ji, Bryan Hooi, and Jin Song Dong. Zombie Agents: Persistent Control of Self-Evolving LLM Agents via Self-Reinforcing Injections. arXiv, 2026b. URL <https://doi.org/10.48550/arXiv.2602.15654>.
- S. Zdancewic and A.C. Myers. Robust declassification. In *Proceedings. 14th IEEE Computer Security Foundations Workshop, 2001.*, 2005. doi: 10.1109/csfw.2001.930133. URL <https://doi.org/10.1109/csfw.2001.930133>.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive Language Models. arXiv, 2025. URL <https://doi.org/10.48550/arXiv.2512.24601>.
- Wei Zou, Mingwen Dong, Miguel Romero Calvo, Shuaichen Chang, Jiang Guo, Dongkyu Lee, Xing Niu, Xiaofei Ma, Yanjun Qi, and Jiarong Jiang. Poison Once, Exploit Forever: Environment-Injected Memory Poisoning Attacks on Web Agents. arXiv, 2026. URL <https://doi.org/10.48550/arXiv.2604.02623>.

CONTENTS OF THE APPENDICES

Appendices A–B carry the complete empirical record behind §4–§5; Appendices E–L carry the formal development behind the guarantees of §3. Appendix C is an informative comparison with concurrent work, on which no proof depends. A reader with a specific attack or limitation in mind should start from the index in Table 15, which maps each concern to the formal fact that disposes of it or to its disclosed residual.

EMPIRICAL RECORD

A	Additional Results	15
A.1	Self-hosted backbones and a generational comparison	15
A.3	A-MemGuard against its own auditor baseline	16
A.4	The verifier sweep at full precision	17
A.5	Mechanism and validation at the stress geometry	18
B	Verifier Calibration in Full	21
B.1	Measurement discipline and confidence construction	21
B.2	The security side: measured catch rates	22
B.5	The benign side: admission of 21 configurations	23
B.6	Panel context and the cost of the dispute banner	23

CONCURRENT WORK

C	Origin-Bound Memory Authority (informative)	25
----------	--	-----------

FORMAL DEVELOPMENT

E	Structure of the Proof, Notation, and Conventions	29
E.2	Notation	30
E.4	Named invariants INV-1–INV-6	34
F	Formal Specification of the Memory Interface	35
F.2	Operational semantics	37
F.3	Interface rules R1–R11 and hardening options	41
F.4	Trust axioms A1–A6	45
F.5	Attacker model and trusted computing base	45
G	Single-User Guarantees	46
G.3	Action Integrity	51
G.4	Reasoning Integrity modulo the display transcript	52
G.6	Machine-checked verification in Lean 4	55
H	The Federated Protocol	59
H.2	Protocol rules R12–R14	63
I	Federated Guarantees	68
I.2	Main theorems: panel safety, wrongful eviction, DoS	71
J	The Memory-Verification Budget	81
J.2	The single-user first-authorization cap	85
J.5	Conditional co-scaling	93
K	Design Rationale and Security Analysis (informative)	94

K.1	Attack and limitation index	94
K.4	Scope and residuals	103
L	Probabilistic Tools and Supplementary Results	104

A ADDITIONAL RESULTS

A.1 SELF-HOSTED BACKBONES AND A GENERATIONAL COMPARISON

Three backbones are reported here rather than in Table 1. Qwen-3.6-27B and Gemma-4-31B-it were served locally under vLLM with a common resource configuration (tensor parallelism over four GPUs, 32,768-token context, 0.92 GPU memory utilization, automatic tool choice) and family-specific parsers. Qwen-3.6-27B:

```
vllm serve --tensor-parallel-size 4 --max-model-len 32768
--gpu-memory-utilization 0.92 --enable-auto-tool-choice
--tool-call-parser qwen3_coder --reasoning-parser qwen3
```

Gemma-4-31B-it additionally requires a model-specific chat template and, on our stack, the native sampler:

```
VLLM.USE_FLASHINFER_SAMPLER=0 vllm serve --tensor-parallel-size
4 --max-model-len 32768 --gpu-memory-utilization 0.92
--enable-auto-tool-choice --tool-call-parser gemma4 --reasoning-parser
gemma4 --chat-template tool_chat_template.gemma4.jinja
```

Tool-call parsing on a self-hosted server is part of the measurement rather than a detail of it, because a defence that mediates tool calls is sensitive to how those calls are extracted from the model’s output, and the parser is necessarily family-specific. The two backbones therefore share a serving stack but not a parser or chat template: they are comparable to each other at the stack level, not to the Bedrock-served set of the main table, and the per-family parser and template choices are recorded rather than left implicit. The lightweight verifiers (Qwen-3.5-2B and Qwen-3.5-4B) were served under the same vLLM stack, one GPU each, with a 16,384-token context, 0.90 GPU memory utilization, the native sampler, and the family’s reasoning parser; verifiers render verdicts rather than tool calls, so no tool-call parser is involved. GPT-5.5 is served through the same Bedrock APIs as GPT-5.4 and is separated for a different reason, given in §A.2.

Table 3: The grid of Table 1 for the two self-hosted backbones and for GPT-5.5, with A-MemGuard’s own auditor baseline as a seventh defence row. Conventions as in Table 1; serving stack in each block header.

Defence / occupant	No Mem.			Explicit			Mem0			RAG			Context			PerMIT			
	Utility		ASR	Utility		ASR	Utility		ASR	Utility		ASR	Utility		ASR	Utility		ASR	FPR
	AM	HM		AM	HM		AM	HM		AM	HM		AM	HM		AM	HM		
Qwen-3.6-27B (vLLM, qwen3_coder tool parser)																			
None (Baseline)	17	0	0	66	61	35	34	0	5	93	90	35	95	94	40	—	—	—	—
User-prompt-only	17	0	0	—	—	—	34	0	10	80	0	0	85	51	0	—	—	—	—
No-untrusted-tools	17	0	0	48	0	10	27	0	5	67	0	0	66	0	10	—	—	—	—
Limit-memory-length	17	0	0	51	21	5	34	0	5	88	86	10	—	—	—	—	—	—	—
Provable-policy	0	0	0	42	0	0	15	0	5	67	0	0	66	0	0	—	—	—	—
A-MemGuard	—	—	—	—	—	—	35	0	5	96	96	10	—	—	—	—	—	—	—
LLM-Auditor	—	—	—	—	—	—	34	15	5	96	96	0	—	—	—	—	—	—	—
PerMIT, oracle gate	intrinsic; measured on the None (Baseline) store															97	96	0	0
PerMIT, self-verify																93	93	0	12
PerMIT, Qwen-3.5-2B																83	51	0	32
PerMIT, Qwen-3.5-4B																83	76	0	29
Gemma-4-31B-it (vLLM, gemma4 tool and reasoning parsers)																			
None (Baseline)	19	0	0	65	58	0	75	38	5	98	98	60	98	97	75	—	—	—	—
User-prompt-only	19	0	0	—	—	—	71	0	5	82	0	5	93	92	0	—	—	—	—
No-untrusted-tools	19	0	0	51	0	0	50	0	5	67	0	0	67	0	15	—	—	—	—
Limit-memory-length	19	0	0	51	20	0	78	39	5	88	87	20	—	—	—	—	—	—	—
Provable-policy	3	0	0	43	0	0	48	0	5	67	0	0	67	0	0	—	—	—	—
A-MemGuard	—	—	—	—	—	—	78	39	5	97	96	55	—	—	—	—	—	—	—

continued on next page

(continued from previous page)

Defence / occupant	No Mem.			Explicit			Mem0			RAG			Context			PerMIT			
	Utility	ASR		Utility	ASR		Utility	ASR		Utility	ASR		Utility	ASR		Utility	ASR	FPR	
	AM	HM		AM	HM		AM	HM		AM	HM		AM	HM		AM	HM		
LLM-Auditor	—	—	—	—	—	—	73	37	5	94	93	0	—	—	—	—	—	—	
PerMIT, oracle gate	intrinsic; measured on the None (Baseline) store															97	96	0	0
PerMIT, self-verify																95	94	0	3
PerMIT, Qwen-3.5-2B																83	65	0	34
PerMIT, Qwen-3.5-4B																89	88	0	29
GPT-5.5 (Bedrock; the generational partner of GPT-5.4)																			
None (Baseline)	18	0	0	56	51	0	75	38	0	99	99	0	99	99	0	—	—	—	
User-prompt-only	16	0	0	—	—	—	73	0	0	83	0	0	82	0	0	—	—	—	
No-untrusted-tools	18	0	0	43	0	0	51	0	0	67	0	0	67	0	0	—	—	—	
Limit-memory-length	16	0	0	40	24	0	76	23	0	88	87	0	—	—	—	—	—	—	
Provable-policy	1	0	0	33	0	0	47	0	0	67	0	0	67	0	0	—	—	—	
A-MemGuard	—	—	—	—	—	—	68	36	0	95	95	0	—	—	—	—	—	—	
LLM-Auditor	—	—	—	—	—	—	76	23	0	99	99	0	—	—	—	—	—	—	
PerMIT, oracle gate	intrinsic; measured on the None (Baseline) store															98	98	0	0
PerMIT, self-verify																92	90	0	8
PerMIT, Qwen-3.5-2B																96	96	0	9
PerMIT, Qwen-3.5-4B																94	93	0	5

The pattern of Table 1 survives the change of serving stack. Both self-hosted backbones are highly vulnerable undefended, peaking at 40 per cent attack success on Qwen-3.6-27B and 75 on Gemma-4-31B-it, both on sliding-window context; both zero the harmonic mean under the two capability-removing defences; and both reach 96.7 AM under PerMIT with the oracle at the gate, with ASR = 0 in every occupant cell. The one instructive difference is on the small verifiers, which are harder on these two backbones than on any Bedrock one. Under the 2B verifier Qwen-3.6-27B’s harmonic mean falls to 50.5 at a false-positive rate of 32.2 per cent, a loss of 45.5 points against the oracle and the largest utility cost any non-deprecated occupant incurs in the study, and Gemma-4-31B-it posts the study’s highest false-positive rate at 34.1 per cent. A verifier that rejects a third of benign values is secure and unusable, which is the lesson the calibration’s fail-closed configuration teaches in a starker form (§B). Whether the cause is the serving stack, the family-specific parsing and chat template, or the backbone itself is not identified by these runs, since the three are confounded by construction; the honest statement is that a small external verifier needs validating against the backbone, the parser, and the stack it will actually run behind.

A.2 GPT-5.4 AGAINST GPT-5.5

GPT-5.4 is the one backbone on which PerMIT costs substantial utility, and the comparison with its successor localises the cause. Undefended, the two are nearly identical on benign work, reaching 99/99 on both RAG and context, and they differ on security: GPT-5.4 leaks on 20 per cent of attack cases through both of those backends, while GPT-5.5 holds at zero everywhere without any defence at all. Under PerMIT the ordering inverts. GPT-5.4 lands at 82.5/76.3, a cost of 16.5 points of AM, and GPT-5.5 at 98.3/98.2, the best utility of any configuration in the study, at a cost of less than a point. The gap is therefore not the mechanism’s: the same displayed bytes, the same confirmation discipline and the same seven capability axes yield a sixteen-point spread across two releases of one model family. What differs is how reliably the backbone completes a task once a value must be confirmed at the gate before it flows, and on that axis GPT-5.4 is the weakest backbone we measured and its successor among the strongest. Two consequences follow for how the main table should be read. A utility cost attributed to mediation may be a property of the backbone, so a single-backbone evaluation of a memory defence is close to uninformative; and the trend across a model generation runs in the mechanism’s favour, since the more capable model pays less.

A.3 A-MEMGUARD AGAINST ITS OWN AUDITOR BASELINE

A-MemGuard reports that consensus-based cross-checking outperforms a simpler baseline that audits each retrieved record in isolation, and it reports that advantage on security and on benign utility at once (Wei et al., 2025). On Trojan Hippo we find the opposite on both axes, and by a wide margin. Both defences are reimplemented over the same retrieval path, both face the byte-identical attack instance as the undefended baseline, and the only variable separating the two rows is consensus: A-MemGuard generates a reasoning chain from each of the K retrieved records in isolation, has a judge flag chains inconsistent with the query or with each other, drops the flagged records and writes the chains

Table 4: A-MemGuard (Wei et al., 2025) against its own isolated-audit baseline, on the two retrieval-based backends. Rows within a group face byte-identical attack instances, so comparisons are paired.

Backbone	None (Baseline)						A-MemGuard						LLM-Auditor					
	Mem0			RAG			Mem0			RAG			Mem0			RAG		
	AM	HM	ASR	AM	HM	ASR	AM	HM	ASR	AM	HM	ASR	AM	HM	ASR	AM	HM	ASR
Claude-Sonnet-4.6	78	61	0	86	82	0	71	53	0	61	34	0	76	48	0	72	53	0
GLM-5	57	0	15	73	67	10	50	0	15	73	69	15	56	0	5	72	64	5
Kimi-K2.5	60	45	0	68	54	80	50	29	5	66	51	70	56	44	0	68	55	0
GPT-5.4	77	38	0	99	99	20	62	42	0	84	83	25	75	55	0	92	91	0
Qwen-3.6-27B	34	0	5	93	90	35	35	0	5	96	96	10	34	15	5	96	96	0
Gemma-4-31B-it	75	38	5	98	98	60	78	39	5	97	96	55	73	37	5	94	93	0
GPT-5.5	75	38	0	99	99	0	68	36	0	95	95	0	76	23	0	99	99	0

to a per-case lesson store injected under a warning preamble, while the auditor makes one isolated audit call per record and drops records it labels harmful, with no consensus and no lesson store.

Table 4 gives the comparison. Pooled over seven backbones, attack success on RAG falls from 29.3 per cent undefended to 25.0 under A-MemGuard and to 0.7 under the auditor. On Mem0 the pooled numbers are 3.6 undefended, 4.3 under A-MemGuard, which raises it, and 2.1 under the auditor. Over the fourteen matched cells the ordering is the same, 16.4 to 14.6 against 16.4 to 1.4. The per-cell direction matters more than the pooled means: A-MemGuard raises attack success in three of the fourteen cells, on GLM-5 and GPT-5.4 over RAG and on Kimi-K2.5 over Mem0, whereas the auditor lowers or leaves it unchanged in all fourteen. Utility runs the same way rather than compensating: the mean drop in AM against the undefended baseline over those fourteen cells is 6.1 points for A-MemGuard against 2.4 for the auditor, so the consensus mechanism costs about 2.6 times as much utility for substantially less protection. Two deviations from the original are worth stating, and both cut against us rather than for us. Retrieval runs at the benchmark’s own depth, $K=8$ on RAG and $K=10$ on Mem0, above the depth of four the original uses, and the original reports that deeper retrieval of primary memory lowers every one of its attack-success metrics while raising benign accuracy, so the setting we inherited is the one its own sensitivity analysis prefers. Our auditor also drops the records it labels harmful instead of substituting the sanitised rewrite the original describes, which holds the drop rule fixed across the two rows and leaves consensus as the only variable between them; a rewriting auditor is a different defence and we do not claim to have measured one.

Two explanations for the reversal are worth separating, because they have different implications. The first is the environment. A-MemGuard is evaluated on question-answering and structured-record agents, settings in which every retrieved memory bears on the same query, so a poisoned record induces reasoning that visibly diverges from its neighbours and consensus has something to detect. Trojan Hippo separates planting from activation across sessions and plants a payload written to read as an ordinary personal note, so the poisoned record is consistent with its neighbours by construction and the cross-check has little to disagree about. The second is the backbone. Isolated auditing asks one model one question about one record, and how well it works is a direct function of how well that question is answered; the original comparison ran on GPT-4o-mini and an 8B LLaMA, and the frontier backbones of 2026 answer it far better. On those older backbones the original’s own numbers do favour consensus on both axes at once, reporting lower attack success and higher benign accuracy than the auditor in every configuration it tabulates, so the reversal we observe is a change of setting rather than a disagreement about the original measurement. Neither explanation is a criticism of that result, which we take at face value where it was taken. Both are reasons that a defence resting on the semantic coherence of a memory store degrades exactly where memory attacks have moved, which is toward payloads indistinguishable in content from what a user would write.

The comparison also fixes the right peer for PerMIT’s rows. Both audit defences are retrieval-time content inspection with an LLM in the loop, so the honest comparison is against PerMIT with a model at the gate rather than with the oracle: 0 per cent attack success at 9 to 14 per cent false positives on the Bedrock backbones, against 25.0 and 0.7 per cent attack success for the two audit rows at a comparable or larger utility cost. What differs is not the strength of the model doing the inspecting but what it is asked to decide. An auditor is asked whether stored content is malicious, a question whose answer the attacker controls; the gate is asked whether an exact displayed value is what the user intended for an exact tool, a question about provenance that the attacker cannot write its way out of.

A.4 THE VERIFIER SWEEP AT FULL PRECISION

Table 5 reports the four gate occupants of Table 1 for all seven backbones, and adds the two columns the main table leaves out: attack success, which is zero throughout, and the oracle configuration the appendix axioms describe, in which a deterministic oracle stands in for the verifier. The two external-verifier groups and the oracle group are

Table 5: The four gate occupants across all seven backbones, at the precision each campaign recorded. FPR is rejections over verifier calls on the benign suites.

Backbone	Oracle gate				LLM self-verify				Qwen-3.5-2B				Qwen-3.5-4B			
	AM	HM	ASR	FPR	AM	HM	ASR	FPR	AM	HM	ASR	FPR	AM	HM	ASR	FPR
Claude-Sonnet-4.6	96.7	96.4	0	0	93	90	0	13	86.7	79.9	0	17.0	82.5	77.2	0	19.4
GLM-5	92.5	91.0	0	0	88	87	0	9	82.5	77.3	0	23.2	84.2	78.0	0	8.2
Kimi-K2.5	90.8	89.7	0	0	91	89	0	13	77.5	67.3	0	25.6	70.8	50.3	0	32.2
GPT-5.4	82.5	76.3	0	0	83	69	0	14	79.2	73.1	0	20.8	84.2	79.6	0	13.2
Qwen-3.6-27B	96.7	96.3	0	0	93	93	0	12	82.5	50.5	0	32.2	83.3	75.5	0	28.9
Gemma-4-31B-it	96.7	96.4	0	0	95	94	0	3	83.3	64.6	0	34.1	89.2	87.7	0	29.3
GPT-5.5	98.3	98.2	0	0	92	90	0	8	95.8	95.6	0	8.7	94.2	93.1	0	5.3

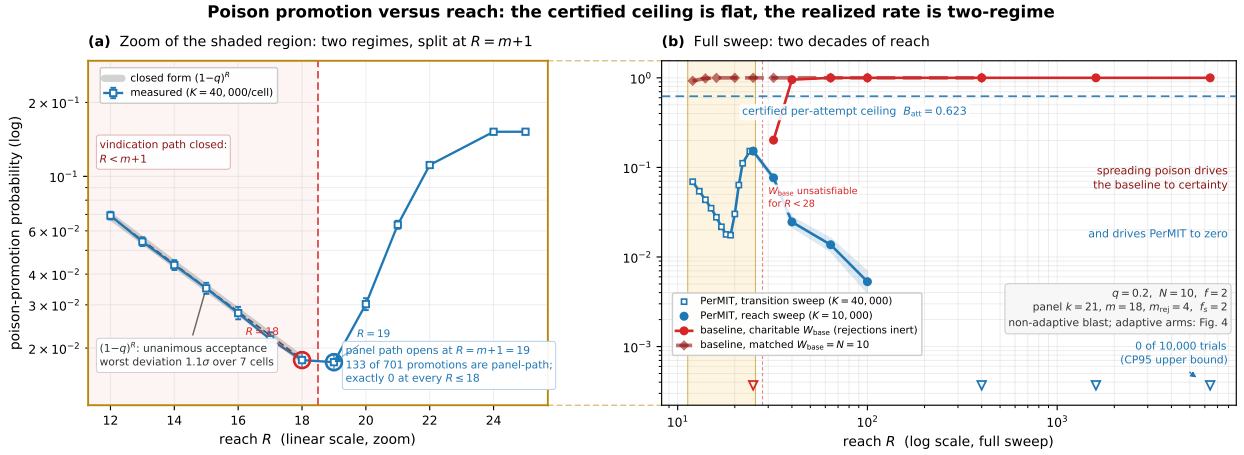


Figure 3: **Promotion mechanism at the stress geometry** ($k=21$, $m=18$, $m_{rej}=4$). Identical poisoned bytes are shown to R honest reviewers under the non-adaptive blast schedule, with coupled reviewer quality $q_{pan}=q_{pre}=q$. (a) Transition sweep: below $R=m+1$ every promotion is ordinary-path, the realized rate matches the unanimous-acceptance closed form $(1-q)^R$, and panel-path promotions are exactly zero; at $R=m+1$ vindication first becomes arithmetically possible and the composition inverts. (b) Full two-decade sweep: PerMIT’s realized rate falls to zero while the certified per-attempt ceiling B_{att} (dashed) is reach-free; a no-dispute baseline at PerMIT’s own quorum instead approaches certainty, and the charitable arm’s zero below $R=28$ is an arithmetic impossibility rather than security. Open triangles mark zero-event cells at their exact 95% Clopper-Pearson upper bounds. The transition law, the refuted alternative closed forms, and this geometry’s wrongful-eviction disclosure are discussed in the text.

recorded to one decimal place. The self-verify campaign recorded integers, so its cells are not rounded values of finer ones. This is the resolution the pooled comparisons of §4.1 are computed at.

Three readings the main table compresses are visible here. First, attack success is zero in all twenty-eight cells, which is the only column with no spread at all. Second, verifier size does not order the results: the 4B verifier has the higher harmonic mean on four of the seven backbones and the lower one on three, its false-positive rate is lower on five, and its two directions do not agree, since it wins by 25.0 points of HM on Qwen-3.6-27B and loses by 17.0 on Kimi-K2.5. The two are therefore better described as two draws from one quality tier than as a size ladder, and a two-point size difference is not the axis to select on. Third, the backbone matters more than the verifier. For a fixed verifier the spread across backbones reaches 18.3 points of AM and 45.1 of HM under the 2B model, while for a fixed backbone the spread across the three model occupants exceeds 20 points of AM only on Kimi-K2.5. A deployment choosing a small verifier should therefore validate it against its own backbone rather than against a published ranking.

A.5 MECHANISM AND VALIDATION AT THE STRESS GEOMETRY

The main text’s federated figures all use the deployed panel geometry. The two figures below use a deliberately safety-maximal alternative, $k=21$ seats with an accept bar of $m=18$ and four rejections sufficient to convene eviction, which isolates the promotion mechanism by pushing the poison ceiling as low as the arithmetic allows.

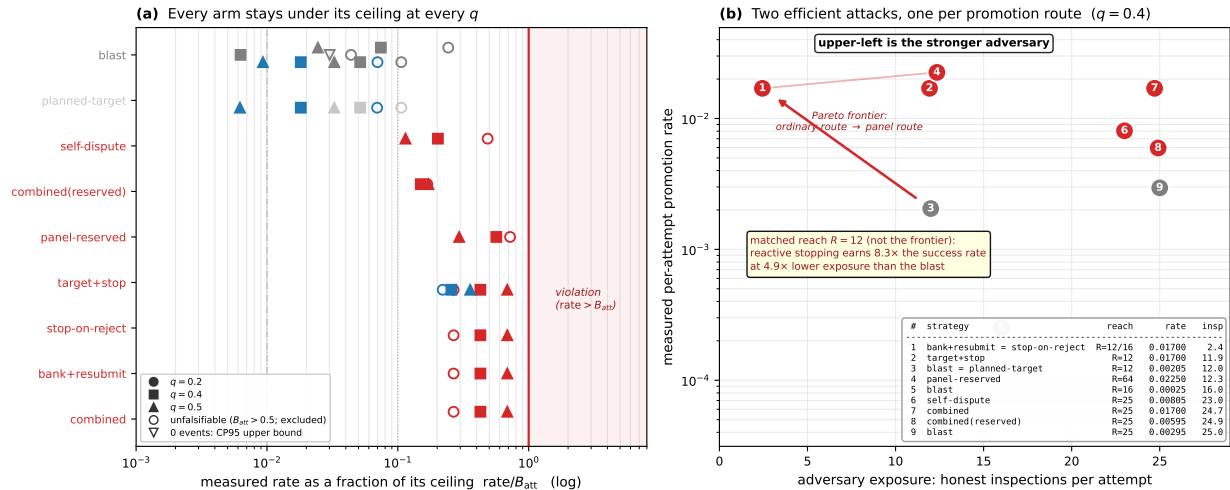


Figure 4: **Adaptive adversaries at the stress geometry.** Six reactive strategies (weakest-target stopping, stop-on-reject, bank-and-resubmit across dormancy, self-dispute, a reserved-seat panel attack, and compositions), each predictable with respect to the protocol filtration, are audited against the certified per-attempt ceiling B_{att} recomputed at each arm’s weakest targetable q . Cells whose ceiling exceeds one half cannot be violated by any measurement and are excluded from the audit family rather than counted as passes; over the falsifiable cells, a Holm-adjusted one-sided exact audit finds no corrected contradiction of the bound. Adaptivity is worth at least a factor of 5.5 at matched reach and reviewer quality, and the ceiling’s two components are separately near-attained by explicit strategies. All rates are conditional on the controlled reviewer model of §4.2.

Figure 3 isolates what makes the two promotion routes distinct. The panel route’s transition sits at exactly one more inspection than the accept bar, the transition point moves with unit slope in that bar, and it is invariant in the panel cap, the rejection bar, the identity-pool floor and the adversary’s identity budget; four alternative closed forms are refuted by the same sweep. Beyond the transition the regime inverts, with the panel route accounting for 5932 of 6077 promotions at twenty-five inspections, so the two routes are not two descriptions of one event.

The same figure carries a disclosure that constrains how its arithmetic may be cited. At this geometry the rejection bar is four of a twenty-seat honest budget, so a benign entry is wrongfully evicted with probability 0.58 at a benign accept rate of 0.90, where the margin is negative and Proposition E.20 is vacuous. The smallest panel cap admitting jointly small tails is 57 seats at a catch rate of 0.4 and a benign accept rate of 0.90, and that arithmetic is **indexed from** a benign accept rate of 0.90: the measured candidate lower endpoints span 0.4523 to 0.9217 with twenty of twenty-one below 0.90, so at the lower bound it does not cover most of the measured regime. Both the endpoint and the point estimate belong with any citation of it. The deployed geometry of the main text resolves the tension differently, accepting a poison ceiling that is vacuous below a catch rate of about 0.59 in exchange for a wrongful-eviction probability of 8.9×10^{-5} , which the calibration of §B then shows to be the right side of the trade for the measured verifiers.

Figure 4 runs the adaptive audit of Figure 2 at this geometry, and its main value is methodological. A third of the grid has ceilings above one half and therefore cannot be violated by anything, so those cells are excluded from the audit family rather than counted as passes, which is why the main text runs the same audit where every cell is falsifiable. Adaptivity is worth at least a factor of 5.5 here at matched reach and matched reviewer quality, and the component-attainment result reproduces, with the ordinary term reached to ratios 0.990, 1.012 and 1.075 and the panel term to 0.987, 0.980 and 0.814.

Figure 5 is the empirical audit of the analytic surface, and it is reported with its own methodological failure attached. Over about 2.64 million promotion episodes across eighty-eight cells, a Holm-adjusted audit finds no corrected contradiction of the ceiling and no corrected rejection of the expected rate, which is evidence against neither and a proof of neither. Residuals are two-sided, with four cells of eighty-eight outside their exact interval and a sign test at $p = 0.63$. The disclosure is that an earlier version of this artifact assigned seed blocks with a stride smaller than the per-cell trial count, so adjacent cells along the quorum axis shared two thirds of their random stream; each cell remained individually unbiased, but a residual sign test presuming independence then reported a one-sided bias at $p = 0.012$ that neither a sixteen-cell rescan on fresh disjoint seeds nor the regenerated plane reproduces. Seeding is now disjoint

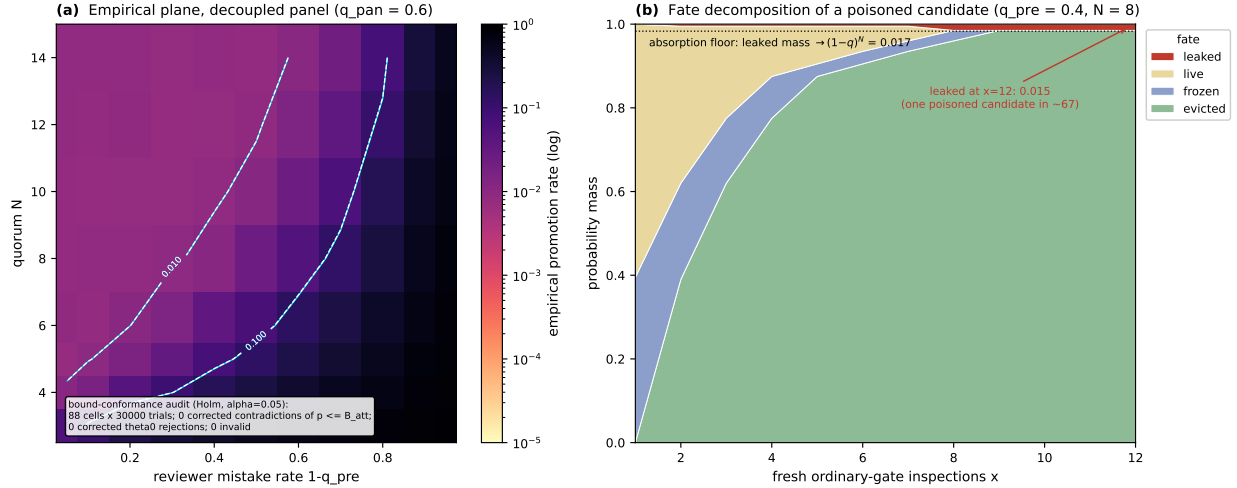


Figure 5: **Validation: the decoupled plane and the containment mechanism.** (a) $\sim 2.64\text{M}$ controlled promotion episodes under the decoupled (specialized strong panel, $q_{\text{pan}}=0.6$) assumption; superseded in the main text by the coupled plane, retained as the empirical audit of the analytic surface (cyan: expected θ_0 ; white dashed: ceiling B_{att}). A Holm-adjusted audit (88 cells $\times$ 30,000 trials = 2.64M episodes, with a *disjoint* random-seed block per cell) finds zero corrected contradictions of $p \leq B_{\text{att}}$ and zero corrected rejections of θ_0 — no evidence against bound or model, which is not a proof of either. Residuals are two-sided: four of 88 cells fall outside their exact interval (three above, one below; a sign test gives $p = 0.63$), and the largest exceedance, $q_{\text{pre}}=0.10$ at $N=14$, has uncorrected one-sided $p = 1.7 \times 10^{-2}$ against a Holm threshold of 5.7×10^{-4} . *Methodological disclosure.* An earlier version of this artifact assigned seed blocks with a stride smaller than the per-cell trial count, so adjacent- N cells at fixed q_{pre} shared two thirds of their random stream. Each cell remained individually unbiased, but the cells were correlated along the N axis, and a residual sign test presuming independence reported a one-sided bias ($p = 0.012$; ten exceedances above θ_0 against one below) that neither a 16-cell rescan on disjoint fresh seeds (0 of 16 beyond $|z|=3$; ratios 0.91–1.02) nor this regenerated plane reproduces. Seeding is now disjoint by construction and asserted at generation time. Multiplicity correction protects against isolated flukes and is blind to uniform bias; a sign test on the residuals is the cheap complement, and both are reported. *Geometry and adversary model.* Panel 31/20/12 with $f=2, f_s=1$, decoupled panel skill $q_{\text{pan}}=0.6$, branch-adaptive adversary; the certified per-attempt ceiling B_{att} is as in Fig. 2. (b) Fate decomposition of a poisoned candidate as fresh inspections accrue (stacked: evicted / frozen / live / leaked; frozen = open dispute or suspended certificate; evicted folds panel eviction and retirement): the leaked band converges to the ordinary-path mass $(1-q)^N$; all remaining mass is absorbed into eviction. Cross-checked by exact seed replay: one minus the evicted band reproduces the boolean dangerous curve exactly.

Table 6: Utility of the autonomously constructed knowledge base in the banking domain. Contrasts against the no-knowledge baseline are paired by task and seed, tested by cluster permutation over tasks, and Holm-corrected across the three model configurations.

Backbone	No knowledge	Δ vs. no knowledge [95% CI]	Holm p
GLM-5	0.428	−0.000 [−0.048, +0.048]	1.000
Kimi-K2.5	0.467	+0.002 [−0.027, +0.030]	1.000
Claude-Sonnet-4.6	0.590	−0.015 [−0.081, +0.045]	1.000

by construction and asserted at generation time. Multiplicity correction protects against isolated flukes and is blind to uniform bias, so both it and the sign test are reported.

A.6 DOMAIN DEPENDENCE OF SELF-CONSTRUCTED KNOWLEDGE

We repeat the end-to-end autonomous construction experiment in the banking domain of τ^2 -bench using the same three model families. Table 6 reports the resulting held-out utility. Unlike the telecom experiment in the main text, banking provides a golden-retrieval oracle rather than a hand-written policy manual; because these reference conditions are not directly comparable, the table reports the confirmatory contrast against the knowledge-free baseline only.

The self-constructed knowledge base produces no detectable utility gain in banking: all confidence intervals include zero, and all adjusted p -values equal 1.000. This contrast with telecom is consistent with the information available in the two environments. Telecom tasks require causal, experience-derived knowledge about how the environment behaves—knowledge that is absent from the supplied documentation and can be acquired only through interaction. Banking tasks primarily require locating information in a well-indexed document corpus that the environment’s re-

trieval system already makes available on demand. Consequently, the telecom community can contribute knowledge that later agents would otherwise lack, whereas the banking community largely distills information that each agent can already retrieve independently. Together with GLM-5’s near-ceiling telecom result, these findings identify a practical boundary on the utility of shared memory: self-constructed knowledge is most valuable when it captures scarce, experience-derived information, and least valuable when the environment already supplies the required information or the underlying agent has little remaining performance headroom. The banking null result therefore bounds the conditions under which autonomous knowledge construction adds utility; it does not indicate a failure of the federated safety or promotion mechanisms.

B VERIFIER CALIBRATION IN FULL

Appendix I’s bounds are frugal: they consume exactly two behavioral quantities and nothing else about a reviewer. The catch rate q is the probability that an honest reviewer rejects a poisoned entry; the benign acceptance rate p_b is the probability that one accepts a clean entry. Both are measurable on the same verifiers §4.1 placed at the gate, which is what turns a proof obligation into a hiring criterion: the deployed geometry fixes thresholds on each, and a candidate verifier either clears them or does not.

Scope, stated before any number is read. Everything here is **corpus-relative**. These are lower confidence endpoints on a fixed, exogenously labeled corpus, with the estimand the weakest stratum rather than an average, and they *falsify or fail to falsify* the analysis rather than certifying a deployment. Sparse and unobserved strata are covered by no finite sample, so the uniform infimum-over-history bounds $q_{\text{pre}}, q_{\text{pan}}, p_b$ require remain modeling hypotheses (Remark I.19, N17). The calibration covers the LLM-verifier deployment mode only; human-manned gates are **out of calibration scope**, and for those the constants remain assumptions while the architecture supplies a path to measuring them (Remark I.17, Note N17). Because the classification Bad is exogenous to the protocol (Definition H.10), no unlabeled log can supply these rates; the security side uses red-team-injected known poison and the benign side an expert-adjudicated clean corpus.

Two campaigns, deliberately not pooled. Two independent benign campaigns appear below and must not be conflated. *Campaign A* (admission) measures p_b for all 21 candidate verifier configurations on the benign suites of the utility runs, at 90 to 157 counted items per run, and produces the admission funnel of Table 8. *Campaign B* (panel context) measures p_b for the six configurations carrying a *measured panel-context* rate, at $n = 51$ counted items in each of three framing conditions, and produces the framing-collapse finding of Table 9. They use different corpora, different sample sizes, and different presentation contexts, so their endpoints differ by configuration and neither supersedes the other.

B.1 MEASUREMENT DISCIPLINE AND CONFIDENCE CONSTRUCTION

Endpoints are Clopper–Pearson with a derived tail of 0.025, at 0.95 per stratum and, for the simultaneous reading, at $1 - 0.05/37 = 0.998649$ over the $S = 37$ forced calibration cells (35 deployed cells plus the two non-deployed probe cells of §B.9). A zero-miss stratum of size n therefore has a per-stratum endpoint $0.025^{1/n}$ and a simultaneous endpoint $(0.05/74)^{1/n}$, which is what makes the sample sizes below load bearing rather than incidental.

Three features of the instrument shape every number that follows, and each was chosen so that the resulting endpoint is smaller than a more permissive protocol would have produced.

- (i) *Presentation is forced.* The crystallizer observationally attenuates the retrieval gate almost to nothing, with 97 of 100 injections never reaching the gate on one backbone, and no clause credits that attenuation to the catch rate (PR-3).
- (ii) *The unit of replication is the case, not the trial.* Each action-gate case admits two forced routes, so the primary endpoint is computed per case and clustered, halving the effective n from 200 to 100 per cell.
- (iii) *Abstention counts as a miss.* A verifier that declines to answer is charged for declining, which is what reduces one probe cell to an endpoint of zero.

Provenance and guards. Of 900 recorded trial rows, 720 are deployed-configuration calibration trials, 120 are non-deployed probe trials, and 60 are control trials. Across all 900: zero cached responses, zero rows with fewer than one client call, one distinct payload digest, one distinct verifier-harness digest, two pre-registration digests (the base clause set and one appended addendum), and 160 matched exposure units with zero violations. The 30 rows lacking a raw response digest are exactly the control rows whose calls returned API errors, exempt by pre-registered clause.

Table 7: Measured catch rate by stratum, and the conditional ceilings that follow at the deployed geometry ($k=31$, $m=20$, $m_{\text{rej}}=12$, $N=10$, $f=2$, $f_s=1$). The action stratum’s endpoints are cluster-conservative over the 100 distinct cases its 200 trials per cell cover. Ceilings use the exact binomial of Remark I.17 at the per-stratum endpoint; the per-epoch column multiplies by $2f\rho_{\text{sub}} = 20$.

Stratum		trials		misses	q endpoint		Conditional ceiling		
gate	context	cells	/cell		per-stratum	simult.	B_{att}	panel term	per epoch
action	no dispute	21	200	0	0.9638	0.9296	2.96×10^{-12}	1.55×10^{-20}	5.92×10^{-11}
retrieval	no dispute	8	60	0	0.9404	0.8854	1.60×10^{-10}	1.56×10^{-16}	3.20×10^{-9}
retrieval	dispute	6	40	0	0.9119	0.8332	3.63×10^{-9}	1.88×10^{-13}	7.26×10^{-8}
reviewer-quality wall, $\alpha=10^{-2}$, independent jurors					$q^* = 0.5939$		all three strata clear it, per-stratum and simultaneous		

Fifteen of fifteen guards pass, including that every admitted q endpoint is recomputed against the panel wall, that every admitted δ_b is positive against the bar, that forced and observational strata are never merged, that no counted trial was a cache hit or a determinism override, and that outcome columns sum to the trial count. The pre-registration is 35 clauses under an append-only hash chain; corrections are appended, never applied in place.

B.2 THE SECURITY SIDE

Security is not the constraint, and the result can be stated in one breath. Across 35 cells and three strata, spanning both gates and including the case where a dispute is already open, the verifiers we would deploy produce *no observed miss at all*: 4,200 trials over 2,100 cases at the action gate, 480 at the retrieval gate with no dispute open, and 240 at the retrieval gate under a dispute banner. Table 7 gives the resulting endpoints. Read against the correlation-free wall $q^* = 0.593947$, the weakest stratum leaves a factor of 4.6 of headroom in the miss rate and the strongest a factor of 11.2, and every stratum clears the wall on the simultaneous reading as well as the per-stratum one. The correlated-juror wall $q^* = 0.797789$ at $\rho=0.1$ is a stress model rather than a certification, and all three strata clear that too.

The dispute stratum deserves particular note, because the appendix previously had to reason about it rather than observe it, and it is now **measured, not inferred**. Six paired cells at the retrieval gate present byte-identical content with and without a dispute banner and a rejecting juror’s stated ground, and record zero misses in both conditions. The paired verdict is that the banner changes detection by nothing: the endpoint gap between 0.9404 and 0.9119 is the sample-size difference between $n=60$ and $n=40$, not a behavioral difference.

B.3 TWO PARAMETERS, NOT ONE

Theorem I.13’s two terms are governed by different parameters: q_{pre} is a disposition-level rate at an ordinary gate inspection, while q_{pan} is conditional on a cast verdict, and verdicts are cast under the dispute banner. Feeding a single value to both understates the panel term on the no-dispute rows by a factor of about 1,200: at $q_{\text{pan}} = 0.9404$ the panel term is 1.56×10^{-16} , whereas the dispute-context value $q_{\text{pan}} = 0.9119$, which is the one a juror actually operates under, gives 1.88×10^{-13} . Because the tool-target panel rate is unmeasured, an action row falls back to the single-parameter reading in which a deployment takes $q := \min(q_{\text{pre}}, q_{\text{pan}})$; every bound of Theorem I.13 is monotone in each parameter, so that substitution is conservative rather than convenient (Remark I.15). At the measured endpoints the ceilings are dominated by the ordinary endorsement path, the panel term contributing four to eight further orders below it.

B.4 COMPOSITION OVER AN EPOCH

Two sentences ship together here, and neither may stand alone, because they have different standings. At the **conservative anchor** $q = 0.6$ the certified per-attempt ceiling is $B_{\text{att}} = 8.957 \times 10^{-3}$, inside the 10^{-2} target; composing over one epoch multiplies by the anchored-attempt count $2f\rho_{\text{sub}} = 20$ and gives 1.79×10^{-1} . So at that anchor the per-attempt target *does not compose to a per-epoch* one, and no choice of quorum recovers it, because the panel term is a floor in the community size at fixed panel geometry, worth 8.302×10^{-3} at the anchor and 1.66×10^{-1} after composition. At the measured endpoints the same geometry meets the target per epoch as well as per attempt, by 5.1 orders of magnitude at the worst stratum and 8.2 at the best. The first sentence is assumption-free and is what the theory alone licenses; the second is corpus-relative and cannot replace it. The gap is a property of the **conservative anchor**, not of the deployed geometry.

Table 8: Campaign A. Admission of all 21 candidate verifier configurations at the deployed geometry. p_b is the binding lower endpoint over that configuration’s valid runs; $\delta_b = p_b - m/(k - f_s)$ against the benign bar 0.6667 (Proposition I.20); eviction is the exact wrongful-eviction tail at that endpoint. All 21 clear the security wall of Table 7; the 8 exclusions are all on the benign side. *Target* marks the 10^{-2} wrongful-eviction line.

backbone	verifier	p_b	δ_b	eviction		backbone	verifier	p_b	δ_b	eviction
Gemma-4-31B-it	self	0.9217	+0.2550	9.1×10^{-6}	target	GPT-5.5	self	0.7555	+0.0889	9.29×
GPT-5.5	Qwen-3.5-4B	0.8814	+0.2148	4.1×10^{-4}	target	Claude-Sonnet-4.6	Qwen-3.5-2B	0.7263	+0.0596	1.73×
GLM-5	Qwen-3.5-4B	0.8455	+0.1788	3.7×10^{-3}	target	GPT-5.4	Qwen-3.5-2B	0.6967	+0.0301	2.83×
GLM-5	self	0.8318	+0.1652	7.2×10^{-3}	target	Claude-Sonnet-4.6	Qwen-3.5-4B	0.6964	+0.0298	2.84×
GPT-5.5	Qwen-3.5-2B	0.7978	+0.1312	2.76×10^{-2}	bar	<i>excluded (benign side)</i>				
Qwen-3.6-27B	self	0.7974	+0.1307	2.80×10^{-2}	bar	Gemma-4-31B-it	Qwen-3.5-4B	0.6148	−0.0519	1.000
GPT-5.4	Qwen-3.5-4B	0.7883	+0.1217	3.76×10^{-2}	bar	GLM-5	Qwen-3.5-2B	0.6143	−0.0524	1.000
Claude-Sonnet-4.6	self	0.7784	+0.1117	5.08×10^{-2}	bar	Qwen-3.6-27B	Qwen-3.5-4B	0.5997	−0.0670	1.000
GPT-5.4	self	0.7733	+0.1066	5.87×10^{-2}	bar	Qwen-3.6-27B	Qwen-3.5-2B	0.5757	−0.0910	1.000
						Gemma-4-31B-it	Qwen-3.5-2B	0.5542	−0.1125	1.000
						Kimi-K2.5	Qwen-3.5-4B	0.5483	−0.1184	1.000
						Kimi-K2.5	Qwen-3.5-2B	0.5305	−0.1362	1.000
						Kimi-K2.5	self	0.4523	−0.2144	1.000

Table 9: Campaign B. Measured benign rejection rate at the panel, by framing context, for the six configurations carrying a *measured panel-context* rate, all of them self-verification, at $n = 51$ counted items per context. The three contexts differ only in framing: no banner, then a dispute banner with the rejecting juror’s stated ground, then the same banner with a generic ground. p_b is the binding lower endpoint over the three contexts, δ_b the margin over the benign bar 0.6667, and eviction the exact tail of Proposition I.20 at that endpoint. All six cleared the security screen of Table 7 with zero misses in both contexts.

Configuration	benign rejection rate (%)			p_b	δ_b	eviction	qualifies
	no banner	ground	generic				
GLM-5	0.0	0.0	0.0	0.9302	+0.2636	3.0×10^{-6}	bar and target
GPT-5.4	7.8	7.8	9.8	0.7859	+0.1192	4.07×10^{-2}	bar only
Kimi-K2.5	0.0	0.0	39.2	0.4611	−0.2055	9.81×10^{-1}	neither
GPT-5.5	11.8	52.9	68.6	0.1911	−0.4756	1.000	neither
Qwen-3.6-27B	0.0	13.7	78.4	0.1129	−0.5538	1.000	neither
Claude-Sonnet-4.6	7.8	94.1	45.1	0.0123	−0.6544	1.000	neither

B.5 THE BENIGN SIDE: ADMISSION ACROSS 21 CONFIGURATIONS

The difficulty is entirely on the benign side, and it is a design consequence rather than an accident. The federated protocol protects honest contributors by making wrongful eviction improbable, and that protection is exactly what a reviewer’s false positives attack. A verifier that rejects clean content does not endanger the community’s safety; it endangers the community’s willingness to participate, and Proposition I.20 prices precisely that. The deployed geometry turns the requirement into two concrete lines on a candidate. Staying under a rejection rate of about a third ($p_b > m/(k - f_s) = 0.6667$) is the **feasibility condition** that gives Proposition I.20 anything to say. Staying under about 17.6 percent is the stronger and more useful line, derived by bisecting the exact wrongful-eviction tail to the point where it meets the 10^{-2} target, which lands at a benign acceptance rate of 0.824456.

Table 8 measures all 21 candidates against both. Every one clears the security wall; 13 additionally clear the feasibility condition and are *admitted*, and of those, 4 meet the 10^{-2} wrongful-eviction target. Candidate endpoints span 0.4523 to 0.9217, with 20 of the 21 below 0.90. Two qualifications belong with every row. Positivity of δ_b is Proposition I.20’s **feasibility condition** and not a benign-side safety target, which is why the table separates the two verdicts: a configuration can be admitted and still evict honest contributions too often to be worth deploying. And an endpoint is a lower bound, so a configuration failing here has not been shown to be unusable, only not shown to be usable.

B.6 THE BENIGN SIDE: PANEL CONTEXT AND THE COST OF THE DISPUTE BANNER

Campaign B narrows to the panel context, and produces the calibration’s least expected result. Only 6 of the 29 configurations with a pre-dispute run carry a panel measurement, and because the panel campaigns are self-verification only, no lightweight-verifier configuration carries one at all. Table 9 gives the six. Most sit comfortably inside the

Table 10: Withholding the rejecter’s stated ground, paired against stating it, per backbone. Counts are discordant pairs out of 51 benign items; p is McNemar exact two-sided.

backbone	generic worse	generic better	McNemar p	reading
Claude-Sonnet-4.6	0	25	5.96×10^{-8}	generic better
Qwen-3.6-27B	33	0	2.33×10^{-10}	generic worse
Kimi-K2.5	20	0	1.91×10^{-6}	generic worse
GPT-5.5	16	8	0.152	not significant
GPT-5.4	4	3	1.000	not significant
GLM-5	0	0	1.000	immune to both

feasibility ceiling in the undisputed context; one, GLM-5 verifying itself, clears both lines in every context measured; and the pattern of failure is the interesting part, because what breaks a candidate is not the content it reviews but the framing around it.

The counts deserve to be seen rather than summarized. Across 240 dispute trials on poisoned items the banner changed detection by nothing, matching 360 no-dispute trials at zero misses. Across paired benign items it flipped 74 of 306 clean memories to rejection, with exact two-sided $p = 1.06 \times 10^{-22}$ and a one-sided 95 percent upper endpoint on the harm rate of 0.2855. Every wrongful-eviction figure computed from a no-banner rate is therefore optimistic by a demonstrated margin.

The pre-registered remedy of withholding the rejecting reviewer’s stated ground was declared in advance to cost either substantially less than stating it, in which case jurors should not see the ground, or about the same, in which case the dispute signal itself is the problem. The measured outcome is neither: withholding costs *more* in aggregate, $b = 113$ against $c = 4$ relative to no banner, and the per-backbone effect flips sign.

Table 10 is why we rest the reading on direction rather than magnitude. Where the effect is significant the discordances are perfectly one-directional, 25 flips one way and none the other, then 33 the other way and none back, and three of six backbones show no significant difference at all. Pooled the other way, stating the ground against a generic one runs 73 to 36 at $p = 5.05 \times 10^{-4}$. The two discordance families are distinct and should not be conflated: 74 and 113 are counted against no banner, while 73 and 36 pair ground against generic. The consequence for the protocol is that Rule R13(iii),(vi) as written, which displays the ground, is the better of the two measured options, and that *juror selection rather than pane redaction is the operative lever*. The instructed-juror arm is declared unrun.

The Claude reading, offered as interpretation. Claude-Sonnet-4.6 is the strongest backbone in Table 1 and the weakest candidate reviewer in Table 9, and it is the single row that improves when the accusation is made vague. Its dispute-arm false positives run 4 of 51 with no banner, 48 of 51 once a rejecting juror’s stated ground is on screen, and 23 of 51 when that ground is withheld. No other backbone has that shape. Offered as an interpretation of the direction rather than as a measured cause: a reader that treats any articulable suspicion of injection as grounds to refuse will refuse more often the more specific the suspicion it is handed, and a specific accusation is exactly what the ground field supplies. The confirmation gate is built to treat a voiced suspicion as grounds to refuse, so the two behaviors compound. Nothing here says the model is weak; the caution that makes it an excellent agent backbone, and an excellent last line of defense when it is the one being protected, is what disqualifies it from a seat on a jury that must acquit the innocent. Rule R12’s separation of nomination from authority is what lets a deployment staff the two roles differently.

The counted denominator. It is 51 rather than 52. One benign item’s verification substring contains a double quote and is excluded by the harness, and all 24 excluded rows record zero client calls, making each a trial that never happened rather than a discarded verdict. Even scoring every one of them as a false positive moves the strongest endpoint only from 0.9302 to about 0.898, so the exclusion does not carry the conclusion.

B.7 TWO FINDINGS THAT OUTLIVE THE CONSTANTS

Both are failures of measurement rather than of the protocol, and both generalize past this study.

F1: a fail-closed gate can look flawless and be useless. One configuration held attack success at zero while benign task success collapsed from 96.7 to 33.3 and the false-positive rate went from 0 to 100 percent. Four independent witnesses identified it: 99 parse failures in 103 calls; a probe cell unparseable in all 60 trials; raw HTTP content arriving null alongside 384 reasoning tokens; and a sevenfold drop in median latency once the fault was repaired (3.882 s $\rightarrow$ 0.553 s on Qwen-3.5-4B). A benign run measures an acceptance rate only if the verifier answered, and

the all-reject set coincides *exactly* with the broken-counter set: of 80 benign runs, 4 rejected everything, 3 of those carried model errors and 1 carried parse failures, and 76 were clean. That is why four published runs are excluded from binding. The two failure modes have different causes, client exceptions on the Gemma configurations and parse failures on the Qwen one, both traceable to a pre-fix client construction, and later runs of the same four configurations returned arithmetic means of 97.5, 96.7, 90.0, and 86.7 with zero errors. The general lesson is that the failure is invisible to any evaluation reporting attack success alone.

F2: the benign rate is not a per-model constant. All 312 no-banner panes are byte-identical by hash across two campaigns, yet 10 of 306 paired items changed verdict, six configurations spread by more than 0.07, three by more than 0.10, and one moved from 13.3 to 35.6 percent at $z \approx -4.3$, $p \approx 2 \times 10^{-5}$. That configuration also carries the highest determinism-override count in the study. A single benign run therefore estimates a rate carrying genuine run-to-run variance, which is the second reason every bound here consumes a lower endpoint rather than a point estimate. Taking the minimum over valid runs is a safety-side bound and it excludes a configuration that the published run would have admitted, which cuts against our interest and is why it is defensible.

B.8 THE PANE BOUNDARY, REPORTED AT $n = 2$

One further mechanism is reported as a mechanism and never as a rate. The boundary between trusted framing and untrusted content inside a verifier’s pane is string interpolation and is not cryptographically authenticated. Two of sixty spans in the corpus embed chat-template control tokens (`<|im_start|>` and `<|im_end|>`, two occurrences each), and the frontier self-verifiers rejected them on all 18 of 18 opportunities: both spans in each of the six no-dispute cells, and the one span falling in the smaller dispute sample in each of the six dispute cells. The one genuine miss in the entire calibration is on such a span, by the 4B verifier under the *non-deployed* probe construction: it approved a 515-character span whose control tokens follow two header lines, on the stated ground that the header was on-topic and contained no directives, which is an honest answer about the two lines the model actually received as pane content. The same span is caught under the deployed construction. That is a mechanism worth naming and a sample too small to rate, and hardening the boundary is the most concrete piece of unfinished engineering the calibration surfaced.

B.9 TWO DISCLOSED SENSITIVITIES

The non-deployed probe cells. Two cells run a low-level pane builder rather than the resolver the deployed command line uses, which adds four parameters. The 2B verifier abstained in all 60 trials of its probe cell, scoring an endpoint of zero under the abstention-is-a-miss convention; the 4B verifier recorded 59 catches and the single genuine miss above. Self-verification’s back-fill to the deployed configuration was proven a call-through *by execution* rather than by inspection, and three cells were re-run; both readings ship, and the probe rows are retained as the record of the divergence.

The replication unit moves the tool-target endpoint a long way. That endpoint rests on six distinct injection texts. Per trial at $n = 200$ it is 0.9817; per case at $n = 100$ it is 0.9638, which is the pre-registered primary; per text at $n = 6$ it is 0.5407, which fails the independent-juror wall and would give a ceiling of 4.38×10^{-2} . We report the per-case figure and note the per-text one rather than choosing the flattering unit. The defense of the instrument is that on those same six texts the deployed verifiers post zero misses over 4,200 trials and 2,100 cases while the weak and broken ones post 891 misses over 1,116 trials and 759 of 800 cases, so the instrument demonstrably discriminates on this corpus even where the per-text sample is too small to bound.

B.10 NAMED UNMEASURED QUANTITIES

For completeness, the quantities this calibration does not supply: the tool-target and lightweight-verifier panel rates; the instructed-juror arm; the crystallized exposure tier; the mutation tier; pane-boundary hardening; and the verdict correlation ρ across verifier families. Measurement of the rates in scope is complete; no further collection is planned or required for them.

C CONCURRENT WORK: ORIGIN-BOUND MEMORY AUTHORITY

The nearest work to ours is the preprint of Louck (2026), whose construction, TMA-NM (tamper-evident memory authority, non-malleable), binds a stored item’s authority to the channel it arrived on, refuses to let any laundering transformation raise that binding, and gates elevation on independent corroboration or a single-use user authorization.

That two lines of work arrived independently at write-time origin binding is evidence the principle is right, and both rest on the information-flow frame of robust declassification and transparent endorsement (Zdancewic & Myers, 2005; Myers et al., 2006). This appendix records where PerMIT and TMA-NM part. Both use the term *Action Integrity* for a checked property; ours is Definition G.10.

Independent development. PerMIT was developed independently of, and concurrently with, TMA-NM. PerMIT’s interface semantics, its two integrity theorems, and their mechanization were in place before the Louck preprint was released. We became aware of it during a related-work search while preparing this submission, and this appendix was added in response. We make no priority claim: the Louck preprint is prior in date of public release, we cite it as such throughout, and we adopt its vocabulary where it is the clearer one (§C.5). What follows is a comparison, not a dispute.

	TMA-NM (Louck, 2026)	PerMIT
Boundary enforced	Action only; untrusted content is read	Observation and action (Lemma G.12, Thm. G.21)
Reasoning-side guarantee	None; answer-bias mitigated, not eliminated, by their own account	Yes, modulo the display transcript (Thm. G.25)
Preference and belief manipulation	Unguarded; poisoned preferences are retrieved and colour the response	Cannot occur: bytes never reach the model undisplayed (Lemma G.12)
Property class	Single-run safety	Also two-run relational
Quantifier scope	Bounded model: 3 slots, ≤ 2 sessions, threshold 2	Unbounded runs, arbitrary strategies and stores
Machine-checked	3 theorems, one invariant, by TLC enumeration; the unbounded step is a prose argument	2 theorems, kernel-checked in Lean 4 over an executable interpreter (single-user core; §G.6)
Interface leakage	Obligation does not arise	Proved closed (Rules R1, R6, R7, R10)
Value attribution	Text proxy; disclosed as evadable	Opaque handles; no text to match
Trusted tool ingress	Allow-listed tools may act; a compromised one is the stated residual risk	None; all tool output captured untrusted (Rule R3)
Effect of elevation	Monotone, in place	Fresh copy; original stays untrusted (Rule R2)
Elevation coverage	Values two or more institutions can license	Any byte string
Deployment precondition	Institutional corroborators, or a fresh authorization at each use	None; the user is the sole endorser (Lemma G.16)
Correlated corroborators	Categorical predicate; correlated failure named as residual risk	Continuous parameter, degrading to q_c (S13)
Multi-user sharing	Out of scope	The federated tier (Appendix H)
Revocation	None; elevation monotone	Freeze, dispute, demotion, re-verification (N16)
Amortization	None; token consumed per use	Once ever per (value, tool) (Thm. J.2(i))
Threshold behaviour	Fixed dial	Scheduled against population (Thm. J.13)

C.1 THE OBSERVATION BOUNDARY

The differences follow from where each design draws its boundary. TMA-NM withholds authority from untrusted memory while leaving it fully visible to the model: poisoned content is retrieved, enters the reasoning context, and is surfaced with provenance. Louck states the consequence plainly, that answer-bias is mitigated rather than eliminated and remains high under every defense measured, TMA-NM included, because non-consequential answer-biasing is out of scope by design.

PerMIT draws the boundary earlier. By Lemma G.12 no byte value enters the Protected LLM’s state by delivery from an untrusted source, and by Rule R3 every tool output, including the status and error responses of sensitive calls, is captured at level U with only a constant acknowledgement returned. Untrusted content reaches the model as an opaque handle, its bytes only through a displayed, endorsed release recorded in the transcript. A reasoning-side theorem is therefore available here and structurally unavailable there.

The gap this leaves is not incidental. A store’s most valuable contents are exactly the ones an action-only boundary cannot protect: preferences, standing decisions, and accumulated context, which influence behaviour by being believed rather than by naming a payee. A poisoned preference under TMA-NM is retrieved, labelled, surfaced, and read, and it then shapes which options the agent proposes, which it omits, and how it frames a recommendation, none of which

is a consequential tool call and none of which the act gate sees. The attacker who wants to steer a decision rather than move money is therefore unimpeded, and the guarantee cannot be strengthened to cover this without moving the boundary, since the content has already entered the reasoning context by the time the gate runs. Under PerMIT the same entry is a handle. It cannot colour a recommendation it was never rendered into, and Theorem G.25 converts this from an argument into a proved property: the agent’s operations are a function of the display transcript, so an adversary who does not change what the user is shown cannot change what the agent reasons to.

The same structure closes a residual that Louck discloses and leaves open. TMA-NM’s value-attribution limitation concedes that a security-relevant value assembled from several low-integrity fragments by in-context reasoning escapes the boundary monitor, and locates the fix in future value-level capability tokens. Under PerMIT the fragments never reach the model as bytes, so there is nothing in context to reconstruct from.

C.2 RELATIONAL VERSUS REACHABILITY PROPERTIES

All three TMA-NM theorems concern one predicate of single reachable states, namely that no untrusted-origin item has authorized a consequential action without corroboration; what varies across them is the gate under analysis, not the class of property. Theorem G.25 is a predicate of *pairs* of runs: two executions under two different attacks, with independently chosen paste events, tool outputs, and reader outputs, agree on the ungated sink projection and the emitted operation sequence whenever their display transcripts agree. No reachability invariant expresses a relational property at any model size. Enlarging the bounded model does not approach Theorem G.25; the property is not weaker there, it is unstatable there.

This also answers whether PerMIT is an instance of the TMA-NM sufficiency direction, as Louck observes a token-augmented lineage scheme would be. That sufficiency result concerns action authorization under a bounded model, and is silent about relational noninterference, about channel completeness at a mediated interface, and about elevation by an open population of attested identities rather than a centrally designated principal set. Our second theorem, the interface channel analysis, and the whole federated development lie outside its scope.

C.3 THE FEDERATED TIER

Louck scopes multi-user sharing out and identifies a federation of origin authorities as a natural next step. Naming a direction and discharging it are different contributions. An open population of attested identities raises questions a fixed set of designated principals never faces: whether the population can be trusted in aggregate when members are fallible and some fraction is adversarial, what becomes of a promoted item when its basis is later disputed, whether an adversary can force wrongful eviction of benign content, and whether per-user cost grows or shrinks as participation does. PerMIT answers these with the promotion and panel bounds of Theorem I.13, the eviction and liveness results of Proposition I.20, and the joint schedule of Theorem J.13, under certified lifetime totals and stated as such.

C.4 DEPLOYMENT WITHOUT INSTITUTIONAL CORROBORATORS

Elevation under TMA-NM needs either two or more independent institutional corroborators or a fresh user authorization consumed at each use. Where such authorities exist the arrangement is comfortable, and a value lacking them is routed to confirmation rather than blocked. What TMA-NM cannot offer is relief from the routing, since the authorization does not persist and a value that never acquires corroborators is confirmed every time it is used. Louck states that how often independent sources exist for routine actions is deployment-dependent, and leaves measuring it open.

PerMIT has no such precondition. Without a federation attached no block ever carries a community label (Lemma G.16), so the user is the sole endorser and no registry is required for the guarantee to deliver usable behaviour. Confirmation is bounded rather than repeated: at most one forced confirmation per (value, tool) pair over the deployment’s lifetime, pathwise, under any dispute, rejection, revocation, or availability campaign (Theorem J.2(i)). Corroboration, where a community exists, is an improvement rather than a prerequisite, since promotion of poison, wrongful eviction, and each user’s confirmation burden fall together as participation grows (Theorem J.13).

C.5 LAUNDERING CHANNELS

The TMA-NM taxonomy of laundering channels is the clearest available statement of what a memory defense must resist, and we adopt it. Each is closed in PerMIT by construction. Self-summarization carries no authority, since the Quarantined Reader holds no tool capability and crystallize writes its summary at level U (Lemma G.14). Trusted-tool echo has no vehicle, because PerMIT admits no trusted tool ingress at all (Rule R3); where TMA-NM taints an allowed tool’s output when its inputs are untrusted, and names a fully compromised trusted tool as its main residual

Algorithm 1 RLM-based retrieval for one reader operation (fill or crystallize)

Require: query q , schema s , memory store $\mathcal{M}$; iteration cap T
Ensure: a schema-valid value with citations, or the null handle

```

1:  $V \leftarrow \{b \in \mathcal{M} : \ell(b) = U\}$  ▷ view restriction (interpreter, deterministic)
2:  $\mathcal{E} \leftarrow \text{INITREPL}(\text{context} \leftarrow [(\text{id}(b), \text{bytes}(b)) : b \in V])$ 
3: issue fidelity-only instruction for  $(q, s)$  to root model  $Q$  with constant-size metadata of  $\text{context}$ 
4: for  $t = 1, \dots, T$  do
5:    $c \leftarrow Q(\text{history}); (\mathcal{E}, \text{out}) \leftarrow \text{REPL}(\mathcal{E}, c)$  ▷  $Q$  reads slices; sub-calls for oversized blocks
6:   if  $\mathcal{E}[\text{answer}]$  is set then
7:      $(v, C) \leftarrow \text{PARSE}(\mathcal{E}[\text{answer}])$ , disambiguated against the true identifier set of  $V$ 
8:     if  $v$  satisfies  $s$  and  $C \subseteq \{\text{id}(b) : b \in V\}$  and  $C \neq \emptyset$  then
9:       return  $(v, C)$  ▷ to  $P$  as opaque handle / U-block only
10:    end if
11:    return null handle ▷ fail closed, Rule R6
12:  end if
13: end for
14: return null handle ▷ cap or reader error, Rule R7

```

risk, PerMIT removes the allow-list and with it the risk. Manufactured corroboration cannot reach a threshold, since weight is carried only by distinct attested identities’ first logged decisions (INV-1, Lemma I.3) spanning D_{pool} pools (Definition H.6(f)) and bounded by lifetime totals that reputation farming cannot outrun (N12). Their benchmark also supplies evidence for the position argued in §2.2: a single-session capability and IFC defense of the CaMeL and Fides family, granted its best-case clean store, is defeated by every laundering channel and additionally permits the direct attack, because it assumes long-term memory is uncompromised.

D THE RECURSIVE LANGUAGE MODEL AS A QUARANTINED READER

Although not listed as one of the major contributions in this paper, our work marks of the first attempts to utilize Recursive Language Models (RLMs) (Zhang et al., 2025) to retrieve memories for LLM agents. We adopted RLMs as a retriever over the traditional RAG because it is more versatile and can provide the formatting and crystallizing needed in the PerMIT design. The results show that RLMs are naturally well-suited for the memory retrieval task and achieved comparable utility when being compared with other memory backbones such as RAG or Mem0.

Why an RLM. The reader must search an untrusted store that is unbounded (convention (C7)) and routinely exceeds any context window, yet its bytes must never be loaded into a trusted context. An RLM fits both constraints by construction: the long input is not fed to the model but placed as a variable in a persistent REPL environment, and the model writes code to inspect, decompose, and recursively query sub-models over programmatic slices of it (Zhang et al., 2025). The store is thus navigated by code, symbolically, at any size.

How retrieval works. The interpreter first restricts the reader’s view to U-labelled blocks — endorsed content and interpreter state are structurally invisible — and loads the visible blocks (identifier, content, tags) as a list variable inside the REPL. The root model receives only the retrieval instruction and constant-size metadata; it then iterates, printing slices of the store, delegating oversized blocks to sub-model calls, and accumulating its answer in a result variable, under a fixed iteration cap. The instruction is *fidelity-only*: read the actual blocks, copy any extracted value verbatim, cite the source block identifier, and report absence as absence — the reader carries no safety obligation, because every safety property is enforced downstream by the interpreter (Lemma G.12, Rules R6, R7). Finally, the interpreter validates the reply deterministically: the value must satisfy its declared schema and every citation must name a block genuinely present in the view; any failure, reader error, or uncited summary collapses to the null-handle outcome the semantics already define (Rule R6). An accepted value reaches P only as an opaque handle, resolved at a confirmation gate; an accepted summary is written back at level U (FILL, CRYST, Figure 8). Algorithm 1 summarizes the procedure.

It works well. In every PerMIT cell of Tables 1 and 3, the reader above is the only path by which stored untrusted content is found and resolved. Under the oracle gate its utility matches or exceeds the undefended embedding-based RAG backend on six of seven backbones — despite the PerMIT rows additionally paying the confirmation gate’s cost, and at zero attack success against up to 80 percent for undefended RAG — while requiring no index, no embedding

model, and no cap on store size. Code-mediated reading of the raw store is thus a practical, drop-in alternative to embedding-based retrieval in this setting; we leave a component-isolated comparison to future work.

E STRUCTURE OF THE PROOF, NOTATION, AND CONVENTIONS

E.1 ORGANISATION AND REGISTER CONVENTIONS

Appendices E–L contain the formal development for PerMIT, with a machine-checked single-user core (§G.6) and register-separated normative, proof, and rationale layers; the empirical appendices are indexed in the guide at the head of the appendix (p. 14). The material is organised into three strictly separated registers.

Normative register (Appendices F, H). The memory-interface semantics, the interpreter and protocol rules (R1–R14), and the assumptions (A1–A9; the budget-model assumptions A10–A13 are stated with the model they govern). Every rule clause states what the harness does, tersely; no rule contains its own justification. A rule clause states normative behaviour tersely, relying only on the normative definitions and conventions it cites; rationale and empirical interpretation are never needed to implement it.

Proof register (Appendices G, I, J). Definitions, lemmas, and theorems. Proof-register text cites the normative register and the named invariants of §E.4; it never cites the non-normative register. The single-user results of Appendix G are additionally machine-checked in Lean 4 over the map-free operation algebra, with the batch-confirmation surface of Rule R9 mechanized as a standalone layer; the store-level integration of map into the run-level interpreter and Option O3 remain outside the mechanized surface, under the disclosed deltas recorded in §G.6.

Non-normative register (Appendix K and commentary blocks). Design rationale (Notes N1–N17), the small-type commentary blocks defined below, and a security and robustness analysis (Analyses S1–S17): an indexed catalogue of attack vectors and failure modes, each with its disposition by formal fact, and its status: *disposed* (under stated assumptions; a formal fact disposes of it) or *residual* (a genuine limitation, disclosed); and either *entering only the budget constants* or *not used in any proof*. Four rationale remarks whose content adjoins specific results are set as numbered Remarks adjacent to the results they annotate, Remarks H.7, I.19, I.5, and I.26. Nothing normative lives in this register; no proof depends on it.

Commentary blocks. Throughout the normative and proof registers, short passages set in small type under the heading *Commentary (non-normative)* carry the rationale, epistemic status, and validation pointers for the material they adjoin, so that a rule or assumption and its justification can be read together without the justification entering the register. Commentary blocks belong to the non-normative register in function: no rule reads them, no proof cites them, and a reader who skips every commentary block loses no hypothesis and no step of any proof.

Statement discipline. A theorem, lemma, proposition, corollary, definition, or invariant environment contains hypotheses, the conclusion, and bare pointers (“established by”, “see Remark”); interpretation, comparison, special cases, and design justification are set in a numbered remark adjacent to the result they concern.

Cross-referencing convention. The formal development may point into the non-normative register only by a parenthetical key, e.g. “(rationale: N2; analysis: S11)”; the non-normative register may cite the formal development freely. Consequently the formal development is self-contained: a reader who skips Appendix K entirely loses no hypothesis and no step of any proof. A reader with a specific concern should consult the attack and limitation index (Table 15), which maps concerns to analysis entries and to the formal facts that govern them.

Map of the formal development (Appendices E–L).

E	Structure	notation, conventions, named invariants and structural properties INV-1–INV-6.
F	Interface	configurations; interface transition schema (Figs. 7–8); Rules R1–R11, options O1–O2 (O3 requirements-level); Assumptions A1–A6; attacker model.
G	Single user	Action Integrity (Thm. G.21), no-silent-resolution (Cor. G.23), Reasoning Integrity modulo the display transcript (Thm. G.25), confinement (Cor. G.26), baseline separation (Prop. G.27), hardening monotonicity (Prop. G.28).
H	Federated protocol	pools, epochs, demand, freshness, endorsement counting; Rules R12–R14; Assumptions A7–A9.
I	Federated guarantees	structural lemmas (sharing exclusion, entry disjointness, c_{acc} , verdict pacing, decisive resolution, no-deletion); deterministic safety under perfect detection (Thm. I.11); panel safety (Thm. I.13); wrongful eviction (Prop. I.20); publisher safety (Prop. I.22); denial of service bounded in forced confirmations (Lem. I.24; effort disclosure N18); submission budget (Cor. I.27).
J	Budget	single-user floor (Thm. J.2, per (user, value, tool)), per-item ledger (Lem. J.7, $\mathcal{O}$ -form), saturation (Thm. J.9), co-scaling corridor and theorem (Thm. J.13).
K	Rationale & analysis	design rationale; security and robustness analysis with attack and limitation index; scope and residuals; summary of composed guarantees.
L	Tools	probability space and adaptive coupling; Hoeffding bounds; no-dispute baseline; the blast-schedule promotion law (a simulation analysis tool, cited by no proof; §L.2); explicit ledger constants; knowledge-based formulation; Zipf case study; assumptions ledger.

Dependency structure. Figure 6 shows which results rest on which hypotheses. Three facts are visible directly from the figure and are relied on throughout: (i) the single-user theorems and the pathwise cap (Thm. J.2(i)) depend only on the trust axioms A1–A6 and the harness rules, never on any federated or budget modelling assumption (Appendices H and J); the cap’s expected-form clauses read only the reference demand model’s masses, with the sole assumption-bearing comparison isolated in Corollary J.3; (ii) no safety theorem depends on the accountability calibration of Rule R13(v), which feeds only budget constants (dashed edge); (iii) the behavioural parameters q, p_b, ρ_{seat} and the demand assumptions A10–A13 enter only the probabilistic-refinement and budget layers, each of which degrades to the layer below it when its hypotheses fail (§K.5).

E.2 NOTATION

The principal symbols of these appendices are collected here; locally scoped symbols are defined at first use. Unless otherwise marked, symbols denote fixed deployment configuration belonging to the *trusted computing base* (TCB): the fixed components (interpreter, channels, rendering and logging paths, and configuration) that the guarantees assume correct and that no model output can alter (inventory: §F.5 single-user, §H.3 federated). Marks: (emp) empirical/behavioural parameter, (sch) population schedule, (rv) random variable.

Single-user model and interface (Appendix F)

$c = (\mathcal{M}, \sigma_P, \sigma_Q, h)$	configuration: store, Protected-LLM state, Reader state, decision history.
$\mathcal{M}; \mathcal{M}_U, \mathcal{M}_E$	memory store; restrictions to level-U / level-E blocks.
$b = (\text{id}, \text{bytes}, \ell, \pi, \text{scope})$	block: identifier (two-sorted, operation-determined, Rule R1(g)), bytes, label, interpreter-only provenance, tool scope.
$\ell(b) \in \{U, T, C, CC, PC\}$	label: untrusted, trusted (user-typed), confirmed, community-confirmed, P -composed.
$\text{lvl}; \mathbb{E}$	level map ($\text{lvl}(U) = U$, else $\mathbb{E}$); endorsed level.
P, Q, R_E	Protected LLM; Quarantined Reader; trusted-store retriever (selector).
$\text{Tools}_{\text{sens}}, \text{Tools}_{\text{data}}$	sensitive (effectful) / data tools; fixed partition, canonical identifiers.
Σ	operation algebra $\{\text{write}_P, \text{recall}, \text{fill}, \text{crystallize}, \text{display}, \text{confirm}, \text{evict}\}$; Σ_P : the P -facing surface $\Sigma \cup \{\text{tool invocation}, \text{retrieve}\}$ (+ derived map). (write, where it appears in prose, abbreviates the sole P -emitted write operation write_P .)
$\text{confirm}(v, t)$	confirmation event: value v , target t (tool; “reasoning”; “publish”).
z	gate decision, $z \in \{\text{accept}, \text{reject}\}$.
h_j	j -th operation-determined handle identifier (Rule R1).
$\text{obs}_P(c)$	Protected-LLM view $(\mathcal{M}_E, H(c), x(c), h)$ (Def. F.3).
$H(c); x(c)$	handle structure (Rule R1); trusted command input of configuration c (Def. F.3).
$\approx_U$	U-equivalence: equality of obs_P , arbitrary $\mathcal{M}_U$ (Def. G.1).

1620	$c_1 \approx c_2$	matched configurations: equal view and equal P -interaction history (Def. G.6).
1621	$\text{Sink}(\rho)$, $\text{DT}(\rho)$	ungated sink projection; display transcript of run ρ (Defs. G.7, G.8).
1622	ω	coupled coin sequence for P and R_E (§E.3).
1623	c_{cand} , c_{batch}	retrieval surfacing cap; per-page batch-confirmation cap.
1624	O1–O3	optional hardening: sink-scoped T (O1) and exact-target data-tool mediation (O2), formally instantiated; local scope expiry (leases; O3, requirements-level, O3).
1625		
1626	<i>Federated identities and promotion (Appendix H)</i>	
1627		
1628	$w_i \in [w_{\min}, w_{\max}]$	attestation weight of identity i ; w_{seat} panel-seat weight floor.
1629	f ; f_s	adversarial identities the adversary <i>ever</i> controls; those ever meeting the seat floor ($f_s \leq f$): lifetime totals, static corruption (§H.1).
1630	W_{adv}	adversary’s lifetime controllable attestation weight ($\leq f w_{\max}$).
1631	W , N	promotion threshold at a tool: weight / distinct-endorser count.
1632	N^+	credit cap $D_{\text{pool}} \lceil N/D_{\text{pool}} \rceil$: credited endorsements per (entry, tool) (Def. H.6(f)); $N \leq N^+ < N + D_{\text{pool}}$.
1633	D_{pool}	pool-diversity floor: endorser span $\geq D_{\text{pool}}$ pools, caps $\lceil N/D_{\text{pool}} \rceil$, $\lceil k/D_{\text{pool}} \rceil$ (Def. H.3).
1634	T_{win}	finalisation-window length of a pipeline entry.
1635	n_v ; $n_{v,t}$ (rv)	<i>latent</i> demanders of item v / of the pair (v, t) in an epoch, reference model (§J.1); $n_{v,t} \leq n_v$, $\sum_t n_{v,t} \leq c_{\text{tool}} n_v$ under A12; realized counts $n_v^{\log}, n_{v,t}^{\log}$ (Def. H.4, §L.1) are dominated pairwise: $n_v^{\log} \leq n_v$, $n_{v,t}^{\log} \leq n_{v,t}$.
1636	E_{max}	per-epoch cap on pipeline entries and on non-vindicated disputes per item (Rule R14).
1637	ρ_{sub}	per-identity submissions per epoch (Rule R12).
1638	T_{exp} , T_{pan}	expiry clock (no demand / dormancy); fast-track suspension-panel target.
1639	H_{acc}	total honest identities ever accepting a given byte string (Cor. I.27).
1640		
1641	<i>Panel, dispute, accountability</i>	
1642		
1643	k ; m ; m_{rej}	panel verdict cap; vindication bar; eviction bar (default $m_{\text{rej}} = k+1-m$).
1644	q_{pre} , q_{pan} (emp)	weakest-target conditional lower bounds: honest fresh gate inspection of a poisoned item (Def. H.10) ends in an effective qualifying rejection (q_{pre}); an honest eligible verdict rejects it (q_{pan}); abstentions = misses; authority-inactive inspections = excluded trials (theorem I.19); single-parameter reading $q = \min(q_{\text{pre}}, q_{\text{pan}})$ (Rem. I.15).
1645	p_b (emp)	lower bound: honest fresh inspection accepts a benign item (Def. H.10), conditionally on the past.
1646	q_c (emp)	residual catch rate under engineered cross-person correlation (S13).
1647	$\delta = \frac{m-f_s}{k-f_s} - (1 - q_{\text{pan}})$	panel safety margin (Thm. I.13).
1648	$\delta_b = p_b - \frac{m}{k-f_s}$	wrongful-eviction margin (Prop. I.20).
1649	B_{att}	certified per-attempt ceiling: the exact-binomial right-hand side of Theorem I.13, = ordinary term + panel term (Rem. I.17); the panel term is a floor in N at fixed panel geometry; never the Hoeffding relaxation.
1650	ρ_{seat} (emp)	lower bound: next demander of a disputed item casts an eligible verdict, conditionally on the past (Assumption A9).
1651	$\kappa = (k+1)/\rho_{\text{seat}}$	expected demanders during an unresolved dispute (Lem. I.6).
1652	r_{max}	open (unresolved) rejections per identity.
1653	θ_{acc} , c_{ceil}	rate trigger (per fresh inspection, calibrated $> 1 - p_b$) and hard per-epoch contradiction ceiling (Rule R13(v)).
1654	$c_{\text{acc}} = c_{\text{ceil}} + r_{\text{max}}$	deterministic per-identity bound, per filing epoch, on effective contradicted dispute-initiations (Lem. I.4).
1655	c_p	voluntary re-inspection prompts per suspension: $\leq c_p k$.
1656	$\kappa' = \kappa + c_p k$	pacing constant with suspension prompts absorbed (Lem. L.18).
1657		
1658	<i>Budget (Appendix J)</i>	
1659		
1660	M ; $V(M)$; β	population; expected distinct demanded items; Heaps exponent $V(M) = \Theta(M^\beta)$.
1661	$p_{u,v,t}$; $p_{v,t}$; p_v	per-user pair-demand probability; population-average pair / item demand probabilities per epoch (heterogeneous IRM, §J.1); $p_{v,t} \leq p_v$.
1662	$d = \sum_v p_v$; $d_{\text{tool}} = \sum_{v,t} p_{v,t}$	item-level and tool-indexed single-epoch demand mass; $d_{\text{tool}} \leq c_{\text{tool}} d$ under A12 (Cor. J.3); d_{tool} is the isolated single-user shareable cost (Thm. J.2(ii)); uniformly bounded along the population scaling under A13.
1663	$p_{v,t}^\infty$; d_{tool}^∞	lifetime (ever-demand) pair probability and mass (§J.1); distinct from the single-epoch mass.

d_{priv}	personal cap: expected distinct personal (<i>value, tool</i>) pairs a user ever confirms, once each (lifetime; per-pair per INV-3).
$C_u, \bar{C}_M, C_v$ (rv)	user u 's shareable confirmations; population mean; confirmations attributable to v .
forced / voluntary / redundant	gate fires for the user's own use of an uncovered <i>pair</i> ; accepting decision at an interpreter-issued invitation surface; accepting decision at a covered gate (handle-expressed reuse of a covered pair, protocol-inert, excluded from C_u) (§J.1, Rule R13(i)).
$\bar{\gamma}; \gamma_v; s_0$	front-loading bound (fraction of an item's demand per window or dormant interval); item's realised fraction; additive slack (A10).
$A(M); B_{\text{fl}}$	demand-independent per-item cost; front-loading coefficient (Lem. J.7; explicit forms §L.3).
$D_0(M), \varepsilon(M), \eta$	coverage sufficiency threshold, failure rate, slack (A11).
c_{tool}	distinct tools per item (A12).
g_N, g_k (sch)	margin schedules: $N = f + g_N, k = f_s + g_k$ (Thm. J.13).
$E_{\text{lease}}; E_{\text{rv}}$	O3 lease periods per item ($\mathcal{O}(1)$; extension-level accounting, O3(e)); re-verification rounds (N16).

Probability (Appendix L)

$(\mathcal{F}_s)$	filtration of logged protocol and display events; adversary strategies predictable with respect to it.
s	discrete time index of the filtration $(\mathcal{F}_s)$; the proof-local step counter of Thm. G.25 is unrelated.
τ_i, Z_i, Y_i (rv)	inspection stopping times; decision indicators; coupled Bernoulli majorants/minorants (Lem. L.2).
U_i (rv)	i.i.d. uniform coupling variables (Lem. L.2).
Bin, Geom	binomial; geometric (number of trials).
λ_{c_0}	tail constant of Lem. I.6.

Blast-schedule analysis (§L.2; cited by no proof)

$\pi_{\text{ord}}, \pi_{\text{pan}}$	realised promotion probability of the blast schedule by route, at reach R (Prop. L.11); sum π_{blast} ; <i>not</i> the terms of B_{att} (Rem. L.14); always subscripted, cf. $\pi(b)$.
$S(J)$	capped generating sum of the panel component, J post-freeze decisions (Prop. L.11).

Zipf case study (Appendix L.5)

$\alpha, Z, R_M, H_M, L_M, \mu_{\text{tail}}, \hat{N}$	rank exponent; normaliser; head boundary; head/tail sets; expected distinct tail demand; per-item cap $\hat{N} := A(M)$.
--	---

Disambiguation. h (decision history) vs. h_j (handles): subscripted occurrences are always handles; $H(c)$ (handle structure), H_{acc} (honest acceptors), and H_M (Zipf head set) are unrelated to one another and to h . C (label) vs. $C_u, C_v, \bar{C}_M$ (confirmation counts): the label is sans-serif and unsubscripted. Z (Zipf normaliser) vs. Z_i (decision indicators): subscripted occurrences are always indicators. $\pi_{\text{ord}}, \pi_{\text{pan}}, \pi_{\text{blast}}$ (blast-schedule promotion probabilities, §L.2) vs. $\pi(b)$ (interpreter-only block provenance, Definition F.2): the former are always subscripted and take a reach argument, the latter never is; no bare $\pi(\cdot)$ denotes a probability anywhere in this document. $S(J)$ (blast generating sum) is unrelated to S_{anc} (anchored attempts, Cor. I.27), to $S_{\text{sh}}(L)$ (shared-tier replay, Definition H.1), and to the bare S of A5 (a crystallized summary, which takes no argument).

E.3 STANDING CONVENTIONS

- (C1) *Determinism modulo coins.* A single coin sequence ω jointly covers the sampling randomness of P and of $R_{\mathcal{E}}$; given ω , both are deterministic functions of their inputs. All two-run comparisons (Thm. G.25, Prop. I.10, Prop. L.20) couple ω , the user's typed stream (A6), and (in the federated setting) the exogenous federation-log prefix (A8; local suffixes derived equal, Def. H.2), identically across the compared runs.
- (C2) *Probability space and adversary.* All federated-protocol randomness lives on the filtration $(\mathcal{F}_s)$ generated by logged protocol events and display events; the adversary's strategy (presentation, submission, stopping, self-dispute) is predictable with respect to it. The construction, and the two structural constraints on adaptivity it implies, are given once in §L.1 and used everywhere.
- (C3) *Log order.* The federation maintains a single linearizable, append-only log of protocol events: a TCB *log service* (a trusted sequencer or a consensus layer) supplies the total order, and its history is materialized as a Certificate-Transparency-style Merkle tree for tamper-evident audit — the tree authenticates the history; the

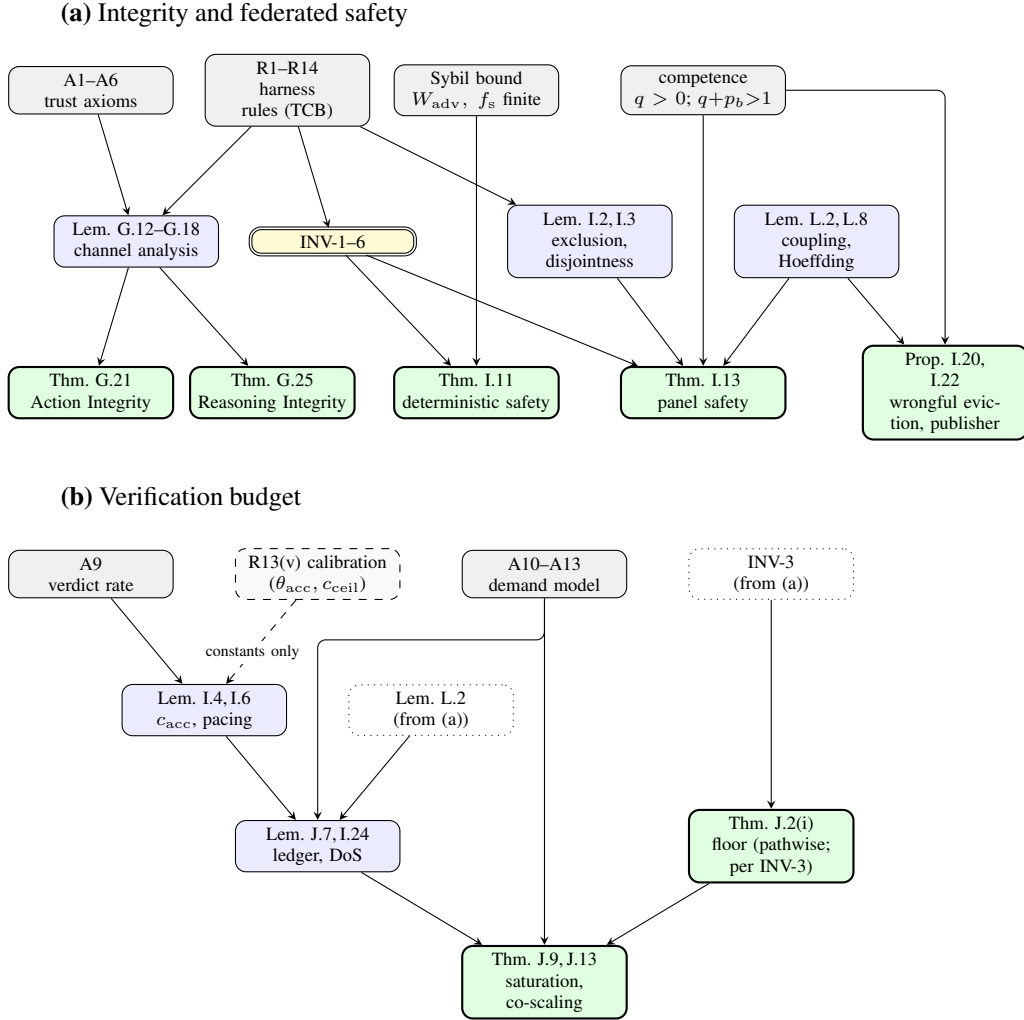


Figure 6: Dependency structure (edges denote “is used in the proof of”). **(a)** The single-user theorems rest on the trust axioms and harness rules alone (the channel-analysis lemmas use R1–R11; the structural lemmas use R12–R14); the coupling and Hoeffding lemmas are assumption-free. A7–A8 enter only Proposition I.10, which supplies their delegated extension. **(b)** Dotted boxes import results proved in panel (a) and Appendix L. The dashed edge is the only role of the accountability calibration: it grounds the constant c_{acc} in the budget ledger and never enters any safety theorem. The pathwise floor, Thm. J.2(i), depends on INV-3 alone, visible here as its sole incoming edge; the expected-cap clauses (ii)–(iii) additionally read the reference demand model’s masses, and the tool-spread comparison $d_{tool} \leq c_{tool}d$ is quarantined in Cor. J.3 (A12), consumed by no other result. (TCB = trusted computing base; defined in §E.2, inventoried in §F.5.)

sequencing service, assumed correct as TCB, provides the linearizability. It supplies the total order on decisions, demand events, and verdicts that Rules R12–R14 invoke. “The federation log,” “the log,” and “log order” all refer to this object and the order it induces. The hash-chained audit log that materializes the display transcript (Definition G.8) is a distinct, per-deployment TCB component and is never called “the log.”

- (C4) *Epochs, leases, rounds*. Three multiplicities the development keeps distinct. (a) *Epochs*. An epoch is a fixed span of the federation log (a fixed number of sequence numbers or a fixed calendar period), common to all items and identities, readable off the log (TCB); every renewable per-epoch quantity (ρ_{sub} , E_{max} , notification deduplication, the accountability allowances of Rule R13(v), (vii)) is indexed to it. (b) *Leases*. A lease is the deployment-selected local scope lifetime of the requirements-level option O3 (O3; not part of the formal semantics); $E_{\text{lease}} = \mathcal{O}(1)$ counts lease periods per (user, item) in the extension-level accounting of O3(e). (c) *Rounds*. A re-verification round re-presents a community-confirmed item to inspectors with no prior decision on its bytes; E_{rv} counts rounds. (d) *Permanence and boundaries*. Decision freshness (Def. H.5) and verdict capacity (Rule R13(iii)) are *permanent* and attach to none of the three: they never renew. Promotion weight and banked credits persist across epoch boundaries until the item retires. Finalisation windows are contained within epochs: a window that would cross a boundary closes at the boundary, banking its accrued endorsements to the item’s next entry exactly as a dormant-interval decision would (Def. H.6); a boundary never finalises a promotion early and never discards honest labour. A boundary-spanning dispute counts toward Rule R14(i) in the epoch in which it opened.
- (C5) *Byte identity*. Items, freshness, disputes, and panels are keyed to exact byte strings. Paraphrase variants are distinct items (consequences: S1).
- (C6) *Confirmations*. A *confirmation* is an accepting gate decision; it is durable (its fresh endorsed copy is written even when another review aborts the enclosing invocation, Rule R2) and is counted once per (value, tool) pair per gate transaction (Definition F.5). Rejecting decisions, abandoned dialogs, batch-element page approvals, and repeated within-transaction reviews of an already-accepted pair are attention cost, counted separately (N18). *Forced, voluntary, and redundant* confirmations are defined in §J.1: the confirmation cap (Thm. J.2) counts forced pair-confirmations, the ledger (Lem. J.7) forced and voluntary; redundant confirmations are protocol-inert (Rule R13(i)) and excluded from both.
- (C7) *Store idealization*. The store is idealized as unbounded: no operation’s defined behaviour depends on aggregate store volume. A deployment enforcing quotas must apply them to the untrusted tier by silent eviction, never by an error visible to P (otherwise store exhaustion reopens exactly the channel Rule R7 closes; S7).
- (C8) *Constants vs. schedules*. $N, k, m, W, D_{\text{pool}}, T_{\text{win}}, T_{\text{exp}}, E_{\text{max}}, \rho_{\text{sub}}, r_{\text{max}}, c_{\text{ceil}}, c_{\text{batch}}, c_{\text{cand}}, c_{\text{p}}$ are fixed TCB configuration at any deployment; Appendix J additionally studies *schedules* $N(M), k(M), \dots$ along a growing population, and every scheduled claim is confined to the corridor stated there. All caps and multiplicity constants are positive integers where integral; in particular $E_{\text{max}} \geq 1$ and $c_{\text{tool}} \geq 1$ (used by Lemma L.18).
- (C9) *Environment scheduling*. An interpreter step has a fixed slot structure: first the channel slots deliver at most one typed event (W-TYPED) and at most one paste event (W-PASTE), in that order; then P emits exactly one operation, whose environment interaction (tool returns, reader outputs, retriever selection, gate decisions) occurs synchronously within that operation’s atomic transition. Two-run comparisons (Theorem G.25, Proposition I.10) couple only this *slot skeleton* and the typed stream (A6): paste *presence and content* are uncoupled and may differ arbitrarily across the compared attacks, as may every tool output and reader output. Channel-slot transitions acknowledge the channel, never P : nothing is appended to P ’s reply stream by W-TYPED or W-PASTE (Figure 8, convention (vi)). Two-run comparisons are always taken at a *common horizon*: the compared runs consist of the same number of interpreter slots, and transcript equality means equality of the two full transcripts at that horizon.

E.4 NAMED INVARIANTS AND STRUCTURAL PROPERTIES

The following six named facts are cited constantly and are collected here; together they are the structural core of the design. They are heterogeneous in kind, and each entry declares its kind: *rule property* (a direct consequence of the cited rule and definition clauses); *conditional inductive invariant* (preserved by every transition, under the named first-party exceptions); *protocol property* (a fact of the counting and pacing rules); *bound metaproperty* (a property of the *bounds* of the cited theorems, holding under those theorems’ hypotheses). None is an axiom: each is established at the location cited, and nothing beyond the cited hypotheses is assumed. Invariants INV-1–INV-6 are stated in the vocabulary of the federated protocol, defined in Appendix H.

Invariant INV-1 (Byte-permanence of freshness). An attested identity’s first *logged* decision on a byte string — its first decision taken while the bytes have a shared-tier presence (Definition H.5) — is its only weight-bearing inspection

of those bytes, ever: every later accepting decision by that identity (at any tool, in any pipeline entry, in any epoch, in any later shared-tier presence of the same bytes) carries no promotion weight and no verdict capacity. Freshness is decidable from the federation log (convention (C3)) and never renews; personal decisions outside any shared-tier presence are not logged and are outside this guarantee (Definition H.5). Kind: rule property. *Established by* Definition H.5 and the counting rule Definition H.6. (Byte-keyed: paraphrase variants are distinct items, (C5), S1.)

Invariant INV-2 (Rejection asymmetry). A rejecting decision can freeze promotion, corroborate an open dispute, or trigger post-promotion review; it can never promote and never vindicate. A *repeat* acceptance carries no promotion weight and no verdict (INV-1). Hence re-exposure of bytes to a *previously logged-decided* inspector can only add rejection opportunities; exposure to a log-fresh inspector adds both a rejection opportunity and a possible weight-bearing acceptance, priced by the hypotheses of Theorem I.13. Kind: rule property. *Established by* Rule R13(i)–(iii) with Definition H.5.

Invariant INV-3 (Once-forced cap, per (user, value, tool)). No user is ever forced to confirm a given (*value, tool*) pair more than once over the deployment’s lifetime: a user’s first accepting review of value v at tool t — at any gate transaction, including one whose enclosing invocation aborts (Rule R2) — durably writes the local C copy with $t \in \text{scope}$, which covers all their later use of v at t and survives every third-party protocol event; and within one transaction, confirmations are counted once per pair (Definition F.5). A first use of the same bytes at a different tool t' is a distinct forced confirmation, of the pair (v, t') , itself once ever; the pairs per item are bounded by A12 in the budget model. The disclosed exception within the formal semantics is first-party: the owner’s own agent’s ungated evict (N13); a deployment adopting a conforming instantiation of the requirements-level Option O3 additionally admits lease expiry, an extension outside this document’s semantics (O3). Kind: conditional inductive invariant (exception: first-party evict, N13; extension-level: conforming-O3 lease expiry, O3). *Established by* Lemma J.1 (from Rules R2, R4, A1–A2, and Lemma I.8); reasoning pairs covered per the reasoning-coverage clause of Rule R13(i); *consumed by* Theorem J.2 and Lemma J.7(i).

Invariant INV-4 (Banking safety). A banked credit (a first decision logged during dormancy or during a freeze with no open entry) raises only the *starting* weight of the item’s next pipeline entry. Nothing finalises promotion except an open entry’s window end, so banking never bypasses a window’s rejection opportunity and never accelerates finalisation past a window boundary; and each banked decision remains its identity’s single fresh inspection of the bytes, so successive entries’ weight-bearing sets stay identity-disjoint (Lemma I.3). Banked credits expire if the item retires first. Every banked decision is logged after its attempt’s anchor (Definition I.1), so banking never lets a credited decision precede the conditioning point of Theorem I.13. Kind: protocol property. *Established by* Definition H.6 with Rule R13(iv).

Invariant INV-5 (Verdict non-renewal). An identity casts at most one verdict on a given dispute instance, for the instance’s entire life, across any number of epoch boundaries. Verdict capacity is consumed only by cast verdicts (abandonment consumes nothing) and, like freshness, never renews. Kind: rule property. *Established by* Rule R13(iii).

Invariant INV-6 (Reach-freeness). The federated safety bounds depend only on protocol constants (N, k, m, D_{pool}) and the adversary’s resources (f, f_s, W_{adv}), never on the number of honest identities to whom a poisoned item is presented: the same stated upper bound applies at every reach. (No monotonicity in reach is claimed: presenting the bytes to log-fresh inspectors creates both rejection opportunities and possible weight-bearing acceptances, INV-2; what is proved is that the bound does not degrade with reach.) Reach-freeness is relative to the stated parameters: the bounds’ *form* is reach-independent, while the behavioural constants $q_{\text{pre}}, q_{\text{pan}}$ are weakest-target infima over all cohorts the adversary can reach (theorem I.19), so widening reach can lower the applicable constants without changing the bound’s form. Kind: bound metaproperty. The property is Definition H.8; it is a property of the stated bounds of Theorems I.11 and I.13, holding under their hypotheses, not a property of reachable configurations; contrast with the reach-dependent no-dispute baseline, Theorem L.9.

F FORMAL SPECIFICATION OF THE MEMORY INTERFACE

PerMIT places no restriction on which tools the Protected LLM may invoke or on how it reasons; the *harness* (the fixed interpreter code of Figures 7–8 together with the display and logging paths) bounds two things only: what P can *observe*, and what may *cross* a tool boundary. The bounded surface is the memory interface: the P -facing operation surface

$$\begin{aligned}\Sigma_P &= \Sigma \cup \{\text{tool}(\bar{a}) : \text{tool} \in \text{Tools}_{\text{sens}} \cup \text{Tools}_{\text{data}}\} \cup \{\text{retrieve}\}, \\ \Sigma &= \{\text{write}_P, \text{recall}, \text{fill}, \text{crystallize}, \text{display}, \text{confirm}, \text{evict}\},\end{aligned}$$

(write, where it appears in prose, abbreviates the sole P -emitted write operation write_P .) together with one interpreter-internal derived form (map). Every completeness claim below (Lemmas G.17, G.18) enumerates over Σ_P , which coincides arm-for-arm with the mechanized operation type (POP, §G.6). Because this surface is finite, its *interface transition schema* is specified *clause-complete at the interface alphabet*: every operation of Σ_P and channel event has

its store effect, reply, transcript events, and sanitized-history events fixed by Figures 7–8 with these rules, with totality per Rule R10(b); the full run state these transitions carry is fixed by Definition F.6. What the figures deliberately do not formalise is P ’s internal computation (a strategy, §F.1) and the UI-side rendering internals of Rule R8; the channel analysis of Appendix G needs only the interface schema, and is verified clause by clause against it, with the totality clause of Rule R10 ensuring that no invocation has undefined or distinguishing behaviour. (Model lineage and design rationale: N1.)

F.1 CONFIGURATIONS, LABELS, AND ROLES

Definition F.1 (Trust lattice and labels). The integrity lattice has two levels, $\mathbf{U} \sqsubset \mathbf{E}$. Each block carries a label $\ell(b) \in \{\mathbf{U}, \mathbf{T}, \mathbf{C}, \mathbf{CC}, \mathbf{PC}\}$; the level map lvl sends $\text{lvl}(\mathbf{U}) = \mathbf{U}$ and $\text{lvl}(\ell) = \mathbf{E}$ for $\ell \in \{\mathbf{T}, \mathbf{C}, \mathbf{CC}, \mathbf{PC}\}$. The four $\mathbf{E}$ -level labels are provenance-and-reuse tags, not a finer trust ordering: $\mathbf{T}$ (user-typed; reusable by its owner), $\mathbf{C}$ (single-user confirmed; admissible only to the tools in its scope, reusable without re-confirmation only by its owner), $\mathbf{CC}$ (community-confirmed, Appendix H; admissible only to the exact targets endorsed (tools, or the reasoning pseudo-target, §H.1), reusable by anyone), $\mathbf{PC}$ (P -composed; empty tool scope, hence never eligible for the sensitive fast path of Rule R4). Label combination is the meet on levels: an operation combining inputs yields level $\mathbf{U}$ if any input is at level $\mathbf{U}$, else level $\mathbf{E}$; no operation in Σ raises an untrusted value to level $\mathbf{E}$ by combination.

Definition F.2 (Store and configuration). A block is $b = (\text{id}, \text{bytes}(b), \ell(b), \pi(b), \text{scope}(b))$ with interpreter-only provenance $\pi(b) \in \{\text{typed}, \text{pasted}, \text{ingress}, \text{cryst}, \text{pcomposed}, \text{confirmed}, \text{cache}\}$, assigned only by the interpreter (provenance is interpreter-private and not part of obs_P), and, for endorsed blocks, a tool scope $\text{scope}(b) \subseteq \text{Tools}_{\text{sens}} \cup \text{Tools}_{\text{data}}$ (a set of individual tool identifiers). Every block-creating transition of Figures 7–8 instantiates one of the total constructors

$$\begin{aligned} \text{newT}(v) &:= (\text{id}_{\bullet}^{\mathbf{E}}, v, \mathbf{T}, \text{typed}, \emptyset), & \text{newPC}(v) &:= (\text{id}_{\bullet}^{\mathbf{E}}, v, \mathbf{PC}, \text{pcomposed}, \emptyset), \\ \text{newC}(v, t) &:= (\text{id}_{\bullet}^{\mathbf{E}}, v, \mathbf{C}, \text{confirmed}, \sigma(t)), & \text{newCache}(v, t) &:= (\text{id}_{\bullet}^{\mathbf{E}}, v, \mathbf{C}, \text{cache}, \sigma(t)), \\ \text{newU}(v, \pi) &:= (\text{id}_{\bullet}^{\mathbf{U}}, v, \mathbf{U}, \pi, \emptyset), \end{aligned}$$

where $\text{id}_{\bullet}^{\mathbf{E}}$ (resp. $\text{id}_{\bullet}^{\mathbf{U}}$) is the next identifier of the endorsed (resp. untrusted) sort (Rule R1(g)) and $\sigma(t) = \{t\}$ for a tool target, $\sigma(\text{reasoning}) = \emptyset$; the cache confirmations of Options O1–O2 use newCache , all other accepting confirmations newC . Tool identifiers are canonical, fixed TCB configuration, so scope membership is a decidable syntactic test. Block identifiers are two-sorted and operation-determined (Rule R1(g)); the sorts separate the tiers: an endorsed-sort identifier resolves only to an $\mathbf{E}$ -level block, an untrusted-sort identifier only to a $\mathbf{U}$ -block.

A configuration is $c = (\mathcal{M}, \sigma_P, \sigma_Q, h)$: the store, the Protected-LLM state, the Reader state, and the interpreter’s sanitized decision history h : a sequence of (target, decision) records, one appended at each reached gating review (one per reached gating position; for a batch, one per gating non- $\diamond$ body position plus one aggregate element decision) — never bytes, never a block identifier of either sort, never a selection ordinal, never a candidate multiplicity, collection cardinality, or page count; selections, candidate lists, page renderings, and per-page decisions are recorded in the display transcript only (Definition G.8). A null-handle abort appends the same (target, reject) record a user rejection appends (Rule R6). Hence h contains no $\mathbf{U}$ -bytes, exposes no untrusted-sort identifier (Rule R1(g)), and carries no occupancy, cardinality, or pagination information (Rules R6, R7); this is the form machine-checked as HEvent (§G.6). $\mathcal{M}_{\mathbf{U}}$ and $\mathcal{M}_{\mathbf{E}}$ denote the restrictions of $\mathcal{M}$ to levels $\mathbf{U}$ and $\mathbf{E}$.

Endorsement unit. The endorsement unit throughout is the (*value, tool*) pair: every confirmation names an exact byte string and an exact tool, and there is no endorsement transfer between tools, however similar. (Exact-target discipline governs the sensitive fast path and all endorsement, promotion, and counting; the *baseline* data-tool path of Rule R4(b) is scope-blind, Corollary G.23(b), and Option O2 extends the exact-target discipline to data tools.) Combinations, orderings, and timings of separately endorsed values are ungated by design (rationale and residual: N2, S11; hardening: Option O1).

Individuation of trusted blocks. $\mathbf{T}$ -blocks are individuated at write time: each field the user explicitly supplies (or, absent field structure, the command as one atomic unit) is one $\mathbf{T}$ -block. The individuating parser is fixed interpreter code inside the TCB; the fast-path test of Rule R4 is byte-equality against these stored blocks.

Two scopes and one selector. The agent is a transition system over configurations whose steps are tool invocations, control flow, and the operations of Figures 7–8, executed in one of two scopes: the *privileged scope* P may invoke any tool and reaches memory only through Σ . Throughout the formal development P is modelled as a *strategy*: a function from its interaction history, its own emitted operations, the reply stream of Rule R10, its view $\text{obs}_P(c)$, and the coins ω

(convention (C1)), to its next operation. Its internal state σ_P is therefore an abbreviation for that history and carries no independent degrees of freedom: two runs agreeing on the history agree on σ_P by definition. This is not a restriction on P (the strategy is arbitrary, adversarial included) but the standard determinism-modulo-coins reading of (C1), and it is the form in which P is machine-checked (§G.6). The *quarantined scope* Q has read-only access to the whole store and no tool access. A third model role, the *trusted-store retriever* R_E (Rule R11), is a stateless selector over $\mathcal{M}_E$: it reads only the endorsed store and returns only block identifiers, which the interpreter dereferences. (The asymmetry between Q ’s and R_E ’s vision is forced by the theorems: N3.)

Definition F.3 (Protected-LLM view). For a configuration c , let $H(c)$ denote the handle structure of Rule R1 and $x(c)$ the trusted command input received so far; h is the decision history. The *Protected-LLM view* is

$$\text{obs}_P(c) = (\mathcal{M}_E, H(c), x(c), h).$$

By Figures 7–8 and Rules R1–R11, $\text{obs}_P(c)$ together with the reply stream of Rule R10 is the entirety of what P can observe (Lemma G.17).

Remark F.4 (Information interface vs. effect interface). Σ is the interface of *effects*; $\text{obs}_P(c)$ is the interface of *information*. Granting P direct sight of $\mathcal{M}_E$ in its view is a deliberate, conservative strengthening: every theorem quantifies over all strategies of this better-informed P , so it a fortiori covers any P that must acquire the same data operationally, and the two-run theorem is *harder* under it (a more informed P has more ways to diverge). recall on endorsed identifiers is consequently informationally redundant and is retained as the operational access path; the sentence “ P reaches memory only through Σ ” concerns effects, not information. The mechanization adopts exactly this reading (§G.6: obs is computed through memEOF).

Definition F.5 (Sensitive sinks, gate transactions, and confirmations). A *sensitive sink event* is a call $\text{tool}(a_1, \dots, a_n)$ with $\text{tool} \in \text{Tools}_{\text{sens}}$.

A *gating position* of a tool invocation is an argument position that does not pass the fast path of Rule R4. The invocation’s gating positions are processed left to right as one atomic *gate transaction* (Rule R2): at each reached position the interpreter resolves the argument, displays its bytes with zoned provenance (Rule R8), and reads the user’s selection (if among candidates) and decision $z \in \{\text{accept}, \text{reject}\}$; a null resolution renders the fixed no-value display of Rule R6 and takes the standard reject outcome, consuming no decision; a rejecting outcome aborts the review of the remaining positions.

An accepting review of resolved bytes v at target t is a *confirmation event* $\text{confirm}(v, t)$. Confirmations are *durable*: at the close of the atomic transaction the interpreter writes, for every accepted gating position, one fresh endorsed copy of its displayed bytes scoped to the target (Figure 8, convention (vii)), *whether or not the transaction as a whole succeeds*. The tool itself executes iff every gating position was accepted and none was null; on execution the bytes that flow at each position are exactly the bytes displayed for it, and on abort no argument flows.

Counting unit. For all accounting (Appendices H, J), confirmations are counted per *(value, tool) pair per gate transaction*: accepting reviews of the same exact pair at several positions of one transaction are one pair-confirmation; the repeated reviews themselves are attention cost (N18).

A standalone $\text{confirm}(v, t)$ with t a tool or “reasoning” is the single-position special case. The “publish” target is not a local confirmation: it creates no local block and is defined by the PUBLISH transitions of Figure 8 and Rule R12. Batch-element page approvals under Rule R9 are call-local: they authorize the displayed elements for that map instance only and create no durable copy; gating non- $\diamond$ arguments of a map body are ordinary durable confirmations (convention (vii’)).

F.2 OPERATIONAL SEMANTICS

Figures 7–8 give the *interface transition schema* of the memory interface: complete at the level of the interface alphabet (every operation’s store effect and reply is fixed here, with totality per Rule R10(b)), with the remaining components of the run state — fixed by Definition F.6 — evolving by the deterministic bookkeeping stated in the rules and conventions. Every transition is fixed interpreter code; no model participates in any label assignment or any reply. Replies to P are drawn from the grammar at the top of Figure 7 and nothing else (Rule R10); the side conditions cite the Rules of §F.3, which constrain naming, atomicity, display, and gating. Transitions triggered by the user or the environment (channel events, gate decisions, tool returns) are marked with their trigger; all others are P -emitted.

Definition F.6 (Extended configuration). The full run state comprises: the store $\mathcal{M}$; the two identifier allocators of Rule R1(g) (endorsed-sort and untrusted-sort); the interpreter-private handle-resolution table (index = handle identifier; never read by any reply, Rule R7); P ’s reply stream (Rule R10); the decision history h ; the trusted command input $x(c)$; the executed-tool, confirmation, and reader-event counters that index the environment’s oracles; and three ghost

Replies to P

(Rule R10)

$$r ::= \text{ack} \mid h_j \mid \text{fail} \mid \text{dec}(\text{accept}) \mid \text{dec}(\text{reject}) \mid \text{bytes}(b), b \in \mathcal{M}_{\mathbb{E}}$$

Replies to P form a *stream*; the grammar above is its *alphabet*. A single transition may append more than one token (e.g. TOOL-GATE-ACC appends $\text{dec}(\text{accept}) \cdot \text{ack}$), per convention (iii).

Judgment. $\mathcal{M} \xrightarrow{o \Downarrow r} \mathcal{M}'$: operation o yields reply r ; components of c other than $\mathcal{M}$ are suppressed when unchanged; h_j is the j -th operation-determined handle (Rule R1). Transitions triggered by the user or the environment are marked by their trigger; all others are P -emitted. Conclusions suppress the operation label over the arrow where the premises display it; the components of the extended configuration (Definition F.6) other than $\mathcal{M}$ evolve by the deterministic bookkeeping stated in the rules and conventions, and the figures are the store/reply projection of that full-state transition, realized executably (for the map-free fragment) by the mechanization (IState, §G.6).

Tool invocation.

TOOL-FAST P emits $\text{tool}(\bar{a})$	every a_i passes the fast path of Rule R4	environment returns o (attacker-chosen bytes and count)
$\mathcal{M} \xrightarrow{\text{tool}(\bar{a}) \Downarrow \text{ack}} \mathcal{M} \cup \text{capture}(o)$		
TOOL-GATE-ACC		
P emits $\text{tool}(\bar{a})$ some a_i gate (Rule R4)	every gating position is confirmed and accepted (Rules R2, R8) environment returns o (attacker-chosen bytes and count)	
$\mathcal{M}' = \mathcal{M} \cup \{\text{newC}(\text{displayed } a_i, \text{tool}) : \text{per accepted gating position (conv. (vii))}\}$		
$\mathcal{M} \xrightarrow{\text{tool}(\bar{a}) \Downarrow \text{dec}(\text{accept}) \cdot \text{ack}} \mathcal{M}' \cup \text{capture}(o)$		
ATOMIC (R2); FRESH IDS (R1(G))		
TOOL-GATE-REJ		
P emits $\text{tool}(\bar{a})$ $\mathcal{M}' = \mathcal{M} \cup \{\text{newC}(\text{displayed } a_i, \text{tool}) : \text{per gating position accepted before the abort}\}$	gate fires; some gating position is rejected by the user, or null-aborts (Rule R6)	
$\mathcal{M} \xrightarrow{\text{tool}(\bar{a}) \Downarrow \text{dec}(\text{reject})} \mathcal{M}'$		
NO INVOCATION; ACCEPTED POSITIONS ENDORSED (R2); h EXTENDED		

Writes.

W-TYPED channel event: $\text{typed}(x)$ $\text{parse}_T(x) = b_1, \dots, b_n$ (TCB)	W-PASTE channel event: $\text{paste}(x)$
$\mathcal{M} \xrightarrow{\Downarrow \text{ack}} \mathcal{M} \cup \{\text{newT}(b_1), \dots, \text{newT}(b_n)\}$	$\mathcal{M} \xrightarrow{\Downarrow \text{ack}} \mathcal{M} \cup \{\text{newU}(b_x, \text{pasted})\}$
W-P	
$b \in \sigma_P$	
$\mathcal{M} \xrightarrow{\text{write}_P(b) \Downarrow \text{ack}} \mathcal{M} \cup \{\text{newPC}(b)\}$	

Memory reads and display.

RECALL-E id resolves to a readable $\mathbb{E}$ -block b	RECALL-O otherwise: U-block, unknown, or nonexistent (uniformly)
$\mathcal{M} \xrightarrow{\text{recall}(\text{id}) \Downarrow \text{bytes}(b)} \mathcal{M}$	$\mathcal{M} \xrightarrow{\text{recall}(\text{id}) \Downarrow h_j} \mathcal{M}$
RETRIEVE	
P emits $\text{retrieve}(q)$	$R_{\mathcal{E}}(q, \mathcal{M}_{\mathbb{E}}, \omega)$ returns an id resolving to $b \in \mathcal{M}_{\mathbb{E}}$
$\mathcal{M} \xrightarrow{\Downarrow \text{bytes}(b)} \mathcal{M}$	
VERBATIM DEREFERENCE (R11)	
RETRIEVE-F	
$R_{\mathcal{E}}$ returns no id, or an id not resolving in $\mathcal{M}_{\mathbb{E}}$	DISPLAY rendering per Rule R8; nothing enters σ_P
$\mathcal{M} \xrightarrow{\text{retrieve}(q) \Downarrow \text{fail}} \mathcal{M}$	$\mathcal{M} \xrightarrow{\text{display}(x) \Downarrow \text{ack}} \mathcal{M}$

Figure 7: Interface transition schema of the memory interface (1/2): tool invocation, writes, memory reads, and display. Every transition is fixed interpreter code; no model assigns any label or composes any reply. Continued in Figure 8 (search, crystallization, confirmation, eviction, iteration, and failure); conventions for both figures follow Figure 8.

logs used only by the theorems — the display transcript DT (Definition G.8), the sensitive-sink log (Definition G.7), and P 's emitted-operation log. P -visible components are exactly $\text{obs}_P(c)$ and the reply stream (Lemma G.17); all others are interpreter-private or ghost. A block is *readable* iff it is at level $\mathbb{E}$, i.e. dereferenceable to bytes by recall (RECALL-E); U-blocks are never readable to P . For the map-free fragment, the executable interpreter of §G.6 realizes this extended configuration verbatim (IState), and the figures are its human-readable projection onto the store and reply components.

Search and crystallization.

FILL
 Q searches $\mathcal{M}_U$ interpreter binds $\leq c_{\text{cand}}$ candidates internally (possibly none) REAL/NULL INDISTINGUISHABLE (R6, R7)

$$\mathcal{M} \xrightarrow{\text{fill}(q) \Downarrow h_j} \mathcal{M}$$

CRYST
 Q writes schema-bound summary s interpreter-side

$$\mathcal{M} \xrightarrow{\text{crystallize}(t) \Downarrow h_j} \mathcal{M} \cup \{\text{newU}(s, \text{cryst})\}$$

Confirmation and eviction.

CONF-ACC
gate confirm(v, t), t a tool or reasoning user selects and accepts on the decision pane (Rule R8) h EXTENDED; FRESH COPY, SCOPE $\sigma(t)$ (CONV. (VII))

$$\mathcal{M} \xrightarrow{\Downarrow \text{dec}(\text{accept})} \mathcal{M} \cup \{\text{newC}(v, t)\}$$

CONF-REJ
gate confirm(v, t), t a tool or reasoning user rejects, or null handle (Rule R6) BYTES NOT RETURNED; h EXTENDED

$$\mathcal{M} \xrightarrow{\Downarrow \text{dec}(\text{reject})} \mathcal{M}$$

PUBLISH-ACC
gate confirm(v , publish) v byte-equals a local C-block of the owner with provenance confirmed
owner accepts on the decision pane (Rule R8) SHARED-TIER ACT; NO LOCAL BLOCK, NO ENDORSED ID (R12)

$$\mathcal{M} \xrightarrow{\Downarrow \text{dec}(\text{accept})} \mathcal{M} \text{ emitting submit}(v) \text{ to the federation log}$$

PUBLISH-REJ
gate confirm(v , publish) owner rejects, or v is ineligible (Rule R12), or null (Rule R6) UNIFORM REJECT; INELIGIBILITY, REJECTION, NULL ALIKE (R7)

$$\mathcal{M} \xrightarrow{\Downarrow \text{dec}(\text{reject})} \mathcal{M}$$

EVICT
 P emits evict(id) interpreter-side: if id resolves to a block of the owner's local store,
that block is removed; otherwise the operation is a no-op resolution status consumed interpreter-side (Rules R7, R10(b)) UNIFORM REPLY; NO LABEL CHANGE

$$\mathcal{M} \xrightarrow{\text{evict}(\text{id}) \Downarrow \text{ack}} \mathcal{M} \ominus \text{id}$$

Iteration and failure.

MAP-TOOL-ACC
 P emits map($\hat{o}, H$), $\hat{o} = \text{tool}(\dots, \diamond, \dots)$ well-formed (Rule R1(f))
batch confirmation accepted on every page (Rule R9)
environment returns $o_1, \dots, o_{|H|}$ per element call (attacker-chosen bytes and count)

$$\mathcal{M}' = \mathcal{M} \cup \{\text{newC}(a_i, \text{tool}) : a_i \text{ an accepted gating non-}\diamond \text{ argument}\}$$

$\mathcal{M} \xrightarrow{\Downarrow \text{dec}(\text{accept})\text{-ack}} \mathcal{M}' \cup \text{capture}(o_1, \dots, o_{|H|})$ ATOMIC BATCH (R2, R9); FRESH COPIES (CONV. (VII))

MAP-TOOL-REJ
 P emits map($\hat{o}, H$), $\hat{o}$ of tool form
some page rejected; null collection (Rule R6); or over-cap collection (Rule R9(b)) NO ELEMENT EXECUTES; NON- $\diamond$
 $\mathcal{M}' = \mathcal{M} \cup \{\text{newC}(a_i, \text{tool}) : a_i \text{ an accepted gating non-}\diamond \text{ argument}\}$ ENDORSEMENTS PERSIST (R2); NULL COLLECTION (R1(A), R6)

$$\mathcal{M} \xrightarrow{\Downarrow \text{dec}(\text{reject})} \mathcal{M}'$$

MAP-CRYST
 P emits map($\hat{o}, H$), $\hat{o} = \text{crystallize}(\diamond)$
 Q writes schema-bound summaries $s_1, \dots, s_{|H|}$ interpreter-side, one per element (CRYST) NO DECISION; ONE HANDLE, ONE ACK (R1(C))

$$\mathcal{M} \xrightarrow{\Downarrow \text{ack}} \mathcal{M} \cup \{s_1, \dots, s_{|H|} : \ell = U\}$$

FAIL
 P emits well-formed o a precondition fails, deterministically in P 's emitted operations and $\mathcal{M}_E$ alone (Rule R7)

$$\mathcal{M} \xrightarrow{o \Downarrow \text{fail}} \mathcal{M}$$

Figure 8: Interface transition schema of the memory interface (2/2): search, crystallization, confirmation, eviction, iteration, and failure, continued from Figure 7. Every transition is fixed interpreter code; no model assigns any label or composes any reply. Conventions for both figures follow below.

Conventions (apply to Figures 7–8). (i) $\text{capture}(o)$ stores the outputs o as U-blocks $\text{newU}(\cdot, \text{ingress})$ bound to a single fresh collection handle h_j (Rules R1, R3); the identifier is operation-determined, computable by P , and never returned. A call whose captured output count is zero binds its collection handle with *null* resolution (Rule R6): a zero-cardinality capture and an empty search result are the same null-resolution case. The capture operation exposes no direct cardinality or resolution bit to P (Rule R7); any later distinction surfaces only through the logged null-collection display and its standard aggregated reject (Rule R9(b)). (ii) Element calls inside MAP are interpreter-iterated, not P -emitted: individually neither handle-producing nor acknowledged. If the argument collection has null resolution (an empty search result or zero-output capture, convention (i)), the tool-free form $\text{map}(\text{crystallize}(\diamond), H)$ writes no summaries, binds the map’s operation-determined output collection handle with null resolution (Rules R1(a),(c), R6), returns the ordinary ack, and appends no display or decision event; the tool-bearing form takes the null-collection batch treatment of Rule R9(b). (iii) In TOOL-GATE-ACC and TOOL-GATE-REJ, gating positions are reviewed left to right; one decision is recorded per reached gating position, and the displayed bytes of an executed call are exactly the bytes that flow; the entire step is atomic (Rule R2). Every accepted position’s fresh C copy is written at the close of the step, on the accepting and the aborting branch alike; rejected and null positions never flow and write nothing. The reply stream carries *one* decision token per gated invocation, regardless of the number of gating positions; per-position decisions are recorded in the transcript and in h only (Rule R10(a)(i),(iv)). Whether an invocation is gated at all is a deterministic function of P ’s own emitted argument list and $\mathcal{M}_{\mathbb{E}}$ (Rule R4), hence already P -computable; the token-shape difference between TOOL-FAST, TOOL-GATE-ACC, and TOOL-GATE-REJ therefore carries no information beyond trusted data and the logged decisions (Rule R7) (the CC branch’s live-entry bit is $\mathbb{E}$ -level log data, coupled under A8 in two-run comparisons (exogenous prefix coupled; local suffixes derived equal, Def. H.2)), and within the reject class, user rejection and null-handle abort remain indistinguishable in the reply and in h (Rule R6). (iv) *Totality* (Rule R10(b)): every well-formed invocation with any argument is covered: unresolvable arguments take RECALL-O-style opaque-handle replies, failed deterministic preconditions take FAIL; no invocation is undefined and none distinguishes attacker-influenced states. (v) $\mathcal{M} \ominus \text{id}$ denotes $\mathcal{M} \setminus \{\text{id}\}$ when id resolves to a block of the owner’s local store, and $\mathcal{M}$ otherwise. The reply is the constant ack in every case; the realised outcome (removal of the owner’s block, at either level; no-op on an identifier not resolving to a block of the owner’s local store, including an unoccupied one) is recorded in the audit log interpreter-side and never returned to P . The effect on $\mathcal{M}_{\mathbb{E}}$ is P -visible trusted data (Rule R7); any effect on $\mathcal{M}_{\mathbb{U}}$, and the absence of any effect, are indistinguishable to P . A *handle* argument to evict is a deterministic malformation of P ’s own emission and takes the generic failure outcome (Rule R10(a)(iii)). (vi) Channel-triggered transitions (W-TYPED, W-PASTE) acknowledge the channel; nothing is appended to P ’s reply stream, so the presence or absence of a paste event is not P -observable (Rule R7, convention (C9)). (vii) *Fresh-copy endorsement; block immutability.* An accepting confirmation never relabels a block in place: CONF-ACC and TOOL-GATE-ACC write, per accepted value, one *fresh* C-labelled block carrying exactly the displayed bytes, at the next endorsed-sort identifier (Rule R1(g)), with scope $\sigma(t) = \{t\}$ for a tool target and $\sigma(\text{reasoning}) = \emptyset$ (readable, but never eligible for the sensitive fast path of Rule R4); the resolved source block, where one exists, is unchanged and remains at its level, and the publish target (Rule R12) is a shared-tier act creating no local block. Endorsement is *durable*: the accepted positions’ fresh copies are written at the close of the atomic step whether or not the enclosing invocation executes (Rule R2). Consequently no transition of Figures 7–8 alters an existing block’s bytes, label, or scope: blocks are immutable once written, and the store changes only by creation of fresh blocks and by EVICT. (vii’) *Batch acceptance is call-local for elements.* An accepting batch confirmation (Rule R9) authorizes each $\diamond$ -element’s bytes for its own element call of that map instance only: no persistent endorsed copy of any element is created, and the elements remain U-labelled in $\mathcal{M}_{\mathbb{U}}$. Fresh C-copies are written for the accepted *gating non- $\diamond$ arguments*, one each, scoped $\{\text{tool}\}$, exactly as at TOOL-GATE-ACC, and durably so: they persist even when a later element page rejects and no element call executes (MAP-TOOL-REJ). This is forced by the identifier discipline of Rule R1(g): per-element copies would allocate $|H|$ endorsed-sort identifiers with $|H|$ attacker-chosen, exposing an attacker-dependent quantity in $\mathcal{M}_{\mathbb{E}}$; the non- $\diamond$ count is fixed by P ’s emitted body. Convention (vii)’s per-accepted-value fresh copy, and the endorsed-copy clause of Definition F.5, are read with this carve-out: they govern CONF-ACC and TOOL-GATE-ACC; batch-accepted elements flow under Rule R2’s displayed-bytes binding without entering $\mathcal{M}_{\mathbb{E}}$. Consequently re-processing the same elements in a later map is gated afresh — call-local authorization caches nothing — and the once-forced cap (INV-3) does not extend to map elements, whose confirmations are per-task cost (Appendix J, Scope).

Commentary (non-normative; the MAP rules and the mechanization boundary). The split fixes, per body form, the decision token (none for the tool-free form), the store effect (capture of element outputs vs. schema-bound U-summaries $s_1, \dots, s_{|H|}$ written as in CRYST), and the endorsement effect (fresh C-copies for gating non- $\diamond$ arguments only; elements are authorized call-locally, convention (vii’)). In every case, including rejection, exactly one collection-handle index is consumed, on rejection with null resolution — so handle structure is independent of the outcome (Rule R1(a)), closing the identifier channel a skipped index would open. map is normative here; its batch-confirmation surface — pagination, per-page decisions, the aggregated decision, all-or-nothing displayed-bytes flow, and batch determinacy — is mechanized

as a standalone layer (`BatchGate.lean`, §G.6), while its store-level integration into the run-level interpreter remains outside the mechanized surface (delta (6)).

F.3 INTERFACE RULES

The rules below constrain the transitions of Figures 7–8. Each clause is normative and implementable as stated; all rationale lives in Appendix K under the keys cited, or in the marked commentary blocks of §E.1.

Rule R1 (Operation-determined naming: handles and block identifiers).

- (a) Handle identifiers (including collection-handle identifiers) are drawn from the sequence of P -emitted operations alone: the j -th handle-producing operation of a run binds identifier h_j . Identifiers never encode, count, or depend on store contents.
- (b) Handle-producing operations: ingress capture (the triggering tool invocation is P -emitted; P computes the collection handle’s identifier from its own operation count; nothing is returned); recall returning a handle; fill; crystallize; map.
- (c) A map invocation is exactly one handle-producing operation regardless of collection cardinality: its per-element outputs are captured as U-blocks bound to a single fresh collection handle determined by the map’s position in P ’s operation sequence. Element calls are interpreter-iterated, not P -emitted, and are individually neither handle-producing nor acknowledged.
- (d) P reaches captured element outputs only through fill/crystallize searches. No handle identifier depends on collection size.
- (e) A collection handle admits exactly two uses: as the argument of map, or as a tool argument — the latter defined by desugaring: an invocation `tool(..., h, ...)` with a collection-handle argument h is the instance `map(tool(...,  $\diamond$ , ...), h)`, the desugaring applied before the invocation matches any transition of Figures 7–8 (it never instantiates TOOL-FAST or TOOL-GATE-*), batch-gated per Rule R9 and subject to the body grammar of clause (f) (in particular at most one collection handle per invocation and no other handle argument; violations take the generic failure outcome, deterministic in P ’s own emission, Rule R7). A collection handle exposes no length, no indexing, and no per-element access.
- (f) *Map bodies.* The body $\hat{o}$ of `map( $\hat{o}$ ,  $H$ )` is a single operation template containing exactly one occurrence of the distinguished element placeholder $\diamond$, of one of exactly two forms: (i) one tool invocation `tool( $a_1, \dots, a_n$ )` in which one argument is $\diamond$ and every other argument is a P -supplied non-handle value, subject per argument to the ordinary mediation of Rule R4 (fast-path values flow; a gating argument is confirmed once for the whole batch, at the batch confirmation of Rule R9, alongside the elements); or (ii) one tool-free `crystallize( $\diamond$ )` template. The body may not contain map, recall, fill, evict, confirm, display, the write operation `writeP`, or a handle argument (the interpreter-side captures of forms (i)–(ii) are transitions of the interpreter, not body operations); nesting is not defined. Well-formedness is a syntactic test on P ’s emitted operation alone, and a body outside this grammar takes the generic failure outcome, deterministic in P ’s own emission (Rule R7). Multi-step per-element pipelines are expressed as successive maps: each map yields a fresh collection handle (clause (c)), which clause (e) admits as the argument of a further map, each step batch-gated independently; the grammar restricts the shape of one step, never the depth of a pipeline.
- (g) *Two-sorted block identifiers.* Block identifiers are drawn from two disjoint sorts, *endorsed* ($\text{id}_0^{\mathbb{E}}, \text{id}_1^{\mathbb{E}}, \dots$) and *untrusted* ($\text{id}_0^{\mathbb{U}}, \text{id}_1^{\mathbb{U}}, \dots$), each allocated in sequence. Endorsed-sort identifiers are allocated by exactly the local endorsed-block-creating transitions: W-TYPED, W-P, CONF-ACC, and, per *accepted gating position*, TOOL-GATE-ACC, TOOL-GATE-REJ, MAP-TOOL-ACC, and MAP-TOOL-REJ (for maps, gating non- $\diamond$ body positions only — a count fixed by P ’s emitted operation, never by the collection cardinality, convention (vii’)). A publish decision allocates no identifier of either sort (PUBLISH-ACC). The j -th endorsed block of a run sits at $\text{id}_j^{\mathbb{E}}$, a position determined by the coupled event sequence (the typed stream, P ’s emitted operations, and the logged decisions) alone, hence computable by P and identical across runs whose coupled events agree. Untrusted-sort identifiers are allocated by the untrusted-writing transitions (W-PASTE, ingress capture, CRYST) and may depend on attacker-chosen multiplicities; no untrusted-sort identifier is ever exposed to P by any reply or view (Rules R6, R7, R10(b)). An endorsed-sort identifier never resolves to a U-block and an untrusted-sort identifier never resolves to an $\mathbb{E}$ -block, so the sorts separate the tiers syntactically. Like handle identifiers, block identifiers of either sort never encode, count, or depend on store contents beyond this allocation order.

Commentary (non-normative; role of the identifier discipline). Clause (g) closes the identifier channel that in-place endorsement would open: identifiers visible to P (those of $\mathcal{M}_{\mathbb{E}}$) depend only on coupled events, while every attacker-dependent multiplicity can influence only untrusted-sort identifiers, which no reply and no view exposes. It is consumed by Theorem G.25 (Step 4) and is machine-checked as the two-sorted id space of the mechanization (§G.6).

Rule R2 (Atomic gating; durable per-value endorsement). When confirmations gate a tool call, the reviews of all gating positions, the writing of the accepted positions’ fresh endorsed copies, and the invocation (or its abort) form one atomic interpreter step: no step may interleave between display, decision, endorsement, and flow. The bytes that flow at an executed call are the displayed bytes, bound to that call instance. If any gating position is rejected or null, the invocation aborts entire: no argument flows; the accepted positions’ endorsed copies are nevertheless written, their accepting decisions being recorded in the transcript like any confirmation. When a batch confirmation gates a map (Rule R9), the aggregated accepting decision and the batch of element invocations form one atomic step, and the bytes that flow at each element call are exactly the element bytes displayed for that call; accepted gating non- $\diamond$ arguments are endorsed durably exactly as at a direct call, whatever the element pages’ outcome.

Commentary (non-normative; severability and durable endorsement). Soundness of durable endorsement on an aborting transaction is the severability clause of A4: intent is per displayed value, severally. A commit-on-success variant — discarding accepted positions’ copies on abort — would be safety-monotone but *sacrifices the once-per-pair cap* (INV-3): a transaction that accepts one pair and aborts on a later position leaves no durable copy, so re-presentations of the same pair re-force it without bound. Durability is adopted precisely because the cap is load-bearing (Theorem J.2).

Rule R3 (Ingress: all tool outputs captured). Tools partition into $\text{Tools}_{\text{sens}}$ (sensitive, effectful) and $\text{Tools}_{\text{data}}$ (data readers); the partition and the canonical tool identifiers are fixed deployment configuration, per capability, inside the TCB. The interpreter captures *every* tool output, including the status and error responses of sensitive calls, to $\mathcal{M}_{\text{U}}$ with label U, and returns to P only a single constant acknowledgement token per invocation: never the bytes, never a success bit, never a count, size, or identifier derived from captured content.

Rule R4 (Whitelist mediation at every tool boundary).

- (a) A value a passed to a tool in $\text{Tools}_{\text{sens}}$ flows without confirmation *iff* a consists of readable $\mathbb{E}$ -level bytes and the interpreter verifies that a byte-equals a stored T-block, or byte-equals some C/CC-block b with tool $\in \text{scope}(b)$ (for CC, the matched block is the live shared-tier entry, checked valid at call time; §H.1). (PC blocks have empty scope and never pass this test.)
- (b) A value passed to a tool in $\text{Tools}_{\text{data}}$ flows without confirmation *iff* it consists of readable $\mathbb{E}$ -level bytes.
- (c) A handle argument never flows silently to any tool: it always triggers $\text{confirm}(a, \text{tool})$, at data tools exactly as at sensitive ones, regardless of whether its resolved bytes coincide with stored endorsed content. There are no per-tool exceptions.
- (d) The fast-path check is membership in $\mathcal{M}_{\mathbb{E}}$ plus a scope test, not taint-tracking of P ’s code.

Rule R5 (Readable local endorsement). Every accepting confirmation with target $t \in \text{Tools}_{\text{sens}} \cup \text{Tools}_{\text{data}} \cup \{\text{reasoning}\}$ writes a fresh block $\text{newC}(v, t)$ (Options O1–O2: $\text{newCache}(v, t)$) carrying exactly the displayed bytes, labelled C (or, under community promotion, the shared tier carries CC status; Appendix H), at the next endorsed-sort identifier (Rule R1(g)); its bytes are readable by P via recall, the identifier being computable by P from its own coupled event history. The originating untrusted block, where one exists, is unchanged and remains untrusted. Before confirmation, a crystallized summary is U-labelled, lives reader/interpreter-side, and is not in σ_P . An accepting *publish* decision creates no local block and allocates no endorsed-sort identifier: it emits only the shared-tier submission event of Rule R12 (PUBLISH-ACC).

Rule R6 (Null handle). A fill or crystallize search over $\mathcal{M}_{\text{U}}$ that finds nothing returns a *null handle*: the handle’s existence and identifier are fixed by the requesting operation (Rule R1); only its resolution differs, and that difference is hidden from P . Likewise, recall on an identifier that does not resolve to a readable $\mathbb{E}$ -block returns a handle of the same kind: whether it resolves to U-bytes or to nothing (in particular, whether the identifier is occupied at all) is hidden from P . If a null handle is passed to any tool, the gate fires as usual (Rule R4), the decision pane renders a fixed “no value resolved” display, recorded in the display transcript like any confirmation event, the call aborts, and P receives the standard reject decision, indistinguishable in the reply stream and in the sanitized history h from a user rejection (Definition F.2); the no-value display itself is recorded on the transcript like any confirmation event.

Rule R7 (Failure indistinguishability). The success/failure of a U-store search, and the resolution status of any recall, is consumed interpreter-side to choose the null-vs-real handle; no reply and no sanitized-history record exposes it. A handle’s private resolution may later be displayed at a gate: that display and the user’s logged decision are an explicit transcript release (Definition G.8), not a failure channel. Any reader error, reader crash, or reader budget exhaustion

arising during an operation over $\mathcal{M}_U$ is likewise consumed interpreter-side: the operation returns the same operation-determined handle it would return on success (Rules R1, R6), whose resolution to bytes, to nothing, or to an aborted computation is hidden from P . The generic failure outcome (fail, Figures 7–8) is reserved for failures that are a deterministic function of P 's own emitted operations, $\mathcal{M}_E$, and the coupled coins ω alone (malformed invocations, failed preconditions on σ_P , and the retrieval failure of RETRIEVE-F). Search outcomes over $\mathcal{M}_E$ are trusted data and may be exposed. (Rationale and the store-quota corollary: N4, S7, convention (C7).)

Rule R8 (Faithful display on a split, zoned surface).

- (a) The display surface is split into a *decision pane* and auxiliary *selection surfaces*.
- (b) The decision pane shows exactly the resolved bytes that will flow (or be recorded as endorsed content), plus a provenance line in two visually separated zones: (i) *attested facts* (channel, label history, timestamps, signature-verification status), rendered in an interpreter-controlled region: TCB-authenticated, not attacker-forgeable or -alterable in content (event *timing* may be attacker-influenced, e.g. a captured tool output's timestamp reflects when the attacker acted; what is guaranteed is that the recorded facts are true and unmodified); (ii) *source-identification strings* originating in untrusted content (sender names, subjects, citations), rendered verbatim and visibly marked *unverified*. Where a transaction aborts after some accepting reviews, the pane states that the accepted values remain authorized for their targets.
- (c) The decision pane contains no reader-generated prose outside the confirmed payload region; where the payload itself is a crystallized summary, it is reader-generated by construction, rendered verbatim, and governed by Assumption A5.
- (d) Selection surfaces may show reader-generated previews, visibly marked unverified; previews are rendered UI-side only and never returned to P , and carry no endorsement authority.
- (e) Both surfaces are recorded in the display transcript and the audit log (Definition G.8).
- (f) On a rejection, the displayed bytes are not returned to P ; P receives only the decision. (Preview rationale: N8, S9.)

Rule R9 (Batch gate under untrusted-driven iteration).

- (a) A tool call reached inside map over a U-collection — including the desugared form of a collection-handle tool argument (Rule R1(e)) — takes no fast path of Rule R4.
- (b) The map is gated as a single *batch* confirmation whose decision pane displays the collection's cardinality *and the full resolved bytes of every element*, together with the resolved bytes of any gating non- $\diamond$ argument of the body (Rule R1(f)), displayed once for the batch, paginated as needed, the cardinality displayed and recorded on the first page; per-page cardinality is capped at c_{batch} ; the interpreter performs any split, presenting a larger collection as successive pages of one batch confirmation. A deployment may additionally fix a total-cardinality cap (TCB configuration): a map over a collection exceeding it renders a fixed oversize display — recorded in the transcript like the no-value display of Rule R6 — and takes the standard aggregated reject of clause (c), consuming no decision, indistinguishable in the reply stream and in h from a page rejection or a null collection. A *null collection* (a collection handle with null resolution: an empty search result, or a zero-output capture, convention (i); Rules R1(a), R6) is handled identically: the interpreter renders the fixed no-value display of Rule R6, recorded in the transcript as a batch event exactly like the oversize display, and the map takes the standard aggregated reject of clause (c), consuming no decision (MAP-TOOL-REJ; mechanized as `batchGateNull`, §G.6). The no-value and oversize displays are distinct event kinds (Definition G.8), so the first recorded batch event determines the size class (Lemma G.20).
- (c) P receives a single aggregated decision for the whole map (accept iff every page was accepted, else the single standard reject), regardless of page count. The number of pages, like the cardinality itself, is never returned to P in any form; both are displayed to the user and recorded in the transcript only.
- (d) Each page's rendering and decision is recorded in the transcript; the aggregated decision is recorded once. By Rule R2, the bytes that flow at each element call are exactly the element bytes displayed for it. The user may reject at any page, aborting the remainder. The oversize and null-collection displays are recorded under their respective fixed-display event kinds (Definition G.8), each with its standard reject and the aggregated batch decision.
- (e) Per-element results are captured to $\mathcal{M}_U$ and never returned to P (the map returns one ack token). (All-or-nothing rationale and c_{batch} sizing: N6, S12.)

Rule R10 (Constant-alphabet interface, with totality).

- (a) Every value the interpreter returns to P is of one of two kinds. The first kind is drawn from a fixed finite alphabet independent of the attack and the environment: (i) one constant acknowledgement per *executed* tool invocation or map, independent of outcome and of the number, size, or content of captured blocks; a gate-rejected invocation does not execute and returns only the reject decision of item (iv) (TOOL-GATE-REJ); (ii) opaque handles named by Rule R1; (iii) the generic failure outcome of Rule R7, deterministic in P 's emitted operations, $\mathcal{M}_{\mathbb{E}}$, and the coupled coins ω ; (iv) the user's decision at a confirmation, with the null-gate abort of Rule R6 rendered as the reject decision. The second kind is (v): verbatim $\mathbb{E}$ -bytes dereferenced from $\mathcal{M}_{\mathbb{E}}$.
- (b) *Totality*. Every syntactically well-formed invocation of every operation of Figures 7–8, with any argument, has its behaviour defined by the figures and these rules; failed preconditions and unresolvable arguments are rendered, uniformly, as the generic failure outcome or as an operation-determined opaque handle, never as a distinguishing error. In particular, whether a given identifier is occupied by a U-block is never observable to P through any operation: recall and evict alike return operation-determined replies (an opaque handle; the constant ack) uniformly across resolving, non-resolving, U-occupied, and $\mathbb{E}$ -occupied arguments, resolution consumed interpreter-side (Rules R6, R7; Figure 8, convention (v)).

Rule R11 (Trusted-store retrieval). $R_{\mathcal{E}}$ (i) reads only $\mathcal{M}_{\mathbb{E}}$, never any U-bytes; (ii) has no tool access; (iii) returns only block identifiers, which the interpreter dereferences against $\mathcal{M}_{\mathbb{E}}$ verbatim; non-dereferencing output is discarded. $R_{\mathcal{E}}$'s sampling randomness is coupled alongside P 's (convention (C1)). (Why neither reader's vision can be relaxed: N3.)

Deployment-selectable hardening. Options O1–O2 are formally instantiated: each strictly gates more flows, and their monotonicity is proved as Proposition G.28. Option O3 is a requirements-level extension, outside the formal semantics and outside Proposition G.28's quantifier (O3).

Option O1 (Sink-scoped trusted values). T-blocks require a first-use confirmation per sensitive sink, after which scope-caching applies. Blocks created by these confirmations carry the interpreter-only provenance mark *cache* and are ineligible for submission (Rule R12).

Option O2 (Exact-target mediation at data tools). The data-tool clause of Rule R4(b) is replaced by the sensitive-tool test of clause (a): a value passed to a tool in $\text{Tools}_{\text{data}}$ flows without confirmation *iff* it byte-equals a stored T-block, or byte-equals some C/CC-block b with tool $\in \text{scope}(b)$ (for CC, the live shared-tier entry, checked valid at call time, §H.1). Any other data-tool argument — a freshly composed value, or endorsed bytes whose scope does not admit this data tool (in particular every reasoning-target copy, whose scope is empty, convention (vii)) — is confirmed once; the fresh copy is scoped $\{\text{tool}\}$ per convention (vii), so it scope-caches and covers all the user's later use of the pair (INV-3). Blocks created by these confirmations carry the interpreter-only *cache* mark and are ineligible for submission (Rule R12); their gate events are cache confirmations, accept-inert per Definition H.6.

Option O3 (Local scope leases (requirements level; not formally instantiated)). C-scopes carry a deployment-set lease (convention (C4)): expiry withdraws the expired copy's tool authorization, so the pair's next scope-checked use with no other currently authorizing witness re-confirms once. Unlike Options O1–O2, O3 is specified at the requirements level only: it is *not* part of the formal semantics of this document, no theorem quantifies over it, and Proposition G.28 does not cover it. A conforming instantiation is a future extension obligated to satisfy all of the following, and to prove analogues of Lemma G.20, Theorem G.25, Lemma J.1, Theorem J.2, and the ledger accounting of Lemma J.7 for the extended semantics.

- (a) *Immutability*. Blocks stay immutable (A2): a lease is a validity predicate computed interpreter-side from creation-time metadata (creation epoch or slot index, never wall-clock time) and the current index; expiry is never a block mutation or removal.
- (b) *Determinism on matched data*. Validity inputs are functions of the coupled event sequence, and lease state is added to the matched components of Definition G.6 and carried through the unwinding, so matched runs gate identically.
- (c) *Witness-level expiry at tool targets*. For tool targets, expiry invalidates only the expired local authorization witness (an ordinary or cache-marked C copy), and the two authorization tests it touches are kept distinct. *Literal authorization*, on a scope-checked literal path, is existential over whichever witnesses the deployment's selected formal options recognize there — byte-equal T-blocks (per O1's per-sink treatment where selected), live CC entries, and lease-valid in-scope C copies; the baseline data-tool path of Rule R4(b) is scope-blind, so lease enforcement at data tools requires O2 — and a literal use gates *iff* no such witness currently authorizes the pair. *Handle coverage* (Rule R13(i)) is narrower: a handle use gates in every case (Rule R4(c)) and is *covered* *iff* the identity holds a lease-valid, in-scope local C copy; a T or live-CC witness never makes a handle use covered, exactly as in the base semantics. The reasoning-coverage branch of Rule R13(i) remains readability-based and lease-insensitive: no reasoning-target reuse is ever re-forced.

- (d) *Classification and renewal.* An expiry-induced gate whose pair lacks any currently authorizing witness is *forced* — whether a literal gate fired by expiry or a handle gate reclassified from covered to forced because no lease-valid in-scope C copy remains. Any accepting forced confirmation so caused writes the fresh copy prescribed by the *triggering rule* — an ordinary C copy at an ordinary confirmation, a cache-marked, federation-inert copy at an O1/O2 cache gate (Rule R12, Definition H.6(iii)) — and starts a new lease, so subsequent literal or handle use within the new period is fast-pathed or covered: at most one forced confirmation per pair per lease period. The class is determined by the gate that fired, never by any expired duplicate witness; rejection and abandonment renew nothing; duplicate matching copies are resolved by the existential tests of clause (c), never by a selection among witnesses.
- (e) *Accounting.* With each (user, item) seeing $E_{\text{lease}} = \mathcal{O}(1)$ lease periods, the extension’s pathwise floor is the once-per-pair cap per lease period, an $\mathcal{O}(1)$ multiplicative factor on Theorem J.2’s constants — an extension-level claim, to be proved with the extension. No saturation-vanishing conclusion follows from $E_{\text{lease}} = \mathcal{O}(1)$ alone: expiry re-fires of promoted pairs are residual forced confirmations, so the extension additionally requires an O3-augmented analogue of the *complete* residual bound of A11 and Theorem J.9(iii) — the per-pair $D_0 + \varepsilon n$ form with expiry re-fires included, together with its product and residual-fraction rate conditions — not merely a substitute for $\varepsilon(M) \rightarrow 0$.
- (O3 is the single-user twin of the federated re-verification queue: N16.)

F.4 TRUST AXIOMS

Assumption A1 (Deterministic source labelling). $\ell(b)$ is a function of the channel/source of b alone, never of bytes(b). Hence the attacker cannot change any label.

Assumption A2 (Label persistence and block immutability). Blocks are immutable once written: no party and no transition alters an existing block’s bytes, label, or scope; the store changes only by the deterministic block-creating transitions of Figures 7–8 (convention (vii)) and by evict. Trust is raised only by the creation of a fresh endorsed block at an accepting confirm, or by deterministic community promotion of a shared-tier entry (Appendix H); the attacker can do neither.

Assumption A3 (Faithful, zoned display). At a confirmation, the decision-pane bytes are exactly the bytes that will flow or be recorded; the attested-fact zone is TCB-authenticated and not attacker-forgeable (Rule R8(b)); untrusted source-identification strings appear only in the marked zone; previews are confined to selection surfaces, visibly marked (Rule R8).

Assumption A4 (Sound atomic endorsement). The user returns *accept* on a displayed value v at tool tool only if the user intends exactly v to be authorized for tool; the intent is per displayed value, severally, and does not depend on the outcomes of the transaction’s other reviews — which is what makes the durable endorsement of Rule R2 sound when a later review aborts the invocation. At a batch confirmation (Rule R9), the user returns *accept* on a page only if this intent holds for *each* displayed element and each displayed gating non- $\diamond$ argument, severally, at the batch’s tool: batch acceptance is the conjunction of the per-value intents, never an aggregate judgment of the collection.

Assumption A5 (Sound narrative endorsement). The user returns *accept* on a displayed crystallized summary S only if, having checked S against its displayed source citations, the user intends S ’s content to inform reasoning.

Assumption A6 (Exogenous command stream). The user’s typed command-channel input is an exogenous input stream, coupled identically across the runs compared in Theorem G.25.

Commentary (non-normative; epistemic status of A4–A6). A4 and A5 are trust axioms in the sense of CaMeL DeBenedetti et al. (2025); A6 is the human-channel idealization that any displayed-content carve-out requires. Their epistemic status, the regime that makes them defensible, and the discharge of A6’s practical content by logging both display surfaces are discussed at N5 (see also S9, S10, S12); A5’s realistic weakness at scale is what the community mechanism of Appendix H addresses.

F.5 ATTACKER MODEL AND TRUSTED COMPUTING BASE

Attacker. The attacker controls the *content* of every U-labelled block, all tool outputs (including sensitive-tool responses and error strings) and paste-channel input, including the byte-length and the number of U-blocks captured per call. It does not control the user’s typed command input, the labelling, the interpreter, or the user’s decisions. P , Q , and $R_{\mathcal{E}}$ are *isolated from the attacker, not assumed benign*: the single-user theorems hold under adversarial behaviour of either reader, and misdirection among the user’s own typed or endorsed values is a utility failure outside the threat model (Option O1 offers defence in depth; scope discussion: §K.4).

Trusted computing base (TCB), single-user. The TCB comprises: an authenticated command channel distinguishing typed from pasted from tool input; the canonical tool identifiers and the $\text{Tools}_{\text{sens}}/\text{Tools}_{\text{data}}$ partition; the interpreter’s implementation (including byte-equality and dereference checks) and the individuating parser; the zoned rendering path of Rule R8; and the hash-chained audit log materializing the display transcript. The store idealization and quota policy are convention (C7). Federated TCB additions are listed in §H.3.

G SINGLE-USER GUARANTEES

This appendix proves two conditional interface properties. *Action Integrity* is an exact-byte, exact-tool authorization property: no untrusted-origin *value* reaches a sensitive action unless the user endorsed exactly that value for exactly that action; it does not authorize the choice to invoke a tool (§K.4). *Reasoning Integrity modulo the display transcript* is a fixed-horizon, termination-insensitive trace-equivalence property: holding fixed everything the user ever saw and decided on the display surface, together with the coupled exogenous inputs, untrusted memory has no further effect on behaviour; it is not a claim of reasoning correctness or of resistance to manipulation through displayed content (Remark G.9, N7). The statements are in the noninterference-modulo-declassification tradition Goguen & Meseguer (1982; 1984); Sabelfeld & Sands (2009); Sabelfeld & Myers (2004); Askarov & Sabelfeld (2007); the first is the PerMIT instance of qualified robustness Zdancewic & Myers (2005); Myers et al. (2006). Both theorems, the supporting lemmas, the baseline separation, the flow half of the hardening monotonicity, and the batch-confirmation surface of Rule R9 are machine-checked (§G.6: scope, axiom audit, artifact, correspondence Table 13, and the disclosed modelling deltas). Lineage and positioning: N1.

G.1 DEFINITIONS

Definition G.1 (U-equivalence). $c_1 \approx_U c_2$ iff $\text{obs}_P(c_1) = \text{obs}_P(c_2)$: agreement on $\mathcal{M}_{\mathbb{E}}$ (content and scopes), $H(c)$, $x(c)$, and h , while $\mathcal{M}_U$ may differ arbitrarily Volpano et al. (1996); Sabelfeld & Myers (2003).

Definition G.2 (Sink-event safety). Fix a finite sequence of display events — decision-pane confirmation events, each carrying its displayed bytes, target, and decision (Definition F.5, Rule R8), and renderings — and a trusted typed-input stream x . A recorded sensitive sink event is *safe* for that history if every argument value of the event is: (i) the exact bytes of a typed input unit of x (a T-block’s bytes, individuation per §F.1); or (ii) endorsed at a prior accepting decision-pane confirmation event of the history displaying exactly those bytes at exactly the event’s tool (for element calls of a batch-gated map, at the batch confirmation displaying those exact element bytes, Rule R9); or (iii) gated at the event’s own call, the flowed bytes being the displayed bytes of that call’s accepting confirmation (Rule R2). The *delegated* form extends case (ii) by one alternative, relative to a designated federation-log prefix (in a run, the prefix delivered at the event’s step, Definition H.2): the bytes byte-equal a live community entry of that prefix’s replay naming the event’s exact tool, backed by a currently valid promotion certificate (Definition H.1, A7); the personal form is the single-user predicate, machine-checked as `SinkSafe` (§G.6). It is consumed by Definition G.3(v) and Definition G.10.

Definition G.3 (Well-formed configuration). A configuration c is *well-formed*, written $\text{WF}(c)$, if: (i) every T-block’s bytes occur in the typed stream $x(c)$; (ii) every C-block b with $t \in \text{scope}(b)$ is backed by an accepting decision-pane confirmation event displaying exactly $\text{bytes}(b)$ at target t in the audit transcript, with the matching sanitized (target, accept) record in h ; (iii) no block carries label CC (single-user; the federated counterpart replaces (iii) by a valid promotion certificate on the federation log, Appendix H); (iv) the handle structure $H(c)$ and the endorsed-sort identifier allocation are the operation-determined structures of Rule R1 for the configuration’s interaction history, σ_P is the strategy state of that history (§F.1), and the recorded reply stream is the interface’s reply to that history (Figures 7–8); (v) every recorded sink event is safe (Definition G.2; federated form: the delegated predicate). The empty configuration is well-formed. A *seeded* configuration (nonempty $\mathcal{M}_{\mathbb{E}}$, empty interaction history) is well-formed only together with a *seed record*: a designated initial-authorization prefix of the audit transcript (with its sanitized projection in h) backing every seeded T-block per (i) and every seeded scoped C-block per (ii) — operationally, the authorizations imported with a migrated store; formally, the record is read as part of $x(c)$, h , and DT, whereupon (i)–(v) hold verbatim. A seeded configuration without such a record is in general *not* well-formed; it is *sorted* (Definition G.5), which is what the two-run results require. Well-formedness is preserved by every transition of Figures 7–8 (this is the induction of Lemmas G.12–G.14, machine-checked as the invariant INV of §G.6). Action Integrity and its corollaries quantify over runs from well-formed configurations; the two-run results quantify over matched pairs of sorted configurations (Definitions G.5, G.6).

The history against which clause (v) reads Definition G.2 is the configuration’s full audit history: any seed-record prefix followed by the run transcript (Definition G.8).

Remark G.4. Clause (v) is the standard inductive strengthening required to prove Theorem G.21 by induction on run length (mirrored verbatim as the `sinkSound` field of `Inv` in the mechanization, §G.6): it tracks, through the induction, exactly the per-event predicate of Definition G.2, whose universal form over reachable configurations is the theorem’s conclusion — not an independent assumption, and not circular: the predicate is defined without reference to the theorem. The from-arbitrary-well-formed forms are *preservation* statements; the empty-initial-configuration forms, where the invariant is established rather than assumed (`inv_init`, §G.6), are the creation forms.

Definition G.5 (Sorted configuration). A configuration c is *sorted* if clauses (iii) and (iv) of Definition G.3 hold of it: no CC label (federated form as there), operation-determined handle and endorsed-sort identifier structure with σ_P the strategy state, and an interface-conformant reply stream. Every well-formed configuration is sorted. Every *endorsed-seed* configuration — an arbitrary $\mathbb{E}$ -level $\mathcal{M}_{\mathbb{E}}$ at endorsed-sort identifiers (Rule R1(g)), an arbitrary $\mathcal{M}_U$, and an empty interaction history — is sorted, with no seed record required. Sortedness is preserved by every transition (machine-checked as `WFs`, with the endorsed-seed condition `EStore`; §G.6).

Commentary (non-normative; why two classes). The split is forced by the theorems, in opposite directions. Action Integrity is an endorsement-soundness statement, so its quantifier must carry endorsement soundness: admitting a bare seed as well-formed would *falsify* Theorem G.21 (Remark G.15) — a seeded C-block with $t \in \text{scope}$ flows to t ungated, with no endorsing confirmation anywhere in the run. Reasoning Integrity, by contrast, never consumes endorsement soundness: its unwinding needs only the matched view, the operation-determined naming of Rule R1, and the strategy reading — precisely the sorted clauses — so its quantifier is deliberately the weaker class, admitting arbitrary shared endorsed seeds. The mechanization enforces exactly this split: `action_integrity_from` requires the full invariant `Inv`, while `reasoning_integrity_from` requires only `EStore/WFs` (§G.6).

Definition G.6 (Matched configurations). Two sorted configurations (Definition G.5) are *matched*, $c_1 \approx c_2$, if $\text{obs}_P(c_1) = \text{obs}_P(c_2)$; P ’s interaction histories coincide (the same emitted-operation sequence, the same reply stream, and the same consumed coin prefix — hence $\sigma_{P,1} = \sigma_{P,2}$, by the strategy reading of §F.1); and the ghost display transcripts and sink logs coincide. Matched views and histories also fix the endorsed-sort allocation state (Rule R1(g)), so subsequent endorsed blocks are created at equal identifiers in the two runs. $\mathcal{M}_U$ and the interpreter’s private handle *resolutions* may differ arbitrarily; the handle *count* is matched through $H(c)$ (Rule R7). Matched implies $\approx_U$; the converse fails on σ_P and the ghost logs, which is why the two-run results are stated over matched pairs. The canonical matched pair is two runs launched from a shared endorsed-seed configuration (Definition G.5) with empty histories and empty ghost logs under two different attacks (the form mechanized as `initE`, §G.6); matched pairs need not be well-formed, and Theorem G.25 holds without any endorsement-soundness hypothesis on the seed. This relation is mechanized, field for field, as `MatchW` (§G.6).

Definition G.7 (Ungated sink projection). For a finite run ρ , $\text{Sink}(\rho)$ is the sequence of sensitive sink events, each with its arguments, except that arguments displayed and accepted in the immediately preceding atomic gating confirmation (Rule R2, including, for element calls of a map, the batch confirmation of Rule R9) are stripped from the annotation. Stripping makes $\text{Sink}(\rho)$ well-defined for baseline comparisons in which transcripts may be empty (Proposition G.27); under DT-equality the stripped arguments are equal in any case, so the projection hides nothing in Theorem G.25.

Definition G.8 (Display transcript). For a run ρ from a designated starting configuration, $\text{DT}(\rho)$ is the ordered sequence of *all display events appended during* ρ (the run suffix; an audit prefix carried by a seeded starting configuration belongs to its seed record, Definition G.3, not to $\text{DT}(\rho)$): (i) selection-surface renderings, each with its candidate identifiers, rendered preview bytes (marked unverified), and the user’s selection, if any; (ii) decision-pane confirmation events, one per reached gating review, each with the resolved bytes (or the fixed no-value display of Rule R6), both provenance zones, the target, and the decision; (iii) for a batch, *either* the fixed oversize display of Rule R9(b) with its standard aggregated reject (where the deployment cap applies), *or* the fixed no-value display of a null collection with its standard aggregated reject (Rules R6, R9(b)), *or* every page’s rendering and per-page decision, the first page recording the collection’s cardinality — which, with c_{batch} , determines the deterministic pagination and hence the page count (Rule R9(b)) — and the aggregated decision, recorded once. Transcript entries carry interpreter-assigned event *kinds* (selection-surface rendering; decision-pane confirmation; batch page; the fixed no-value display; the fixed oversize display; aggregated batch decision), fixed constructors of the TCB logging path: the fixed displays are distinguished from ordinary pages by kind, never by comparing rendered byte strings, so page content colliding textually with a fixed message cannot confuse the classification (consumed by Lemma G.20). Rejections are recorded as well as acceptances, and so are the accepting reviews of transactions that subsequently abort (Rule R2). The harness materializes the full audit history — any seed-record prefix followed by $\text{DT}(\rho)$ — in a hash-chained audit log (operational reading: N7; previews: N8, S9).

Remark G.9 (Coverage of the display transcript). By Rule R8(e), $\text{DT}(\rho)$ comprises the totality of attacker-influenceable content ever rendered to the user, together with everything the user decided. Recording rejections is necessary: two runs in which the user rejects and later accepts the same content differ in behaviour during the inter-

val, and a projection that forgets rejections cannot distinguish them. Previews carry no endorsement authority; their inclusion records them as part of the release history.

Definition G.10 (Action Integrity, personal and delegated). PerMIT has *personal Action Integrity* (equivalently: *argument-value integrity*) if in every run from a well-formed configuration, under every attack, every sensitive sink event is safe in the personal form of Definition G.2: every value reaching a sensitive sink is a user-typed T-value, or was endorsed by *this* user at its exact bytes for that exact sink (now, or at a prior in-scope confirmation; for element calls of a batch-gated map, at the batch confirmation that displayed those exact element bytes, Rule R9). It has *delegated Action Integrity* if the same holds with the endorsement authority extended to a qualifying community confirmation of the exact bytes at the exact tool (CC, Appendix H) (the delegated predicate of Definition G.2); delegation is an assumption (A7), never a theorem of the single-user model.

Definition G.11 (Reasoning Integrity modulo the display transcript). PerMIT has *Reasoning Integrity modulo the display transcript* if, for every pair of *matched* configurations $c_1 \approx c_2$ (Definition G.6), every pair of attacks (coupled per convention (C9): slot skeleton and typed stream only; paste presence and content, tool outputs, and reader outputs uncoupled), and every coupled coin sequence ω (convention (C1)), any two runs of a common horizon (convention (C9)) with $\text{DT}(\rho_1) = \text{DT}(\rho_2)$ satisfy $\text{Sink}(\rho_1) = \text{Sink}(\rho_2)$ and emit the same sequence of Protected-LLM operations and tool invocations. Informal reading (a conditional statement, not an unconditional inertness claim): any influence of untrusted memory on the listed behaviours must pass through a display event or an endorsed release — conditional on the transcript, hidden untrusted state has no further effect. An unrendered untrusted byte is not claimed inert outright: it may steer which candidates or previews the Reader surfaces, and thereby the transcript itself; what is excluded is influence that bypasses the transcript.

The statement is trace-wise per coin sequence; the knowledge-based formulation in the style of gradual release Askarov & Sabelfeld (2007) that it induces is recorded in §L.4.

G.2 STRUCTURAL LEMMAS

Lemma G.12 (Protected-LLM isolation / provenance preservation). *In every configuration reachable from a sorted configuration (Definition G.5; a fortiori from a well-formed one), no byte value enters σ_P by delivery from a U-labelled source: every bytes-carrying reply P receives is, at the step of its emission, the verbatim content of a non-U block of the store, and U-derived data appears in σ_P only as opaque handles. This is a provenance statement, not a disjointness of byte strings across tiers: a typed or endorsed byte string in σ_P may be extensionally equal to the bytes of an unrelated U-block, and the flow it licenses is the trusted copy's. (Machine-checked, run-level: `bytes_reply_isolation`, §G.6.)*

Proof. Induction on run length with invariant $I(c)$: every bytes-carrying value ever delivered to P was delivered as the content of a non-U block (reply grammar case (v)) or as trusted command input; everything else in P 's input stream is a handle or constant token. At a sorted start, σ_P is the strategy state of the initial interaction history (§F.1, Definition G.5), whose reply stream is interface-conformant: every bytes-carrying reply in it is $\mathbb{E}$ -level by the reply grammar, and its typed input is trusted command input at level T; an endorsed-seed configuration has empty history and satisfies I trivially. For the inductive step, case-analyse the transitions of Figures 7–8. TOOL-FAST, TOOL-GATE-ACC/REJ, MAP: by Rules R3 and R9 all outputs land in $\mathcal{M}_U$ and only the constant acknowledgement (and, for gated calls, the decision) returns; the collection handle's identifier is operation-determined and computed by P , not returned (Rule R1(b)). W-TYPED stores T; W-PASTE is captured to $\mathcal{M}_U$. W-P has precondition $b \in \sigma_P$, hence $\mathbb{E}$ -level by the hypothesis, and stores at PC with empty scope. RECALL-E returns $\mathbb{E}$ -bytes; RECALL-O returns an opaque handle: uniformly for U-blocks, unknown, and nonexistent identifiers (Rules R1, R6, and the totality clause Rule R10(b)); R_E -retrieval returns verbatim $\mathbb{E}$ -bytes (Rule R11). FILL and CRYST return one handle and, by Rule R7, no branchable bit. DISPLAY and EVICT inject nothing (EVICT replies the constant ack uniformly, Figure 8, convention (v)); previews are UI-side only (Rule R8(d)). CONF-ACC/REJ return only the decision (the null-gate abort returning the reject decision, Rule R6) and on accept produce an $\mathbb{E}$ -block. FAIL returns the constant fail token. P -generated content is a function of σ_P and the coins, hence $\mathbb{E}$ -level by the meet rule (Definition F.1). By A1–A2 and convention (vii), no transition raises the level of any existing block; an accepting confirm writes a fresh $\mathbb{E}$ -block and leaves its untrusted source, if any, at U. Thus every delivery to P carries non-U provenance; the existence of extensionally equal untrusted copies is irrelevant. I is preserved. $\square$

Lemma G.13 (Whitelist mediation, by tool class). *Every value a passed to any tool satisfies the following, according to the tool's class. (Sensitive tools.) If $\text{tool} \in \text{Tools}_{\text{sens}}$, at least one of: (i) a consists of readable $\mathbb{E}$ -bytes byte-equal to a stored T-block; (ii) a consists of readable $\mathbb{E}$ -bytes byte-equal to some C/CC-block b with $\text{tool} \in \text{scope}(b)$; (iii) the call is gated by an atomic confirm(a , tool) displaying a 's resolved bytes (or, for a batch, every element's bytes; or, for*

a null handle, the fixed no-value display, whereupon the call aborts). For a literal-expressed argument, in the baseline semantics of Rule R4(a) — before the formally instantiated Options O1–O2 are applied — case (iii) is the complement of the union of (i) and (ii): the fast path passes iff (i) or (ii) holds, and the gate fires otherwise. Under Option O1, a sensitive-tool argument satisfying case (i) may nevertheless gate (first use of the T-value per sensitive sink); Option O2 leaves this sensitive case analysis unchanged and modifies only the data-tool case (D) below; the option-modified analyses are Proposition G.28’s. Option O3 is requirements-level, outside this lemma and outside Proposition G.28 (O3). Cases (i) and (ii) themselves may hold simultaneously — the store may contain a T-block and an in-scope C/CC-block with identical bytes, a co-occurrence Option O1 manufactures by design — and either suffices for the baseline fast path; no consumer of this lemma uses exclusivity of (i) against (ii), each case being separately sufficient in Theorem G.21. Handle-expressed arguments take case (iii) unconditionally (Handles clause below). (Data tools.) If $\text{tool} \in \text{Tools}_{\text{data}}$, then either (D) a consists of readable $\mathbb{E}$ -level bytes held by P — that every byte value P holds carries non-U provenance, is Lemma G.12’s contribution, not Rule R4(b) alone — which flow without confirmation (Rule R4(b)) — in particular PC-labelled and freshly P -composed $\mathbb{E}$ -bytes flow ungated at data tools; this is precisely the clause Option O2 hardens — or the call is gated per case (iii). (Handles.) Every handle argument, at every tool, falls under (iii): an ordinary handle via Rule R4(c); a collection handle via the batch gate of Rule R9, by the desugaring of Rule R1(e). (Under Option O2, case (D) is replaced by the clause-(a) case analysis applied at the data tool; Proposition G.28.) (Machine-checked core: `processArgs_spec` and `bytes_reply_isolation`, §G.6: an ungated flowed argument is a verbatim P -supplied literal, and at a sensitive tool it passes the whitelist test over $\mathcal{M}_{\mathbb{E}}$; that P holds only $\mathbb{E}$ -level bytes in case (D) is `bytes_reply_isolation`’s contribution; witness for case (D): Scenario 7.)

Proof. Immediate from Rule R4 with Rules R2, R6, R9: clause (a) gives the sensitive case analysis: in the baseline semantics the fast path passes iff the argument consists of readable $\mathbb{E}$ -bytes matching (i) or (ii), and the gate fires otherwise, so case (iii) is exhaustive over the complement, while (i) and (ii) may co-occur, harmlessly (the membership and scope tests are decidable against $\mathcal{M}_{\mathbb{E}}$, Definition F.2; PC blocks have empty scope and cannot take the sensitive fast path); clause (b) gives case (D) at data tools; clause (c) gives the handle case at every tool. For element calls of a map, the $\diamond$ -substituted element falls under (iii) via the batch confirmation (Rule R9), and each non- $\diamond$ argument, a non-handle P -supplied value by Rule R1(f), falls under the case its tool class assigns exactly as at a direct call, its gating branch resolved at the batch confirmation. $\square$

Lemma G.14 (Endorsement is the only trust raise).

- (a) (Transition-level; any run.) *The only C-creating transitions of Figures 7–8 are the accepting gating reviews — CONF-ACC and the accepted positions of TOOL-GATE-ACC, TOOL-GATE-REJ, MAP-TOOL-ACC, MAP-TOOL-REJ — each writing a fresh copy of exactly the displayed bytes with scope $\sigma(t)$ (Rule R2, convention (vii)); the only CC-assigning transition is deterministic community promotion (Appendix H); and no transition alters an existing block’s label or scope (A1–A2).*
- (b) (Backing; well-formed runs.) *In every run from a well-formed configuration (Definition G.3), every C-block b and every $t \in \text{scope}(b)$ is backed by an accepting decision-pane confirmation displaying exactly bytes(b) at target t , in the run suffix (by clause (a)) or in the seed record (Definition G.3(ii)); and every CC scope in force is backed by its promotion certificate, whose threshold was met by endorsements at the scoped tool itself (Rule R13(iv), Definition H.6).*

Proof. (a) is the case analysis below; (b) adds the base case: at the initial configuration, clause (ii) of well-formedness — with the seed record where the store is seeded — supplies the backing for every existing scoped C-block, and clause (iii) (federated form) the certificates. By A1–A2 and convention (vii), no transition alters an existing block’s label or scope, so a block’s label and scope are those of its creating transition. No write transition creates a block at C/CC; W-P creates PC with empty scope; the only endorsement-creating transitions are the accepting gating reviews — CONF-ACC, and the accepted positions of TOOL-GATE-ACC, TOOL-GATE-REJ, MAP-TOOL-ACC, and MAP-TOOL-REJ (Rule R2: endorsement is durable, written on the accepting and aborting branches alike) — each of which writes scope $\sigma(t)$ per convention (vii) for exactly the displayed bytes. The only CC-assigning transition is promotion Myers et al. (2006), which (Rule R13(iv), Definition H.6) adds tool to a CC scope only when the community threshold was met by endorsements at tool itself. $\square$

Remark G.15. Clause (b) is deliberately *not* asserted over the sorted class (Definition G.5): an endorsed-seed configuration may hold scoped C-blocks with no backing confirmation anywhere, and clause (b) fails there, while clause (a), a property of the transitions alone, holds in every run. Consumers of clause (b) quantify over well-formed runs (via Definition G.10); Theorem G.25 consumes neither clause. This is the `Inv-vs-EStore` split of Definitions G.3/G.5.

Lemma G.16 (No community label single-user). *In every run from a well-formed configuration with no federation attached, no block ever carries label CC: promotion is the only CC-assigning transition (Lemma G.14) and it does not exist in the single-user semantics. (Machine-checked as Inv.noCC , §G.6.)*

Proof. $\text{WF}(c_0)$ (iii) gives the base case; no transition of Figures 7–8 assigns CC (Lemma G.14’s case analysis). $\square$

Lemma G.17 (Interface completeness). *The sequence of values the interpreter returns to P is a function of the trajectory of $\text{obs}_P(c)$, P ’s own emitted operations and tool invocations, the coupled coins ω , and the user’s decisions.*

Proof. By the reply grammar of Figure 7, every return is one of: (a) a constant token (ack or fail, per Rule R10(a)(i),(iii)), including the uniform acknowledgement of evict, whose resolution status is consumed interpreter-side (Figure 8, convention (v)); (b) $\mathbb{E}$ -bytes from $\mathcal{M}_{\mathbb{E}}$, a component of $\text{obs}_P(c)$; (c) a handle whose existence and identifier are fixed by the emitted-operation sequence (Rules R6, R1), including collection handles, whose identifiers P computes and which are never returned, and the handles of RECALL-O, indistinguishable across resolution statuses (Rule R10(b)); (d) the user’s decision, with the null-gate abort rendered as the reject decision (Rule R6); (e) the verbatim dereference of an $\mathbb{E}$ -block selected by $R_{\mathcal{E}}$, a function of the query, $\mathcal{M}_{\mathbb{E}}$, and ω (Rule R11). The grammar has no other productions, and Rule R10(b) defines every invocation (totality); no return depends on $\mathcal{M}_{\mathbb{U}}$ bytes except through an accepting confirmation, which writes the endorsed copy into $\mathcal{M}_{\mathbb{E}}$. $\square$

Lemma G.18 (Control-flow confinement to trusted data and releases). *P ’s choice and ordering of operations and tool invocations is a function of $\text{obs}_P(c)$, the reply stream, and the coupled coins alone. No direct channel exists from U-block bytes, U-search outcomes, recall resolution statuses, tool-invocation outcomes, candidate multiplicities, collection cardinalities, block-identifier occupancy, or handle-identifier structure beyond the operation-determined naming, to any reply P receives. Information about hidden state reaches P only through logged release events: the user’s recorded decisions and the endorsed copies they create — each carried, with its displayed bytes, in the display transcript (Definition G.8) — which is precisely the declassification boundary Theorem G.25 conditions on.*

Proof. We verify against Figures 7–8 and Rules R1–R11 that every channel is closed. (a) *U-content*: absent from σ_P (Lemma G.12). (b) *U-search bits and recall resolution status*: consumed interpreter-side (Rules R6, R7, R10(b)); the null-gate abort returns the same reject decision a user rejection would, so P cannot distinguish them from the reply or the sanitized history; the resolved value, where the user later inspects it, is released only through the displayed gate event and its logged decision. (c) *Candidate multiplicity*: bound interpreter-side (FILL), absent from $\text{obs}_P(c)$. (d) *Reader exceptions and budget exhaustion*: absorbed into the null-vs-real handle indistinguishability (Rule R7); the observable fail outcome occurs only on conditions that are a deterministic function of P ’s emitted operations, $\mathcal{M}_{\mathbb{E}}$, and the coupled coins, identical across matched runs. (e) *Timing/step count*: the reply grammar contains no clock or step-count value, so no such value exists in σ_P to branch on; wall-clock channels are outside the model (§K.4, S8). (f) *Collection cardinality and block occupancy*: a collection handle exposes no length or indexing (Rule R1(e)); whether an identifier is occupied by a U-block is not observable (Rule R10(b)), so the attacker-chosen number of captured blocks is invisible; map is one emitted operation returning one acknowledgement with per-element results captured, never returned (Rule R9(e)); tool-call multiplicity inside a map is closed by the batch gate, and pagination above c_{batch} is interpreter-side with a single aggregated decision returned (Rule R9(c)), so neither the cardinality nor the page count is ever returned to P directly; the single aggregated decision is a logged release, on the transcript, like any gate decision; both are displayed to the *user* and recorded in the transcript only. Under the body grammar of Rule R1(f) — which also covers collection-handle tool arguments, normalized to map before transition matching (Rule R1(e)) — the check is exhaustive per step: each map is exactly one handle-producing operation whatever $|H|$ (Rule R1(c)); its single per-element tool call is covered by the batch gate, which displays every element’s bytes and each gating non- $\diamond$ argument’s bytes (Rules R9(b), R2); per-element outputs are captured, never returned (Rule R9(e)); tool-free bodies write U-blocks and reply once; and no admissible body operation can generate a data-dependent reply, recall, fill, map, confirm, and evict being excluded from bodies and tool replies inside the batch being captured (Rule R3). (g) *Handle identifiers*: fixed by the operation sequence (Rule R1(a)), identical across U-equivalent runs with matched operations. (h) *Reply values*: Lemma G.17. (i) $R_{\mathcal{E}}$: a function of query, $\mathcal{M}_{\mathbb{E}}$, ω ; never sees U-bytes; $\mathbb{E}$ -search outcomes are trusted data (Rule R7). (j) *Tool outcomes*: captured (Rule R3); the acknowledgement is constant. (k) *Eviction*: evict replies with the constant ack uniformly (Figure 8, convention (v)); whether the argument resolved, and to which tier, is consumed interpreter-side. Its $\mathbb{E}$ -tier effect is a function of P ’s emitted operation and $\mathcal{M}_{\mathbb{E}}$ alone, matched across $\approx_{\mathbb{U}}$ -related runs; its U-tier effect alters only $\mathcal{M}_{\mathbb{U}}$, which no reply exposes. In particular an evict-then-recall probe learns nothing: RECALL-O is uniform across occupied, vacated, and never-occupied identifiers, before and after any evict (Rule R10(b)). The enumeration is complete relative to the written interface: Σ_P (§F) is the entire operation surface, matching Figures 7–8 arm-for-arm and the mechanized POp type, the reply grammar types every reply, and the totality clause Rule R10(b) leaves no invocation with undefined or distinguishing behaviour. $\square$

Lemma G.19 (Append-only transcript; prefix comparability). *Every transition of Figures 7–8 extends the display transcript by a (possibly empty) suffix and never removes or alters an existing entry; hence along any run the transcript after t steps is a prefix of the transcript after $n \geq t$ steps. Consequently, if two lists are prefixes of one common list, one is a prefix of the other.* (Machine-checked: `dt_execOp_prefix`, `dt_runFrom_prefix`, `prefix_or_prefix`; §G.6.)

Proof. Case inspection of the figures: each transition appends its own display events at the end of the transcript or appends nothing (Rules R2, R8, R6); the transcript is write-once by construction (the audit-log reading of convention (C3) and Definition G.8). The comparability clause is elementary. $\square$

Lemma G.20 (Display-segment determinacy). *Let two configurations be matched (Definition G.6) immediately before a common P -emitted operation, and execute that operation in both. If the display segments the operation appends are prefix-comparable (one a prefix of the other), then they are equal, and so are the operation’s extensions of the reply stream, the sanitized history, the endorsed store (identical fresh blocks at identical endorsed-sort identifiers, Rule R1(g)), the confirmation counter, and the sink log; only $\mathcal{M}_U$ and the private handle resolutions may differ.* (Machine-checked core: `processArgs_det` and `match_execOp` for direct invocations, and `batchGate_det` for the batch-confirmation case, §G.6; the batch case’s store-level integration is established by the paper proof only.)

Proof. Case analysis on the common operation, whose inputs ($\mathcal{M}_E$, the argument list, the confirmation counter, the coins) are matched. For `display( $x$ )`: both segments are the single render event of the same x . For a gate transaction (confirm, or the gating positions of an invocation, processed left to right): the set of gating positions is a function of the argument list and $\mathcal{M}_E$ alone (Rule R4), both matched, so the runs gate at the same positions, and every reached position emits exactly one confirmation event (a null resolution emits the fixed no-value event, Rule R6), so both segments are nonempty at each reached position. Prefix-comparability forces the first events equal; equality of that event pins the same resolution class (real value vs. no-value display), the same displayed bytes, the same selection, and the same decision, hence the same sanitized record, the same reply token contribution, the same continuation (accept: proceed; reject or null: abort the remaining positions in both runs), and, by Rule R2, the same durable fresh C-copies at the same identifiers on either branch. Induction along the remaining reached positions gives segment equality and equal effects. For a batch confirmation (Rule R9): equality of the first appended events pins the size class by event kind (Definition G.8) — both null (the fixed no-value display, standard reject, no decision consumed), both oversize (the fixed oversize display, likewise), or both paginated, the cap being fixed TCB configuration — and in the paginated case the first page event records the collection’s cardinality, and the pagination is the deterministic c_{batch} -split of Rule R9(b), so equality of the first page events pins the page partition; the first-event argument applied page by page gives equal rendered elements, equal decisions, and the same stopping page on rejection, whence equal aggregated decisions, equal durable non- $\diamond$ endorsements (convention (vii’)), and equal captures to $\mathcal{M}_U$ only. Selection-surface renderings within these events touch only the UI and the transcript, never σ_P (Rule R8(d)). All other operations append no display events and their effects on the matched components are functions of matched data (Lemma G.17). $\square$

G.3 ACTION INTEGRITY

Theorem G.21 (Personal Action Integrity). *Under A1–A4, the well-formedness invariant (Definition G.3) is*

- (a) (Establishment) *satisfied by the empty configuration and by every seeded configuration with a valid seed record (Definition G.3); and*
- (b) (Preservation) *preserved by every transition of Figures 7–8, under every attack, with no control-flow or distributional hypothesis, and under adversarial behaviour of either reader.*

Consequently every sink event of every configuration reachable from a well-formed configuration is safe (Definition G.2, via clause (v)): PerMIT has personal Action Integrity (Definition G.10), per execution. In the single-user semantics the CC branch of Lemma G.13 is vacuous (Lemma G.16); the delegated extension is Proposition I.10. (Machine-checked for clauses (i)–(iii),(v), whose conjunction is the mechanized invariant `Inv` (Table 13): established by `inv_init`, preserved by `inv_execOp` with `inv_ingestTyped/inv_ingestPaste`, composed as `inv_runFrom`; `action_integrity`, `action_integrity_from`. Clause (iv) is enforced by the executable state and transition structure and the separate sortedness invariant `WFs`; §G.6.)

Proof. (a) The empty configuration satisfies (i)–(v) vacuously; a seeded configuration’s seed record supplies (i)–(ii) by Definition G.3, (iii)–(iv) hold of it by construction, and (v) is vacuous (no recorded sink events). (b) Preservation of clauses (i)–(iv) is the induction of Lemmas G.12–G.14; for clause (v), take any newly appended sink event and argument a ; the event is sensitive (Definition F.5), so by Lemma G.13 one of the three sensitive-tool cases holds. (i)

a byte-equals a T-block: by well-formedness (Definition G.3(i)) and A1, those exact bytes were typed by the user on the command channel. The fast path is deliberately extensional: attacker-supplied content *may* be byte-equal to a stored T-block, in which case the flowing bytes are still a value the user personally typed, and the flow is user-authorized content regardless of which equal copy motivated the call; provenance of other equal copies is irrelevant to the authorization of these bytes. (ii) a byte-equals some C/CC-block b with $\text{tool} \in \text{scope}(b)$: by Lemma G.14 an accepting $\text{confirm}(a, \text{tool})$ occurred; by A3 the endorsed bytes were exactly a ; by A4 the user intended a at tool . (iii) a fresh atomic $\text{confirm}(a, \text{tool})$ gates the call, in particular every handle argument, and by Rule R2 the bytes that flow are the displayed bytes, bound to this call instance, even if they coincide with stored content; for a batch-gated map, every element's bytes were displayed at the batch confirmation (Rule R9) and accepted under the per-element intent clause of A4, so each element call flows exactly the bytes displayed for it; a null handle aborts (Rule R6).

For reader adversariality: an adversarial Q can at worst surface a wrong value or decorate it with a preview; it holds no tool capability, its output is U , the decision pane shows the true resolved bytes in an interpreter-controlled region the attacker cannot influence (Rule R8(b)), and A3–A4 reject the wrong value. R_E selects only among $\mathbb{E}$ -blocks, each independently in case (i)/(ii). This is qualified robustness Zdancewic & Myers (2005); Myers et al. (2006). $\square$

Remark G.22 (Extensional trust of typed values). Case (i)'s fast path is extensional and tool-blind at the baseline: a user-typed value is authorized as a *literal* at every sensitive sink, including sinks the user never contemplated when typing it, and attacker content byte-equal to a typed value flows as the typed value (the proof's provenance reading). The handle discipline is unaffected: a handle-expressed argument always gates, even when its hidden resolution byte-equals a typed value (Rule R4(c)). Deployments for which per-sink authorization of typed values matters select Option O1, which re-gates T-values once per sensitive sink (Proposition G.28; budget: Corollary J.6).

Corollary G.23 (No silent resolution; the post-endorsement boundary). *Under the same assumptions:*

- (a) (Never-endorsed bytes.) *An untrusted-derived value that has never been endorsed at any target is resolved at a tool boundary, sensitive or data, only inside an accepting confirmation displaying exactly those bytes at exactly that tool. In particular, no never-endorsed untrusted-derived value can be exfiltrated through a data-tool parameter (a URL, a search query, a file path) without the user endorsing that exact value at that exact tool, in that call.*
- (b) (Post-endorsement boundary.) *Once bytes have been endorsed at any target (including "reasoning"), the fresh C-copy makes them readable $\mathbb{E}$ -level bytes (Rule R5), and their subsequent literal-expressed flow to data tools is ungated (Lemma G.13, case (D)); a handle-expressed use of the same bytes gates at every tool regardless (Rule R4(c), S18); tool-specific endorsement is enforced only at sensitive tools (the scope test of Rule R4(a)). This is the disclosed baseline post-endorsement channel (S10); Option O2 closes it: under O2 the data-tool fast path applies the same exact-target test as the sensitive path, so a reasoning-target copy (empty scope) and any out-of-scope endorsed copy gate at every data tool, each such flow then requiring its own per-pair confirmation. (Machine-checked: the gateCheckO2 closure witnesses of §G.6.)*

Remark G.24 (Reading of "untrusted-derived"). Clause (a) is a provenance statement about byte *values*: the never-endorsed untrusted bytes themselves cross a tool boundary only inside an accepting confirmation displaying them. It is not a claim of information-theoretic independence: P may branch on a logged gate decision (a transcript release, Lemma G.18) and encode that released bit into P -composed data-tool arguments; all such influence is transcript-carried and governed by Theorem G.25 and Corollary G.26, not by this corollary.

Proof. (a) By Lemma G.12, P holds never-endorsed untrusted-derived data only as opaque handles, so such bytes reach a tool argument only by handle resolution; by Rule R4(c) every handle argument at every tool gates; by A3–A4 and Rules R2, R9 the resolved bytes flow only on an accepting decision over exactly those bytes at exactly that call's tool. (b) Immediate from Rule R5 and Rule R4(b) for literal-expressed use; handle arguments gate in every case by Rule R4(c); a reasoning-target copy has empty scope, so the *sensitive* fast path remains closed to it, and Theorem G.21 is unaffected. (Machine-witnessed: Scenario 8, §G.6.) $\square$

G.4 REASONING INTEGRITY MODULO THE DISPLAY TRANSCRIPT

Theorem G.25 (Reasoning Integrity modulo the display transcript). *Under A1, A2, A3, A6, and the scheduling convention (C9), PerMIT has Reasoning Integrity modulo the display transcript (Definition G.11): for every matched pair $c_1 \approx c_2$ (Definition G.6), every pair of attacks, and every coupled coin sequence ω (convention (C1)), any two runs of a common horizon (convention (C9)) with $\text{DT}(\rho_1) = \text{DT}(\rho_2)$ agree on the ungated sink projection, the emitted sequence of Protected-LLM operations and tool invocations, the reply stream, and the final Protected-LLM views. (Machine-checked, with these strengthened conclusions and with paste presence and content uncoupled across*

the compared attacks, as `reasoning_integrity_from`; the batch case’s determinacy core is checked separately (`batchGate_det`) while the store-level integration of `map` is covered by the paper proof only — its element-call sink events are gated, hence stripped (Definition G.7), so the ungated sink projection is unaffected; §G.6.)

Proof. We use Lemmas G.17–G.18, which hold under the harness rules, A1–A2, and the strategy convention of §F.1 alone. Fix a matched pair $c_1 \approx c_2$ (Definition G.6; sorted suffices, Definition G.5), two attacks, and a coin sequence ω ; by A6 and convention (C9) the typed streams and the slot skeleton are coupled, while paste events (presence and content), tool outputs, and reader outputs may differ arbitrarily across the two attacks. Suppose, for contradiction, that $DT(\rho_1) = DT(\rho_2)$ but the runs differ in some Protected-LLM operation, tool invocation, or ungated sink event; let t^* be the first step at which they differ. Define the invariant $I(t)$: after t steps, (1) the configurations are $\approx_U$ -related; (2) the two runs have emitted identical sequences of Protected-LLM operations and tool invocations, received identical reply streams, and consumed identical coin prefixes — hence, by the strategy reading of §F.1, hold identical states σ_P ; (3) the display-transcript prefixes and sink logs are equal. $I(0)$ is Definition G.6 verbatim: matched pairs agree on the view, the interaction histories, and the ghost transcript and sink logs (for the canonical endorsed-seed pairs, all ghost logs are empty). The argument is the unwinding method Goguen & Meseguer (1984) with a gradual-release carve-out Askarov & Sabelfeld (2007); Sabelfeld & Sands (2009); alignment of operations and coins is *derived* at each step from $I(t)$ via Lemmas G.17–G.18, never assumed. Steps 3–4 use the append-only transcript, Lemma G.19.

Step 1: $I(t)$ holds for all $t < t^$, and step t^* is matched, a contradiction.* The induction is Steps 2–4; Step 5 concludes the argument.

Step 2 (Determinacy). Given $I(t)$ with $t < t^*$, step $t+1$ opens with the channel slots of convention (C9), disposed of first. The typed events are equal (coupled by A6), extending $x(c)$ and $\mathcal{M}_{\mathbb{E}}$ identically in both runs. The paste events may differ in presence and in content; but each lands in $\mathcal{M}_U$ with label U and appends nothing to P ’s reply stream (Figure 8, convention (vi)), so after the slots the configurations remain matched, with $\mathcal{M}_U$ differing freely exactly as Definition G.6 permits. P ’s next operation is then, by the strategy convention of §F.1, a function of its emitted operations, its reply stream, its view $\text{obs}_P(c)$, and the coins; by $I(t)$ (1)–(2) all four are matched across the runs. Hence both runs emit the same operation at step $t+1$, determined *before* any display event of that step.

Step 3 (Display-free step). Channel-slot effects were disposed of in Step 2; the cases below are the operation’s own transition. If the common operation triggers no display event, its effects preserve $\approx_U$: tool output lands in $\mathcal{M}_U$ with constant acknowledgement (Rule R3); handles have operation-determined structure (Rules R6, R1); W-P writes identical PC-blocks at identical endorsed-sort identifiers (Rule R1(g), the allocation state being matched by $I(t)$); $\mathbb{E}$ -reads return equal bytes (RECALL-E, and $R_{\mathcal{E}}$ under matched query, $\mathcal{M}_{\mathbb{E}}$, and ω , Rule R11), including its failure branch: RETRIEVE/RETRIEVE-F select or fail as a function of $(q, \mathcal{M}_{\mathbb{E}}, \omega)$ alone, matched across the runs; evict preserves $\approx_U$: its reply is constant, an evicted $\mathbb{E}$ -block is the same block in both runs (the identifier sorts separate the tiers, Rule R1(g), and $\mathcal{M}_{\mathbb{E}}$ is matched), and an evicted or non-resolving U -identifier changes only $\mathcal{M}_U$ (Figure 8, convention (v)); the fail outcome is deterministic in matched data (Rule R7). Transcript prefixes extend equally (by nothing), the coin prefixes advance identically, and the reply streams extend equally (Lemma G.17: every reply of a display-free step is a constant token, an operation-determined handle, or matched $\mathbb{E}$ -bytes). $I(t+1)$ holds.

Step 4 (Display step). Suppose the common operation (Step 2) triggers display events. Let D_r denote run r ’s transcript after step t and E_r the segment the operation appends. By $I(t)$ (3), $D_1 = D_2 =: D$. By Lemma G.19, DE_r is a prefix of run r ’s final transcript, and the final transcripts are equal by hypothesis (at the common horizon, convention (C9)); hence DE_1 and DE_2 are prefixes of one common list, so one is a prefix of the other, and cancelling D makes E_1 and E_2 prefix-comparable. Lemma G.20 then gives $E_1 = E_2$ together with equality of the operation’s reply, history, endorsed-store (including the durable endorsements of an aborting transaction, Rule R2), counter, and sink-log extensions; tool outputs, reader outputs, and private handle resolutions may still differ, but they alter only $\mathcal{M}_U$ and the resolution table, exactly as Definition G.6 permits. Hence $I(t+1)$ holds. (*The determinacy core of the batch case is machine-checked: prefix-comparable page transcripts, over possibly different private handle resolutions, force equal pagination, page decisions, aggregated decision, and flowed elements — `batchGate_det`, §G.6; the store-level integration of `map` into the run-level interpreter remains established by the paper proof only, and the raw sink events of element calls are gated and hence stripped, Definition G.7, so the ungated sink projection is unaffected.*)

Step 5 (Output consistency at t^).* By Step 2, step t^* ’s operation is matched, so the divergence must lie in an ungated sink annotation. Sink events are sensitive (Definition F.5), so by the sensitive-tool case analysis of Lemma G.13 their arguments are (a) readable T or in-scope C/CC values, equal across $\approx_U$ -related configurations (they lie in $\mathcal{M}_{\mathbb{E}}$; selection by P or $R_{\mathcal{E}}$ is a function of matched data and ω); or (b) freshly confirmed, hence stripped by Definition G.7, and by Step 4 the same value was endorsed (or the call aborted) in both runs in any case. The gate-or-no-gate decision is itself matched: by Rule R4 it is a function of storage class and membership/scope in $\mathcal{M}_{\mathbb{E}}$, all components of $\text{obs}_P(c)$; hidden handle contents cannot flip it. Calls inside maps are batch-gated and appear in the transcript (Rule R9); all

other sinks have occurrence fixed by P 's matched, cardinality-independent operation sequence (Lemma G.18). So no component of step t^* differs, contradicting the choice of t^* .

At the final horizon, I additionally yields equality of the full reply streams (clause (2)) and of the final views (clause (1)), the two strengthened conclusions of the statement.

Hence runs with equal transcripts never diverge. In particular, varying untrusted bytes in a way that leaves every display event unchanged leaves the behaviour unchanged; contrapositively, a hidden untrusted byte influences behaviour only by changing some rendered display event (e.g. a Reader-surfaced candidate or preview) or an endorsed release — influence that is transcript-carried and logged (Corollary G.26) — while the byte value itself never enters σ_P undisplayed (Lemma G.12). $\square$

Corollary G.26 (Attack-surface confinement). *If two runs of a common horizon from a matched pair (Definition G.6) under any two attacks (coupled per conventions (C1) and (C9)) differ in any ungated sink event, Protected-LLM operation, or tool invocation, then their display transcripts differ. Every attacker-induced difference in these behaviours — ungated sink events, Protected-LLM operations, and tool invocations — is therefore visible in a logged, user-facing display event: on the selection surface, on the decision pane, or in a decision. (Differences confined to untrusted memory, reader state, or tool outputs lie outside the compared behaviours; timing and termination are outside the model, §K.4(IV).)*

Proof. Contrapositive of Theorem G.25. $\square$

Commentary (non-normative; reading of Corollary G.26). Corollary G.26 is a visibility guarantee, not a manipulation-proofness guarantee: any attacker influence on the protected behaviours (Definition G.11) must pass through the logged display transcript or an explicitly accepted endorsement, where each relevant release or decision is recorded and attributable to a transcript entry — not necessarily to an attacker identity — after the fact; prevention is Theorem G.21's gate, which stops any value that is neither user-typed nor endorsed — personally, or, under delegation, by a qualifying community quorum — for the exact sensitive tool from ever crossing that tool's boundary (argument-value integrity; tool choice itself: §K.4(X)). The operational force of the hash-chained audit log, the residual human channel of A6, and what previews can and cannot do are discussed at N7, N8, S9, S10; the scope and residuals of both single-user guarantees are collected in §K.4.

G.5 BASELINE SEPARATION AND HARDENING MONOTONICITY

Let B be the following baseline system: the transition system of Figures 7–8 with three modifications. (i) RECALL-O on an identifier resolving to a U-block returns bytes(b), so untrusted bytes enter σ_P ; (ii) Rule R4(c) is dropped, handle arguments resolving silently; (iii) Rules R4(a)–(b) gate a tool argument iff it carries U-taint under standard dynamic data-taint propagation through σ_P (data taint only, not control taint; the standard practice the witness below exploits). DT and Sink (Definitions G.8, G.7) are read over B 's runs verbatim, B 's confirmation events being recorded exactly as PerMIT's.

Proposition G.27 (The taint-gating baseline fails Reasoning Integrity). *Let B be the baseline system above. There exist $c_1 \approx_U c_2$ and attacks with $\text{DT}(\rho_1) = \text{DT}(\rho_2) = \emptyset$ yet $\text{Sink}(\rho_1) \neq \text{Sink}(\rho_2)$.*

Proof. Task: “check the vendor's latest message; if it concerns the overdue invoice, fetch the vendor's payment history,” with $\text{fetch_history} \in \text{Tools}_{\text{sens}}$ and a user-typed argument. Attack $\mathcal{A}$ plants “invoice overdue”; $\mathcal{A}'$ plants “already settled.” In B the agent reads the U-bytes and calls the sink in one run but not the other; no confirmation occurs; the transcripts are empty and equal while the sink projections differ. PerMIT cannot express the divergence: the U-bytes never enter σ_P and P 's operation is determined by matched data (Lemmas G.12, G.18); the legitimate version is recovered through crystallize-and-confirm, whereupon the deciding content appears in the transcript, as Corollary G.26 requires: any difference in the compared behaviours would require a transcript difference. $\square$

Proposition G.28 (Hardening is monotone). *Let a deployment select any subset of Options O1–O2, under the following per-option hypotheses: for O1, each T-value is used at $\mathcal{O}(1)$ distinct sensitive sinks over the deployment's lifetime (bounded per-value sink-spread); for O2, freshly composed data-tool arguments belong to the per-task stream outside the reusable-memory ledger (scope, Appendix J), and each user's endorsed-value reuse touches $\mathcal{O}(1)$ distinct data tools per value over the deployment's lifetime (bounded per-value data-tool spread, the data-tool analogue of O1's hypothesis). Option O3 is requirements-level and outside this proposition's quantifier (O3). Then:*

- (i) (Flow monotonicity.) *Each option removes fast paths of Rule R4 and introduces none, so the relation of flows the interface admits ungated strictly shrinks.*

- (ii) (Safety preservation.) *Every single-user and federated safety theorem of this document is preserved: the options add no promotion path and change no counting rule; option-induced cache acceptances are federation-inert and option-induced rejections are qualifying rejections like any other (Definition H.6(iii), INV-2) (theorem preservation, not pathwise monotonicity; see proof).*
- (iii) (Budget.) *Every budget theorem holds with its constants multiplied by $\mathcal{O}(1)$: under O1, an $\mathcal{O}(1)$ factor on d_{priv} ; O2's confirmations of freshly composed arguments lie in the per-task stream by hypothesis, outside every ledger row; its confirmations of endorsed-value reuse number at most one per (user, value, data-tool) pair (the fresh copy scope-caches, INV-3), are federation-inert (cache class, Definition H.6), and are folded into the pair accounting exactly as A12 folds tool pairs — an $\mathcal{O}(1)$ factor under the data-tool-spread hypothesis. All asymptotics are unchanged (the routing of option-induced confirmations into the per-task stream and the floor accounting is collected as Corollary J.6).*

Proof. (i) Each option replaces a fast path of Rule R4 by a confirmation; strictness is at the level of the interface's admitted-flow relation (a particular run may exercise none of the removed paths) and is witnessed by any configuration containing a T-block passed to a sensitive tool (O1), or a data-tool argument that is neither byte-equal to a stored T-block nor to an in-scope C/CC-block (O2: a freshly composed value, or endorsed bytes out of scope at the data tool — e.g. a reasoning-target copy). (The flow half for O1/O2 is machine-checked; §G.6.)

(ii) Single-user: gating strictly more flows preserves Lemma G.13's case analysis (sensitive cases only move from (i)/(ii) to (iii); under O2 data-tool flows move from case (D) to the clause-(a) case analysis applied at the data tool: fast for T- or in-scope C/CC-matched bytes, else gated per case (iii)) and adds confirmation events to the transcript, which Theorem G.25 conditions on. Federated: the options add no promotion path and change no counting rule. Cache confirmations create submission-ineligible blocks (Rule R12, Lemma I.2), so the shared candidate set cannot expand; and their accepting branch is federation-inert (Definition H.6, log-scoping clause), so no option-induced acceptance is ever credited toward any threshold: selecting an option adds logged *rejection* opportunities on presence-coinciding bytes and nothing else to the shared-tier acceptance side. Preservation of the federated safety theorems is therefore *theorem preservation*, not pathwise monotonicity: the modified inspection schedule is one more adaptive presentation schedule, and Theorems I.11 and I.13 quantify over all such schedules, their hypotheses ($q = 1$; the uniform $q_{\text{pre}}, q_{\text{pan}}$ bounds over targetable contexts, O1/O2 cache gates are excluded from the trial classes, Theorem I.13(c), their rejections one more effective qualifying rejection, which can only deny the ordinary path) covering it verbatim; the added logged decisions on a candidate are cache-gate rejections (accepting cache branches are never logged, Definition H.6(iii)); Rule R13(i) processes each as a dispute initiation, corroboration, or advisory flag according to the current state, and they add no promotion credit and no verdict capacity, so every counting fact of Appendix I is unchanged and the stated bounds apply to the optioned deployment as to the baseline.

(iii) O1 adds at most one first-use confirmation per (T-value, sensitive sink) pair, an $\mathcal{O}(1)$ factor on d_{priv} by the sink-spread hypothesis; O2's confirmations of freshly composed arguments lie in the per-task stream by hypothesis, outside every ledger row; its confirmations of endorsed-value reuse number at most one per (user, value, data-tool) pair (the fresh copy scope-caches, INV-3), are federation-inert (cache class, Definition H.6), and are folded into the pair accounting exactly as A12 folds tool pairs — an $\mathcal{O}(1)$ factor under the data-tool-spread hypothesis. $\square$

G.6 MACHINE-CHECKED VERIFICATION OF THE SINGLE-USER GUARANTEES

The core single-user fragment of this appendix is mechanized and kernel-checked in Lean 4 (toolchain `leanprover/lean4:v4.31.0`) against Mathlib pinned at tag `v4.31.0` (revision `fabf563a`, per the included `lake-manifest.json`). The build is the verification: every theorem is kernel-checked, and every executable scenario is a build-time assertion that fails the build if false.

Artifact. The complete Lean development is included as an anonymized archive (`permit_formal.zip`) in the supplementary material and contains no author-identifying information. The archive pins the toolchain (`lean-toolchain: leanprover/lean4:v4.31.0`) and the full dependency manifest (`lake-manifest.json`; Mathlib at tag `v4.31.0`, revision `fabf563a`), so the verification is reproducible from the archive alone by `lake exe cache get && lake build`: the build fails on any unproved goal or false `#guard` assertion, and the final `#print axioms` lines report the axiom footprint of every named theorem. The statement-level correspondence is Table 13, mirrored in the archive's `CORRESPONDENCE.md`; the archive additionally includes the verbatim `#print axioms` output of a clean build (reproduced in its `README`), a single final build log, and a SHA-256 checksum manifest of the sources. A deanonymized repository and an archival release will be made public upon acceptance. Scope: the batch-confirmation surface of the batch-gated map of Rule R9 is mechanized as a standalone layer (`BatchGate.lean`: deterministic c_{batch} -pagination with first-page cardinality,

per-page decisions with abort, the single aggregated decision, all-or-nothing element flow bound to exactly the displayed bytes, the null-collection display, and the batch determinacy fact consumed by Theorem G.25 Step 4); its store-level integration into the run-level interpreter (ingress capture of element outputs; durable endorsement of gating non- $\diamond$ body arguments) and Option O3 remain outside the mechanized surface. For avoidance of doubt, the machine-checked claim is: *the single-user, map-free core, plus the batch-confirmation surface as a standalone layer*. Paper-only (not mechanized): the run-level map integration; the federated product system (Definition H.2) and all of Appendices H–I; every probabilistic and budget result (Appendices J, L); Option O3 (requirements-level, O3) and the batch cardinality cap; arbitrary mid-run matched pairs (delta (7)); and the index-oracle reduction (Proposition G.29). All statements below cover the remaining operation algebra, with the collection-handle delta disclosed as item (6) below. The budget layer — A11 and the ledger of Lemma J.7 — remains outside the mechanized surface, as before.

Model. An executable interpreter: configurations, the operation algebra, the reply grammar of Rule R10 realized as an inductive type (so the constant-alphabet property holds by construction: no other reply can be formed), and one total transition function per SOS rule, the totality witnessing Rule R10(b); the display transcript and sink projection are carried as ghost state. P and R_E are arbitrary strategy functions over exactly the observables the definitions grant them (§F.1, Rule R11); the attacker is an arbitrary function supplying every untrusted input, *including the Quarantined Reader’s outputs*, so the mechanized adversary subsumes the paper’s, and no behavioural assumption on any model enters any statement.

Checked statements. The kernel has checked:

- **Personal Action Integrity** (Theorem G.21), as an inductive invariant of the transition system, in two forms: from the empty initial configuration (`action_integrity`) and from an *arbitrary well-formed* configuration (`action_integrity_from`), the machine-checked invariant `Inv` being Definition G.3(i)–(iii),(v) verbatim (T-blocks typed; C-scopes transcript-backed; no CC, Lemma G.16; recorded sinks safe). A precedence corollary places the endorsing confirmation in the transcript prefix closed no later than the sink event (the transcript being append-only, Lemma G.19), and the semantics-level core of Corollary G.23(a) is checked: a handle argument never fast-passes, definitionally, and every ungated flow is verbatim P -supplied literal bytes.
- **Reasoning Integrity modulo the display transcript** (Theorem G.25) and Corollary G.26, by a lockstep simulation whose relational invariant `MatchW` is Definition G.6 verbatim: Protected-LLM view, reply stream, emitted-operation sequence, handle *count* (deliberately not handle *contents*), transcript, and sink log. Again in two forms: from empty stores, and from an arbitrary shared endorsed seed (`reasoning_integrity_from` over `initE`), realizing the matched-pair quantifier of Definition G.6 at its canonical sorted class (Definition G.5; Lean side condition `EStore`), with no endorsement-soundness hypothesis — exactly the `Inv-vs-EStore` split of Definitions G.3/G.5. The mechanized conclusions are *stronger* than the paper statement requires: transcript equality forces equality of the *raw* sink events, the reply streams, and the final Protected-LLM views — confirming that the stripping of Definition G.7 hides nothing under transcript equality (the run-level surface is map-free; the batch case’s determinacy core is checked separately, `batchGate_det`, with the store-level integration’s paper-only status and its sink-projection scope stated at the proof’s Step 4).
- **Isolation as a named kernel fact** (`bytes_reply_isolation`): every byte string P ever receives originated, at the step of its emission, as the content of a non-U block of the store — the reply-grammar half of Lemma G.12, checked rather than inspected.
- the **batch-confirmation surface of Rule R9** (`BatchGate.lean`), as a standalone layer over the same user, decision, and reply types: deterministic `c_batch`-pagination whose pages flatten back to the collection, the first page recording the collection cardinality (`chunks_flatten`, `batchGate_first_page`); per-page decisions with reject-aborts-remainder; all-or-nothing element flow bound to exactly the displayed bytes (`batchGate_accept_flow`, `batchGate_elem_displayed`, `batchGate_reject_no_flow`); the null-collection fixed display consuming no decision (`batchGate_null`); the single aggregated reply, by type a function of the aggregated decision alone — neither cardinality nor page count can reach P ’s reply stream (`batchReply_decision_only`); and the batch *determinacy* fact consumed by Theorem G.25 Step 4 (`batchGate_det`, the batch analogue of `processArgs_det`).
- the **baseline separation** (Proposition G.27), by computation and with the taint machinery *inside the model*: the baseline carries a dynamic data-taint bit through its reply stream; the witness realizes the control-taint blind spot (a P -supplied constant emitted under poison-dependent control flow is data-untainted, hence ungated); a companion check confirms the taint gate does bite when the tainted bytes themselves flow; and the PerMIT interpreter under the same strategy and both attacks is checked attack-invariant in operations, transcript, and sink projection;
- the **flow-monotonicity of Options O1/O2** (Proposition G.28(i)): every flow admitted ungated under O1/O2 is admitted by the baseline gate, with strictness witnessed computationally; for O2, the closure of the post-endorsement

channel of Corollary G.23(b) is likewise witnessed computationally — a reasoning-scoped copy (Scenario 8’s endorsed store, verbatim) and a cross-tool-scoped copy gate at a data tool under O2, while T-blocks and in-scope cache copies flow.

Paper item	Lean identifier	File	Axioms
Thm. G.21	<code>action_integrity(_from)</code>	<code>ActionIntegrity.lean</code>	†
precedence corollary	<code>action_integrity_precedence</code>	<code>ActionIntegrity.lean</code>	†
Cor. G.23(a) core	<code>noexfil_semantics;</code> <code>handle_always_gates</code>	<code>ActionIntegrity.lean</code>	†
Lem. G.12 (reply half)	<code>bytes_reply_isolation</code>	<code>Invariants.lean</code>	<code>propext</code>
Lem. G.13 (sens./ungated)	<code>processArgs_spec</code>	<code>Invariants.lean</code>	†
Def. G.3(i)–(iii),(v)	<code>Inv</code>	<code>Invariants.lean</code>	—
Def. G.2	<code>SinkSafe</code>	<code>Invariants.lean</code>	—
Defs. G.5, G.6	<code>WFs/ESTore; MatchW</code>	<code>ReasoningIntegrity.lean</code>	—
Lem. G.19	<code>dt_runFrom_prefix;</code> <code>prefix_or_prefix</code>	<code>Invariants/</code> <code>ReasoningIntegrity</code>	†
Thm. G.25 Step 4 core	<code>processArgs_det; match_execOp</code>	<code>ReasoningIntegrity.lean</code>	†
Thm. G.25	<code>reasoning_integrity(_from)</code>	<code>ReasoningIntegrity.lean</code>	†
Cor. G.26	<code>confine(_from)</code>	<code>ReasoningIntegrity.lean</code>	†
Rule R9(b),(d)+R2 (batch flow)	<code>batchGate_accept_flow;</code> <code>batchGate_elem_displayed</code>	<code>BatchGate.lean</code>	†
Rule R9(c) (decision-only reply)	<code>batchReply_decision_only</code>	<code>BatchGate.lean</code>	†
Lem. G.20, batch case (core)	<code>batchGate_det</code>	<code>BatchGate.lean</code>	†
Prop. G.27	<code>Baseline #guard block</code>	<code>Baseline.lean</code>	<code>evaluator</code>
Prop. G.28(i); Cor. G.23(b) closure under O2	<code>gateCheckO1/O2_mono;</code> <code>Hardening.lean #guards</code>	<code>Hardening.lean</code>	†/evaluator
Scenarios 1–8	<code>Sanity.lean #guards</code>	<code>Sanity.lean</code>	<code>evaluator</code>

Table 13: Statement correspondence. †: axiom footprint $\subseteq \{\text{propext}, \text{Quot.sound}\}$ per `#print axioms`; “—”: definition (no axioms); “evaluator”: build-time `#guard`, trusting Lean’s evaluator.

Executable scenarios additionally check, by computation: the occupancy-hiding uniformity of Rules R6/R7/R10(b) (identical reply streams on U-occupied versus nonexistent identifiers); the atomic endorse-then-fast-path lifecycle of Rule R4(a); the data-tool ungated flow of freshly *P*-composed PC bytes passed as literals, with the sensitive fast path closed (Lemma G.13 case (D), Scenario 7); the post-endorsement boundary of Corollary G.23(b) — a reasoning-target endorsement makes the bytes readable and, literal-expressed, data-tool-ungated while the sensitive fast path stays closed (Scenario 8) — the baseline residual whose closure under Option O2 is witnessed by the `Hardening.lean` guards; the batch surface’s deterministic pagination, first-page cardinality, all-or-nothing flow, and cardinality-independent aggregated reply (`BatchGate.lean` guards); and a finite instance of Corollary G.26 exhibiting the quadrant structure stated by Theorems G.21 and G.25 — raw sink events differ, stripped projections equal, transcripts differ, *P*’s reply streams identical: the poison flowed only through the accepted, logged gate, invisibly to *P*.

Axiom audit. Lean’s `#print axioms` reports that every named theorem above depends only on `propext` and `Quot.sound`, the foundational axioms of Lean’s kernel; no `sorry`, no classical choice, and no domain-specific axiom occurs. Executable scenario checks additionally trust Lean’s evaluator.

Disclosed modelling deltas. Eight, none silent.

- (1) *Q* is folded into the attacker, which chooses *Q*’s outputs directly: a strict strengthening.
- (2) One typed event is one T-block (the individuating parser simplified).
- (3) An out-of-range user selection clamps to the first candidate; the flowed bytes are still exactly the displayed bytes.

- (4) The user’s decisions and selections, and the attacker’s output fields, are functions of event *ordinals*, not of the displayed bytes or call arguments. Coverage of display-reactive behaviour is argued by diagonalization: a run is a deterministic function of its inputs, so a reactive actor is covered by the index-oracle equal to its realized sequence, and for the two-run theorem transcript equality pins exactly the data a display-reactive user could react to. This w.l.o.g. is stated precisely as Proposition G.29: a disclosed *meta*-argument about the mechanization, not a machine-checked fact.
- (5) Compared runs share only the slot skeleton of convention (C9): paste *presence and content* are independent per attacker. This is strictly stronger than a coupled-timing hypothesis and coincides with the paper’s environment model.
- (6) With the run-level map deferred, a handle over a multi-element capture is confirmed at a *direct* tool call by user selection of one displayed element — a distinct, strictly more restrictive semantics standing in for *both* normative surface forms (map and the collection-handle tool argument, which desugars to map before transition matching, Rule R1(e)) — under which Action Integrity holds verbatim and nothing untrusted flows undisplayed. The batch-confirmation surface of Rule R9 itself is mechanized separately (`BatchGate.lean`): all-or-nothing displayed-bytes flow, the decision-only aggregated reply, and the batch determinacy fact (`batchGate_det`) consumed by Step 4 of Theorem G.25; the store-level integration (ingress capture of element outputs; durable non- $\diamond$ endorsements) remains covered by the paper proof only.
- (7) Seeded initial configurations (`initE`) instantiate the matched-pair quantifier of Definition G.6 at the canonical endorsed-seed class of Definition G.5 (`EStore` is that class’s id-sort condition, attacker content entering through the attacker’s own initial untrusted store); arbitrary matched *mid-run* pairs are covered by the paper proof only. `action_integrity_from` instead requires the full invariant `Inv`, so a bare seed is not admitted there — matching the paper’s quantifier split verbatim.
- (8) The reasoning-coverage clause of Rule R13(i) is a federated accounting convention over the local readability fact: the mechanized interpreter writes reasoning-target copies with empty tool scope and gates every reasoning-target confirmation at the interface level (Scenario 8), and it contains no federated counting, so the clause lies outside the mechanized surface by construction — coverage is a counting classification, not a transition change.

With these eight deltas disclosed, the mechanized interpreter and the paper interpreter coincide on the map-free operation algebra: no paper-only interpreter behaviour exists on the mechanized surface; the batch-confirmation surface is mechanized as a standalone layer (`BatchGate.lean`) rather than as an interpreter arm, so this coincidence claim is unaffected by it. The covered-gate clause of Rule R13(i) and the confirmation classes of §J.1 are federated logging and accounting conventions, not interpreter transitions: they alter no reply, no store effect, and no display event, so they lie outside the mechanized surface by construction rather than by omission.

The index-oracle reduction, stated. Delta (4) is used often enough to deserve a precise statement. It is a paper-only meta-proposition *about* the mechanization, not a machine-checked fact.

Proposition G.29 (Index-oracle coverage of display-reactive behaviour). *Fix the mechanized semantics (§G.6) and inputs $(\text{cfg}, A, P, R_E, \omega)$, and a horizon n . Call a user display-reactive if its decisions and selections are functions of the transcript prefix rendered so far (rather than of event ordinals). Then: (i) for every display-reactive user $\hat{U}$, the run to horizon n is constructed step by step, each decision depending only on the already-rendered prefix; record the finite decision-and-selection sequence $\hat{U}$ realizes along it, and extend it arbitrarily beyond its realized length (decision ordinals need not coincide with step indices: one gate transaction can consume several decisions). The index oracle U^* equal to that sequence replays the original run: by induction on steps, with invariant: after t steps the two runs agree on every interpreter/run-state component, and U^* has consumed exactly the first $k(t)$ entries of the recorded sequence, where $k(t)$ is the number of decisions and selections $\hat{U}$ ’s run consumed in t steps. A step consuming no decision preserves the invariant trivially (matched states, same operation, same transition); a step consuming $j \geq 1$ user responses (decisions or selections) consumes them at gate events determined by the matched state, in order, so U^* ’s entries $k(t)+1, \dots, k(t)+j$ are consumed at exactly the events where $\hat{U}$ gave them, and the states re-align at the step’s close. Hence every mechanized theorem, quantifying over all index-oracle users, covers $\hat{U}$ ’s run. (ii) For the two-run theorem (`reasoning_integrity_from`): given two attacks and a display-reactive $\hat{U}$, transcript equality pins every datum $\hat{U}$ could react to (each consumed decision and selection is recorded with its displayed bytes, Remark G.9), so the two realized finite sequences coincide; instantiating the mechanized statement at that common index oracle yields the conclusion for $\hat{U}$. The same construction covers history-adaptive attackers via their realized output sequences.*

Specification points sharpened during mechanization. Four points of this appendix were corrected or made precise in the course of mechanization. First, the two-run argument requires endorsed blocks to receive *operation-determined* identifiers: an in-place relabel at the untrusted source’s identifier would expose an attacker-dependent identifier in $\mathcal{M}_{\mathbb{E}}$ and break the matched-pair invariant. The fresh-copy endorsement of convention (vii) and the two-sorted identifier discipline of Rule R1(g) are the normative form of this correction, adopted into the figures when the mechanization surfaced it; the prose proof had previously used the discipline tacitly, and the mechanized semantics and the paper semantics now coincide on this point. Second, the simulation invariant matches the handle *count* but deliberately not the handle *contents*, which may diverge between transcript-equal runs until a display event — making precise the sense in which Rule R7 confines resolution status. Third, channel-slot transitions must acknowledge the channel rather than P (Figure 8, convention (vi)): without this, an uncoupled paste event would emit a P -visible acknowledgement and the two-run comparison would fail — the clause was added to the figures when the mechanization surfaced it. Fourth, the fast-path case analysis of Lemma G.13 is disjunctive, not exclusive: the mechanized `processArgs_spec` proves exhaustiveness with no mutual-exclusivity clause between the typed and endorsed matches — which indeed co-occur on byte-equal copies, e.g. under Option O1 — and an earlier prose statement asserting mutual exclusivity has been corrected to the mechanized form.

H THE FEDERATED PROTOCOL (PERMIT-ATTEST)

Only *Confirmed* content is shared: generalizable atomic values and crystallized knowledge an owner has verified and explicitly published; T- and U-blocks are never candidates (Lemma I.2). The tier is built on machinery network security has already formalised: threshold endorsement Maurer (1996), transparency and revocation Laurie et al. (2013; 2021), and the identity-cost requirement of the Sybil literature Douceur (2002); Dwork & Naor (2007), the precise reason the scaling results are conditional on an attested or cost-bounded setting. The dispute mechanism is a bounded, demand-paced jury whose competence condition is the classical one Grofman et al. (1983). (Lineage: N10.)

The federated layer is specified as a protocol over the log service of convention (C3): its state transitions (promotion, suspension, demotion, freeze) are given by Rules R12–R13 as deterministic log-event semantics, each transition a function of the prior log and the triggering event. This layer is *not* integrated into the single-user small-step relation of §F.2; no theorem in this document requires that integration. The single-user theorems consume the shared tier only through the live-entry test of Rule R4(a); in the federated two-run comparison the log is an exogenous coupled input stream (A8), and Proposition I.10 extends Theorems G.21 and G.25 on exactly this basis.

H.1 IDENTITIES, POOLS, EPOCHS, AND DEMAND

Weighted attestation and the corruption model. Each attested identity i carries weight $w_i \in [w_{\min}, w_{\max}]$, $w_{\min} > 0$, assigned by the attestation layer on cryptographic facts, never by a model. Because endorsements and verdicts *persist* on the log, the adversary’s resources are counted over the deployment’s lifetime: f is the number of distinct identities the adversary *ever* controls, $W_{\text{adv}} \leq f \cdot w_{\max}$ the total attestation weight those identities ever carry while controlled, and $f_s \leq f$ the number of ever-controlled identities that ever meet the panel-seat floor w_{seat} . This is a static-corruption reading: an identity once controlled is counted adversarial forever, so a *mobile* adversary, controlling different identities at different times, is covered with f counting every identity it ever touches. Peak-simultaneous bounds would not suffice here, precisely because persistent endorsements accumulate across control periods. Because f , W_{adv} , and f_s are lifetime totals, reputation farming cannot carry the adversary past any cap, and penalties that shrink adversarial capacity leave every bound stated against them valid (N12, S3). Deployments whose attestation layer revokes compromised identities may additionally cancel their endorsements (Definition H.6(c) counts only *un-revoked* endorsers), tightening the effective f below the lifetime total; no proof relies on this.

Canonical tools; singleton scoping. Tool identifiers are canonical across the federation: the same capability carries the same identifier at every deployment, fixed in TCB configuration and covered by attestation. All community endorsement, promotion, and reuse is per (value, tool) pair: an endorsement names the exact tool displayed on the endorser’s decision pane, promotion accrues one tool at a time, and a CC scope contains exactly the tools at which the community threshold was met. There is no endorsement transfer between tools, however similar. (Scope of this discipline: as at the endorsement-unit paragraph of §F.1.)

The reasoning pseudo-target. The target name reasoning is registered as one more canonical accounting target of the federation: it belongs to neither $\text{Tools}_{\text{sens}}$ nor $\text{Tools}_{\text{data}}$, is the subject of no invocation, and is never matched by the scope test of Rule R4(a) for any tool. Every per-(value, tool) counting rule of this appendix (Definitions H.4–H.6, Rules R12–R14) applies verbatim to the pair $(v, \text{reasoning})$: reasoning-context inspections gather promotion

weight at reasoning, and a reasoning-context qualifying rejection freezes the item like any other. Promotion to CC at reasoning makes the entry’s bytes readable community-wide, through the live endorsed view (Definition H.1), with no personal narrative confirmation (A5); it authorizes no tool flow through any scope test: a $\{\text{reasoning}\}$ scope matches no member of $\text{Tools}_{\text{sens}} \cup \text{Tools}_{\text{data}}$, so the sensitive fast path remains closed to it at every tool; at data tools, the baseline Rule R4(b) admits any readable $\mathbb{E}$ -bytes when literal-expressed — the disclosed post-endorsement channel of Corollary G.23(b); handle-expressed use gates regardless (Rule R4(c)) — and under Option O2 the data fast path is closed to it likewise (the exact-target test). *Locally*, an accepting reasoning confirmation continues to write its fresh C copy with *empty* tool scope (convention (vii)): the pseudo-target names the logged decision, never a whitelist scope; and a repeated confirm on locally readable bytes at reasoning is a covered gate (Rule R13(i), reasoning coverage), so reasoning pairs obey the once-forced cap exactly as tool pairs (INV-3), the budget’s c_{tool} count including reasoning on this basis (A12).

Two-tier storage of community status. Community status is a property of the *shared tier*, never of any local store. A promoted item is represented by a shared-tier entry (bytes, per-tool scope, status), backed by its promotion certificate on the federation log; what a user’s agent holds locally is at most a reference. The fast-path test of Rule R4(a) for the CC branch is evaluated against the *live* shared entry at call time: a value flows on community authority iff the shared entry is currently valid (promoted, not suspended, not demoted, not retired) and names the tool. Demotion, suspension, and retirement therefore act by invalidating or flagging the shared entry, withdrawing community authority from every user who has not personally confirmed, while no local block of any user is created, altered, or removed by any of them: a personally confirmed C copy is a distinct local object and continues to license the pair it was endorsed for (Lemma J.1). This is the storage reading under which Lemma I.8 and Proposition I.10 are stated, and it is the federated form of Definition G.3(iii): every CC authority in force is backed by a currently valid certificate.

Definition H.1 (Federated product configuration; live endorsed view). A federated configuration is $\hat{c} = (c, L)$: a local extended configuration c (Definition F.6) and the federation log L (convention (C3)). The shared-tier state $S_{\text{sh}}(L)$ is the deterministic replay of L under Rules R12–R14; $\text{LiveCC}(L)$ denotes the set of *virtual* community entries of $S_{\text{sh}}(L)$ that are currently promoted, unsuspended, undemoted, and unretired, each carrying its exact per-tool scope from its promotion certificate. Virtual entries are shared-tier objects in a namespace disjoint from both local identifier sorts; they are members of no local store. The federated endorsed view is $\mathcal{M}_{\mathbb{E}}^{\text{fed}}(\hat{c}) := \mathcal{M}_{\mathbb{E}}(c) \uplus \text{LiveCC}(L)$, and the federated Protected-LLM view replaces $\mathcal{M}_{\mathbb{E}}$ by $\mathcal{M}_{\mathbb{E}}^{\text{fed}}$ in Definition F.3. Local transitions read $\text{LiveCC}(L)$ only through the CC branch of Rule R4(a); log transitions change only L (hence the virtual view) and never any local block (Lemma I.8).

Definition H.2 (Federated step; log slot). The federated product system extends the slot structure of convention (C9) with a *log slot* preceding the typed slot: a federated interpreter step first appends the step’s exogenous log-event prefix (possibly empty) to L — under A8, coupled identically across compared runs — then delivers the typed and paste slots, then P emits one operation, executed against the federated view $\mathcal{M}_{\mathbb{E}}^{\text{fed}} = \mathcal{M}_{\mathbb{E}} \uplus \text{LiveCC}(L)$ with L as delivered by this step’s log slot. *Locally generated federation events* — all events the federated definitions specify for local acts (Definitions H.4, H.6, Rule R12): the submit(v) of PUBLISH-ACC and the signed protocol-bearing gate decisions being the principal examples, realized demand events (e.g. a dereference of a live community entry) included — are appended to L atomically at the close of the emitting operation’s transition, after its local effects; the CC branch of Rule R4(a) is evaluated, within the invocation’s atomic step, against the prefix delivered by that step’s log slot, so a step’s own appended events never affect its own gate. In the two-run comparison these locally appended events are equal across the compared runs — each is a function of matched components (the emitted operation, the displayed bytes, and the logged decision, all transcript-pinned; a demand event, arising from a matched operation against the matched federated view, likewise) — so the full delivered logs coincide: the exogenous prefixes by A8, the local suffixes by this derivation. Log-slot transitions acknowledge the log service, never P : nothing is appended to P ’s reply stream, no local block is created, altered, or removed (Lemma I.8), and the only P -visible effect is through $\text{LiveCC}(L)$, the deterministic replay of the delivered prefix (Definition H.1). The product system is specified on paper only; it lies outside the mechanized surface (§G.6).

Commentary (non-normative; distributed realizations of the live-entry test). The abstract model linearizes each invocation at a definite log position: the call’s CC check reads the prefix delivered to its step, so a call and a concurrent demotion are ordered — the call linearizes before or after the demotion, never against a stale view. A distributed deployment realizes this by pinning each invocation to its interpreter’s delivered log position (a per-call certificate); the fidelity of that delivery is a freshness obligation of the TCB log service (convention (C3)), and the delegated guarantee (Proposition I.10(i)) is stated relative to the delivered prefix.

Definition H.3 (Identity pools). The attestation layer partitions attested identities into *pools* on cryptographic and organizational facts (attestation provenance, organization, deployment cohort), fixed TCB configuration, never as-signed or influenced by any model. An identity’s pool is set at attestation and immutable thereafter; the adversary’s

f identities occupy whatever pools their attestations place them in and can concentrate in at most f pools. The diversity requirement demands that ordinary promotion’s N endorsers span at least D_{pool} distinct pools with at most $\lceil N/D_{\text{pool}} \rceil$ counted per pool, and panels accept at most $\lceil k/D_{\text{pool}} \rceil$ verdicts per pool (Rule R13(iii)). Deployment parameters satisfy $1 \leq D_{\text{pool}} \leq N$ (with N endorsers one cannot span more than N pools, so larger D_{pool} makes ordinary promotion unsatisfiable) and $D_{\text{pool}} \leq k$ where panels are read against the same pool partition. (What diversity does and does not buy: N11, S13.)

Definition H.4 (Demand). A user *demands the pair* (v, t) at the first interpreter event in the epoch (convention (C4)) at which the user’s own attempted use of v at tool t fires the gate of Rule R4 (because v is unpromoted at t , disputed, outside the user’s local scopes, or expressed through a handle — which gates at every tool regardless of any live community entry, Rule R4(c)) — a covered gate (Rule R13(i)) is not a demand event, nor is a cache confirmation of Options O1–O2 a log-demand event on either branch (accept-inert on acceptance, Definition H.6(iii); its rejecting branch, though protocol-bearing, initiates or corroborates without registering demand — while the attempted use firing it realizes the user’s *latent* indicator per the forced-confirmer clause, §J.1) — or dereferences v as a CC-block at t , or the user’s own accepting or gating *non-covered* reasoning-context confirmation of v (a covered reasoning gate, Rule R13(i), is not a demand event, exactly as a covered tool gate is not), or a dereference of a live CC-at-reasoning entry of v — demand of the pair $(v, \text{reasoning})$. A user *demands the item* v at their first demand of any pair (v, t) ; $n_v^{\log}$ counts distinct realized demanders of v within the epoch and $n_{v,t}^{\log}$ counts distinct demanders of the pair: a user who uses v at several tools contributes to each corresponding $n_{v,t}^{\log}$ and once to $n_v^{\log}$, so $n_{v,t}^{\log} \leq n_v^{\log}$; the reference model’s *latent* counts $n_v, n_{v,t}$ (§J.1) dominate these pathwise (§L.1). Demand order is the log order of these first events (convention (C3)). The owner’s creating confirmation of v , and any publish decision (itself federation-inert as an inspection, Definition H.6), is a demand event, so every item has, in each epoch of its publication or re-entry, $n_v^{\log} \geq 1$ with its owner among its realized demanders; in the reference model the owner-coupling convention (§J.1) mirrors this latently, so $n_v \geq 1$ holds in those epochs as well. A retired item is no longer surfaced (Rule R14), so no further confirmations accrue to its shared-tier ledger; the same bytes encountered later through a user’s own channels are ordinary personal content, counted in the single-user stream, not in v ’s ledger (this ledger-residual demand is bounded pathwise by Theorem J.2(i), persistence exceptions per INV-3; the aggregate expected per-user contribution of its benign, A11-conforming component vanishes under the vanishing conditions of Theorem J.9(iii), while poisoned or assumption-violating demand remains in $r_{\text{bad}}^{(e)}$, Corollary J.10).

Definition H.5 (Shared-presence freshness). An attested identity’s inspection of a shared candidate v is *fresh* if the identity has no prior *logged* decision (a protocol-bearing decision: accepting endorsement, qualifying rejection, or panel verdict; Definition H.6) on v ’s exact bytes; otherwise it is *repeat*. Decisions are logged exactly when protocol-bearing (Definition H.6), so freshness is *shared-presence-first*, not *life-first*: a personal decision on the same bytes taken outside any shared-tier presence (before first publication, or after retirement) is not logged and does not consume freshness — a disclosed boundary of the byte-permanence guarantee (INV-1); the behavioural parameters $q_{\text{pre}}, q_{\text{pan}}, p_b$ are accordingly read over the log-defined fresh class (theorem I.19). Freshness is decidable from the federation log, is permanent while so defined, and never renews: across entries, dormancy, freezes, suspensions, and epochs within and across the bytes’ shared-tier presences, one logged decision per identity is final. All promotion weight and all panel verdicts are carried by fresh inspections (Definition H.6, Rule R13(iii)); a repeat inspection can contribute only a rejection. This is INV-1; the safety of the asymmetry is INV-2 (rationale: theorem H.7).

Definition H.6 (Endorsement counting and banking). A published item enters the promotion pipeline by submission (Rule R12); each pipeline entry opens a fresh finalisation window of length T_{win} at (re-)submission. A gate decision is *protocol-bearing* for a shared candidate iff all of the following hold: (i) its exact bytes have, at decision time, a shared-tier presence (a candidate or CC entry that is not retired; frozen and suspended presences included, since freeze- and suspension-time decisions are precisely the verdicts, corroborations, and bankable credits the counting rules consume); (ii) its target is a canonical tool in $\text{Tools}_{\text{sens}} \cup \text{Tools}_{\text{data}}$ or the canonical pseudo-target reasoning (§H.1); (iii) the gate is not a covered gate (Rule R13(i)); and, if it is a cache confirmation of Options O1–O2, its outcome is rejecting — cache gates are *accept-inert* (their accepting branch is never protocol-bearing), while their rejecting branch on presence-coinciding bytes is protocol-bearing like any rejection (INV-2); (iv) its outcome is a durable accepting pair-confirmation or a rejecting review (Definition F.5). Only protocol-bearing decisions are appended to the federation log as decisions on the candidate: only they consume shared-presence freshness, carry promotion weight, initiate or corroborate disputes, or become panel verdicts. A *publish* decision is logged as a submission event (PUBLISH-ACC, Rule R12), not as an inspection: it carries no promotion weight and consumes no freshness, while remaining a demand event for its item (Definition H.4). All other confirmations — personal content — are purely local events with no federated effect and no freshness consequence for shared-tier counting. Cache confirmations are *accept-inert* regardless of whether their bytes coincide with a shared-tier presence: their accepting branch is never logged as a shared-candidate decision (no promotion weight, no credit, no notification cohort, no freshness consumption); their rejecting branch on presence-coinciding bytes remains a qualifying rejection like any other (INV-2), consuming

freshness as any logged rejection does. (Their accepting branch, being unlogged, retains the identity’s freshness on the bytes; the acceptance itself carries no weight, and any later non-cache decision by the same identity is one more fresh decision inside the log-defined class the weakest-target parameters $q_{\text{pre}}, q_{\text{pan}}, p_b$ are already required to cover, theorem I.19.) Inertness and coverage are decided by interpreter-only, attacker-immutable attributes (provenance, Definition F.2; the owner-side coverage test, Rule R13(i)).

- (a) *Once per identity per item, keyed to bytes, log-scoped, authority-gated.* An identity’s first logged accepting decision on v ’s bytes (Definition H.5) is its one weight-bearing endorsement *provided the identity holds dispute-initiation authority at the moment of the decision* (not rejection-suspended, below its ceiling c_{ceil} , with a free r_{max} slot, and within its per-item initiation allowance, Rule R13(i),(v)); it is counted at the single tool that decision names and toward the pipeline entry open at the time of the decision, subject to the credit caps of clause (f). An accepting decision by an identity lacking that authority is logged, consumes freshness, and creates the decider’s local C copy (Lemma J.1), but is *authority-inert*: it carries no promotion weight, is never credited (in-entry, banked, or vindication-credited), and joins no notification cohort. Authority is a deterministic predicate of the prior log (Rule R13(v)), evaluated interpreter-side at the decision’s header atom and recorded with it. Every later acceptance by the same identity (at any tool, in any entry, in any epoch, in any shared-tier presence) carries no weight (INV-1).
- (b) *Banking.* A first decision recorded during dormancy (window closed, not yet resubmitted or retired) or during a freeze with no open entry is *banked*: credited as endorsement weight at its tool when the owner’s next entry opens (subject to clause (f)), a deterministic relabel of the already-logged event, not a new confirmation and not a finalisation. Banked credits expire if the item retires first (INV-4).
- (c) *Finalisation only at window end.* Within an entry, v is promoted to CC at a specific tool when the weight of distinct, un-revoked accepting endorsers at that tool reaches W in credited endorsements (unweighted: N credited endorsers; crediting per clause (f)), drawn from at least D_{pool} diverse pools, with finalisation only at the end of the window; any qualifying rejection logged within the window intervenes.
- (d) *Dormancy.* Between entries the item remains surfaced and personally usable; a gate confirmation during dormancy is logged as a decision, consuming freshness, counted in the item’s ledger (Lemma J.7), banked per (b), and creating the user’s local C copy, after which their gate never re-fires *at that tool* (INV-3).
- (e) *Consequence.* The weight-bearing endorsement sets of successive entries of the same bytes are identity-disjoint across the entire deployment history, and every weight-bearing endorsement is its endorser’s fresh inspection of the bytes (Lemma I.3).
- (f) *Credit caps (per entry, per tool).* Crediting is greedy in log order and capped: a qualifying accepting decision (in-entry, or banked and credited at the entry’s opening in log order of the original decisions) is *credited* to the entry at its tool iff, at the moment of crediting, the entry holds fewer than $\lceil N/D_{\text{pool}} \rceil$ credits from the decider’s pool at that tool (Definition H.3) and fewer than $N^+ := D_{\text{pool}} \lceil N/D_{\text{pool}} \rceil$ credits at that tool in total. Crediting requires authority at decision time (clause (a)); the caps bind over authority-active credits only. An uncredited accepting decision retains every personal effect (the decider’s local C copy, Lemma J.1; freshness consumed, Definition H.5) but carries no promotion weight, joins no notification cohort, and is never re-credited. The caps are promotion-neutral: a pool-capped decision never counted toward promotion under Definition H.3 in any case, and the total cap binds only once the entry holds $N^+ \geq N$ credits which, under the per-pool cap, necessarily span at least D_{pool} pools — i.e. only when finalisation at window end is already enabled. Weighted deployments cap credited weight analogously (per pool and in total), bounding credited endorsers by $\mathcal{O}(W/w_{\text{min}})$: constants change, never exponents (Remark I.30).

An endorsement is not a separate protocol act: it is the endorser’s own accepting confirmation of (v, tool) at their gate (Theorem G.21), signed and appended to the federation log. On re-entry, the interpreter notifies exactly the weight-bearing endorsers credited to the *immediately preceding* entry; no earlier cohort is ever re-notified, so the notified set per re-entry is bounded by that single entry’s credited endorsements, at most N^+ per demanded tool (clause (f)), independently of the item’s history. Notifications are deduplicated jointly with the suspension prompts of Rule R13(vi), at most one invitation per identity per item per epoch. An invitation is valid only in the epoch of its issuance: an unopened invitation expires at the epoch boundary and is not reissued for the same re-entry; acting on an invitation (accepting, rejecting, or declining) is an event of the issuance epoch. Narrowing the cohort weakens nothing: no safety bound relies on repeat inspections (INV-1, INV-2), and notifications carry no protocol authority. A notified endorser’s re-inspection is repeat: its accepting branch carries no weight, its rejecting branch is a qualifying rejection like any other (Rule R13(i)). Declining costs nothing (Theorem J.2; theorem H.7).

Commentary (non-normative; role of the credit caps). Clause (f) supplies the counting the budget ledger consumes. Without it, finalisation only at window end would let a popular entry accrue credited endorsers without bound inside one window, and the weight-bearing row of Table 14, the notified re-entry cohort, and the reconsideration term of Lemma J.7 would all grow with the item’s popularity. With it, credited endorsers per (entry, tool) number at most $N^+ < N + D_{\text{pool}} \leq 2N$ (using $D_{\text{pool}} \leq N$, Definition H.3), deterministically; the overflow — in-window accepting demanders beyond the caps — is within-window demand, counted by the window-bucket row of Table 14 under A10’s demand clause. Safety is unaffected a fortiori: promotion still requires N credited fresh endorsements, Theorem I.13 lower-bounds honest *decisions* (of which credits are a subset), rejections are never capped (INV-2), and promotion-neutrality (clause (f)) leaves Assumption A11’s sufficiency reading unchanged.

Remark H.7 (Why the freshness asymmetry is safe). The asymmetry of Definition H.5 (repeat acceptances inert, repeat rejections still qualifying) is deliberate: a rejection can only freeze or corroborate, never promote (INV-2), so re-exposure of bytes to a previously fooled inspector can help the defence but never the attack, and the defence never rests on a person catching on a second *logged* look what they missed on the first: no logged repeat inspection carries weight. (Unlogged personal inspections outside a shared-tier presence are outside this discharge, Definition H.5; their calibration consequence is theorem I.19.) Banking is safe in both directions per INV-4 (as restated there: starting weight only, window’s rejection opportunity never bypassed, identity-disjointness preserved). Declining a re-entry notification costs the decliner nothing: a user who has personally confirmed holds a local in-scope C copy and reuses it at that tool without any gate (INV-3); the floor distinguishes such voluntary repeat acts from forced confirmations (Theorem J.2(ii), Remark J.4).

Definition H.8 (Reach-free bound). A safety bound is *reach-free* if it does not depend on the number of honest identities to whom the item is presented (its *reach*), only on protocol constants and the adversary’s resources (INV-6), and the stated behavioural parameters $q_{\text{pre}}, q_{\text{pan}}$, read as weakest-target infima (INV-6).

H.2 PROTOCOL RULES

The rules below are normative; all rationale lives in Appendix K under the keys cited, or in the marked commentary blocks of §E.1. Clause numbering of Rule R13 preserves the roman numerals (i)–(vii) cited throughout.

Rule R12 (Sharing eligibility and the publish gate). Only C-labelled blocks with provenance confirmed (never cache, Options O1–O2) are eligible for submission, by a deterministic label-and-provenance membership test (provenance is interpreter-only, Definition F.2, hence attacker-immutable); a model may *nominate* but cannot submit; submission requires the owner’s accepting confirm(v , publish) on the decision pane (PUBLISH-ACC), recorded in the owner’s transcript, which emits one signed submit(v) event to the federation log and creates the candidate’s shared-tier presence; it creates no local block and consumes no freshness (Definition H.6). Each identity emits at most ρ_{sub} submission events per epoch; re-entry of an item into the promotion pipeline counts as a fresh submission, and a submission of bytes whose per-epoch entry cap is exhausted (Rule R14(i)) is ineligible.

Required deployment parameters. A deployment instantiating Rule R13 fixes, as TCB configuration: the measurement window and minimum sample size of the charged-rejection rate trigger (v)(c); the suspension and cool-down durations of (v)(c) and (vii); the escalation schedule and the attestation-weight reduction function of (vii); the derivation-cascade relation of (iv); and the lifetime-account trigger of (v)(f) where selected. The theorems below consume only the explicit caps and threshold properties stated in their hypotheses ($r_{\text{max}}, c_{\text{ceil}}, \theta_{\text{acc}} > 1 - p_b, k, m, E_{\text{max}}, c_p$); no proof depends on the unspecified schedule shapes.

Rule R13 (Dispute and demand-paced panel resolution).

- (i) *Qualifying rejection = protocol-bearing gate rejection.* Every attested identity’s rejecting decision on a shared candidate at its own *protocol-bearing* confirmation gate (Definition H.6(iii): tool- or reasoning-target, non-covered; cache gates included on their rejecting branch) is automatically a signed, logged qualifying rejection. There is no separate filing step and no rejection weight floor: inspection and report are one act. A user who merely abandons the dialog records nothing (abstention is counted as a miss inside $1 - q$; theorem I.19). Each identity holds at most r_{max} open (unresolved) rejections. *Covered gates are inert.* A gate event fired by a handle-expressed use (Rule R4(c)) of a pair for which the identity already holds an in-scope local C copy — a *covered gate*; a literal-expressed reuse takes the fast path (Rule R4(a)) and never gates — is protocol-inert for the shared tier, on both branches: its accepting decision carries no promotion weight, consumes no freshness, and joins no notification cohort (a *redundant* confirmation, §J.1); its rejecting decision is logged as corroboration if the item is frozen and as an advisory flag otherwise — no dispute initiation, no verdict, no accountability exposure. Coverage is decided owner-side on the identity’s own local store, a deterministic test independent of any shared-tier or attacker-influenced state. *Reasoning coverage.* A gate event with target reasoning on bytes of which the identity’s local store holds *any* readable $\mathbb{E}$ -level block (T, PC, or C of any scope; Rule R5, Definition F.6) is likewise a

covered gate, protocol-inert on both branches exactly as a tool-target covered gate: its accepting decision is a redundant confirmation (§J.1) — no promotion weight, no freshness consumption, no notification cohort — and its rejecting decision is logged as corroboration if the bytes are frozen and as an advisory flag otherwise. The coverage test is owner-side and deterministic: it reads only the identity’s own local store’s readability facts, never any shared-tier or attacker-influenced state. Within an epoch, only an identity’s *first* rejecting decision on a given byte string can take effect as a dispute-initiating rejection; every later rejecting decision by the same identity on the same bytes within the epoch is logged as corroboration if the item is frozen, and otherwise as an advisory flag (the zero-authority class above): no protocol effect, no accountability exposure. The allowance renews at the epoch boundary. A rejecting decision that is verdict-eligible under clause (iii) is cast as a panel verdict regardless of this cap, exactly as under clause (v)(e); the cap restricts dispute *initiation* only, and capped rejections remain visible as flags. An identity whose rejection rights are under suspension (clause (v)) files *advisory flags* instead: no protocol effect, no penalty; flags alert other users.

- (ii) *Disputed state*. A qualifying rejection freezes promotion of v at every tool, immediately, and visibly flags the item. (A qualifying rejection on bytes holding CC status at any tool is instead a post-promotion rejection, routed to clause (vi).) A frozen item remains retrievable (surfaced flagged, Rule R14(ii)) and personally usable through each user’s own gate confirmation; the freeze suspends community promotion only. The item’s pre-dispute endorsers are notified. Pre-dispute endorsements are retained on the log for audit and reputation but carry no weight in resolution.
- (iii) *Panel formation (demand-paced, fresh-juror)*. The panel is not a body convened in advance but a capped tally of decisions: it comprises the first k *eligible verdicts* logged on the disputed item, with no global deadline. Because a disputed item is unpromoted, every demander must personally inspect-and-confirm it to use it (Theorem G.21); the interpreter displays the exact bytes at the demander’s own gate, visibly carrying the dispute flag and the rejecter’s stated ground. At the moment a demander records a decision, and only then, the interpreter checks eligibility on log facts:
 - (a) attestation weight $\geq w_{\text{seat}}$;
 - (b) fresh (Definition H.5);
 - (c) no verdict already cast by this identity on this dispute instance; verdict eligibility is scoped to the dispute instance for its entire life: an identity casts at most one verdict on a given dispute, ever (INV-5);
 - (d) at most $\lceil k/D_{\text{pool}} \rceil$ verdicts per identity pool, likewise scoped per dispute instance;
 - (e) fewer than k verdicts so far.

A cache gate (Options O1–O2) is never a verdict surface: its rejecting decision may initiate or corroborate a dispute per clause (i), but is not verdict-eligible and occupies no panel seat. (Only a cache gate’s rejecting branch is logged, Definition H.6(iii); admitting such gates as verdict surfaces would make verdict membership outcome-selected there, against the benign-panel hypothesis of Proposition I.20. The exclusion is theorem-compatible and resolves an otherwise outcome-selected surface: Theorem I.13 and Proposition I.20 bound tallies over verdicts actually cast, and the eligible class is simply defined without cache gates; no pathwise monotonicity is claimed.) A dispute instance opens at its qualifying rejection and closes at resolution or expiry; its verdict tally and pool caps persist unchanged across epoch boundaries (convention (C4)), and a boundary-spanning dispute counts toward Rule R14(i) in the epoch in which it opened. If eligible, *that gate decision is the verdict*, recorded once, verdicts ordered by log order of decisions; otherwise the decision is an ordinary personal confirmation (a bystander rejection still logs as corroboration). A user who opens the dialog and abandons it records nothing and occupies nothing: verdict capacity is consumed only by cast verdicts, so abandonment cannot exhaust or stall the panel, and no seat allocation, timer, or release mechanism exists. The dispute, like freshness, is byte-level: the panel adjudicates the bytes, each juror inspecting them at the juror’s own attempted tool, with the dispute flag and the rejecter’s stated ground displayed. Pre-dispute endorsers cast no verdicts: a notified endorser’s re-inspection is repeat (Definition H.5); its accepting branch has no protocol effect, and its rejecting branch is a further qualifying rejection with no additional protocol effect while the item is frozen (logged as corroboration). Consequently every verdict-bearing inspection is the juror’s first *logged* decision on the bytes (Definition H.5; INV-1, INV-2).

- (iv) *Resolution (early and decisive)*. Verdicts are tallied as cast. The moment rejecting verdicts reach m_{rej} : *evicted*: demoted to U in the shared tier, cool-down, cascade to shared derivations. The moment accepting verdicts reach m : *vindicated*: the byte-level freeze lifts and the interrupted pipeline entry resumes, its window clock excluding the freeze; vindication consumes neither the owner’s rate limit nor an entry of Rule R14(i), and it never directly promotes. Each juror’s accepting verdict is simultaneously that juror’s gate confirmation at the juror’s own tool

(its one fresh decision on the bytes), credited toward the entry’s per-tool endorsement sets subject to the authority gate and credit caps of Definition H.6(a),(f); it joins those sets together with the retained pre-dispute endorsements, which carry no weight in resolution but keep their promotion weight, being fresh inspections the panel’s vindication never contradicted. Freeze-time bystander confirmations, fresh accepting gate confirmations by demanders whose decisions are not verdicts (under-weight, pool-capped, or post-capacity), are credited identically on vindication at their own tools (per Definition H.6(a),(f)); crediting adds no confirmation event (Lemma J.7). If no entry was open at the freeze, vindication lifts the freeze and the item returns to dormant status, its verdict-confirmations credited on the next entry’s opening subject to Definition H.6(a),(f), or expiring with it (Definition H.6(b), INV-4). Every promotion, after any number of disputes, therefore still requires N fresh-inspection endorsements at the promoted tool itself (Lemma G.14).

Default bars. The default bars are $m_{\text{rej}} = k + 1 - m$ with $m > f_s$ and $m_{\text{rej}} > f_s$ (jointly forcing $k \geq 2f_s + 1$); their consequences (mutually exclusive outcomes and decisive, tally-deadlock-free resolution) are Lemma I.7. Deployments choosing looser bars ($m + m_{\text{rej}} \geq k + 2$) retain a neither-bar outcome, in which the item returns to unpromoted candidate status and re-entry consumes the owner’s rate limit (Rule R12) and gathers a fresh endorsement set. A disputed item that attracts no new demand for T_{exp} expires (Rule R14(i)): retired unpromoted, all open rejection slots on it freed, no penalty to any party.

(v) *Rejecter accountability.*

- (a) Panel verdicts are never penalised and never charged.
- (b) A dispute-initiating rejection is *charged* when its dispute ends in vindication, at the moment the vindication is logged, to the epoch in which that vindication is logged. Rejections whose disputes end in eviction or expiry charge nothing.
- (c) The first charge per identity per epoch carries no penalty. Repeated charges escalate to suspension of *rejection rights only* (endorsement weight and ordinary use untouched), restored after cool-down. Escalation is rate-based, never count-based: the trigger is the identity’s charged-rejection rate *per fresh inspection*, over a minimum sample size, at a threshold θ_{acc} calibrated strictly above the honest baseline $1 - p_b$ (theorem I.5).
- (d) *Hard ceiling.* Independently of (c): once c_{ceil} charges have accrued to an identity within an epoch, the identity’s further dispute-initiating rejections for the remainder of the epoch are filed as advisory flags (the zero-authority class of clause (i)); the allowance renews at the epoch boundary.
- (e) Suspension and the ceiling restrict *dispute-initiating* rejections only: a rejecting decision that is verdict-eligible under clause (iii) is cast as a panel verdict regardless of the identity’s accountability state, so the panel arithmetic of Theorem I.13 and Proposition I.20 is independent of accountability state; freeze-time corroborating rejections, having no protocol effect, are logged either way.
- (f) Deployments may additionally maintain a *lifetime* charged-rejection account per identity, with the same per-inspection normalisation and a comparative (population-outlier) trigger, suspending rejection rights directly for extreme outliers.
- (g) Both layers touch rejection rights only, never endorsement weight or any local store (Lemma I.8).

(vi) *Post-promotion rejection.* A qualifying rejection against a promoted item moves it to *Suspended*. Suspension invalidates the shared entry for the fast-path test of Rule R4(a) (§H.1): community authority is withdrawn for the duration of the review, so every user’s use of the pair gates — except a user holding a personally confirmed in-scope local C copy, who continues through that copy (Lemma J.1), unaffected. Suspension additionally imposes the byte-level freeze of clause (ii) on promotion at every tool. A fast-track panel (same parameters) attempts completion within T_{pan} ; failure to complete by T_{pan} leaves the item *Suspended* with verdict collection continuing demand-paced, exactly as an ordinary dispute. Its verdicts arise at the gates suspension re-arms:

- (a) *Demanders during suspension:* any user demanding the pair while the entry is suspended (new adopter and prior ungated adopter alike) gates, and verdict eligibility is decided by the log-fact check of clause (iii) exactly as in an ordinary dispute. In particular a prior ungated adopter with no logged decision on the bytes is fresh, while one with *any* prior logged decision (a rejection, or an acceptance at another tool) is repeat and ineligible, its rejecting branch a qualifying rejection like any other (logged as corroboration while frozen).
- (b) *Prompted re-inspection:* the suspension notification carries a voluntary re-inspection prompt to current adopters not presently demanding the pair; accepting it re-displays the exact bytes at the adopter’s own gate under the dispute flag, eligibility again per clause (iii); an accepting inspection earns the adopter a local C copy, covering all later use of the pair. Declining costs nothing beyond the suspension itself: the adopter’s next actual use of the pair gates once, at the floor (Corollary J.5). Prompts are issued to at most $c_p k$ adopters per suspension; re-inspection invitations of either kind, re-entry notifications (Definition H.6) and suspension

prompts, are deduplicated jointly, at most one invitation per identity per item per epoch. Prompts expire at the end of their issuance epoch; a review left unresolved at the boundary continues demand-paced but issues no new prompts merely for having crossed it. At most $E_{\max}$ fast-track reviews are opened per item per epoch; once the cap is reached, the item remains Suspended for the remainder of the epoch, verdict collection continues demand-paced exactly as in an ordinary dispute, and further post-promotion rejections are logged as corroboration of the open review (the zero-authority class of clause (i): no protocol effect, no accountability exposure).

The panel decides demotion-with-cascade or reinstatement. If k fresh verdicts do not arrive within T_{pan} , the item remains Suspended and verdict collection continues demand-paced, exactly as an ordinary dispute; a Suspended item attracting no fresh demand for T_{exp} is retired from the shared tier (Rule R14(i)): no reinstatement, no eviction, no accountability consequence for either side, and every personally confirmed local copy untouched (Lemma I.8).

- (vii) *Publisher accountability.* When a panel *evicts* an item, the publishing identity incurs an escalating penalty: the first eviction per identity per epoch triggers a submission cool-down; repeated evictions within an epoch escalate to suspension of *submission rights* and reduction of attestation weight. The penalty touches shared-tier rights and attestation weight only, never any local store, label, or scope (Lemma I.8), and is triggered only by a resolved panel’s eviction verdict, never by rejections alone and never by expiry (Proposition I.22).

Commentary (non-normative; role of clause (v)). Clause (v)’s induced deterministic bound $c_{\text{acc}} = c_{\text{ceil}} + r_{\text{max}}$ is Lemma I.4; these clauses enter only the budget constants of Appendix J, never any safety theorem (Figure 6). Calibration guidance and the honest cost analysis: theorem I.5, theorem I.19, S15.

Remark H.9 (Per-pair dispute scoping (deployment variant)). A deployment may scope disputes and panels per (value, tool) pair: a qualifying rejection then freezes promotion at the rejected pair only, and jurors are drawn from that pair’s demanders, adjudicating the rejected tool’s context. This removes the cross-tool-jury residual of S4: the applicable q_{pan} becomes the targeted tool’s. The costs are per-pair panel formation and freeze scope, panels and prompts multiplying by up to c_{tool} per item, and a re-keying decision — whether freshness and verdict capacity remain byte-level or become pair-level — that this document does not make; an analogous per-attempt bound is plausible but not formally instantiated here, the freshness and verdict-capacity re-keying being deliberately undecided. No theorem of this document consumes the variant.

Rule R14 (Candidate hygiene). The shared candidate store is curated by three deterministic, interpreter-applied policies.

- (i) *Entry cap and expiry.* At most $E_{\max}$ pipeline entries open per exact byte string per epoch — promoted, evicted, and unpromoted entries alike, republications and re-entries included; a submission of bytes whose per-epoch entry cap is exhausted is ineligible (Rule R12) and takes the uniform reject of PUBLISH-REJ, eligibility returning at the epoch boundary. The cap is availability-only: banked credits and any live promotion status persist across the boundary (convention (C4); N14). Additionally, a published candidate is retired from the shared tier when any of the following holds:
 - (a) it completes $E_{\max}$ pipeline entries without promotion within the epoch (the count is cumulative across republications within the epoch and resets only at the epoch boundary);
 - (b) it accumulates $E_{\max}$ *non-vindicated* disputes within the epoch (disputes ending in vindication are exempt; under the default bars this trigger fires only through the optional neither-bar outcome of Rule R13(iv); rationale: N15);
 - (c) it remains dormant (window closed, not resubmitted) for T_{exp} ;
 - (d) it attracts no demand for T_{exp} , including a *disputed* candidate attracting no new demand for T_{exp} .

Entries and dormant intervals alternate: each pipeline entry is followed by at most one dormant interval, ending at the next entry or at retirement; by the entry cap, entries per item per epoch number at most $E_{\max}$ and dormant intervals begun within the epoch at most $E_{\max}$, deterministically (one further dormant interval may be carried across the boundary; the ledger’s conservative common bound is $E_{\star} = E_{\max} + 1$, Lemma J.7). Because the cap and the count of clause (a) are per epoch, a persistently demanded benign item may re-enter in later epochs; safety is unaffected, every federated safety bound being per-attempt (Definition I.1) with lifetime exposure counted by Corollary I.27. The owner’s local C copy is untouched (Lemma I.8); the owner may republish, subject to Rule R12 and the entry cap. Retirement of a disputed item frees all open rejection slots on it and is neither an eviction nor a vindication: no accountability consequence for any party.

- (ii) *Surfacing order*. The $\leq c_{\text{cand}}$ candidates a retrieval surfaces are ordered by trust status (CC, then undisputed candidates by endorsement weight, then disputed items, flagged), computed on log facts, never by a model.
 - (iii) *Demotion hygiene*. Items evicted by panel verdict are removed from retrieval entirely for the cool-down period.
- These policies only refuse entry openings, remove items, or reorder items in the shared tier; their safety- and budget-preservation is Lemma I.9.

H.3 FEDERATED ASSUMPTIONS, ADVERSARY, AND TCB ADDITIONS

Assumption A7 (Federated endorsement transfer). A Confirmed item endorsed *at a specific tool* by a qualifying weighted set of distinct attested identities is authoritative for that exact tool for any user (and likewise at the pseudo-target: a qualifying quorum at reasoning is authoritative for informing any user’s reasoning), *provided the item is context-independent*, a semantic property judged by the publishing owner and the inspecting community, not decided by the type system. Tool identifiers are canonical across the federation (TCB), so “the same tool” is well-defined; there is no transfer between tools. (Scope: as at §F.1, endorsement unit; the baseline data-tool path is governed by Rule R4(b) and Corollary G.23(b).) The type-level half is discharged by Lemma I.2 and is not assumed. (All formal guarantees are integrity guarantees; semantic confidentiality of published content rests on the publish gate and is not separately proved: §K.4.)

Assumption A8 (Exogenous federation log). In the two-run comparison of Theorem G.25 extended to the federated setting (Proposition I.10), the *exogenous log prefix* delivered at each step’s log slot (Definition H.2) — every federation-log event not generated by the compared local run itself: other parties’ endorsements, rejections, promotions, suspensions, panel verdicts, and accountability events — is an exogenous input stream, coupled identically across the compared runs, exactly as A6 couples the user’s typed stream. The compared run’s *own* locally generated federation events (Definition H.2) are not assumed coupled: their equality across the compared runs is *derived* in Proposition I.10(ii) from the matched components, so the full delivered logs coincide without constraining either attack. The federated Reasoning Integrity statement therefore isolates the effect of *unendorsed*, *undisplayed* untrusted bytes; attacker influence exercised through *legitimate* log activity is governed not by that theorem but by the safety bounds of Appendix I (Theorems I.11, I.13).

Assumption A9 (Verdict rate). There is $\rho_{\text{seat}} \in (0, 1]$ that lower-bounds, uniformly over disputed items, times, and histories, the conditional probability, given all prior protocol events, that the next arriving demander of a disputed item casts an eligible verdict (Rule R13(iii)): weight $\geq w_{\text{seat}}$, fresh, no prior verdict on the dispute instance, within the pool caps, and recording a decision rather than abandoning. The bound is asserted *jointly over behaviour and protocol state*: eligibility is consumed endogenously (freshness spent, Definition H.5; pool caps filled, Rule R13(iii)(d); capacity exhausted, (iii)(e)), so A9 entails in particular that the item’s remaining fresh, seat-weighted, pool-diverse demand does not run out while the dispute is open — a demand-composition property, not a psychological one. The bound is read per the conditioning convention of §L.1: conditional on all logged atoms prior to the arrival, and uniformly over demand realisations.

Commentary (non-normative; epistemic status and failure mode of A9). A9 has two components with different characters. The behavioural component (a demander who *could* cast an eligible verdict does so at rate $\geq \rho_{\text{seat}}$) has the same epistemic status as $q_{\text{pre}}, q_{\text{pan}}, p_b$. The structural component is stronger and can fail *deterministically*, with no human variation at all: freshness consumption and the pool caps are protocol mechanics, so an item pre-exposed to most of a pool’s identities, or whose demand is concentrated in few pools, can reach a state where *no* arriving demander is eligible — the conditional probability is then 0, and no behavioural assumption restores it. The consequences of failure are confined by design: every result resting on A9 is availability-side (Lemma I.6, Lemma I.24(ii), the ledger’s bucket rows, and through them the budget asymptotics); no safety theorem consumes it (Figure 6), the unresolved dispute stalls *frozen* (Rule R13(ii)), and each affected demander still pays at most the floor (Theorem J.2(i), Lemma I.24(ii)’s unconditional clause). Validation: verdict-eligibility rates on disputed items, stratified by pool and by the item’s prior exposure (N17). Quantitative consequences: Lemma I.6. The eligible-verdict class excludes cache-gate decisions (Rule R13(iii)); ρ_{seat} is read over the class as so defined. q_{pre} and ρ_{seat} condition on the same human act consistently: an abstaining demander is a miss inside $1 - q_{\text{pre}}$ and a non-verdict inside ρ_{seat} ; high abstention degrades resolution speed, never the validity of the safety parameter (theorem I.19).

Adversary (federated). The adversary controls, over the deployment’s lifetime, f qualifying identities of total weight $\leq W_{\text{adv}}$, of which at most f_s are ever seat-eligible (static corruption, §H.1). It does not control honest users’ decisions, the labelling, the interpreter, or the log. It may present a poisoned item to whom it likes, in what order it likes, and may stop or continue as a function of past outcomes, but never as a function of an inspection not yet performed: formally, its strategy is a predictable process with respect to the protocol filtration (§L.1, convention (C2)),

against which Theorem I.13 is stated. Two structural constraints on its adaptivity (within-entry byte immutability, and the reading of q as a uniform bound over the honest identities the adversary can target) are recorded in §L.1 and theorem I.19.

Definition H.10 (Exogenous item classification). Fix a classification $\text{Bad} \subseteq \mathcal{V} \times (\text{Tools}_{\text{sens}} \cup \text{Tools}_{\text{data}} \cup \{\text{reasoning}\})$, exogenous to the protocol: no rule reads it, no transition depends on it, and no model computes it. An item v is *poisoned* if $\text{Bad}(v, t)$ holds for some tool t , and *benign* otherwise; “harmful at t ” means $\text{Bad}(v, t)$; harm at reasoning means content that misleads reasoning if adopted (A5). Every behavioural parameter ($q_{\text{pre}}, q_{\text{pan}}, p_b, q_c$) and every safety and budget statement of Appendices I–J is read relative to this classification; the classification is a modelling device for stating the bounds, not a protocol object, and the protocol’s behaviour is identical under any choice of it.

TCB additions (federated). Beyond §F.5: the attestation layer and its weight assignment on cryptographic facts; the pool assignment of Definition H.3; canonical tool identifiers shared and attested across the federation; and the log service of convention (C3) (sequencing and authenticated history). The Sybil conditions (finiteness of W_{adv} and the seat margin $m > f_s$) require an attestation authority or identity cost Douceur (2002); Dwork & Naor (2007). Because f , W_{adv} , and f_s are *lifetime* totals (§H.1), they are enforceable only to the extent that the attestation or identity-cost mechanism supplies certified lifetime bounds: the issuance policy must bound the identities and weight any party can ever acquire, and that bound is what the constants denote. Where issuance is open or uncosted, no finite f exists and the federated safety results are compatibility statements — valid for whatever lifetime totals the deployment can certify — while the single-user layer and the cap (Theorems G.21, G.25, J.2) are unaffected, consuming no Sybil bound.

I FEDERATED GUARANTEES

This appendix proves the guarantees of the protocol of Appendix H: the structural lemmas (§I.1), then the integrity lifting, the deterministic safety theorem, panel safety, wrongful eviction, publisher safety, denial-of-service, and submission-budget results (§I.2). Dependencies are as in Figure 6.

I.1 STRUCTURAL LEMMAS

Definition I.1 (Promotion attempt). Fix a byte string v . A *promotion attempt* is a maximal segment of one shared-tier presence of v beginning at an *anchor* and ending at the first *finalisation* thereafter — the close of a pipeline entry’s window at its full length, where promotion is decided (Definition H.6(c)) — or, where no finalisation occurs, at the end of the presence (retirement, or demotion by panel eviction) or at a loose-bar neither-bar resolution returning the item to candidate status (Rule R13(iv)). Anchors are: the presence-creating submission event (first attempt), and each attempt-ending event just listed other than the end of the presence (each later attempt). A window closed early at an epoch boundary is *not* a finalisation (convention (C4)): it banks its accrued endorsements to the item’s next entry, which belongs to the *same* attempt.

Anchors are log events, and every decision credited to an attempt’s entries — in-window, banked in a dormant or frozen interval, boundary-banked, or vindication-credited (Definition H.6(b),(f), Rule R13(iv)) — is logged strictly after the attempt’s anchor: in-window and interval decisions follow the preceding anchor by construction, and boundary-banked decisions were logged in an earlier entry of the same attempt, itself opened after the anchor. Likewise no ordinary dispute and no fast-track review is open at any anchor: either state imposes the byte-level freeze (Rule R13(ii),(vi)), which interrupts a window rather than letting it finalise (Rule R13(iv)), and retirement frees all open rejection slots (Rule R14(i)); so every dispute or review bearing on an attempt opens, and casts all its verdicts, after the attempt’s anchor.

Attempts partition each presence; every entry belongs to exactly one attempt (the attempt whose anchor precedes its opening); at most one entry of an attempt lies in any single epoch (a second entry of the same attempt arises only across an epoch boundary, via a boundary-closed window); and only an attempt’s final entry can finalise. An attempt *succeeds* if its finalisation is a promotion; an attempt with no finalisation — ending at retirement, eviction, or a neither-bar resolution, or with no entry ever opened — succeeds with probability zero. Attempts whose finalisation lies in a given epoch number at most the epoch’s entries of the item, hence at most E_{max} per item (Rule R14(i)), each such finalising entry opened by a rate-limited (re-)submission of the same epoch (Rule R12; windows are epoch-contained, convention (C4)). Dually, the attempts *anchored* in a given epoch are begun at that epoch’s presence-creating submissions and finalisation events (under the default bars; under loose bars also its neither-bar resolutions), which is the counting Corollary I.27 consumes.

Commentary (non-normative; role of the attempt unit). The attempt is the analysis unit at which the per-submission safety bound is conditioned. Banked decisions credited to an entry precede the entry’s opening, and the owner — or an adversary

— may decide whether to open the entry *as a function of* the banked outcomes; a bound conditioned at the entry’s opening would therefore condition on an adversarially selected event (an optional-selection error). The attempt’s anchor precedes every decision creditable to its entries by construction, so conditioning at the anchor is sound; and an attempt whose intervals bank decisions is an attempt whether or not a further entry opens, so the choice not to open is a choice among attempts already counted, at success probability zero. No protocol rule reads attempts; like the ledger’s charging convention (Lemma J.7), they are a hindsight bookkeeping device over the same window/interval alternation the ledger’s buckets partition.

Lemma I.2 (Type-level sharing exclusion). *No T-block, no U-block, no cache-marked block, and no block lacking an accepting publish confirmation by its owner ever enters the shared candidate store.*

Proof. Label and provenance membership are deterministic and attacker-immutable (A1–A2; provenance is interpreter-only, Definition F.2); the publish gate of Rule R12 is a confirmation event under Rule R8 and A3; nomination has no authority. $\square$

Lemma I.3 (Entry disjointness). *For a fixed byte string, the weight-bearing endorsement sets of distinct pipeline entries are disjoint in identities over the deployment’s entire history, and every weight-bearing endorsement (in-entry or banked-and-credited) is its endorser’s fresh, authority-active inspection of the bytes (Definition H.6(a)).*

Proof. By Definition H.6(a), an identity’s weight-bearing endorsement of the bytes is its *first* logged decision on them (Definition H.5), taken while authority-active (Definition H.6(a)), (when credited, Definition H.6(f)), counted toward exactly one entry (the entry open at decision time, or, via banking, Definition H.6(b), the next entry to open). Every later acceptance by the same identity is repeat and carries no weight (INV-1). Hence no identity contributes weight to two entries, and each contribution is by construction a fresh inspection. $\square$

Lemma I.4 (Deterministic accountability bound). *Fix an identity and an epoch. The identity’s dispute-initiating rejections filed within the epoch that both take protocol effect and are eventually charged (contradicted by vindication), in this epoch or any later one, number at most $c_{\text{acc}} := c_{\text{ceil}} + r_{\text{max}}$.*

Proof. By Rule R13(v)(b), a rejection is charged at the moment its vindication is logged. Suppose first that the epoch’s c_{ceil} -th charge to this identity occurs, at some moment T . From T onward, the identity’s further dispute-initiating rejections in the epoch are filed as advisory flags with no protocol effect (Rule R13(v)(d)), hence are never counted. Every counted rejection is therefore either charged no later than T (and the epoch’s charges up to and including T number exactly c_{ceil} , by the choice of T) or still open at T and charged afterward; open rejections number at most r_{max} at any moment (Rule R13(i)). The two classes are disjoint, a rejection being charged exactly once; summing gives $c_{\text{ceil}} + r_{\text{max}}$. If instead fewer than c_{ceil} charges accrue within the epoch, at most $c_{\text{ceil}} - 1$ counted rejections are charged within it, and every one charged in a later epoch is open at the epoch boundary, at most r_{max} again. $\square$

Remark I.5 (Calibration of rejecter accountability). The rate trigger of Rule R13(v)(c) is normalised *per fresh inspection*, never per rejection. The normalisation matters: when poison is rare, almost all of an honest inspector’s rejections are wrongful rejections of benign items and almost all are vindicated, so charges *per rejection* approach one for honest and adversarial identities alike and cannot separate them; charges *per inspection* sit near $1 - p_b$ for an honest identity and near its full rejection rate for a campaign. The ceiling c_{ceil} is calibrated, like the rate trigger, strictly above the honest baseline: an honest identity accrues charges at rate at most $1 - p_b$ per benign fresh inspection, and its fresh inspections per epoch are bounded by its own forced-confirmation stream (small, and shrinking with population, Appendix J), so c_{ceil} set at a high quantile of $(1 - p_b) \times (\text{honest per-epoch fresh-inspection count})$, with margin, is reached by an honest identity only rarely; an honest identity that does reach it retains advisory flags and ordinary use — though its acceptances are authority-inert for the duration (Definition H.6(a), Remark I.18) — loses nothing permanent, is restored at the epoch boundary, and its inspections during that state are excluded trials, not misses (theorem I.19), so no safety statement is affected. Renewal semantics: convention (C4) (the accountability allowances are the only freshness-like quantities that renew). (This calibration concerns the per-fresh-inspection rejection-rate trigger only; suspension-induced forced confirmations are priced separately, once per demander, per Note N14.) Mis-calibration risk: S15.

Lemma I.6 (Verdict pacing under A9). *Under A9, for any disputed item and any adversary strategy (convention (C2)): the number of demanders who arrive while the dispute is unresolved is stochastically dominated by the index of the $(k+1)$ -st success in an i.i.d. Bernoulli(ρ_{seat}) sequence, i.e. by a sum of $k+1$ independent Geometric(ρ_{seat}) variables. In particular its expectation is at most $\kappa := (k+1)/\rho_{\text{seat}}$, and for any $c_0 > 1$ the probability that more than $c_0 \kappa$ demanders arrive while the dispute is unresolved is at most $e^{-\lambda_{c_0} k}$ for a constant $\lambda_{c_0} > 0$ depending only on c_0 and ρ_{seat} .*

Proof. Index trials by arrivals: let O_i be the event that an i -th demander of the item arrives while the dispute is unresolved, measurable at that arrival’s rendering atom, and let Z_i indicate that this arrival’s disposition is an eligible verdict on the dispute instance. Under the atomic reading of §L.1, each arrival’s disposition is realized at its own log position, so the σ -algebra of atoms prior to the i -th rendering atom contains the dispositions and protocol effects of all earlier arrivals; A9 gives $\mathbb{P}[Z_i = 1 \mid \mathcal{G}_i] \geq \rho_{\text{seat}}$ on O_i at exactly these σ -algebras, and Lemma L.2(ii), in the form of Remark L.4, supplies i.i.d. Bernoulli(ρ_{seat}) variables Y_i with $Y_i \leq Z_i$ on O_i . A dispute instance accepts at most k eligible verdicts in its entire life (Rule R13(iii)(e)), and every verdict is cast at an arrival; hence on O_n , $\sum_{i \leq n-1} Z_i \leq k$, so $\sum_{i \leq n-1} Y_i \leq k$, i.e. $n \leq G := \min\{j : \sum_{i \leq j} Y_i = k + 1\}$. The arrival count is therefore pathwise at most G , a sum of $k+1$ independent Geometric(ρ_{seat}) variables. Linearity gives the expectation; the tail is the Chernoff bound for sums of independent geometrics. (Because dispositions resolve at arrival, the sharper count $\sum_{i \leq n-1} Z_i \leq k - 1$ in fact holds while the dispute is unresolved, improving κ to k/ρ_{seat} ; we retain the conservative $\kappa = (k+1)/\rho_{\text{seat}}$, which every downstream constant consumes, the slack being monotone.) $\square$

Lemma I.7 (Decisive resolution under the default bars). *Under the default bars $m_{\text{rej}} = k + 1 - m$ with $m > f_s$ and $m_{\text{rej}} > f_s$: (i) eviction and vindication are mutually exclusive; (ii) every panel that gathers k verdicts reaches exactly one bar, and the first bar reached in log order is the bar reached by the full tally of k verdicts: early resolution changes timing, never the outcome; (iii) both bars are reachable by honest verdicts alone, and the adversary’s at most f_s verdicts (INV-5) alone reach neither; hence no tally deadlock exists once k verdicts are gathered, and neither bar can be reached by the adversary’s verdicts alone; whether k eligible verdicts arrive is the separate, availability-side matter of A9 (Remark I.21).*

Proof. (i)–(ii) $m + m_{\text{rej}} = k + 1$, so k verdicts cannot meet both bars and must meet at least one; verdict counts are non-decreasing along the verdict sequence, so a bar is reached at some prefix iff it is reached by the full tally. (iii) From $k = m + m_{\text{rej}} - 1$ with $m, m_{\text{rej}} > f_s$: $k - f_s \geq m$ and $k - f_s \geq m_{\text{rej}}$, so honest verdicts alone suffice for either bar; and $f_s < m$ means the adversary’s at most f_s verdicts alone cannot reach the m -bar, and symmetrically $f_s < m_{\text{rej}}$ for the other bar. $\square$

Lemma I.8 (Third-party immutability of local stores). *For every user u and every block b in u ’s local store (the user-owned tier: T-, C-, PC-, and U-blocks; community status lives in the shared tier and is referenced, never stored, locally, §H.1), no protocol event initiated by any identity other than u alters bytes(b), $\ell(b)$, or scope(b). Dispute, suspension, vindication, demotion, eviction, expiry, and every accountability penalty of Rule R13(v),(vii) act on the shared tier alone: they invalidate or flag shared entries, restrict shared-tier rights, or reduce attestation weight. Demotion of a CC item withdraws community authority from users who have not personally confirmed it, whose gates simply fire again, within the single-user cap of Theorem J.2, and leaves every personally confirmed C copy intact (with a notification). Suspension likewise withdraws community authority for the review’s duration (Rule R13(vi)), equally without touching any local block. Consequently no third-party protocol transition ever directly alters or removes any block of any user’s local store; first-party removal by the owner’s own agent (evict, including evictions the agent performs in reaction to notifications or other legitimate log events) is the disclosed residual N13. The induced cost bound, that no campaign pushes any user above the single-user cap, is Theorem J.2, which rests on the present lemma.*

Proof. By Figures 7–8, local stores mutate only via owner-invoked Σ -operations and owner confirmations (A1–A2); evict is owner-only (first-party eviction by the owner’s own agent is a disclosed residual: N13, §K.4). Community status is shared-tier state (§H.1), so demotion and suspension update the shared entry and never touch a local block; promotion, demotion, and expiry are interpreter relabels or retirements of the shared log entry (Rules R13, R14); the accountability penalties of clauses (v) and (vii) act on rejection rights, submission rights, and attestation weight, none of which is a local-store attribute; the cascade quarantines shared derivations, not local stores; notifications are advisory. $\square$

Lemma I.9 (Hygiene preserves safety through bound uniformity). *The policies of Rule R14 preserve every theorem of this document. The entry cap only refuses entry openings, and retirement and demotion hygiene only remove shared-tier items; none of these can enable a promotion. Surfacing reordering is not monotone by event-set inclusion — reordering, and removal of one candidate, can increase another item’s reach within the c_{cand} -capped surface — but the safety bounds are unaffected because Theorems I.11 and I.13 are uniform over adaptive presentation schedules and reach-free (INV-6): any demand redistribution hygiene induces is one more presentation schedule inside the quantified class. Lemma I.8 is unaffected because no local store is touched; the ledger of Appendix J is unaffected at the moment of any hygiene action, which adds no confirmation event. Retirement does redirect later demand: a user who encounters the retired bytes afterwards through their own channels confirms them as ordinary personal content, in the single-user stream (Definition H.4), bounded by the cap (Theorem J.2), its benign, A11-conforming*

component vanishing in expectation under the vanishing conditions of Theorem J.9(iii) and the remainder entering $r_{\text{bad}}^{(e)}$ (Corollary J.10); the shared-tier ledger rows are never increased by removal or reordering.

Proof. Each policy is a deterministic interpreter action on the shared tier: the entry cap refuses an entry opening, retirement removes a candidate, surfacing order permutes at most c_{cand} surfaced candidates, demotion hygiene removes an evicted item from retrieval. None relabels any local block, forces any confirmation, or adds any promotion path. For the safety theorems, the relevant quantification is over adaptive presentation and demand schedules (convention (C2)); the schedules realizable under any hygiene policy are a subset of that class, and the bounds of Theorems I.11 and I.13 are uniform over it and independent of reach, so they apply verbatim to the hygiene-modified deployment. The post-retirement reclassification is Definition H.4’s retirement clause. $\square$

I.2 MAIN THEOREMS

Proposition I.10 (Delegated Action Integrity and federated Reasoning Integrity). *Assume A7 and A8, and read the interpreter over federated product configurations (Definition H.1) under the federated step of Definition H.2, with Definition G.3(iii) in its federated form (every virtual CC authority in force backed by a currently valid promotion certificate on L). Then:*

- (i) *a value flows on community authority without personal confirmation only if it byte-equals a live virtual entry naming the exact tool (Rule R4(a), live-entry test), each such entry backed at that tool by qualifying fresh endorsements (Definition H.6, Lemma G.14), each satisfying A3–A4 or A5 for its endorser and authoritative by A7: PerMIT has delegated Action Integrity (Definition G.10);*
- (ii) *under A8 the coupled log slots deliver equal prefixes, and locally emitted federation events are equal across matched runs (Definition H.2), hence equal virtual views $\text{LiveCC}(L)$ at every step; the matched relation of Definition G.6, read over $\mathcal{M}_{\mathbb{E}}^{\text{fed}}$ and extended by equality of delivered log prefixes, is preserved by log-slot transitions — which emit no reply and alter no matched component except the coupled virtual view — and by every local transition exactly as in Theorem G.25; hence Theorem G.25 and Corollary G.23 lift to the product system with “C” read as “C or live CC”;*
- (iii) *suspension, demotion, retirement, and every accountability event act on L alone: they withdraw virtual authority and never create, alter, or remove any local block (Lemma I.8).*

Proof. (i) The single-user results are personal (Lemma G.16); community authority enters only through the live-entry test, whose backing endorsements are fresh accepting confirmations at the named tool (Definition H.6(c), Lemma G.14), authoritative by A7; the scope test is the same singleton-membership test as for C, and handle gating is label-independent (Rule R4(c)). (ii) Log-slot transitions: both runs append the same coupled prefix (A8), producing equal $S_{\text{sh}}(L)$ by deterministic replay (Definition H.1), appending nothing to either reply stream and touching no local store (Definition H.2, Lemma I.8). Locally emitted federation events extend the two logs equally, by cases. (1) submit(v): the publish decision is a transcript event (matched) on bytes of a local C-block of the owner ($\mathcal{M}_{\mathbb{E}}$ matched). (2) Protocol-bearing gate decisions: displayed bytes, target, and decision are transcript-pinned; shared-presence status, coverage, cache classification, freshness, and authority are deterministic predicates of the matched local store and the equal delivered prefix (Definitions H.5, H.6, Rule R13(i),(v)). (3) Realized demand events: fired by the matched operation against the matched federated view, the acting identity common to both runs, and epoch-first-demand status a deterministic predicate of the equal delivered prefixes (Definition H.4). By induction on steps: equal delivered prefixes give equal $\text{LiveCC}(L)$; together with the matched-local-state induction hypothesis and the single-user unwinding (Theorem G.25), this gives matched local transitions, and the case analysis then gives equal appended suffixes, hence equal delivered prefixes at the next step. Local transitions: the unwinding of Theorem G.25 runs verbatim with $\mathcal{M}_{\mathbb{E}}^{\text{fed}}$ for $\mathcal{M}_{\mathbb{E}}$ — the CC branch of the gate is a function of the argument list and the step’s delivered coupled log prefix (Definition H.2), matched across the runs, so the gate-or-no-gate decision of Step 5 remains matched. (iii) is Definition H.1 with Lemma I.8. $\square$

Theorem I.11 (Deterministic safety under perfect honest detection). *Under A1–A5 and Rules R12–R13, with perfect honest detection ($q = 1$: every honest inspection of the poisoned item rejects): if $W > W_{\text{adv}}$ and $m > f_s$, then no poisoned item (Definition H.10) is ever promoted to CC at any tool, at any population size and under any attack, through either the ordinary or the panel path.*

Proof. With $q = 1$, every accepting endorsement and every accepting panel verdict on a poisoned item is adversarial. The ordinary path requires accepting weight at least W at the tool in question, while the adversary’s weight is at

most W_{adv} and $W > W_{\text{adv}}$: impossible. The panel path requires at least m accepting verdicts, while the adversary casts at most f_s verdicts (INV-5), $m > f_s$, and honest accepting verdicts are zero: impossible. Vindication-credited and dormancy-banked endorsements are covered identically: with $q = 1$ every honest decision on the poison rejects, so every credited acceptance is adversarial and already counted inside W_{adv} (respectively inside the f_s verdict cap). Deterministic and population-free. $\square$

Remark I.12 (Scope of Theorem I.11). The hypothesis $q = 1$ is a behavioural idealization, the strongest in this document, not the absence of one: the Sybil conditions $W > W_{\text{adv}}$, $m > f_s$ exclude *adversarial* weight, while honest acceptances of the poison are excluded only by $q = 1$ itself. The theorem is therefore *not* a fallback for the unrecognizable-poison regime $q \rightarrow 0$, where honest inspectors themselves supply the threshold and no community-truthfulness guarantee is available within a human-judgment design. What survives that regime is the local layer — personal Action Integrity, the single-user cap, and no-deletion (Theorem G.21, Theorem J.2, Lemma I.8) — together with the one-sided detection clause (a single honest *effective qualifying rejection*, whenever one occurs, freezes promotion immediately, Rule R13(i),(v)) and re-verification against never-decided inspectors (N16). See S14.

Theorem I.13 (Panel safety under adaptive presentation). *Adopt Rules R12–R13 with the default bars of Rule R13(iv) ($m_{\text{rej}} = k+1 - m$, $m > f_s$, $m_{\text{rej}} > f_s$; Lemma I.7) and the probabilistic framework of §L.1. Suppose:*

- (a) *ordinary promotion at a tool requires N distinct accepting endorsements counted toward one pipeline entry (Definition H.6), with $N > f$;*
- (b) *at most f_s of the adversary’s identities are seat-eligible, with $m > f_s$;*
- (c) *there are $q_{\text{pre}}, q_{\text{pan}} \in (0, 1]$ such that: each honest fresh, authority-active (Definition H.6(a)) inspection of the poisoned item at a non-covered, non-cache confirmation gate ends in an effective qualifying rejection — a recorded rejecting decision that takes protocol effect under Rule R13(i) — with conditional probability at least q_{pre} ; and each honest eligible verdict on the item rejects with conditional probability at least q_{pan} , conditionally on the verdict being cast, i.e. at the verdict’s header-inclusive σ -algebra (§L.1) (Rule R13(iii)). Both bounds are read per the conditioning convention of §L.1 (all prior logged atoms; the demand realisation), and both are uniform lower bounds over the honest identities and presentation contexts the adversary can target — the catch rate of the most fallible targetable honest cohort, not a population average (theorem I.19) — q_{pre} ’s contexts are the unfrozen candidate contexts — in-window and candidate-dormancy inspections, any advisory-flag state — where an effective qualifying rejection is possible (Rule R13(i),(ii)); freeze- and suspension-time fresh inspections carry no initiation and belong to q_{pan} ’s province where verdict-eligible, their non-verdict rejections being corroborations, consumed by no trial of this theorem. q_{pan} ’s contexts include the freeze itself: verdicts are cast while the item is frozen. Both bounds are read over the target contexts (tools and reasoning) in which the trials actually occur: q_{pan} is a uniform bound over the tool contexts of eligible jurors, which under byte-level panels may include tools at which the item is innocuous — for tool-targeted poison the applicable q_{pan} may therefore be small (S4). Authority-inert inspections, covered gates, and cache gates (Options O1–O2) belong to no trial class of this theorem: an authority-inert acceptance carries no promotion weight and an authority-inert rejection no initiation (Definition H.6(a)), covered gates are protocol-inert on both branches (Rule R13(i), theorem I.19), and cache gates are accept-inert and verdict-ineligible (Definition H.6(iii), Rule R13(iii)) — their rejecting branch is one more effective qualifying rejection, which can only deny the case-(a) path — so promotion still requires $N - f$ honest authority-active fresh non-cache credits, and q_{pre} is quantified over exactly the class whose members can both accept with weight and reject with effect. Abstention by an authority-active fresh inspector records nothing and is counted as a miss inside $1 - q_{\text{pre}}$; the two parameters deliberately live at different conditioning levels — q_{pre} at the disposition of a fresh inspection, where abstention must count as a miss; q_{pan} at a cast verdict, where abstention is already excluded by eligibility — and $q_{\text{pre}} \leq q_{\text{pan}}$ is typical but not assumed;*
- (d) *the panel margin is positive: $\delta := \frac{m-f_s}{k-f_s} - (1 - q_{\text{pan}}) > 0$.*

Then for any poisoned (Definition H.10) submission, every promotion attempt (Definition I.1) succeeds with probability at most the display below, conditionally on the σ -algebra generated by all atoms up to and including the attempt’s anchor atom, uniformly over adaptive presentation, stopping, and re-entry strategies and regardless of how many users the item is shown to (INV-6),

$$\mathbb{P}[\text{promotion}] \leq (1 - q_{\text{pre}})^{N-f} + \mathbb{P}[\text{Bin}(k - f_s, 1 - q_{\text{pan}}) \geq m - f_s] \leq (1 - q_{\text{pre}})^{N-f} + e^{-2\delta^2(k-f_s)}.$$

Remark I.14 (Scope of Theorem I.13). Theorem I.11 is the special case $q = 1$. The bound covers a single promotion attempt (Definition I.1); for attempts anchored while the bytes hold live CC status at some tool (additional-tool promotion of already-promoted bytes, possible only after a prior success), hypothesis (c) is read with q_{pre} ’s contexts extended per the closing paragraph of the proof; first-promotion attempts need no extension; entries embed injectively into attempts, so per-entry counting statements read a fortiori; attempts are counted at their anchors, and the

per-anchor-epoch union bound taken, in Corollary I.27. No hypothesis is placed on repeat inspections: none carries promotion weight or a verdict (INV-1, INV-5; Remark H.7). The panel arithmetic rests on INV-5 alone, which is stated independently of the freshness rule precisely so that it never depends on INV-1. The default bars are a hypothesis of the theorem: case (b) uses decisive resolution (Lemma I.7) to cap honest verdicts at $k - a$. Under the looser bars permitted by Rule R13(iv) ($m + m_{\text{rej}} \geq k + 2$), the same argument bounds the vindication event verbatim — vindication still requires m accepting verdicts among at most k , so the case-(b) term stands with $k - f_s$ trials — and the additional neither-bar outcome returns the item to unpromoted candidate status, whose re-entry opens a new attempt with an identity-disjoint credited set, counted separately and marginally by Corollary I.27. The bound therefore extends to loose-bar deployments per entry; only the decisiveness clauses (mutual exclusivity, tally-deadlock-freeness) are specific to the defaults.

Remark I.15 (Single-parameter reading). Every appearance of a single q elsewhere in this document (Theorem I.11’s hypothesis $q = 1$; the corridor row $q + p_b > 1$; Corollary I.27; the composed summary of §K.5; the correlated rate q_c of S13) is read as $q := \min(q_{\text{pre}}, q_{\text{pan}})$: the bounds of Theorem I.13 are monotone in each parameter, so the single- q forms are the special case $q_{\text{pre}} = q_{\text{pan}} = q$ and remain valid verbatim. $q = 1$ means both acts are perfect; $q \rightarrow 0$ (S14) is the regime where the smaller of the two vanishes.

Remark I.16 (Itemwise reading; heterogeneous item difficulty). Theorem I.13 is stated and proved per promotion attempt (Definition I.1): nothing in its trials couples distinct items, so its hypotheses and bound are *itemwise* — for each poisoned item v , the bound holds with v ’s own weakest-target rates $q_{\text{pre}}(v)$, $q_{\text{pan}}(v)$, and it is monotone in each. Heterogeneous item difficulty is therefore inside the model, transparently: easily recognized poison contributes a negligible term; near-unrecognizable poison ($q_{\text{pre}}(v) \rightarrow 0$) degrades its own term toward the disclosed residual S14, leaving every other item’s bound untouched. The per-anchor-epoch union bound of Corollary I.27 holds in the same itemwise form, $\sum_e [(1 - q_{\text{pre}}(v_e))^{N-f} + e^{-2\delta(v_e)^2(k-f_s)}]$ (terms with $\delta(v_e) \leq 0$ read as the exact binomial tail), the single- q display being the uniform special case; a deployment-level constant is thus dominated by the hardest items the adversary can mint — exactly the calibration duty of S1. No parametric ability-or-difficulty model (jury, psychometric, or crowdsourcing) is assumed: the weakest-target conditional floor subsumes any such structure, at the price that the floor, not the average, is the deployed constant (theorem I.19, N17).

Remark I.17 (The certified per-attempt ceiling B_{att}). Write

$$B_{\text{att}} := (1 - q_{\text{pre}})^{N-f} + \mathbb{P}[\text{Bin}(k - f_s, 1 - q_{\text{pan}}) \geq m - f_s]$$

for the right-hand side of Theorem I.13’s bound in its exact binomial form, and call its two summands the *ordinary term* and the *panel term*. The word *certified* names this analytic bound and is unrelated to the promotion certificates of Definition H.1, which are log objects. Three properties are used throughout the empirical evaluation: the first two are immediate from the display, the third from the theorem’s statement.

The panel term is a *floor*: at fixed panel geometry (k, m, f_s) and fixed q_{pan} it does not contain N , so raising the endorsement quorum shrinks only the ordinary term and B_{att} can never fall below it — panel geometry, not quorum, is the binding deployment parameter, which is the frontier Proposition I.20 reads against and which the corridor of §J.5 moves only by scheduling k itself. The two terms fall in *different* protocol constants and must be pushed down independently: the ordinary term geometrically in $N - f$, the panel term as a binomial tail in $k - f_s$ whose Hoeffding relaxation $e^{-2\delta^2(k-f_s)}$ is exponential in that constant. And B_{att} is per promotion attempt (Definition I.1) and reach-free (INV-6); deployment-wide exposure is obtained by summing over anchored attempts and byte-distinct variants, through the union, expectation and deterministic-cap forms of Corollary I.27 and the lifetime ceiling of Remark I.29, never by reading the per-attempt figure as a per-epoch one.

Reporting discipline. Any numerical value quoted for B_{att} is the exact binomial tail above, never the Hoeffding relaxation that Theorem I.13 displays alongside it. The two differ materially, and by different factors on the two sides of the design: at $k = 31$, $m = 20$, $f_s = 1$ with $q_{\text{pan}} = 0.6$ the relaxation is 3.81×10^{-2} against an exact panel term of 8.30×10^{-3} , a factor of about 4.6; on the benign side of Proposition I.20, at $p_b = 0.9$ and the same δ , the numerically identical relaxation stands against an exact wrongful-eviction tail of 8.91×10^{-5} , a factor of about 430. A reader recomputing either relaxation from a quoted exact value will therefore not reproduce it, and — because the two relaxations coincide numerically while their exact counterparts differ by two orders of magnitude — the two slacks must never be interchanged.

Proof of Theorem I.13. Fix the attempt and condition on the σ -algebra through and including its anchor atom (Definition I.1). The behavioural bounds of hypothesis (c) are asserted uniformly over histories, so the argument restarts after any fixed anchor-inclusive history; every trial charged below occurs strictly after the anchor, and each trial’s own conditioning σ -algebra (§L.1) refines the anchor-inclusive one. Partition by whether an effective qualifying rejection was logged after the anchor and before the attempt’s finalisation.

Throughout, the bytes’ promotion status at every tool is constant within the attempt: promotion occurs only at a finalisation, which ends the attempt, and demotion by eviction ends the presence (Definition I.1, Definition H.6(c)), so whether the bytes hold live CC status at any tool is decided at the anchor. Suppose first that they do not — the case of every attempt preceding the bytes’ first promotion, hence of every attempt the ever-promoted events of Corollary I.27 require; the complementary case is scoped at the close of the proof. Then no suspension can arise within the attempt (suspension presupposes promoted bytes, Rule R13(vi)), every qualifying rejection on the bytes within it is dispute-initiating (Rule R13(i),(ii)), and every non-freeze piece of the attempt is an entry window or a candidate-dormancy interval (presence taxonomy, Lemma J.7): every unfrozen header within the attempt is an *unfrozen candidate context* in the sense of hypothesis (c).

(a) *Promotion with no qualifying rejection.* By Rule R13(i),(v), an honest *authority-active* identity’s rejecting gate decision on an unfrozen candidate takes protocol effect, so the case-(a) event entails that no honest authority-active inspection of the item ended in an effective qualifying rejection before the attempt’s finalisation. Promotion at the finalisation requires N credited accepting endorsements at the finalising entry (in-window, interval-banked, boundary-banked, or vindication-credited, Definition H.6), of which at least $N - f$ come from honest identities; each is a distinct honest identity’s fresh, authority-active, non-covered decision on the bytes (Lemma I.3, Definition H.6(a)), and each is logged strictly after the attempt’s anchor and before its finalisation (Definition I.1).

Index the trials by log order within the attempt: let O_i be the event that an i -th protocol-bearing decision at a non-covered, non-cache gate on the item, by an honest, fresh, authority-active identity, is logged after the attempt’s anchor and before the attempt ends, at an unfrozen header — by the opening reduction, an unfrozen candidate context in the sense of hypothesis (c). O_i is measurable at that record’s header atom (§L.1): attempt membership is decided by the anchor, a past log event, and freshness, coverage, authority, and frozen status are deterministic predicates of the prior log (Definitions H.5, H.6(a), Rule R13(i),(ii)) — no trial’s occurrence depends on the attempt’s future. Because no event interleaves between a position’s rendering and header atoms (§L.1), the status at the header is the status under which hypothesis (c)’s disposition-level bound is read. Let Z_i indicate that the trial’s content is an effective qualifying rejection. Hypothesis (c) lower-bounds by q_{pre} the disposition-level probability of such a rejection at the trial’s rendering atom, on all of O_i and irrespective of whether case (a) occurs — the class is confined to exactly the unfrozen candidate contexts over which q_{pre} is quantified; conditioning on a recorded decision only raises the bound (Lemma L.1(iii)). Hence $\mathbb{P}[Z_i = 1 \mid \mathcal{G}_i] \geq q_{\text{pre}}$ on O_i at the header-inclusive σ -algebras of the conditioning convention (§L.1), each of which refines the anchor σ -algebra, so Remark L.4 applies under the anchor-conditional measure directly.

Only now is the case-(a) event used. On it, no effective qualifying rejection is logged after the anchor and before the attempt’s finalisation; since no dispute or review is open at the anchor (Definition I.1) and every freeze arising within the attempt presupposes such a rejection, every trial of the class preceding the finalisation occurs at an unfrozen header, and the $N - f$ honest credited endorsements — in-window, interval-banked in a candidate-dormancy interval, or boundary-banked, by the opening reduction; a vindication-credited endorsement presupposes a panel, hence a case-(b) run — are all logged at trials of the class, each necessarily non-cache (cache acceptances are never logged, hence never credited, Definition H.6(iii)). The case-(a) success event is therefore contained in $O_{N-f} \cap \{Z_1 = \dots = Z_{N-f} = 0\}$: at least $N - f$ trials of the class are logged before the finalisation, so the first $N - f$ trials in log order all precede it, and each has $Z_i = 0$, an effective qualifying rejection before finalisation being a case-(b) run. The joint all-zeros bound of Lemma L.2(ii), in the σ -algebra form of Remark L.4, gives $\mathbb{P}[\text{case (a)} \mid \text{anchor}] \leq (1 - q_{\text{pre}})^{N-f}$. Additional reach creates both rejection opportunities and possible weight-bearing acceptances (INV-2); no pathwise monotonicity in reach is claimed. What the bound uses is only the $N - f$ honest fresh authority-active trials promotion itself requires, so the stated bound is unchanged at every reach (INV-6).

(b) *Promotion after a qualifying rejection.* The byte-level freeze then blocks every tool’s pipeline until an accepting panel resolution (Rule R13(ii)–(iv),(vi)), so the case-(b) success event is contained in the accepting outcome (vindication, or reinstatement under Rule R13(vi)) of the first panel instance opened within the attempt — an ordinary dispute panel or a fast-track suspension review, with the same k, m, m_{rej} parameters and bars; that panel instance opens, and casts all its verdicts, after the attempt’s anchor (Definition I.1), so the argument below runs under the anchor-conditional measure verbatim: the accepting-verdict count must reach m . Suppose the adversary casts $a \leq f_s$ accepting verdicts (at most one per identity for the entire life of the panel instance (INV-5, read over the instance; Rule R13(iii),(vi)), across any number of epoch boundaries, by INV-5; seat floor and pool caps per Rule R13(iii); abandonment casts none). An accepting resolution then requires $m - a$ honest accepting verdicts; with the default bars, at most k verdicts are cast before a bar is reached (Lemma I.7), so at most $k - a$ are honest. Each honest verdict is a recorded decision at a fresh inspection. By the atomic decomposition of §L.1, the event that an i -th honest verdict is cast is measurable at that verdict’s header atom, strictly before its content atom, and honest verdicts are log-ordered with every earlier verdict’s content preceding every later verdict’s header. By hypothesis (c), each honest verdict rejects with conditional probability at least q_{pan} at its header-inclusive σ -algebra (§L.1); equivalently, its

conditional accept probability there is at most $1 - q_{\text{pan}}$ (a cast verdict’s content is binary). Apply Lemma L.2(i), in the σ -algebra form of Remark L.4 at those σ -algebras, to the accept indicators Z_i of the honest verdicts: there are i.i.d. Bernoulli($1 - q_{\text{pan}}$) majorants $Y_1, \dots, Y_k$ with $Z_i \leq Y_i$ pointwise, unrealised verdicts contributing zero, so the early decisive truncation of Rule R13(iv) removes only nonnegative terms (Lemma L.2(ii)). On the case-(b) event, $m - a \leq \sum_{i \leq k-a} Z_i \leq \sum_{i \leq k-a} Y_i \leq \sum_{i \leq k-f_s} Y_i + (f_s - a)$, whence $\sum_{i \leq k-f_s} Y_i \geq m - f_s$: the case $a = f_s$ dominates, uniformly in a , and $\sum_{i \leq k-f_s} Y_i \sim \text{Bin}(k - f_s, 1 - q_{\text{pan}})$. Hoeffding’s inequality (Lemma L.8) at the margin δ gives the second term. The events are disjoint; sum.

Reach-freeness. The bound of case (a) is independent of reach: it counts only the $N - f$ honest fresh decisions promotion itself requires, however many further inspections occur; in case (b), k and m are protocol constants, viewers whose decisions are not verdicts do not vote, and logged-repeat viewers cannot vote at all (INV-1). An expired dispute retires the item unpromoted (Rule R14); republication is a fresh submission whose endorsement set is identity-disjoint from all prior sets (Lemma I.3), counted separately and marginally (Corollary I.27). The case-(b) bound holds regardless of who filed the rejection, so self-dispute gains nothing (S2). Finally, the adversary’s choice whether to open an entry after favourable banked outcomes is a choice among attempts already counted: an attempt whose intervals bank decisions is an attempt whether or not the entry opens, and an unopened attempt succeeds with probability zero (Definition I.1).

Attempts anchored with live CC status. For an attempt at whose anchor the bytes hold live CC status at some tool — possible only after a prior successful attempt of the same bytes, itself bounded marginally at its own anchor — the inter-window spans are stable promoted intervals, first decisions banked there are creditable (Definition H.6(b), Lemma J.7), and a fresh inspector’s rejection there is a post-promotion rejection routed to Rule R13(vi) rather than a dispute initiation. The two-case argument applies to such attempts under a correspondingly extended reading of hypothesis (c): q_{pre} quantified over the attempt’s unfrozen contexts *including* its stable promoted intervals, with the effective qualifying rejection there being the review-opening post-promotion rejection of Rule R13(vi) within its per-epoch review cap — an extension a deployment must calibrate separately, disclosed here (cf. S4). The event that any byte string is ever promoted *at all* is a union of first-promotion events, whose attempts are anchored CC-free and need no extension. $\square$

Remark I.18 (Authority-gated crediting: soundness and cost). The authority gate of Definition H.6(a) restores the quantifier compatibility that deterministic silencing would otherwise break: without it, an adversary steering presentations toward identities whose rejection rights are suspended, ceiling-capped, or slot-exhausted would face inspectors whose effective rejection probability is *zero* while their acceptances still carried full promotion weight, and no $q_{\text{pre}} > 0$ could satisfy hypothesis (c) uniformly over targetable cohorts. With the gate, the two capacities are coupled at one deterministic log predicate: a silenced identity can neither reject with effect nor accept with weight, so targeting it gains the adversary nothing. The split is exact: verdict *capacity* is never authority-gated (Rule R13(v)(a),(e)) — an authority-inactive juror’s accepting verdict counts toward the m -bar like any other, keeping the panel arithmetic of Theorem I.13 and Proposition I.20 calibration-independent — while promotion *crediting* always is (Definition H.6(a)): the same verdict contributes weight only if its caster held authority at decision time, in-entry, banked, and vindication-credited cases alike. Honest cost is negligible: an honest identity is authority-inactive only in the rare calibrated states of theorem I.5, and its acceptance in that state still buys its personal C copy — INV-3 is unaffected, the cap counting forced confirmations, never credits. Safety is monotone: authority-gating removes acceptances from the promotion count and removes nothing from the rejection side (rejections by inactive identities remain advisory flags or corroboration exactly as before, and verdict-eligible rejections are cast regardless of accountability state, Rule R13(v)(e)); Theorem I.11 is unaffected (its counting is adversarial-weight only), q_{pan} needs no gating (verdicts are never silenced, Rule R13(v)(a),(e)), and verdict eligibility, hence A9, is untouched. One availability-side consequence is disclosed: an adversary who engineers charges against honest identities (S15) now also delays their *crediting* capacity, slowing benign promotion — an exposure of the same character, availability-only, bounded by the same calibration margins (theorem I.5) and priced inside $\varepsilon(M)$ where it matters (A11, heuristic sufficiency).

Remark I.19 (Interpretation of the parameter q). q_{pre} is the probability that an honest inspector, presented the item at their own confirmation gate, detects the poison *and records a rejecting decision that takes protocol effect* (quantified over unfrozen candidate contexts, Theorem I.13(c)); q_{pan} is the probability that an honest *eligible juror*’s cast verdict rejects it. The split matters for calibration: the two acts have different filters (seat weight, pool caps, and verdict capacity apply only to jurors) and different honest baselines, so their log validation is stratified separately (N17); a deployment estimating one number reads it as $q := \min(q_{\text{pre}}, q_{\text{pan}})$, under which every bound of Theorem I.13 holds with q in both roles (theorem I.15). Both parameters are *uniform, weakest-target, conditional* lower bounds (§L.1: conditional on all prior logged atoms and the demand factor): because the adversary chooses whom to present the item to, each is the catch rate of the most fallible honest cohort the adversary can reach, never a population average, and aggregate log rates do not directly estimate them (N17). Because a gate rejection is automatically qualifying (Rule R13(i)), detection and reporting are a single act: there is no separate filing step whose friction could decouple

them. Abstention (a user who suspects the item and abandons the dialog) records nothing and is counted as a miss inside $1 - q$. An honest identity whose rejection rights are suspended, whose $r_{\max}$ slots are transiently full, whose per-epoch ceiling c_{ceil} is reached, or whose single per-item initiation is spent is *authority-inactive*: its inspections are excluded trials, not silenced misses — its acceptance carries no promotion weight and its rejection no initiation (Definition H.6(a), Remark I.18) — and such states are rare for honest identities (honest charges accrue at rate at most $1 - p_b$ per benign fresh inspection, both the rate trigger and the ceiling calibrated strictly above that baseline, theorem I.5; slots free on resolution or expiry, Lemma I.24; the ceiling and the per-item allowance renew each epoch). Covered gates (Rule R13(i)) are not silenced misses but *excluded trials*: both branches are protocol-inert, so they belong to no trial class of Theorem I.13, and the identity’s inspection of the bytes, where one is counted at all, is the earlier decision that created its coverage; so, symmetrically, are authority-inactive inspections (Definition H.6(a)), and so are cache gates on the accepting side (accept-inert, Definition H.6(iii)), whose rejections take effect as dispute initiations or corroborations but never as verdicts (Rule R13(iii)). Freshness being shared-presence-scoped (Definition H.5), the fresh class can include identities with a prior *unlogged* personal decision on the bytes (before first publication, or across a retirement gap); q_{pre} and q_{pan} are lower bounds over this log-defined class, so a previously fooled but log-fresh inspector is inside the class the parameters must cover — one more reason the bounds are weakest-target infima, never averages, and one the validation strata of N17 must include (prior-exposure stratification). Case (a) of Theorem I.13 needs only $q_{\text{pre}} > 0$: one honest *effective qualifying rejection* — a rejecting decision at a protocol-bearing gate by an identity currently holding dispute-initiation authority (Rule R13(i),(v)) — denies the ordinary path, a one-sided disjunction, not a vote; coverage-silenced and authority-inactive detections are excluded trials rather than misses (Definition H.6(a)), the corresponding acceptances carrying no weight; post-promotion rejections beyond the review cap of Rule R13(vi) concern the suspended state only and touch no case-(a) trial. The panel is a bounded jury: joint safety and a low wrongful-eviction bound need the better-than-chance condition $q_{\text{pan}} + p_b > 1$; safety alone can be forced for any $q_{\text{pan}} > 0$ by raising m toward k , at the price of a weaker wrongful-eviction bound. The regime $q \rightarrow 0$ is the disclosed limitation S14. The exponents $N - f$ and $k - f_s$ are protocol constants; the bases $1 - q_{\text{pre}}$, $1 - q_{\text{pan}}$ are behavioural parameters of the targeted population. Because panel verdicts are never penalised and first charges are free each epoch (Rule R13(v)), the protocol attaches no penalty to a first good-faith rejection, so it does not disincentivise the gate behaviour that generates q_{pre} . Finally, q_{pre} and ρ_{seat} condition on the same human act consistently: an abstaining demander is a miss inside $1 - q_{\text{pre}}$ and a non-verdict inside ρ_{seat} ; high abstention degrades resolution speed, never the validity of the safety parameter.

Proposition I.20 (Wrongful-eviction bound). *Adopt the default bars, and suppose each honest juror’s fresh inspection of a benign item (Definition H.10) ends in an accepting decision with conditional probability at least p_b given all prior protocol events, read per the conditioning convention of §L.1. If $\delta_b := p_b - \frac{m}{k-f_s} > 0$ (the wrongful-eviction margin), then, uniformly over adversary strategies,*

$$\mathbb{P}[\text{wrongful eviction}] \leq e^{-2\delta_b^2(k-f_s)}.$$

Moreover both margins ($\delta > 0$ and $\delta_b > 0$) are simultaneously achievable by an integer m whenever $(k - f_s)(q + p_b - 1) > f_s + 1$; in particular, whenever inspectors discriminate better than chance ($q + p_b > 1$) and k is large enough.

Proof. Under the default bars, a resolved panel’s outcome is eviction or vindication, exclusively (Lemma I.7(i)–(ii)), and eviction requires $m_{\text{rej}} = k + 1 - m$ rejecting verdicts. Suppose the adversary casts $a \leq f_s$ verdicts (at most one per identity for the dispute’s entire life, INV-5); at most a of them reject, so eviction requires at least $m_{\text{rej}} - a$ honest rejecting verdicts, while honest verdicts number at most $k - a$. For an honest verdict, writing $p_{\text{acc}}, p_{\text{rej}}$ for its conditional accept/reject disposition probabilities at its rendering atom, with $p_{\text{acc}} \geq p_b$ and $p_{\text{acc}} + p_{\text{rej}} \leq 1$ (§L.1), Lemma L.1(ii) bounds the conditional rejection probability at the verdict’s header-inclusive σ -algebra by $1 - p_b$; these σ -algebras satisfy the occurrence and measurability conditions of Remark L.4 exactly as in Theorem I.13(b). Lemma L.2(i), in that form, applied to the rejection indicators Z_i , gives i.i.d. Bernoulli($1 - p_b$) majorants Y_i with $Z_i \leq Y_i$ pointwise, unrealised verdicts contributing zero. On the wrongful-eviction event, $m_{\text{rej}} - a \leq \sum_{i \leq k-a} Z_i \leq \sum_{i \leq k-a} Y_i \leq \sum_{i \leq k-f_s} Y_i + (f_s - a)$, whence $\sum_{i \leq k-f_s} Y_i \geq m_{\text{rej}} - f_s$, uniformly in a . Since $m_{\text{rej}} - f_s = (k - f_s) - (m - 1) = ((1 - p_b) + \delta'_b)(k - f_s)$ with $\delta'_b := p_b - \frac{m-1}{k-f_s} \geq \delta_b > 0$, Hoeffding’s inequality (Lemma L.8) bounds the event by $e^{-2\delta_b'^2(k-f_s)} \leq e^{-2\delta_b^2(k-f_s)}$. Simultaneity: an integer m lying strictly between $f_s + (1 - q)(k - f_s)$ and $p_b(k - f_s)$ exists whenever the gap between those quantities exceeds one, i.e. whenever $(k - f_s)(p_b + q - 1) > f_s + 1$. $\square$

Remark I.21 (Resolution as a demand-side event). Resolution is a demand-side property: the demand-paced panel has no deadline, $g_k(M)$ may grow arbitrarily slowly, and an item without demand expires with no protocol consequence, for which liveness (in the classical, eventual-success sense) is vacuous (theorem I.26). Proposition I.20 and Theorem J.13(ii) bound wrongful eviction unconditionally, asserting no probability for resolution itself. As a corollary reading: conditional on resolution, vindication has probability at least $1 - e^{-2\delta_b^2(k-f_s)} / \mathbb{P}[\text{resolution}]$.

Proposition I.22 (Publisher accountability is safe for benign items). *Under Rule R13(iv),(vii):*

- (i) *the adversary's panel verdicts alone can never trigger a publisher penalty against any publisher, since eviction requires $m_{\text{rej}} > f_s$ rejecting verdicts;*
- (ii) *with the default bars and wrongful-eviction margin $\delta_b > 0$, write $p^* := e^{-2\delta_b^2(k-f_s)}$; each panel on a benign item wrongfully evicts (penalising the publisher) with probability at most p^* conditionally on all prior protocol events (Proposition I.20, whose hypothesis is conditional). Three consequences, in decreasing strength of hypothesis. (Fixed instances.) For any r dispute instances on benign items of the same publisher, predictably specified: membership of an instance in the family is determined no later than that instance's opening atom, measurably in the opening-inclusive log history and before any verdict content of that instance (any advance deterministic choice included); a family selected retrospectively from panel outcomes is not a fixed-instance family — it is covered by the capped tier where the deterministic cap $L \leq \bar{\ell}$ is available, and otherwise only by the linear random-count tier. For such a family, the probability that all r instances end in wrongful eviction is at most $(p^*)^r$, uniformly over adversary strategies and over any dependence among the panels. (Random count, linear.) With L the number of dispute instances opened on the publisher's benign items in the epoch, indexing panel trials at their opening events (an instance unresolved at the epoch horizon evicts nobody and carries indicator zero), the expected number of wrongful evictions is at most $p^* \mathbb{E}[L]$, whence $\mathbb{P}[\text{at least } r \text{ wrongful evictions}] \leq p^* \mathbb{E}[L]/r$ (Markov); $\mathbb{E}[L]$ is bounded by the dispute ledger (Lemma J.7, $\bar{D}_v$) summed over the publisher's items, and, for fast-track reviews, deterministically by $E_{\max}$ per item per epoch (Rule R13(vi)). No r -fold exponent is claimed at this generality. (Random count, capped.) If the deployment enforces a deterministic per-epoch cap $L \leq \bar{\ell}$ on dispute instances against one publisher's items, then $\mathbb{P}[\text{at least } r \text{ wrongful evictions}] \leq \binom{\bar{\ell}}{r} (p^*)^r$, decaying geometrically in r whenever $2\delta_b^2(k - f_s) > \ln \bar{\ell}$;*
- (iii) *every eviction of an adversary-published poisoned item consumes the adversary's allowance and, on repetition within an epoch, its submission rights and attestation weight; eviction of poison published by an honest-but-deceived identity penalises that publisher, by design (S10);*
- (iv) *no publisher penalty ever touches a local store (Lemma I.8).*

Dispute expiry (Rule R14(i)) is neither an eviction nor a vindication and triggers no penalty on either side. Protection is claimed for publishers of benign items: an honest-but-deceived publisher of attacker-crafted content is penalised on eviction, by design (S10).

Proof. (i) Eviction requires at least m_{rej} rejecting verdicts with $m_{\text{rej}} > f_s$; by INV-5 the adversary casts at most f_s verdicts on the dispute over its entire life, however many epoch boundaries it spans, so at least one honest panelist must reject; with no honest rejections there is no eviction and hence no penalty. (ii) Under the default bars a resolved panel's outcome is eviction or vindication, exclusively (Lemma I.7(i)–(ii)), so the penalising event is exactly wrongful eviction, bounded per panel by Proposition I.20. For the fixed-instance product bound, the r predictably specified instances are indexed in opening order, and their honest verdicts jointly in log order of their header atoms; a verdict's instance membership and within-instance honest ordinal are thereby determined at its header atom, $\mathcal{G}_i$ -measurably, as Lemma L.7 requires (§L.1). The benign-item hypothesis of Proposition I.20 bounds each honest verdict's conditional rejection probability by $1 - p_b$ at its header-inclusive σ -algebra — including the outcomes of earlier-resolved panels — so Lemma L.2(i), in the σ -algebra form of Remark L.4, applied once to the joint sequence, supplies coupling variables with $Z_i \leq Y_i$ pointwise on realised verdicts. Allocate cells: assign the j -th specified instance's honest verdicts, in log order, to cells $(j, 1), \dots, (j, k - f_s)$, discarding any beyond the $(k - f_s)$ -th; a verdict's cell is determined at its header atom (its dispute instance and within-instance honest ordinal are log facts prior to its content), hence $\mathcal{G}_i$ -measurably and injectively, the instances' index sets disjoint — not necessarily contiguous, which the argument never needs. By Lemma L.7, the cell array $(X_{j,\ell})_{j \leq r, \ell \leq k - f_s}$ (coupling variables on filled cells, fresh independent uniforms on never-filled cells) is i.i.d. Bernoulli($1 - p_b$). On the all- r -evict event, instance j resolves in eviction: with $a_j \leq f_s$ adversarial verdicts (INV-5), honest verdicts number $h_j \leq k - a_j$, of which at least $m_{\text{rej}} - a_j$ reject. If $h_j \leq k - f_s$, every honest verdict sits among the first $k - f_s$ cells, which then contain at least $m_{\text{rej}} - a_j \geq m_{\text{rej}} - f_s$ rejections; otherwise at most $h_j - (k - f_s) \leq f_s - a_j$ of the rejecting ones lie beyond cell position $k - f_s$, and the first $k - f_s$ honest verdicts contain at least $(m_{\text{rej}} - a_j) - (f_s - a_j) = m_{\text{rej}} - f_s$ rejections. In either case $\sum_{\ell \leq k - f_s} X_{j,\ell} \geq \sum_{\ell \leq k - f_s} Z_{(j,\ell)} \geq m_{\text{rej}} - f_s$, never-filled cells contributing nonnegatively, uniformly in a_j . The all- r -evict event is therefore contained in $\bigcap_{j \leq r} \{ \sum_{\ell \leq k - f_s} X_{j,\ell} \geq m_{\text{rej}} - f_s \}$; the array being i.i.d. with the rows indexed by disjoint cell sets, the intersection's probability is the product of r identical binomial tails, each at most p^* by the margin computation of Proposition I.20 ($m_{\text{rej}} - f_s = ((1 - p_b) + \delta'_b)(k - f_s)$, $\delta'_b \geq \delta_b$, Hoeffding via Lemma L.8), giving $(p^*)^r$. For the linear tier, let W_j indicate wrongful eviction of the j -th instance opened in the epoch, indexed

at its opening event, with $W_j = 0$ for instances unresolved at the horizon; $\mathbb{P}[W_j = 1 \mid \text{prior atoms}] \leq p^*$ by Proposition I.20 (its hypothesis conditions on all prior protocol events, the opening atom included), so the delayed-marks lemma (Lemma L.5, with $\mathcal{G}_j$ generated by all atoms up to the j -th opening event and W_j resolved only at that instance’s resolution, possibly after later openings) gives $\mathbb{E}[\sum_j W_j] \leq p^* \mathbb{E}[L]$ and Markov the tail. For the capped tier, fix any set of r indices $j_1 < \dots < j_r \leq \bar{\ell}$; the fixed-instance argument above, applied to the j_1 -th, $\dots$, j_r -th instances opened in the epoch (the instances being determined by their opening order, and their honest verdicts jointly log-ordered), bounds the probability that all r resolve in eviction by $(p^*)^r$. At least r wrongful evictions entails that some such index set evicts wholly; the union bound over the $\binom{\bar{\ell}}{r}$ index sets gives the claim. No direct stochastic domination of delayed, opening-indexed eviction indicators is asserted. The same-bytes sharpening is INV-5 with Definition H.5: a cast verdict is a logged decision on the bytes, so its caster is ineligible for any later dispute instance on the same bytes under Rule R13(iii)(b). (iii) By construction of clause (vii), read per publisher. (iv) Clause (vii) modifies submission rights and attestation weight, which are shared-tier attributes; Lemma I.8 covers the rest. The expiry clause is Rule R14(i) verbatim. $\square$

Remark I.23 (Reading of Proposition I.22(ii)). The escalating penalties of Rule R13(vii), requiring $r \geq 2$ wrongful evictions within one epoch, are exponentially rare per specified pair of instances, bounded in expectation in general, and carry the r -fold exponent exactly where a deterministic instance cap holds (capped tier). For repeated panels on the *same* byte string the bounds sharpen: every verdict is a logged decision consuming shared-presence freshness (Definition H.5, Rule R13(iii)(b)–(c)), so each identity casts at most one verdict across all of the bytes’ panels ever; in particular the adversary’s f_s seat-eligible identities are spent once across those panels, not once per panel.

Lemma I.24 (Bounds on rejection-based denial of service).

- (i) *At any time, at most $f \cdot r_{\max}$ items are held in Disputed by adversarial rejections. Moreover, adversarial dispute-initiations against any single item number at most f per epoch (Rule R13(i), single-initiation clause), independently of $r_{\max}$.*
- (ii) (Demand-paced resolution.) *Unconditionally, with no behavioural assumption (persistence exceptions per INV-3), a freeze costs each affected demander at most one accepting confirmation per demanded pair, and only if the demander holds no local in-scope copy: exactly the forced confirmation any demander of an unpromoted pair owes regardless (Theorem J.2(i)). A demander who rejects or abandons incurs inspection effort, a distinct resource disclosed in N18, not bounded by the cap. A dispute therefore suspends only the community amortisation for its demanders and never pushes anyone above the single-user cap, however long the freeze lasts. Under A9, the number so affected is additionally bounded: the demanders affected while the dispute is unresolved number at most κ in expectation, with exponential tail (Lemma I.6), independent of the item’s popularity and of the freeze’s wall-clock duration. A dispute on an undemanded item affects no user and expires within T_{exp} of its last demand (Rule R14). (Wide pre-exposure: Remark I.25; S6.)*
- (iii) *No panel-lapse exploitation; slot economics. There is no panel lapse, hence no forced re-entry: an unresolved dispute never drains the owner’s submission budget. A rejection slot stays occupied until resolution or expiry: against demanded items, disputes resolve at demand rate into the accountability schedule; against undemanded items, the slot pins only an undemanded item and frees at expiry. (Remark I.26; S5.)*
- (iv) *A sustained wrongful-rejection campaign alters no local store (Lemma I.8) and costs each user at most the single-user floor (Theorem J.2); beyond the f free first charges each epoch, it expends the adversary’s rejection rights at an escalating rate (Rule R13(v)). Its throughput is bounded by three deterministic caps, each in its own currency: charged dispute-initiations (those eventually contradicted by vindication) number at most $c_{\text{acc}} f$ per epoch (Lemma I.4); concurrent open rejections number at most $f r_{\max}$ at any moment, and freezes against any single item at most f per epoch (clause (i), single initiation); and uncharged terminal disputes (ending in eviction or expiry) number at most $E_{\max}$ per shared-tier presence per item per epoch (Rule R14(i)), each retiring that presence. No single constant bounds all wrongful rejections across all items — a slot freed by expiry may be re-used against another item — and what each re-use buys is one freeze, resolved or expired at that item’s own demand clock, within the single-user cap. The allowances renew at epoch boundaries (N14).*

Proof. (i) Each adversarial identity holds at most $r_{\max}$ open rejections (Rule R13(i)) and the adversary controls at most f identities. The per-item clause caps each identity at one initiation per item per epoch. (ii) The harm of a freeze is exactly the amortisation it suspends: the demanders who must confirm personally while the item is frozen; by Rule R13(iii)–(iv) the eligible deciding demanders cast the verdicts and, with the default bars, resolution occurs by the k -th verdict; each affected demander’s accepting confirmation, where one occurs, is the one forced confirmation they would owe on any unpromoted pair, so the per-user floor is untouched (INV-3); the in-expectation and tail bounds

are Lemma I.6; an item with no new demand meets the expiry clause of Rule R14(i). (iii) By Rule R13(iv) the only re-entry paths are the optional neither-bar outcome and post-expiry republication, both owner-initiated; slots free on resolution or expiry (Rules R13(iv), R14(i)). (iv) Every dispute-initiating rejection contradicted by vindication charges the rejecter’s schedule and ceiling (Rule R13(v)); by Lemma I.4 the adversary’s effective charged dispute-initiations are deterministically at most c_{acc} per identity per epoch, bounding per-epoch *charged-initiation* throughput at $c_{\text{acc}}f$ as a protocol fact; the concurrency, single-initiation, and terminal-dispute caps are Rule R13(i) and Rule R14(i) verbatim. By Lemma I.8 no local store is altered, and each user’s forced cost remains at most the single-user cap (Theorem J.2). $\square$

Remark I.25 (Wide pre-exposure). An item pre-exposed so widely that few verdict-eligible demanders remain simply stalls *frozen*: the freeze blocks promotion at every tool for as long as the panel is unresolved, and the demand-paced panel carries no deadline an adversary could exploit (Rule R13(ii)–(iii)); this is the deterministic failure mode of A9, and its cost is availability, never safety. For a *poisoned* item the stall is borne by the adversary’s own submission. A *benign* item can also be stalled this way — by pre-exposure or pool-skewed demand, with or without adversarial help — and then the availability cost falls on its publisher and demanders; the harm is bounded exactly as for any freeze: each demander pays at most one forced confirmation per pair, within the single-user cap (Lemma I.24(ii), unconditional clause), the item remains personally usable through each user’s own gate (Rule R13(ii)), local stores are untouched (Lemma I.8), and an item whose demand lapses expires harmlessly (Rule R14(i)). (S6.)

Remark I.26 (Roles of T_{win} , T_{pan} , T_{exp} ; the demand-paced design). The window T_{win} forbids the promotion race: finalisation occurs only at window end, so any qualifying rejection intervenes even if N acceptances arrive first: this is what makes the partition of Theorem I.13 well-defined; and each attempt’s identity-disjoint credited endorsement set (Lemma I.3, Definition I.1) is what makes the per-attempt marginal bounds and their union valid (Corollary I.27). T_{pan} survives only as the clock of fast-track suspension panels, whose log-fresh jurors are drawn from demanders during suspension and prior adopters without a logged decision on the bytes (Rule R13(vi)); their supply is not guaranteed by construction — prior adopters with logged decisions are ineligible — but is availability-side, exactly A9’s province; the ordinary panel carries no global deadline; it is demand-paced: verdicts accumulate at the item’s own demand rate, so resolution time scales inversely with demand while the harm of the freeze scales directly with it, and their product is bounded (Lemma I.24(ii)). A disputed item with too little fresh demand simply waits, exactly as it would have waited for ordinary promotion; an item nobody demands expires harmlessly within T_{exp} . Zero-tolerance designs lack an accountability clock entirely; the demand-paced panel provides one wherever fresh demand exists. An evict-and-restart rule would instead multiply per-item cost by the number of rejections; the panel’s decisive, demand-paced resolution is what keeps the dispute load an additive $O(E_{\text{max}}N + c_{\text{tool}}\bar{D}_v\kappa)$ per demanded item, with $\bar{D}_v$ the expected dispute count of Lemma J.7, absorbed by the cross-term and shrinking-front-loading conditions of Theorem J.13 (S5).

Corollary I.27 (Submission budget and publisher accountability). *Promotion occurs only at an attempt’s finalisation (Definition I.1), and every attempt is counted at its anchor — a logged event preceding every decision creditable to the attempt, hence a sound conditioning point (Definition I.1, §L.1). Under the default bars, the anchors lying in a given epoch are of exactly two kinds, each bounded by the submission rate limit: presence-creating submissions, and finalisation events, each of which closes an entry opened by a same-epoch rate-limited (re-)submission (windows are epoch-contained, convention (C4); Rule R12). Hence the promotion attempts of adversary-published poisoned items anchored in a given epoch number at most $2f\rho_{\text{sub}}$ (paraphrase variants and re-entries included). Each such attempt gathers an identity-disjoint credited endorsement set (Lemma I.3) and is bounded marginally by Theorem I.13 at its own anchor (under its hypotheses); the union bound, valid under arbitrary dependence across attempts, gives*

$$\mathbb{P}[\text{any adversary-submitted poison promoted at an attempt anchored in the epoch}] \leq 2f\rho_{\text{sub}} \left((1-q)^{N-f} + e^{-2\delta^2(k-f_s)} \right).$$

(Attempts anchored while the bytes hold live CC status — additional-tool promotions of already-promoted bytes — are covered under the extended context reading at the close of Theorem I.13’s proof; the event that any poison is ever promoted at all consumes only CC-free anchors.) Every promotion belongs to an attempt anchored in some epoch, so summing over the anchor epochs $1, \dots, E$ from deployment inception bounds the probability that any adversary-submitted poison is ever promoted within that horizon by $2f\rho_{\text{sub}} E \left((1-q)^{N-f} + e^{-2\delta^2(k-f_s)} \right)$. For an interval not beginning at deployment inception, no bound conditional on its realised carry-in pool is asserted — the set of attempts still live at the interval’s start is itself outcome-selected, for the reason of Remark I.28 — and the valid statement is the anchor-epoch bound applied to all anchor epochs from deployment inception through the interval’s end. (Under the looser bars permitted by Rule R13(iv), neither-bar resolutions are additional anchors; the numeric count $2f\rho_{\text{sub}}$ is asserted for the default bars, and loose-bar deployments use the expectation form below, with the anchored-attempt count enlarged accordingly; per-attempt coverage per Remark I.14.)

Poisoned content published by an honest-but-deceived identity (S_{10}) is not counted by $2f\rho_{\text{sub}}$. Writing S_{anc} for the (in general random) number of poisoned promotion attempts anchored in the epoch, whatever their publishers: indexing attempts at their anchor atoms and applying the delayed-marks lemma (Lemma L.5) to the per-attempt success indicators — each bounded, conditionally on the anchor-inclusive σ -algebra of its anchor atom, by the per-attempt marginal bound, and each resolved only at the attempt’s finalisation, possibly after later attempts’ anchors, which Lemma L.5 permits —

$$\mathbb{P}[\text{any poison promoted at an attempt anchored in the epoch}] \leq \mathbb{E}[S_{\text{anc}}] \left((1-q)^{N-f} + e^{-2\delta^2(k-f_s)} \right),$$

and a deployment enforcing a deterministic cap $S_{\text{anc}} \leq \bar{S}$ may substitute $\bar{S}$ for the expectation. (No bound keyed to the attempts finalising in an epoch is asserted: that family is outcome-selected, Remark I.28. Nor is a bound conditional on a realised value of S_{anc} asserted: the attempt count may be adaptively determined by earlier outcomes; the expectation and deterministic-cap forms are the valid system-wide statements.) $\mathbb{E}[S_{\text{anc}}]$ is not bounded in terms of the adversary’s resources alone — it depends on how often false endorsement (A5) and publication co-occur among honest identities — though the per-identity rate limit yields the crude default-bar cap $S_{\text{anc}} \leq 2M\rho_{\text{sub}}$ over M eligible identities, and each such item, on eviction, penalises its publisher (Rule R13(vii)), economically suppressing repeat deception. Under permanent freshness, the credited endorsement sets of all attempts of a given byte string are identity-disjoint over the deployment’s entire history, not merely within an epoch (Lemma I.3); consequently the number of attempts of the same bytes that can ever gather N credited endorsements is at most $\lfloor H_{\text{acc}}/(N-f) \rfloor$, where H_{acc} is the total number of honest identities that ever accept those bytes at a gate. Moreover, every eviction of a poisoned attempt triggers the publisher-accountability consequences of Rule R13(vii) (cool-down; on repetition within an epoch, suspension of submission rights and attestation-weight reduction); the quantitative effect on future submission capacity depends on the deployment’s escalation schedule (§H.2, required parameters) and is consumed by no proof, while a benign publisher’s exposure to the same penalty is bounded by Proposition I.22(ii).

Proof. An attempt is counted at its anchor. Under the default bars every anchor is a presence-creating submission or a finalisation (Definition I.1); distinct attempts have distinct anchors; presence-creating submissions of adversary-published items in the epoch number at most $f\rho_{\text{sub}}$ (Rule R12); and each finalisation in the epoch closes an entry whose opening (re-)submission lies in the same epoch (windows are epoch-contained, convention (C4)), so finalisation anchors also number at most $f\rho_{\text{sub}}$: at most $2f\rho_{\text{sub}}$ anchored attempts. Union bound over these attempts, each covered marginally by Theorem I.13 at its anchor on its own credited set (Lemma I.3); a union bound requires no independence across the events it covers. The horizon form sums the per-epoch display over the anchor epochs of the horizon, every promotion belonging to the attempt anchored in some such epoch (from deployment inception; a general interval is covered by summing from inception through its end, no carry-in-conditional bound being asserted). The system-wide form is the delayed-marks lemma (Lemma L.5) applied to anchor-indexed success indicators. Its occurrence events lie in the conditioning σ -algebras: take $\mathcal{G}_i$ generated by all atoms up to and including the i -th anchor atom, so that O_i is $\mathcal{G}_i$ -measurable; the per-attempt bound of Theorem I.13 is stated conditionally on exactly this anchor-inclusive σ -algebra, so it bounds each mark conditionally on $\mathcal{G}_i$, the mark resolving only at the attempt’s finalisation, which Lemma L.5 permits; the deterministic-cap form substitutes $\bar{S}$. The lifetime clause is immediate from identity-disjointness of credited decisions across all attempts. The accountability clauses are Rule R13(vii) and Proposition I.22(ii)–(iii). $\square$

Remark I.28 (Why attempts are keyed to anchor epochs, not finalisation epochs). An earlier version of Corollary I.27 counted the attempts whose *finalisation* lies in the epoch. That attribution is unsound, for the same optional-selection reason the attempt anchor exists (Definition I.1, commentary): banked decisions logged during dormancy are public log events preceding the owner’s resubmission decision, so the set of attempts finalising in a given epoch is outcome-selected — an adversary maintaining many anchored dormant attempts on distinct byte strings and resubmitting only those with favourable banked outcomes makes the per-finalisation-epoch attempt count small while the epoch’s promotion probability is governed by the whole maintained pool. Anchors, by contrast, precede every decision creditable to their attempts, so the anchored-in-epoch family is determined before any of the outcomes it is charged with, and the delayed-marks lemma applies. The per-attempt bound of Theorem I.13 and the lifetime ceiling of Remark I.29 are unaffected: only the calendar attribution of attempts to epochs changes.

Remark I.29 (Lifetime exposure under permanent freshness). The horizon union over anchor epochs in Corollary I.27, and the lifetime ceiling, count anchored attempts — the lifetime ceiling, threshold-reaching attempts — whose honest endorsers are never re-used: every pipeline entry of a given byte string that reaches its threshold consumes $N-f$ honest identities’ first-and-only *logged* decisions on those bytes, so the adversary cannot re-use a previously logged-deciding population, and $\lfloor H_{\text{acc}}/(N-f) \rfloor$ is a hard ceiling on the number of *threshold-reaching* attempts of the same bytes over the deployment’s lifetime, independent of the number of epochs. Unsuccessful attempts — each gathering fewer than $N-f$ new honest credited acceptances — and presentations yielding no logged acceptance are not bounded by this ceiling; they are governed by the per-epoch rate limit ρ_{sub} and the per-attempt marginal bound of Theorem I.13,

each attempt consuming its own union-bound term. The ceiling is per exact byte string (convention (C5)): an adversary minting byte-distinct paraphrase variants holds a fresh ceiling per variant, each consuming its own rate-limited submission and union-bound term (S1, Corollary I.27); the lifetime clause therefore hard-bounds *successful* re-entry of *fixed* bytes, while variant families are governed by the per-epoch rate and calibration clauses, not by this ceiling.

Remark I.30 (Weighted form). With endorsement weights, replace the exponent $N - f$ by $(W - W_{\text{adv}})/w_{\text{max}}$. With seat weights in $[w_{\text{seat}}, w_{\text{max}}]$ and a vindication weight bar W_m , honest accepting weight at least $W_m - f_s w_{\text{max}}$ is required; seat weights are attestation-layer constants fixed at verdict time, hence predictable in the sense of Lemma L.8’s second clause, and Hoeffding for weighted sums (Lemma L.8) gives $\exp(-2t^2/((k - f_s)w_{\text{max}}^2))$ with t the weight margin: constants change, exponents do not, and the bound remains reach-free.

Remark I.31 (Per-source-class thresholds). Promotion thresholds may be set per *source* class of the underlying content (authentication is not trust: a signed phishing message is still an attack; so source identity only tunes the threshold, never bypasses the quorum, and never widens a tool scope). All bounds hold with W, N, m, k read as the minimum over source classes; Theorem I.11 requires $\min_{\text{class}} W > W_{\text{adv}}$ and $\min_{\text{class}} m > f_s$.

J THE MEMORY-VERIFICATION BUDGET

Scope. All results in this appendix count confirmations attached to *reusable memory content*, per (value, tool) pair: a one-time personal cap of private values, and the stream of shareable Confirmed items. Per-task confirmations (for freshly composed sensitive arguments and for crystallizing ephemeral task content) are common to every gated design, scale with usage rather than population, and are orthogonal to the amortisation analysed here. PerMIT’s distinction is layered. Pathwise — independent of every federated and demand assumption, persistence exceptions per INV-3 — no user’s forced cost exceeds once per (value, tool) pair, ever, and the epoch’s expected shared forced load never exceeds the standalone single-user mass d_{tool} — the cost of having no community at all (Theorem J.2(i)–(ii)); this cap is the appendix’s firmest guarantee: pathwise, consuming no federated or demand assumption (its expected clauses read the reference model’s masses), subject only to the stated first-party persistence exceptions. On top of it, the *ledger-charged* shared stream is at most the front-loading multiple $B_{\text{R}} \bar{\gamma}$ of the standalone mass, independently of item popularity (Theorem J.9, Remark J.11); and the *ledger residual* — demand arriving outside every charged piece, in particular handle-expressed arrivals of already-certified bytes, which gate by design (Rule R4(c), S18) — has its benign, assumption-conforming component vanish under the full vanishing conditions of Theorem J.9(iii): the coverage-and-representation assumption A11 together with the product-rate condition $D_0(M)V(M) = o(M)$, $\varepsilon(M)d_{\text{good}}(M) \rightarrow 0$ (under A13, from $\varepsilon(M) \rightarrow 0$; d_{good} per Lemma J.8), and bounded tool spread (A12); these conditions are sufficient, not necessary, and demand on poisoned or assumption-violating pairs remains in $r_{\text{bad}}^{(e)}$ (Corollary J.10). Under the measured corridor of §J.5 — whose front-loading rows describe deployments whose time geometry changes with scale, not consequences of population growth alone (Remark J.12) — both shared components vanish; where the corridor or A11 fails, the universal fallback applies to the *forced* component: the epoch’s expected shared forced load remains at most the standalone mass d_{tool} (Theorem J.2(ii)), the once-per-pair cap binding throughout. Voluntary invitation confirmations remain governed by the ledger rows of Lemma J.7; rejection and attention labour is accounted separately in N18. No standalone-mass ceiling is claimed for those additional resources. Headline readings of this appendix elsewhere in the paper must carry these conditions: “shared verification load vanishes” is asserted only under measured saturation, smoothness, coverage, juror-supply, and bounded-mass hypotheses (A10–A13), never from population growth alone (Remark J.12); the firmest statements are the pathwise once-per-pair cap and the reference-model standalone-mass ceiling (Theorem J.2).

J.1 MODEL

- A population of M users; a universe $\mathcal{V}$ of shareable Confirmed items. The accounting unit is the *(value, tool) pair* (the endorsement unit, §F.1): a user’s demand, forced cost, and local coverage are all per pair. Personal and context-dependent values lie outside $\mathcal{V}$ (Rule R12, Lemma I.2); d_{priv} is the expected number of distinct personal *(value, tool) pairs* a user ever confirms, once each over the deployment’s lifetime (the pair is the endorsement unit, §F.1; a personal value used at j tools contributes j pairs). d_{priv} is a *lifetime* quantity; where it appears alongside single-epoch masses (Theorem J.2(ii)), the sum is read as a lifetime personal term plus a per-epoch shareable term, each bounding its own component.
- The independent reference model (IRM) Fagin (1977), per pair and per user: user u demands the pair (v, t) within the epoch with probability $p_{u,v,t}$, independently across users; the model is heterogeneous, and $p_{v,t} := M^{-1} \sum_u p_{u,v,t}$, $p_v := M^{-1} \sum_u \mathbb{P}[u \text{ demands } v \text{ at some tool}]$ denote the population-average marginals, so $p_{v,t} \leq p_v$ and $\mathbb{E}[n_{v,t}] = Mp_{v,t}$. Every expected-count argument in this appendix consumes only cross-user independence and this identity, never identical distribution across users (the owner’s degenerate indicator, next bullet, is the one

systematic heterogeneity). The single-epoch masses are $d := \sum_v p_v$ and $d_{\text{tool}} := \sum_{v,t} p_{v,t}$, both finite, owner coupling folded in (next bullet), with $d_{\text{tool}} \leq c_{\text{tool}} d$ under A12. Under this model n_v and $n_{v,t}$ denote the *latent* counts $\sum_u D_{u,v,t}$; the log-defined realized counts satisfy $n_{v,t}^{\log} \leq n_{v,t}$ pathwise (§L.1), so every expected bound stated against $Mp_{v,t}$ covers the realized events the ledger charges.

- *Owner coupling.* The owner’s publication schedule for each item — the epochs in which the owner submits or resubmits v (Rule R12) — is carried in the exogenous demand factor alongside the demand indicators (§L.1): it is owner behaviour, exogenous to the protocol and the attacker exactly as the typed stream is (A6). In every scheduled epoch the owner’s own demand indicator for v is set to one, at the owner’s creating tool (formally, the owner’s marginal $p_{u,v,t}$ is degenerate at one in scheduled epochs; other users’ indicators and the cross-user independence are untouched — the heterogeneous form of the previous bullet): submission is modelled as demand-driven, latently mirroring Definition H.4’s clause that the owner’s publish confirmation is a realized demand event. Consequently every epoch containing a submission or re-entry of v has $n_v \geq 1$, and every realized demander is a latent demander, so $n_v^{\log} \leq n_v$ pathwise. A shared-tier presence can, by contrast, be *carried* into an epoch with no current-epoch owner activity (a dormant candidate short of its retirement clock, a live CC entry, or the one carry-in freeze of Lemma J.7); the vanishing clause of Lemma J.7(iii)–(iv) therefore rests on the absence of *events*, not of presence: on $\{n_v = 0\}$ there is no entry (an entry requires a current-epoch submission, hence owner demand), no dispute or review opening (each opens at a *latent* demander’s protocol-bearing gate decision — attempted use realizes the latent indicator, forced-confirmer clause; the opener need not be a realized log-demander, e.g. at a cache gate’s rejecting branch, Definitions H.6(iii), H.4), no freeze-bucket arrival, no invitation (a re-entry notification requires an entry, and a carried-in review issues no new prompts, Rule R13(vi)), and no publish confirmation. The masses $p_{v,t}, p_v, d, d_{\text{tool}}$ are read with this coupling folded in (the owner’s degenerate indicator contributes its own mass like any user’s), so every expected bound stated against them is unchanged in form. *Scope of the exogenous-schedule reading.* The reference model’s expectation-form conclusions (Lemma J.7(iii), Theorem J.9, Corollary J.10) are asserted under exogenously scheduled publication: the owner’s schedule is carried in the demand factor, like the typed stream (A6), not chosen as a function of protocol outcomes. Adaptive republication reacting to banked or dispute outcomes is covered elsewhere: by the deterministic caps (the E_{max} -rows and the pathwise clause of Lemma J.7(iv), which hold for every schedule), and, on the safety side, by the anchor-keyed counting of Theorem I.13 and Corollary I.27, which quantifies over adaptive strategies.
- Promotion threshold N per tool (weighted: distinct endorsers in $[W/w_{\text{max}}, W/w_{\text{min}}]$, changing constants not exponents); panel cap k ; credited endorsements per (entry, tool) are capped at $N^+ = \Theta(N)$ (Definition H.6(f)).
- A *confirmation* is an accepting gate decision (convention (C6)); rejection labour is bounded separately (N18). Accepting decisions fall into three exhaustive classes. A confirmation of the pair (v, t) is *forced* if the gate fires for the user’s own use of v at t and the user holds no local endorsed copy of v whose scope admits t (for the pseudo-target: no local readable $\mathbb{E}$ -level copy of v at all — reasoning coverage, Rule R13(i)). *Forced-confirmer clause (a reference-model coupling).* The reference model is read with attempted use coupled to latent demand: a user’s own attempted use of v at t within the epoch — any event firing a gate for it, inside or outside a shared-tier presence, covered and cache gates included — occurs only on $\{D_{u,v,t} = 1\}$; equivalently, every such event realizes the user’s latent indicator for the epoch. This is a modeling stipulation on the demand factor (§L.1), not a consequence of independent sampling: it is the natural extension of the existing convention that a realized shared-tier demand presupposes attempted use. Every forced confirmer of a pair is therefore a latent demander of it; the log-defined shared-tier demand of Definition H.4 ($n^{\log}$) remains the subclass realized during a shared-tier presence, with $n_{v,t}^{\log} \leq n_{v,t}$ pathwise. It is *voluntary* if it is recorded at an interpreter-issued invitation surface (a re-entry notification or suspension prompt, Definition H.6, Rule R13(vi)), declinable at zero forced cost; voluntary confirmations exist *only* through such surfaces, deduplicated jointly at one invitation per identity per item per epoch. It is *redundant* if the gate fires for the user’s own use while an in-scope local copy is held — possible only through handle-expressed reuse (Rule R4(c)) or a reasoning-target confirmation of locally readable bytes (reasoning coverage, Rule R13(i)), a *covered gate* — in which case the decision is protocol-inert on both branches (Rule R13(i)): no promotion weight, no freshness consumption, no dispute initiation, no verdict. Redundant confirmations are excluded from every count below and disclosed as attention cost in N18. C_u counts user u ’s forced and voluntary shareable confirmations; $\bar{C}_M = \frac{1}{M} \sum_u C_u$. The cap (Theorem J.2) is stated over forced confirmations; the ledger (Lemma J.7) over forced and voluntary.
- *Epoch vs. lifetime masses.* $p_{v,t}$ is a per-epoch probability. The lifetime mass is $d_{\text{tool}}^\infty := \sum_{v,t} p_{v,t}^\infty$ with $p_{v,t}^\infty$ the probability that a user *ever* demands the pair; single-epoch and lifetime expected bounds are stated against the matching mass and never conflated (Theorem J.2(ii)–(iii)). Disputes add the ledger rows of Lemma J.7; revocation adds Corollary J.5 (re-verification mechanics: N16).
- *Saturation:* $V(M) := \mathbb{E}|\{v : n_v \geq 1\}| = o(M)$; concretely Heaps’ form $V(M) = \Theta(M^\beta)$, $\beta \in (0, 1)$ Newman (2005).

Assumption A10 (Temporal smoothness, fractional form). There are a constant $s_0 \geq 1$ and a front-loading bound $\bar{\gamma} = \bar{\gamma}(M) \in (0, 1]$ such that, for every pair (v, t) demanded in the epoch: the number of distinct demanders of (v, t) whose first demand of the pair falls within any single finalisation window (length T_{win}) or any single *candidate-dormancy* interval (an inter-window span carrying no live community entry, of length $\leq T_{\text{exp}}$ by the retirement clock; *stable promoted intervals* — a live, unsuspended CC presence with no open window — are exempt from both clauses: demand there is fast-pathed or ledger-residual, never bucketed, Lemma J.7) is at most $\bar{\gamma} n_v + s_0$; and likewise the number of distinct identities whose *first audit-recorded decision on the pair* (v, t) in the epoch (§L.1: display-transcript decisions at non-covered gates, cache gates on *both* branches included — the accepting branch is audit-recorded though never federation-logged, Definition H.6(iii)) falls within any single such window or interval is at most $\bar{\gamma} n_v + s_0$ (the *first-pair-decision clause*). Demander counts are *realized* shared-tier demand events (n_v^{log} , §L.1); decider counts are audit-recorded first pair-decisions (a class containing every federation-logged first pair-decision), whose deciders are latent demanders (forced-confirmer clause, §J.1) but need not be realized log-demanders. Both are capped against the item’s *latent* demand n_v — the weaker denominator, since $n_v^{\text{log}} \leq n_v$ and every decider is a latent demander.

Commentary (non-normative; reading and validation of A10). The cap is relative to the item’s own demand, never an absolute count: it forbids *front-loading* (demand concentrating inside one window), not popularity. For demand arriving steadily over an epoch of length L , $\bar{\gamma} \approx T_{\text{win}}/L$ for every item, independently of n_v and of M ; the additive s_0 makes the cap satisfiable for items of tiny demand. A10 is a hindsight property of completed epochs, used only in the analysis: no protocol rule reads, estimates, or depends on $\bar{\gamma}$. Its violation is item-local and graceful: an item whose demand spikes into one window ($\gamma_v \approx 1$) simply fails to amortise: each of its demanders pays the one forced confirmation the pathwise cap (Theorem J.2(i)) already bounds, while every other item’s accounting is untouched (S17). Demand-paced panels need no such cap: panels accumulate verdicts across windows, so A10 constrains promotion finalisation windows and dormant intervals only; the dormant-interval clause is the stronger half ($T_{\text{exp}} \gg T_{\text{win}}$ typically), kept plausible by the dormancy-retirement clock of Rule R14(i) and by its restriction to *candidate* dormancy: a promoted item’s demand, however concentrated, is exempt from both clauses (its cost is fast-pathed or ledger-residual, Lemma J.7). Validation: per-item realised fractions γ_v , checked specifically on the most-demanded items (N17). The first-pair-decision clause has the same character and the same validation path: for demand arriving steadily, first pair-decisions track first pair-demands; both realised fractions are logged per item (N17). The audit-recorded class is validated the same way, directly from display-transcript logs: both branches of every gate, cache gates included, are recorded there (Rule R8(e)), so realised first-pair-decision fractions are observable without reference to the federation log. Its pair-level indexing is load-bearing: it is what makes the dispute-initiation classification of Lemma J.7 exhaustive, since a cross-tool repeat decision (a prior acceptance at another pair) and a decision postponed past an abandoned dialog are each the identity’s *first* decision at their own pair, counted where they fall. The pair-level form is the item-level cap read tool by tool, still stated against the item’s *total* demand n_v , the weaker and more plausible denominator; steady demand gives $\bar{\gamma} \approx T_{\text{win}}/L$ per pair exactly as per item, and the same per-item logging of realised fractions γ_v validates it (N17).

Assumption A11 (Coverage sufficiency, operational form). Adopt the presence taxonomy and charging convention of Lemma J.7: within an epoch, an item’s shared-tier presence partitions into entry windows, candidate-dormancy intervals, and freeze/suspension intervals (the *ledger-charged* pieces), and *stable promoted intervals* (a live, unsuspended community entry with no open entry window). For a pair (v, t) and an epoch, let $U_{v,t}$ count the epoch’s *residual* forced accepting pair-confirmations of (v, t) : those charged to no ledger row (option-induced cache confirmations excluded: floor-charged, Corollary J.6) — a subclass of the forced confirmations by demanders whose epoch-first demand of the item falls in a stable promoted interval or outside every shared-tier presence (a credited such decision is weight-row-charged instead, Lemma J.7, only shrinking the residual) (before first publication, after retirement, or after demotion; gates fired during a suspension are freeze-bucket events, ledger-charged, never residual). Each belongs to a distinct demander of the pair (INV-3), so $U_{v,t} \leq n_{v,t}$ pathwise. The residual class is *operational*, keyed to gate behaviour, never to certificate existence: a live CC entry naming (v, t) fast-paths only *literal*-expressed use (Rule R4(a)); a handle-expressed use of the same pair gates at every tool (Rule R4(c)) and, absent a local in-scope copy, is forced — and, in a stable promoted interval, residual. There are $D_0(M) \geq 0$ and $\varepsilon(M) \rightarrow 0$ such that, for every *benign* (Definition H.10) shareable item $v \in \mathcal{V}$, every target t , and every $n \geq 1$, conditional on $n_{v,t} = n$:

$$\mathbb{E}[U_{v,t} \mid n_{v,t} = n] \leq D_0(M) + \varepsilon(M) n.$$

Like A10, this is a hindsight property of completed epochs, read against latent demand (§L.1); no protocol rule reads, estimates, or depends on it, and it is trivially satisfied whenever $n \leq D_0(M)$ (since $U_{v,t} \leq n_{v,t}$). The growth of D_0 is consumed only through the *product condition* $D_0(M) V(M) = o(M)$ of Theorem J.9(iii), a joint property of the coverage threshold and the saturation law, never of either alone; under the Heaps form $V(M) = \Theta(M^\beta)$ (§J.1), the sufficient rate is $D_0(M) = o(M^{1-\beta})$, the form used in the corridor (§J.5) and in the sufficiency sketch below. It bounds jointly: (i) *publication coverage* — a demanded benign item its owner never publishes has every gate-firing accepting demand residual, and satisfies the bound only if that demand is small; (ii) *promotion coverage* — demand at not-yet-promoted pairs arriving outside every ledger-charged piece; and (iii) *representation coverage* — demand for live-CC pairs arriving handle-expressed (untrusted-channel arrival of already-certified bytes), which gates and pays

despite the certificate (S18). An item or arrival pattern violating it never amortises: each residual demander pays the one forced confirmation the pathwise cap (Theorem J.2(i)) already bounds, item-locally and gracefully, entering the residual $r_{\text{bad}}^{(e)}$ of Corollary J.10.

Heuristic sufficiency and compatibility. The following sufficiency sketch is heuristic — a plausibility argument, not a derivation: it does not formally compose intervening freezes, wrongful evictions, the entry cap, the pool-composition requirements, or the arrival-representation mix into a proof of A11, which remains an assumption; the compatibility calculation closing the paragraph is exact. Heuristically sufficient, for a benign item and any fixed $\eta > 0$: the owner publishes the item before more than an ε -fraction of each demanded pair’s demand arrives and resubmits promptly after each unpromoted window (dormant intervals short of the retirement clock, within the per-epoch entry cap of Rule R14(i)); some entry, counting the banked credits of its preceding interval (Definition H.6(b)), accrues at least $(1 + \eta)N/p_b$ fresh byte-first demanders (a positive byte-first fraction per demanded tool — enough demanders whose first decision on the bytes occurs at that tool — is one empirical mechanism supporting this cohort’s existence; it is a heuristic diagnostic for A11, not a hypothesis of any theorem) whose demand spans at least D_{pool} pools with at least $\lceil N/D_{\text{pool}} \rceil$ accepting demanders available per pool (so the per-pool credit caps of Definition H.6(f) do not bind below N), almost all of them authority-active (Definition H.6(a): honest identities are authority-inactive only in the rare calibrated states of theorem I.5, folded into the same ε) — a demand-composition property of the same character as A9’s structural half, validated by the same pool-stratified logging (N17); and demotion of promoted benign entries is exponentially rare (Proposition I.20), so promoted status, once reached, persists; and, for the representation clause, demand for promoted pairs predominantly arrives fast-path-expressed — through trusted retrieval of the live entry (Definition H.1, Rule R14(ii)) or reuse of local readable copies — with the handle-arrival fraction (S18) folded into the same ε : an arrival-channel property of the demand stream, logged directly (N17), never derived from the protocol. For schedules with $N(M) \rightarrow \infty$ — every corridor schedule, §J.5 — the accepting count then crosses N except with probability $e^{-2\eta'^2 N(M)/(1+\eta)^2} \rightarrow 0$ by Hoeffding’s lower tail, for a margin η' depending on η and p_b , giving $\varepsilon(M) \rightarrow 0$; at *fixed* N the same cohort yields only a fixed failure probability, so $\varepsilon(M) \rightarrow 0$ under fixed protocol constants requires instead that available cohorts grow beyond a fixed multiple of N , which A11 then asserts rather than derives; intervening disputes are absorbed by the vindication exemption of Rule R14(i)(b) (N15) and resolve at the item’s own demand rate, freezes excluding themselves from the window clock (Rule R13(iv)). *Consistency with A10*: under the fractional cap the two assumptions do not constrain one quantity from opposite sides. A crossing window is permitted to contain $(1 + \eta)N/p_b$ demanders whenever $\bar{\gamma} n_v + s_0 \geq (1 + \eta)N/p_b$, i.e. whenever $n_v \geq ((1 + \eta)N/p_b - s_0)/\bar{\gamma}$; $D_0(M)$ may therefore be taken of order $N(M)/\bar{\gamma}(M)$, and the product condition’s Heaps form $D_0(M) = o(M^{1-\beta})$ (Theorem J.9(iii)) becomes $N(M)/\bar{\gamma}(M) = o(M^{1-\beta})$, satisfied by every corridor schedule (§J.5).

Commentary (non-normative; scoping, representation dependence, and validation of A11). *Benign scoping is deliberate*: poisoned items are excluded; their promotion probability is bounded above by Theorem I.13, and their residual demand is charged at the single-user cap by design, each such confirmation being a personal gate inspection of an unpromoted or flagged item, exactly the inspection the design intends. *The operational (no-row) form is load-bearing*: an earlier draft keyed coverage to certificate existence, under which a handle-expressed use of a live-CC pair counted as covered while its gate — forced under Rule R4(c) — was charged to no row and to no residual; the operational form charges every forced confirmation somewhere by construction (Lemma J.7, charging convention), and names exactly the demand the amortisation story must cover. *Representation dependence is the disclosed price*: community certification amortises the flows that can take the fast path — literal-expressed reuse of promoted bytes, in particular bytes obtained through trusted retrieval (RETRIEVE returns live entries as readable $\mathbb{E}$ -bytes, Definition H.1, Rule R14(ii), which then byte-match the live entry, Rule R4(a)) — and never the handle-expressed arrivals, which gate by design (S18); $\varepsilon(M)$ must absorb the handle-arrival fraction of certified demand, so the vanishing of the residual is a joint property of publication timing, promotion speed, and arrival representation. Failure is graceful and item-local: residual demand is priced at the cap (Theorem J.2(i)), and the epoch’s shared forced load never exceeds the standalone mass d_{tool} in any case (Theorem J.2(ii)). Incentive sensitivity is the disclosed residual S16. Under permanent freshness the condition weakens in one direction and is explicitly priced in the other: banked credits accumulate across epoch boundaries toward the next entry, so qualifying demand need not concentrate within one epoch; conversely, endorsers consumed by a failed entry never renew, which the identity-disjoint counting already prices (Lemma I.3). Validation: log each item’s realized residual fraction $U_{v,t}/n_{v,t}$ and the fast-path-vs-gate split of live-CC demand (the handle-arrival fraction), stratified on the most-demanded items, together with per-item time-to-promotion against accumulated demand and the per-epoch demanded-but-unpublished mass (N17).

Assumption A12 (Bounded tool-spread). Each shareable item is demanded at no more than $c_{\text{tool}} = \mathcal{O}(1)$ distinct tools over the epoch: at most c_{tool} tools have $n_{v,t} \geq 1$, and hence $\sum_t n_{v,t} \leq c_{\text{tool}} n_v$ (latent reference-model counts, §J.1; the realized counts of Definition H.4 inherit the bound pathwise: $n_{v,t}^{\log} \leq n_{v,t}$ places the realized tool support inside the latent tool support, so at most c_{tool} tools have $n_{v,t}^{\log} \geq 1$, and every realized pair-d demander is a realized item-d demander, so $n_{v,t}^{\log} \leq n_v^{\log}$ (Definition H.4); hence $\sum_t n_{v,t}^{\log} \leq c_{\text{tool}} n_v^{\log}$ pathwise). In particular each user demands

at most c_{tool} pairs per item, and $d_{\text{tool}} \leq c_{\text{tool}} d$ (the $\leq c_{\text{tool}}$ demanded targets per item include reasoning where demanded; constants unchanged).

Commentary (non-normative; reading and validation of A12). The bound formalises the observation that a published item typically addresses the scenario its owner used it in: endorsement, promotion, and scope are all per (value, tool); it licenses the per-tool split in Theorem J.9(iii). Validation: per-item distinct-tool counts (N17). The *byte-first density* of a demanded tool’s stream — the fraction of its demanders whose first decision on the bytes occurs at that tool — is deliberately *not* part of this assumption: no proof consumes it, and its role is heuristic support for the promotion-coverage clause of A11 (see the heuristic-sufficiency paragraph there).

Assumption A13 (Bounded per-user demand mass). Along the population scaling, the single-epoch per-user demand mass is uniformly bounded: $\sup_M d(M) < \infty$, where $d(M) = \sum_v p_v = M^{-1} \sum_u \sum_v \mathbb{P}[u \text{ demands } v]$ is the population-average number of distinct shareable items one user demands per epoch (§J.1); hence $\sup_M d_{\text{tool}}(M) < \infty$ under A12, and $d_{\text{good}} \leq d$ throughout. Where A13 is not adopted, every vanishing ($\rightarrow 0$) budget conclusion below holds verbatim with its rate condition read against the mass: $\bar{\gamma}(M) (1 + \kappa(M)) d_{\text{good}}(M) \rightarrow 0$ in Theorem J.9(i)–(ii) and Theorem J.13(iii), and $\varepsilon(M) d_{\text{good}}(M) \rightarrow 0$ in Theorem J.9(iii) (whose $D_0(M)V(M)/M$ term is independent of the per-user mass and unchanged). The relative bounds (cost at most the front-loading multiple $B_{\text{fl}} \bar{\gamma}$ of the standalone mass, Theorem J.9(i), first clause) and the cap (Theorem J.2) consume no bound on d in any case.

Commentary (non-normative; reading and validation of A13). The assumption is load-bearing and deliberately explicit: without it, the vanishing conclusions would fail — a population-indexed demand law with $d(M) \rightarrow \infty$ (each user demanding ever more distinct shared items as the community grows) satisfies every remaining hypothesis while the front-loading term $B_{\text{fl}} \bar{\gamma} d$ grows — and pointwise finiteness of d at each M does not supply the uniform bound the limits consume. It is an individual-usage property, not a population property: under any fixed per-user demand law the masses are constants in M and A13 is automatic (the Zipf law of §L.5 has $d = \sum_r p_r = 1$ identically), and the Heaps form is standardly read over a corpus accumulated from bounded individual streams Lü et al. (2010), so the saturation hypothesis already carries this reading. The owner-coupling convention cannot violate it: coupled indicators number at most ρ_{sub} per user per epoch (Rule R12). The disclosed failure mode is demand growth induced by the shared tier itself — a richer library drawing each user to more shared items per epoch — a network effect of the same character as the incentive feedback of S16; violation is graceful: the relative bounds and the pathwise cap (Theorem J.2(i)) are unaffected, and the general-form rate conditions of A13 price the growth explicitly. Validation: log per-user distinct shareable-item demand per epoch across deployment growth (N17).

J.2 THE SINGLE-USER FIRST-AUTHORIZATION CAP

The cap’s core is a counting identity. Its pathwise clause, Theorem J.2(i), rests on the harness rules and INV-3 alone: no ledger row, no behavioural parameter, no demand model, and no assumption of this appendix. Its expected clauses restate that identity against the reference demand model’s masses (§J.1), still using no assumption; the single assumption-bearing statement in this subsection is the mass comparison of Corollary J.3 (A12). The cap is therefore stated first. Its dependency chain is Lemma J.1 $\rightarrow$ INV-3 $\rightarrow$ Theorem J.2 $\rightarrow$ Lemma J.7, strictly in that order.

Lemma J.1 (Local scope persistence). *Fix a user u , a value v , and a tool t , and suppose u ’s own accepting review of (v, t) has occurred at any gate transaction — including a transaction another of whose reviews aborted the enclosing invocation (Rule R2: endorsement is durable). Then in every later configuration, unless u ’s own agent has evicted the block (N13), u ’s store contains a C-block with bytes v and $t \in \text{scope}$, and every use of v at t by u expressed as a readable literal takes the fast path of Rule R4(a); a use expressed through a handle gates (Rule R4(c)) as a covered gate, protocol-inert and redundant (Rule R13(i), §J.1). In particular no later gate event of u for (v, t) is forced. (Option O3 is requirements-level and absent from the formal semantics; a conforming instantiation would leave the block in place and read the authorization and coverage tests over currently authorizing witnesses, $O3(a), (c) \text{--} (d)$.) For the pseudo-target: once u holds any local readable E-level copy of the bytes (via RECALL-E, Definition F.6), a later confirm on those bytes at reasoning is a covered gate by the reasoning-coverage clause of Rule R13(i), protocol-inert and redundant; hence no later gate event of u for $(v, \text{reasoning})$ is forced.*

Proof. By Rule R2 and Definition F.5, every accepting review of (v, t) writes a fresh C-block with bytes v and $t \in \text{scope}$ at the close of its atomic transaction, on the executing and the aborting branch alike (CONF-ACC, TOOL-GATE-ACC, TOOL-GATE-REJ; Figures 7–8, convention (vii)). By A1–A2 blocks are immutable once written, so neither this block’s label nor its scope ever changes; local stores mutate only through owner-invoked Σ -operations and owner confirmations, and no protocol event initiated by any other identity alters or removes any block in u ’s local store (Lemma I.8). The only removal path is first-party eviction (N13). Rule R4(a) then byte-matches v against

the stored block with $t \in \text{scope}$. For the pseudo-target: any local $\mathbb{E}$ -level copy of the bytes is readable (RECALL-E, Definition F.6) and persists by A1–A2 and Lemma I.8, the first-party eviction exception aside; the reasoning-coverage clause of Rule R13(i) then classifies every later reasoning-target gate on those bytes as covered. Its owner-side test reads exactly the readability fact Rule R5 establishes: readability is the entirety of what an accepting reasoning confirmation buys, so the clause covers precisely the gates whose acceptance would change nothing locally. $\square$

Theorem J.2 (Single-user first-authorization cap). *The cap counts forced accepting confirmations — first authorizations of reusable (value, tool) pairs — and nothing else: not rejecting decisions, abandoned or repeated dialogs, covered-gate (redundant) acceptances, batch-page attention, or per-task confirmations (classes: §J.1; effort account: N18). Suppose the owner’s agent does not evict the relevant local blocks (the disclosed first-party exception of INV-3); Option O3 is requirements-level and absent from the formal semantics (O3), so no lease expiry exists in the proved system.*

- (i) (Pathwise lifetime invariant.) *Pathwise and independent of every federated and demand-model assumption, subject to the stated first-party persistence exceptions: under any dispute, rejection, revocation, or availability campaign whatsoever, no user is forced to confirm any given (value, tool) pair more than once over the deployment’s lifetime: $C_{u,v,t}^{\text{forced}} \leq 1$ pathwise, counting forced accepting pair-confirmations (Definition F.5, counting unit). This is INV-3, established by Lemma J.1. Rejecting decisions, abandoned dialogs, repeated within-transaction reviews, redundant confirmations (§J.1), and inspection time are a distinct resource, disclosed and partially bounded in N18, and are not bounded by this clause.*
- (ii) (Expected cap, single epoch, reference model.) *In the independent reference model (§J.1), the two components are bounded on their own horizons: $\mathbb{E}[C_{\text{personal}}^{\text{forced},\infty}] \leq d_{\text{priv}}$ (lifetime) and $\mathbb{E}[\bar{C}_{\text{share}}^{\text{forced},(e)}] \leq d_{\text{tool}}$ (per epoch e); the componentwise sum $d_{\text{priv}} + d_{\text{tool}}$ is a valid upper bound for their sum on any single epoch read jointly with the lifetime personal term, not a homogeneous single-horizon cost. The bound d_{tool} is exactly the expected shareable cost a single user incurs in isolation, where every gate necessarily fires: no user is ever pushed above the single-user cost. The comparison to the item-level mass, $d_{\text{tool}} \leq c_{\text{tool}} d$, is Corollary J.3 and is this theorem’s only assumption-bearing companion.*
- (iii) (Expected cap, lifetime.) *Over the deployment’s lifetime, expected forced shareable cost is at most $d_{\text{tool}}^{\infty} = \sum_{v,t} p_{v,t}^{\infty}$; the single-epoch mass bounds the lifetime cost only when it is itself read as the lifetime mass (§J.1).*
- (iv) (Monotonicity.) *At a fixed realised workload with no disputes, adding users never increases any individual’s forced count: promotions only advance, and a promotion of (v,t) removes the gate for every later demander of that pair (Rule R4). With disputes, an added identity can suspend amortisation, forcing later demanders to confirm personally; the pathwise invariant (i) still caps every user at once per pair.*

Proof. (i) By Lemma J.1: u ’s first accepting review of (v,t) writes the covering local copy durably, whatever the enclosing transaction’s outcome, so no later gate event of u for (v,t) is forced (a literal-expressed use fast-paths; a handle-expressed use is a covered gate, redundant by definition); within that first transaction the pair is counted once (Definition F.5); declining voluntary invitations costs nothing (Definition H.6). (ii) By (i), each forced confirmation of (v,t) belongs to a distinct demander of the pair (realized, hence latent, §L.1), so the population’s forced total for (v,t) is at most $n_{v,t}$, with $\mathbb{E}[n_{v,t}] = M p_{v,t}$ in the reference model; summing over pairs and dividing by M gives d_{tool} ; personal pairs add d_{priv} , once each (INV-3). (iii) The pathwise clause (i) is lifetime; taking expectations over lifetime pair-demand indicators gives d_{tool}^{∞} . (iv) Immediate from Rule R4 and (i). $\square$

Corollary J.3 (The cap under bounded tool-spread). *Under A12, $d_{\text{tool}} \leq c_{\text{tool}} d$; hence the shareable cap of Theorem J.2(ii) is at most $c_{\text{tool}} d$ per epoch (componentwise sum $d_{\text{priv}} + c_{\text{tool}} d$, horizons per Theorem J.2(ii)). The lifetime analogue, $d_{\text{tool}}^{\infty} \leq c_{\text{tool}} d^{\infty}$ with $d^{\infty} := \sum_v p_v^{\infty}$, holds when A12 is read over lifetime demand (at most c_{tool} distinct tools per item ever), a strictly stronger reading than the per-epoch form and asserted only here. This corollary restates the cap against the coarser item-level mass; no other result consumes it.*

Proof. A user demanding the pair (v,t) demands the item v (Definition H.4), and under A12 at most c_{tool} tools per item carry demand, so $\sum_t p_{v,t} \leq c_{\text{tool}} p_v$ per item; sum over items. The lifetime form is identical under the lifetime reading. $\square$

Remark J.4 (Voluntary labour and the cap). The cap is stated over forced confirmations. Voluntary community labour (a repeat re-inspection after a dispute or re-entry notification) is declinable at zero personal cost, carries no promotion weight (INV-1), and is counted in the budget (Lemma J.8), never in the cap.

Corollary J.5 (The cap under revocation). *Post-promotion suspensions and re-verification (N16) do not multiply forced cost: a user who relied on ungated CC status at t and loses it on demotion or suspension owes at most their first-ever confirmation of (v, t) , already counted inside d_{tool}^{∞} , after which their local C copy covers all reuse at t for the deployment’s lifetime (Lemma J.1). Hence expected total forced cost is at most $d_{\text{priv}} + d_{\text{tool}}^{\infty}$ lifetime, independent of the number of epochs. A multiplicative factor E_{lease} would arise only under a conforming instantiation of the requirements-level Option O3 — an extension-level claim, stated and to be proved with that extension (O3(e)), outside this corollary.*

Proof. Immediate from Theorem J.2(i),(iii): forced confirmations are once per (user, value, tool) over the lifetime; demotion, suspension, and re-verification never touch local C copies (Lemma I.8). Option O3 is absent from the formal semantics (O3). $\square$

Corollary J.6 (Budget under Options O1–O2). *Option-induced cache confirmations (Options O1–O2) belong to no pipeline ledger row and to no residual (Lemma J.7, exclusions): they are accept-inert for the shared tier (Definition H.6(iii)) and are accounted in two classes. (Per-task.) O2’s confirmations of freshly composed data-tool arguments lie in the per-task stream, outside the reusable-memory accounting entirely, by the scope hypothesis of Proposition G.28. (Floor-charged.) All remaining option-induced cache confirmations — O1’s first-use sensitive-sink confirmations of T-values and O2’s confirmations of endorsed-value reuse at data tools — are forced pair-confirmations of the confirming user, charged to the floor component: each is at most one per (user, value, sensitive sink) resp. (user, value, data tool) (scope-caching, Lemma J.1, INV-3), an $\mathcal{O}(1)$ enlargement of the per-item pair count under the sink-spread and data-tool-spread hypotheses of Proposition G.28 — folded exactly as A12 folds tool pairs. The exhaustiveness of the charging convention (Lemma J.7) is unaffected: option-induced confirmations are charged to the per-task stream or the floor, never to any pipeline row or residual.*

Proof. Immediate from Proposition G.28(iii), Definition H.6(iii), and INV-3: the fresh cache copy scope-caches (Lemma J.1), so each (user, value, target) triple is forced at most once; the spread hypotheses bound the targets per value by $\mathcal{O}(1)$; and the freshly-composed class is per-task by hypothesis. $\square$

J.3 THE PER-ITEM CONFIRMATION LEDGER

Lemma J.7 (Per-item confirmation ledger, $\mathcal{O}$ -form). *Fix an item v and an epoch; adopt the default bars of Rule R13(iv), A9, A10, A12, and, for the bucket rows, that each honest inspection, fresh or repeat, accepts a benign item with conditional probability at least p_b given all prior protocol events (for a repeat inspection of bytes the inspector previously accepted, the bound is if anything the more plausible). Adopt also the standing exception of Theorem J.2 (no owner eviction of the relevant local blocks; Option O3 is requirements-level and absent from the formal semantics, O3). Option-induced cache confirmations (Options O1–O2) are excluded throughout — from C_v^{pipe} , from the residual of A11, and from the pathwise count C_v of clause (iv): they are charged to the floor component, with the $\mathcal{O}(1)$ spread factors of Corollary J.6. Let $E_{\star} := E_{\text{max}} + 1$. Presence taxonomy. Within the epoch, the item’s shared-tier presence partitions into four classes of pieces: entry windows (at most E_{max} , by the per-item entry cap of Rule R14(i), counting promoted, evicted, and unpromoted entries alike; windows are epoch-contained, convention (C4)); freeze/suspension intervals (the current-epoch disputes and fast-track reviews, plus at most one carry-in instance — the byte-level freeze is exclusive, Rule R13(ii); a carried-in instance is counted toward Rule R14(i) in its opening epoch, convention (C4), but its current-epoch demanders are charged here, via the carry-in term of $\bar{D}_v$, and it issues no new prompts, Rule R13(vi)); candidate-dormancy intervals (no open window, no live promotion at any tool, unfrozen); and stable promoted intervals (a live, unsuspended community entry at some tool, no open window, unfrozen). Inter-window spans — each entry is followed by at most one, ending at the next entry, retirement, or the epoch boundary, so at most $E_{\star}$ begin in or carry into the epoch — have promotion status fixed at the preceding window’s close, so each span is candidate-dormant or stable-promoted throughout, with its freeze/suspension pieces carved out; A10’s dormant-interval clause is applied per candidate-dormancy span, freeze-time demanders within a span being charged to the freeze bucket. The first three classes are the ledger-charged pieces; stable promoted intervals are deliberately not bucketed: a matching literal fast-paths there at the promoted tool (Rule R4(a)) at zero cost, while the gates that do fire there — handle-expressed uses of the certified pair (Rule R4(c)) and uses at unpromoted tools — are charged to the ledger residual below, bounded by Theorem J.9(iii) under A11. (A stable promoted interval is “window closed, not yet resubmitted or retired” in the sense of Definition H.6(b): first decisions logged there bank toward the item’s next entry exactly as in candidate dormancy; the taxonomy separates the classes for cost attribution only and changes no protocol rule.) We use the common conservative bound $E_{\star}$ for the window- and dormancy-bucket piece counts. (The attempt structure of Definition I.1 partitions the same presence; the ledger’s buckets and bounds are unchanged by it.) Write*

$$\bar{D}_v := 2(1 - p_b) E_{\star} c_{\text{tool}}(\bar{\gamma} n_v + s_0) + (1 - p_b) E_{\text{max}} c_{\text{tool}} N^+ + c_{\text{accf}} + 2E_{\text{max}} + 1,$$

the final +1 is the carry-in instance; the invited-reconsideration term keeps $E_{\max}$ because invitations are epoch-scoped (Definition H.6). $\bar{D}_v$ is a function of the realised demand n_v : the bucket rows bound conditional expectations given the demand realisation, and the split in (iii) follows by taking the expectation over demand last, licensed by the demand construction of §L.1: the behavioural bounds are asserted uniformly over demand realisations, so Lemma L.2 applies under the demand-conditional measure. For poisoned items no p_b bound is asserted: their entries are counted by Corollary I.27 and their residual demand is charged at the single-user cap (Theorem J.2).

Pipeline component and ledger residual. The rows cover the pipeline component C_v^{pipe} : every confirmation of a pair of v charged to a ledger row under the convention below — credited weight-bearing endorsements, bucketed forced confirmations (window, candidate-dormancy, and freeze/suspension time, carry-in included), and all invited-repeat, prompt, and publish confirmations of v . Forced confirmations charged to no row are the ledger residual: those of demanders whose epoch-first item-demand falls in a stable promoted interval (in particular handle-expressed uses of a live-CC pair, which gate under Rule R4(c) despite the certificate) or outside every shared-tier presence (before first publication, after retirement, or after demotion). The residual is bounded by Theorem J.9(iii) under A11; any overlap between the two accountings only over-counts.

Charging convention (retrospective). At the close of the logging epoch every confirmation of v — a durable accepting pair-confirmation, counted once per pair per transaction (Definition F.5) — is assigned to exactly one row or to the ledger residual; the ledger is a hindsight accounting device, like A10, never protocol state. The ledger rows charge protocol-bearing decisions and invited and publish events (Definition H.6); the residual is defined over all forced confirmations of the bytes, protocol-bearing or not — in particular outside-presence confirmations, which are unlogged as candidate decisions — so that every forced confirmation in this lemma’s accounting universe (option-induced cache confirmations excluded, above) is charged somewhere, to a row or to the residual, by construction. (Definition H.4’s retirement clause governs demand counting, not the confirmation universe of this convention.) Non-covered reasoning-target confirmations satisfying Definition H.6 are protocol-bearing (§H.1) and are charged to the pair $(v, \text{reasoning})$ exactly as tool pairs are; covered reasoning gates are inert (Rule R13(i)) and excluded like any redundant confirmation. Credited weight-bearing endorsements (in-entry, banked-and-credited within the epoch, or vindication-credited; forced or invited alike) are assigned to the weight row — including a banked first decision taken in a stable promoted interval, so a residual-class demander’s credited acceptance is weight-row-charged, only shrinking the residual; uncredited invited confirmations to their invitation row (re-entry or prompt; invitations are epoch-scoped, so no invitation row carries over); publish confirmations to the publish row; every other forced confirmation to the ledger-charged piece (window, candidate-dormancy, or freeze/suspension — current-epoch instances and the at most one carry-in instance) containing its demander’s first demand of the item in the epoch, at most c_{tool} confirmations per demander (one per pair; INV-3, A12), when that first item-demand falls in a ledger-charged piece, and to the ledger residual otherwise (first item-demand in a stable promoted interval or outside every presence). Cross-epoch crediting of an already-logged event (Definition H.6(b), Rule R13(iv)) relabels a prior epoch’s event and adds nothing to any epoch. The four presence classes partition the item’s shared-tier presence within the epoch, so every demander’s epoch-first item-demand falls in exactly one piece or outside every presence; the assignment classes are disjoint and exhaustive over forced, voluntary, and publish confirmations (redundant confirmations excluded throughout, §J.1) — in particular every forced confirmation is charged somewhere, to a bucket row or to the residual, by construction — and each row bound is an upper bound for its assigned class. Then:

- (i) (Once, ever, per pair.) No user’s forced count for any pair (v, t) exceeds one over the deployment’s lifetime (INV-3; exceptions per Theorem J.2), a fortiori one per epoch; forced confirmations of (v, t) belong to distinct demanders of the pair.
- (ii) (Row bounds.) Every pipeline-component confirmation is assigned to exactly one row of Table 14 under the retrospective charging convention above, with the stated per-epoch bounds; cross-epoch crediting of an already-logged event (Rule R13(iv), Definition H.6(b)) is a relabel adding no event to any epoch.

- (iii) (Split.) Summing rows, there are $A(M)$ and B_{fl} depending only on protocol constants, with

$$A(M) = \mathcal{O}(E_{\max} c_{\text{tool}}^2 (N + c_{\text{acc}} f)(1 + \kappa)) \text{ (via } N^+ < N + D_{\text{pool}} \leq 2N),$$

$$B_{\text{fl}} = \mathcal{O}(E_{\max} c_{\text{tool}}^2 (1 + (1 - p_b) \kappa)),$$

such that $\mathbb{E}[C_v^{\text{pipe}}] \leq A(M) \mathbb{P}[n_v \geq 1] + B_{\text{fl}} \bar{\gamma} \mathbb{E}[n_v]$; the pipeline forced total obeys the same bound. Explicit constants: Lemma L.18 (§L.3).

- (iv) (Pathwise, all baseline-counted confirmations.) Let C_v count all forced, voluntary, and publish confirmations of v ’s exact bytes in the epoch, wherever they fall — ledger-charged and residual alike; redundant confirmations and option-induced cache confirmations excluded (§J.1; Corollary J.6). In the single-epoch model, with $K :=$

Row	Kind	Bound per item per epoch
Weight-bearing endorsements	forced/vol.	$\leq E_{\max} c_{\text{tool}} N^+$: at most N^+ credited per tool toward each entry (Definition H.6(f); credited by the close of the logging epoch, within-epoch banked credits included; cross-epoch crediting relates a prior epoch's event and adds nothing), credited $\Rightarrow$ authority-active (Definition H.6(a)); $\leq c_{\text{tool}}$ demanded tools (A12), entries $\leq E_{\max}$ deterministically (entry cap, Rule R14(i)), credited sets identity-disjoint across entries (Lemma I.3); vindication-credited prompted acceptances included, forced or invited alike.
Window-bucket confirmations	forced	$\leq E_{\star} c_{\text{tool}}^2 (\bar{\gamma} n_v + s_0)$: a user's first <i>item-demand</i> in a window implies some <i>pair-first-demand</i> in that window, so $\leq c_{\text{tool}} (\bar{\gamma} n_v + s_0)$ demanders have their first item-demand in it (A10, demand clause, per demanded pair; $\leq c_{\text{tool}}$ pairs, A12), each charged $\leq c_{\text{tool}}$ uncredited forced confirmations (one per pair, INV-3), over $\leq E_{\star}$ windows.
Dormancy-bucket confirmations	forced	$\leq E_{\star} c_{\text{tool}}^2 (\bar{\gamma} n_v + s_0)$: identically, over $\leq E_{\star}$ candidate-dormancy intervals (presence taxonomy, Lemma J.7); freshness consumed, decision banked (Definition H.6(b)), crediting charges nothing. Stable promoted intervals are charged to no row: their forced gates are ledger residual (Theorem J.9(iii)).
Freeze-bucket confirmations	forced	expected current-epoch disputes and reviews, plus the at most one carry-in instance, $\leq \bar{D}_v$; each admits $\leq \kappa$ arriving demanders in expectation while unresolved (Lemma I.6) — item-arrivals during the freeze or suspension, exactly what Lemma I.6 counts; each such demander (including every demander gating during a suspension, Rule R13(vi)) is charged $\leq c_{\text{tool}}$ confirmations; total $\leq c_{\text{tool}} \bar{D}_v \kappa$ in expectation. Each is owed on any unprompted pair regardless (Lemma I.24(ii)).
Repeat re-inspections	voluntary	$\leq E_{\max} c_{\text{tool}} N^+$: notified cohorts are the immediately preceding entry's credited endorsers only (Definition H.6), at most N^+ credited slots per demanded tool (Definition H.6(f)) ($\leq c_{\text{tool}}$ tools, A12), cohorts identity-disjoint across re-entries (Lemma I.3), over $\leq E_{\max}$ re-entries; declinable at zero cost.
Suspension-prompt inspections	re- voluntary	$\leq E_{\max} c_p k$: $\leq c_p k$ prompts per fast-track review, reviews $\leq E_{\max}$ per item per epoch (Rule R13(vi)); invitations deduplicated jointly; absorbed into the κ' term of $A(M)$ (Lemma L.18).
Publish confirmations	owner	$\leq E_{\max}$: one per opened entry (Rules R12, R14(i)).

Table 14: The per-item confirmation ledger. “Forced” and “voluntary” per §J.1. Bucket rows use the conservative $E_{\star} = E_{\max} + 1$ (Lemma J.7); windows need only $E_{\max}$. Stable promoted intervals are charged to no row; their forced confirmations form the ledger residual, bounded by Theorem J.9(iii) under A11.

$$E_{\max}(c_{\text{tool}}N^+ + c_p k + 1),$$

$$C_v \leq c_{\text{tool}} n_v + K \mathbf{1}\{n_v \geq 1\} :$$

every forced confirmation of a pair of v belongs to a demander of the pair — its confirmer's own attempted use fires the gate, hence realizes the latent indicator (forced-confirmer clause, §J.1), inside or outside any shared-tier presence — at most c_{tool} pairs per demander (one per pair, INV-3, A12); voluntary confirmations exist only at invitation surfaces, by definition (§J.1), and invitations per epoch are deterministically capped at $E_{\max} c_{\text{tool}} N^+$ re-entry notifications plus $E_{\max} c_p k$ suspension prompts, independently of current-epoch demand — notified endorsers and prompted adopters need not be current-epoch demanders — with joint deduplication (Definition H.6, Rule R13(vi)); the owner contributes at most $E_{\max}$ publish confirmations (one accepting publish per opened entry: a cap-exhausted submission is ineligible and records no accepting confirmation; Rules R12, R14(i)). Every term vanishes on $\{n_v = 0\}$, by the event-based vanishing clause of the owner-coupling convention and the forced-confirmer clause (§J.1): no latent demander means no attempted use, hence no forced confirmation, and no entry, review, invitation, or publish confirmation occurs in the epoch, whatever presence is carried in.

Proof. (i) After a user's own accepting confirmation of (v, t) , the local in-scope C copy covers all their reuse at t for the deployment's lifetime (Lemma J.1) and no third party can disturb it (Lemma I.8); this is INV-3.

(ii) *Weight-bearing*: within an entry, credited endorsements per tool number at most N^+ by the credit caps (Definition H.6(f)), banked credits filling slots the same caps govern; each credited endorsement is a distinct demander's first protocol-bearing decision, so successive entries' credited sets are disjoint (Lemma I.3), entries per epoch at most $E_{\max}$ (the entry cap of Rule R14(i), republications and post-eviction republications included). *Buckets*: within the epoch, the item's shared-tier presence partitions into the four classes of the presence taxonomy: at most $E_{\max}$ windows, at most E_* candidate-dormancy pieces (one possibly carried in), the freeze/suspension intervals of the current-epoch disputes and reviews plus at most one carry-in instance, and the stable promoted intervals; every demander of v has exactly one epoch-first item-demand, in exactly one piece or outside every presence. A demander whose first item-demand falls in a window or candidate-dormancy piece has a pair-first-demand there for at least one of the item's $\leq c_{\text{tool}}$ pairs, so A10's demand clause (scoped to exactly these two piece types) bounds window/candidate-dormancy occupancy by $c_{\text{tool}}(\bar{\gamma} n_v + s_0)$ per piece; each such demander is charged at most c_{tool} forced pair-confirmations (once per pair per transaction, Definition F.5; once ever forced, INV-3), giving the two bucket rows with the factor E_* . Lemma I.6 bounds each freeze or review's arriving demanders by κ in expectation while unresolved — for the carry-in instance, counting only its current-epoch arrivals, a sub-count of the same dominated arrival sequence — each charged at most c_{tool} ; the freeze-bucket bound $c_{\text{tool}} \bar{D}_v \kappa$ follows by the tower property under the demand-conditional measure (§L.1). Demanders whose first item-demand falls in a stable promoted interval, or outside every presence, are charged to no bucket: their forced confirmations are the ledger residual, bounded by Theorem J.9(iii) under A11 (their credited acceptances, where any, are weight-row events already counted). *Disputes and reviews* ($\bar{D}_v$): disputes on v open only at protocol-bearing dispute-initiating rejections logged while v is unfrozen and *unpromoted at every tool* (freeze-time rejections corroborate, Rule R13(iii); a qualifying rejection on bytes holding CC status at any tool is a post-promotion rejection routed to clause (vi), Rule R13(ii), counted below within the review cap) — hence only during window or candidate-dormancy pieces — at most one initiation per identity per item per epoch (Rule R13(i)). An honest identity's initiating rejection is the rejecting branch of its first *audit-recorded* decision on that pair in the epoch, at a non-covered gate (cache gates included: both branches are audit-recorded, §L.1) with tool or reasoning target while v is unfrozen (an earlier audit-recorded accepting decision on the pair would have created durable coverage, Lemma J.1, making the gate covered, hence inert; an earlier rejecting decision — federation-logged in every case, Definition H.6(iii) — would make the present one non-initiating; abandonment records nothing). The trial is indexed at the pre-outcome header atom of an audit-recorded first pair-decision, so its occurrence never selects the outcome, and the occurrences are capped by A10's first-pair-decision clause: at most $\bar{\gamma} n_v + s_0$ per pair per piece, $\leq c_{\text{tool}}$ pairs, over at most E_* window pieces and E_* candidate-dormancy pieces — conservatively $2E_*c_{\text{tool}}(\bar{\gamma} n_v + s_0)$ candidate occurrences, each rejecting a benign item with conditional probability at most $1 - p_b$ at its header atom (the p_b hypothesis covers every honest inspection, cache gates included; Lemma L.1(ii), Lemma L.2(iii)), contributing the first term. *Invited reconsiderations* are epoch-scoped (Definition H.6): at most $E_{\max}c_{\text{tool}}N^+$ per epoch, each rejecting with conditional probability at most $1 - p_b$ (repeat form of the hypothesis), contributing the second term. *Adversarial initiations* that take effect are either eventually charged, at most c_{acc} per identity per epoch (Lemma I.4), $\leq c_{\text{acc}}f$ in total; or terminal (eviction or expiry), retiring that presence, at most one per presence and $\leq E_{\max}$ presences per epoch. *Post-promotion reviews* — which absorb every rejection logged during a stable promoted interval, honest or adversarial — are capped at $E_{\max}$ per item per epoch (Rule R13(vi)). The carry-in instance adds 1. Summing gives $\bar{D}_v$. *Voluntary and publish rows*: Definition H.6, Rule R13(vi), Rule R12.

(iii) Summing the row bounds (with E_* in the bucket rows and the carry-in term of $\bar{D}_v$) and collecting the coefficient of $\bar{\gamma} n_v$ gives the split; every row's pathwise count vanishes on $\{n_v = 0\}$ (the event-based vanishing clause of the owner-coupling convention, §J.1: no entries, reviews, invitations, or demanders on $\{n_v = 0\}$, whatever presence is carried in), so each demand-free term is carried with the indicator $\mathbf{1}\{n_v \geq 1\}$ before taking expectations (the freeze bucket via Lemma L.2(iii) applied to the indicator-stopped counts), multiplying it by $\mathbb{P}[n_v \geq 1]$; the explicit algebra, with the stated $\mathcal{O}$ -forms verified, is Lemma L.18. Each forced confirmation belongs to a distinct demander, so the forced total obeys the same bound.

(iv) With $K := E_{\max}c_{\text{tool}}N^+ + E_{\max}c_p k + E_{\max}$:

$$C_v \leq c_{\text{tool}} n_v + K \mathbf{1}\{n_v \geq 1\}.$$

Forced: by the forced-confirmer clause (§J.1) every forced confirmer of a pair of v is a latent demander of v , contributing at most c_{tool} pair-confirmations, once per pair per transaction and once ever forced (INV-3, A12); voluntary: confirmations exist only at invitation surfaces (§J.1), which are epoch-scoped and deterministically capped at $E_{\max}c_{\text{tool}}N^+$ re-entry notifications plus $E_{\max}c_p k$ suspension prompts, with joint deduplication and boundary expiry (Definition H.6, Rule R13(vi)), independently of current-epoch demand; publish: at most $E_{\max}$ (one per opened entry; Rules R12, R14(i)). Every term vanishes on $\{n_v = 0\}$: no latent demander means no attempted use, hence no forced confirmation (forced-confirmer clause); an entry requires a current-epoch submission (owner demand); a review or dispute opens at a latent demander's protocol-bearing decision; invitations require a current-epoch entry or review, and

a carried-in review issues no new prompts (Rule R13(vi)); a carried-in freeze with no current-epoch arrivals charges nothing. $\square$

J.4 SATURATION

Lemma J.8 (Distinct-demand bound, two-term form). *Let $\mathcal{V}_{\text{good}} \subseteq \mathcal{V}$ be the benign (Definition H.10), assumption-conforming items — those to which the p_b , A10, A9, and A12 hypotheses of Lemma J.7 apply — and write $V_{\text{good}}(M) := \mathbb{E}|\{v \in \mathcal{V}_{\text{good}} : n_v \geq 1\}| \leq V(M)$ and $d_{\text{good}} := \sum_{v \in \mathcal{V}_{\text{good}}} p_v \leq d$. Then Under A10, A9, A12, and the competence bound of Lemma J.7, with $A(M)$, B_{fl} , κ as there,*

$$\mathbb{E}[\bar{C}_M^{\text{pipe,good}}] \leq A(M) \frac{V_{\text{good}}(M)}{M} + B_{\text{fl}} \bar{\gamma} d_{\text{good}},$$

where $\bar{C}_M^{\text{pipe,good}}$ counts all pipeline-component shareable confirmations, forced and voluntary (Lemma J.7), over $\mathcal{V}_{\text{good}}$; the ledger residual (Lemma J.7, charging convention) is bounded separately by Theorem J.9(iii) and added in Corollary J.10. The first term is carried by the number of distinct demanded items and vanishes under saturation; the second is the front-loading term: item popularity has cancelled out of it, leaving only the front-loading multiple $\bar{\gamma}$ against the standalone single-user shareable mass d . Demand on the complement is charged at the single-user cap and enters the residual r_{bad} of Corollary J.10.

Proof. By Lemma J.7(iii), $\mathbb{E}[C_v^{\text{pipe}}] \leq A(M) \mathbb{P}[n_v \geq 1] + B_{\text{fl}} \bar{\gamma} \mathbb{E}[n_v]$. Summing over $v \in \mathcal{V}_{\text{good}}$: $\sum_{v \in \mathcal{V}_{\text{good}}} \mathbb{P}[n_v \geq 1] = \mathbb{E}|\{v \in \mathcal{V}_{\text{good}} : n_v \geq 1\}| = V_{\text{good}}(M)$ and $\sum_{v \in \mathcal{V}_{\text{good}}} \mathbb{E}[n_v] = M d_{\text{good}}$; divide by M . (E_{max} inside $A(M)$ is a deterministic per-epoch entry bound, Rule R14(i); no expectation over entry counts is needed.) $\square$

Theorem J.9 (Confirmation cost under saturation). *Under A10, A9, A12, the competence bound of Lemma J.7, and — consumed only by the limit ($\rightarrow 0$) clauses below — A13:*

- (i) *if $V(M) = o(M)$ and $A(M)V(M) = o(M)$, then $\mathbb{E}[\bar{C}_M^{\text{pipe,good}}] \leq o(1) + B_{\text{fl}} \bar{\gamma} d_{\text{good}}$: the ledger-counted shareable cost falls to at most the front-loading multiple $B_{\text{fl}} \bar{\gamma}$ of the standalone single-user shareable mass, independently of item popularity; if in addition $\bar{\gamma}(M)(1 + \kappa(M)) \rightarrow 0$, then $\mathbb{E}[\bar{C}_M^{\text{pipe,good}}] \rightarrow 0$ (an upper-bound statement: the displayed bounds vanish);*
- (ii) *with $V(M) = \Theta(M^\beta)$, $\beta \in (0, 1)$, the protocol constants fixed, and additionally $f(M) = \mathcal{O}(1)$ and $\inf_M \rho_{\text{seat}}(M) > 0$ — hence $\kappa(M) = \mathcal{O}(1)$, so $A(M) = \mathcal{O}(1)$ and $B_{\text{fl}}(M) = \mathcal{O}(1)$; the added hypotheses are not implied by fixing the protocol constants, f being an adversary resource and ρ_{seat} a behavioural parameter — (i)’s hypothesis holds automatically: $\mathbb{E}[\bar{C}_M^{\text{pipe,good}}] \leq \mathcal{O}(M^{\beta-1}) + B_{\text{fl}} \bar{\gamma} d_{\text{good}}$, vanishing whenever $\bar{\gamma}(M) \rightarrow 0$ (under these hypotheses equivalent to (i)’s condition $\bar{\gamma}(M)(1 + \kappa(M)) \rightarrow 0$). This is the fixed-adversary, fixed-constants regime; the growing-adversary regime is the corridor (§J.5, Theorem J.13);*
- (iii) (Ledger residual, benign conforming component.) *under A11 and A12, the expected per-user forced cost charged to no ledger row over the benign, A11-conforming items — the ledger residual of Lemma J.7: forced confirmations of demanders whose epoch-first item-demand falls in a stable promoted interval (in particular handle-expressed use of a live-CC pair, which gates under Rule R4(c)) or outside every shared-tier presence (before first publication, after retirement, or after demotion); residual demand on poisoned or A11-violating pairs enters $r_{\text{bad}}^{(e)}$ (Corollary J.10) instead — is at most $c_{\text{tool}}(D_0(M)V(M)/M + \varepsilon(M)d_{\text{good}}(M))$ ($d_{\text{good}} \leq d$, $V_{\text{good}}(M) \leq V(M)$ as in Lemma J.8). This bound vanishes if, in addition, $D_0(M)V(M) = o(M)$ and $\varepsilon(M)d_{\text{good}}(M) \rightarrow 0$; under A13 the latter follows from $\varepsilon(M) \rightarrow 0$. In particular, under the Heaps form $V(M) = \Theta(M^\beta)$ of clause (ii), $D_0(M) = o(M^{1-\beta})$ supplies the product condition. The product condition is an additional rate hypothesis of this vanishing conclusion: neither A11 nor clause (i)’s hypotheses alone implies it.*

Proof. (i)–(ii) Immediate from Lemma J.8: $A(M)V(M)/M = o(1)$ by hypothesis (under Heaps’ form with the hypotheses of (ii) — protocol constants fixed, $f(M) = \mathcal{O}(1)$, ρ_{seat} bounded below — $A(M) = \mathcal{O}(1)$, $\kappa(M) = \mathcal{O}(1)$, and $V(M)/M = \mathcal{O}(M^{\beta-1})$), and $B_{\text{fl}} = \mathcal{O}(E_{\text{max}} c_{\text{tool}}^2(1 + \kappa)) = \mathcal{O}(E_{\text{max}}(1 + \kappa))$ ($c_{\text{tool}} = \mathcal{O}(1)$ absorbed), so the second term is $\mathcal{O}(E_{\text{max}}(1 + \kappa)\bar{\gamma}d_{\text{good}})$, vanishing whenever $\bar{\gamma}(M)(1 + \kappa(M)) \rightarrow 0$, the mass factor being uniformly bounded ($d_{\text{good}} \leq d$, A13) (a sufficient condition: the displayed bound is an upper bound, and degenerate parameter regimes, e.g. $p_b = 1$, can shrink it further). (iii) Fix a benign, A11-conforming item v and a target t . Every forced confirmation of (v, t) belongs to a distinct demander of the pair (INV-3). Under the charging convention of Lemma J.7, each is charged to a bucket row (its demander’s epoch-first item-demand lying in a ledger-charged piece), to the weight row (credited), or to the ledger residual; nothing is uncharged. The residual forced confirmations of (v, t) number

exactly $U_{v,t}$ (A11), with $U_{v,t} \leq n_{v,t}$ pathwise; conditioning on $n_{v,t}$ and applying A11 (whose bound holds trivially on $\{n_{v,t} = 0\}$, where $U_{v,t} = 0$),

$$\mathbb{E}[U_{v,t}] \leq D_0(M) \mathbb{P}[n_{v,t} \geq 1] + \varepsilon(M) \mathbb{E}[n_{v,t}].$$

Now sum over *all* targets t , using A12 pathwise on each term:

$$\sum_t \mathbb{P}[n_{v,t} \geq 1] = \mathbb{E}[\#\{t : n_{v,t} \geq 1\}] \leq c_{\text{tool}} \mathbb{P}[n_v \geq 1], \quad \sum_t \mathbb{E}[n_{v,t}] = \mathbb{E}\left[\sum_t n_{v,t}\right] \leq c_{\text{tool}} \mathbb{E}[n_v],$$

the first because at most c_{tool} tools carry demand and none does on $\{n_v = 0\}$ ($n_{v,t} \leq n_v$), so $\#\{t : n_{v,t} \geq 1\} \leq c_{\text{tool}} \mathbf{1}\{n_v \geq 1\}$ pathwise; the second by $\sum_t n_{v,t} \leq c_{\text{tool}} n_v$. Summing over the benign, A11-conforming items ($\sum_{v \in \mathcal{V}_{\text{good}}} \mathbb{P}[n_v \geq 1] = V_{\text{good}}(M) \leq V(M)$, $\sum_{v \in \mathcal{V}_{\text{good}}} \mathbb{E}[n_v] = M d_{\text{good}}(M)$, Lemma J.8) and dividing by M gives $c_{\text{tool}}(D_0(M)V(M)/M + \varepsilon(M) d_{\text{good}}(M))$, which vanishes under the additional rate hypotheses of the statement ($D_0(M)V(M) = o(M)$; $\varepsilon(M)d_{\text{good}}(M) \rightarrow 0$, supplied under A13 by $\varepsilon(M) \rightarrow 0$), with $c_{\text{tool}} = \mathcal{O}(1)$; under the Heaps form of clause (ii), $D_0(M) = o(M^{1-\beta})$ supplies the product condition. (Beyond A11, the pair-to-item aggregation consumes only the bounded tool-spread of A12; the vanishing limit additionally consumes the stated rate hypotheses and, where $\varepsilon(M) \rightarrow 0$ is invoked, A13. Demand at pairs where A11 fails enters $r_{\text{bad}}^{(e)}$, Corollary J.10.) $\square$

Corollary J.10 (Per-epoch shared cost; lifetime reading). *Fix an analyzed epoch e satisfying the hypotheses of Theorem J.9(i)–(iii) including the additional rate conditions of clause (iii), read per epoch: $D_{0,e}(M) V_e(M) = o(M)$ and $\varepsilon_e(M) d_{\text{good},e}(M) \rightarrow 0$ (under A13, from $\varepsilon_e(M) \rightarrow 0$); let $r_{\text{bad}}^{(e)} = r_{\text{bad}}^{(e)}(M)$ be the expected per-user demand mass in epoch e on Bad (Definition H.10) or A11-violating pairs (each such pair charged at most once ever, Theorem J.2(i)). Then*

$$\limsup_{M \rightarrow \infty} \mathbb{E}[\bar{C}_{M,\text{share}}^{(e)}] \leq \limsup_{M \rightarrow \infty} \left(B_{\text{fl}}(M) \bar{\gamma}(M) d_{\text{good}} + r_{\text{bad}}^{(e)}(M) \right),$$

and under a shrinking-front-loading schedule with bounded mass (§J.5, A13) the first term vanishes. The personal component is a lifetime inventory: at most d_{priv} expected forced pair-confirmations over the deployment’s life (Theorem J.2(ii)). The headline reading “lifetime personal inventory plus current-epoch shared load” therefore satisfies $\limsup_{M \rightarrow \infty} (\text{lifetime personal} + \text{epoch-}e \text{ shared}) \leq d_{\text{priv}} + \limsup_{M \rightarrow \infty} r_{\text{bad}}^{(e)}(M)$. This is not a bound on the sum of shared costs over an unbounded number of epochs: the present accounting yields a lifetime shared upper bound whenever the following series of per-epoch upper bounds is summable — a sufficient condition only: the terms are loose upper bounds, so divergence of this series implies nothing about the true lifetime cost — with the coverage parameters read per epoch ($D_{0,e}, \varepsilon_e$; a deployment asserting A11 with epoch-uniform parameters drops the subscripts),

$$\sum_e \left(A_e(M) \frac{V_e(M)}{M} + B_{\text{fl}}^{(e)}(M) \bar{\gamma}_e(M) d_{\text{good},e} + c_{\text{tool}}(D_{0,e}(M) \frac{V_e(M)}{M} + \varepsilon_e(M) d_{\text{good},e}) + r_{\text{bad}}^{(e)} \right) < \infty,$$

in which case lifetime expected total cost is at most d_{priv} plus that sum; the lifetime shareable mass comparison of Theorem J.2(iii) (d_{tool}^∞) bounds the forced component unconditionally in any case. These are upper bounds; no matching lower bound is claimed.

Proof. Combine Theorem J.9(i)–(iii) over epoch e with Theorem J.2(i)–(iii); the summable case is termwise. $\square$

Remark J.11 (Reading of Theorem J.9). Four observations govern the interpretation of Theorem J.9. First, the residual charges are the design operating as intended: each residual poisoned-demand confirmation is a personal gate inspection of an unpromoted or flagged item, and each residual handle-arrival confirmation is a personal gate inspection of bytes that reached this user through an untrusted channel — the certificate amortises trusted-channel reuse, never untrusted-channel arrival (Rule R4(c), S18); both are exactly the inspections the design intends (cf. A11). Second, the front-loading term is the price of certifying popular items rather than an overhead: each within-window confirmation is a fresh honest inspection *before finalisation*: exactly the pre-promotion scrutiny the exponent $N - f$ of Theorem I.13 counts. Third, the bound is informative when $\bar{\gamma} \lesssim 1/B_{\text{fl}}$; for larger $\bar{\gamma}$ it remains valid but loose, a bounded multiple of d , with Theorem J.2 the binding guarantee throughout. Fourth, these are conditional *accounting* statements. Clauses (i)–(ii) consume the ledger, smoothness, competence, juror-supply, and demand-shape hypotheses; clause (iii) additionally consumes A11 — the assumption carrying publication, promotion, and arrival-representation coverage jointly — together with the product condition $D_0(M)V(M) = o(M)$ and the remaining vanishing conditions of Theorem J.9(iii). The shrinking-front-loading conditions describe changing time geometry, never a consequence of population growth alone (Remark J.12); no clause derives coverage or shrinking front-loading from the protocol. The assumption-free protocol guarantee is the pathwise cap of Theorem J.2(i), with clause (ii) giving its reference-model standalone-mass ceiling.

J.5 CONDITIONAL CO-SCALING

Status of the corridor. The corridor collects, in one place, every constraint the co-scaling result uses; each row exists elsewhere and none is new. Two of its rows are *schedules* of protocol constants (g_N, g_k), which a deployment controls; the rest are empirical or behavioural properties ($\beta, \bar{\gamma}, \rho_{\text{seat}}, q + p_b, d$, the coverage law (D_0, ε) of A11, the Sybil bounds), which a deployment *measures* (N17) and cannot set. The co-scaling theorem is therefore a compatibility statement: *if* the measured environment satisfies its rows, *then* schedules exist under which security, resolution correctness, and budget improve together; and the floor of Theorem J.2 holds outside the corridor unconditionally.

Constraint	Source	Role	Example schedule
$V(M) = \Theta(M^\beta), \beta \in (0, 1)$	Heaps saturation	budget decay rate	β measured
$\sup_M d(M) < \infty$	A13	mass factor of the front-loading term bounded	automatic under a fixed demand law; measured
$f(M) = o(M^{1-\beta})$	Sybil condition	margins can grow	$f = \log M$
$f_s(M) = o(g_k(M))$	Thm J.13	panel margin positive	$f_s = \mathcal{O}(1)$
$\bar{\gamma}(M)(1 + \kappa(M)) = o(1)$	A10, A9	front-loading term vanishes	$\bar{\gamma} = \log^{-3} M$ measured
$D_0(M)V(M) = o(M), \varepsilon(M) \rightarrow 0$ (Heaps: $D_0 = o(M^{1-\beta})$)	A11, Thm J.9(iii)	coverage residual vanishes (with A13: $\varepsilon d_{\text{good}} \rightarrow 0$)	measured: $D_0 \asymp N/\bar{\gamma} = \Theta(\log^4 M), \varepsilon \rightarrow 0$
$N(M)k(M) = o(M^{1-\beta})$	cross-term, Thm J.13	dispute load sublinear	$2 \log^3 M = o(M^{1-\beta})$
$g_N, g_k \rightarrow \infty, = o(M^{1-\beta})$	margin schedules	exponents diverge	$g_N = \log M, g_k = \log^2 M$
ρ_{seat} bounded below	A9	$\kappa = \Theta(k)$	measured, logged
$q + p_b > 1$	competence (Rem. I.15)	joint safety and resolution correctness	measured, logged

Remark J.12 (Operational reading of the shrinking-front-loading rows). For steady demand over an epoch of length L with fixed window T_{win} , $\bar{\gamma} \approx T_{\text{win}}/L$ independently of M (A10, commentary): population growth alone does not shrink $\bar{\gamma}$. The rows $\bar{\gamma}(M)(1 + \kappa(M)) = o(1)$ and $\bar{\gamma}(M)k(M) \rightarrow 0$ therefore describe deployments whose *time geometry* changes with scale — finalisation windows shrinking relative to the epoch, or demand growing more diffuse as the population grows — and are to be read as measured properties of that geometry, not consequences of $M \rightarrow \infty$. Where they fail, the fixed- $\bar{\gamma}$ reading of Theorem J.9 is the operating point: pipeline cost falls to the front-loading multiple $B_{\text{fl}} \bar{\gamma} d_{\text{good}}$ of the standalone single-user shareable mass, with Theorem J.2 binding throughput (Remark J.11).

A concrete schedule satisfying every row simultaneously: $f_s = \mathcal{O}(1)$, $f(M) = \log M$, $N(M) = 2 \log M$, $k(M) = f_s + \log^2 M$, against measured $\bar{\gamma}(M) = \log^{-3} M$ and bounded measured d (A13); then with ρ_{seat} bounded below, $\kappa = \Theta(\log^2 M)$, so $\bar{\gamma}(1 + \kappa) = \Theta(\log^{-1} M) \rightarrow 0$; for an illustrative measured coverage law with $D_0 \asymp N/\bar{\gamma} = \Theta(\log^4 M)$ and empirically validated $\varepsilon(M) \rightarrow 0$, $D_0 V = \Theta(M^\beta \log^4 M) = o(M)$; and $Nk = \Theta(\log^3 M) = o(M^{1-\beta})$.

Theorem J.13 (Joint scaling of safety, wrongful-eviction risk, and cost). *Adopt Rule R13 with the default bars, A9, A10, A11, A12, A13, and saturation $V(M) = \Theta(M^\beta)$, $\beta \in (0, 1)$. Assume also the coverage-rate condition of Theorem J.9(iii), $D_0(M)V(M) = o(M)$ (under the Heaps form, equivalently $D_0(M) = o(M^{1-\beta})$; $\varepsilon(M) \rightarrow 0$ per A11). Suppose $f(M) = o(M^{1-\beta})$ and $q + p_b > 1$; fix $\gamma_0 \in (0, q + p_b - 1)$. Choose margin schedules $g_N(M), g_k(M) \rightarrow \infty$ with $g_N, g_k = o(M^{1-\beta})$ and $f_s(M) = o(g_k(M))$, and set $N(M) = f(M) + g_N(M)$, $k(M) = f_s(M) + g_k(M)$, $m(M) = f_s(M) + \lceil (1 - q + \gamma_0)g_k(M) \rceil$, $m_{\text{rej}}(M) = k(M) + 1 - m(M)$ (so every panel that gathers k verdicts resolves decisively and $m_{\text{rej}} > f_s$ for all large M). All protocol caps and multiplicity parameters not explicitly scheduled here are uniformly $\mathcal{O}(1)$ in M (fixed TCB configuration along the scaling, convention (C8)); in particular $E_{\text{max}}, \rho_{\text{sub}}, r_{\text{max}}, c_{\text{ceil}}, c_p, s_0, D_{\text{pool}}, c_{\text{batch}}, c_{\text{cand}} = \mathcal{O}(1)$, hence $c_{\text{acc}} = c_{\text{ceil}} + r_{\text{max}} = \mathcal{O}(1)$, while $c_{\text{tool}} = \mathcal{O}(1)$ by A12. This is load-bearing: the conclusions $A(M) = \mathcal{O}(E_{\text{max}} c_{\text{tool}}^2 (N + c_{\text{acc}} f)(1 + \kappa)) = o(M^{1-\beta})$ and $K(M) = o(M^{1-\beta})$ consume it. Assume further the cross-term condition $N(M)k(M) = o(M^{1-\beta})$ (which, since $N > f$, implies $f(M)k(M) = o(M^{1-\beta})$) and the shrinking front-loading condition $\bar{\gamma}(M)k(M) \rightarrow 0$ ($\kappa = \Theta(k)$ for ρ_{seat} bounded below); both are achievable by choosing g_k to grow slowly against the measured front-loading of the demand stream. Then:*

- (i) (Safety, per poisoned entry.) $\mathbb{P}[\text{promotion of a fixed poisoned entry}] \leq (1 - q)^{g_N(M)} + e^{-2\gamma_0^2 g_k(M)} \rightarrow 0$ (Theorem I.13), against adaptive presentation and at any reach (INV-6). (System-wide, per anchor epoch.) If additionally $2f(M)\rho_{\text{sub}}((1 - q)^{g_N(M)} + e^{-2\gamma_0^2 g_k(M)}) \rightarrow 0$, the per-anchor-epoch bound of Corollary I.27 over

all adversary-published attempts anchored in the epoch also vanishes; the example schedule satisfies this: with $f = \log M$, $g_N = \log M$ (natural logarithm throughout; a fixed change of logarithm base only rescales the schedule by a constant and leaves every limit below unaffected), $2f\rho_{\text{sub}}(1-q)^{g_N} = 2\rho_{\text{sub}} \log M \cdot M^{\ln(1-q)} \rightarrow 0$, and $2f\rho_{\text{sub}} e^{-2\gamma_0^2 \log^2 M} \rightarrow 0$ a fortiori;

(ii) (Wrongful eviction.) with $\delta_b := \frac{q+p_b-1-\gamma_0}{2} > 0$, for all sufficiently large M a benign item is evicted, penalising its publisher, with probability at most $e^{-2\delta_b^2 g_k(M)} \rightarrow 0$ (Propositions I.20, I.22); equivalently $\mathbb{P}[\text{vindication}] \geq \mathbb{P}[\text{resolution}] - e^{-2\delta_b^2 g_k(M)}$, resolution being the demand-side event of Remark I.21;

(iii) (Budget, per analyzed epoch.) with ρ_{seat} bounded below, $\mathbb{E}[\bar{C}_M^{\text{pipe,good}}] \leq A(M) \mathcal{O}(M^{\beta-1}) + B_{\text{fl}} \bar{\gamma}(M) d_{\text{good}} = o(1)$, and the benign ledger residual is $o(1)$ (Theorem J.9(iii)): the analyzed epoch's shared confirmation load vanishes, while the lifetime personal inventory remains at most d_{priv} (Theorem J.2) and demand on Bad or assumption-violating pairs adds that epoch's residual $r_{\text{bad}}^{(e)}$, bounded pathwise by Theorem J.2(i). No claim is made about the sum over an unbounded number of epochs without the summability condition of Corollary J.10.

All three hold with no assumption on how widely a poison spreads.

Proof. (i) $N - f = g_N$, and $\delta = \frac{m-f_s}{k-f_s} - (1-q) \geq \gamma_0$ by the choice of m , so Theorem I.13 gives the bound with exponents g_N and $2\gamma_0^2 g_k$. (ii) $m/(k-f_s) \leq (1-q+\gamma_0) + (f_s+1)/g_k(M)$; since $1-q+\gamma_0 = p_b - 2\delta_b$ by the definition of δ_b (using $\gamma_0 < q+p_b-1$, so $\delta_b > 0$), and $(f_s+1)/g_k(M) \rightarrow 0$ is eventually at most δ_b , we get $m/(k-f_s) \leq p_b - \delta_b$ for all large M , and Proposition I.20 applies; Proposition I.22(ii) restates the same event as the publisher penalty. (iii) By Lemma J.8: $A(M) = \mathcal{O}(E_{\text{max}} c_{\text{tool}}^2 (N(M) + c_{\text{acc}} f(M)) (1 + \kappa(M))) = o(M^{1-\beta})$ by the margin schedules, the cross-term condition, and $\kappa = \Theta(k)$, so the first term is $o(M^{1-\beta}) \cdot \mathcal{O}(M^{\beta-1}) = o(1)$; the second term is $\mathcal{O}(E_{\text{max}} \kappa(M) \bar{\gamma}(M)) d = o(1)$ by the shrinking front-loading condition and the bounded mass (A13); the residual clause of Theorem J.9(iii) applies by the coverage-rate hypothesis $D_0(M) V(M) = o(M)$ assumed in the statement, with $\varepsilon(M) d_{\text{good}}(M) \rightarrow 0$ from $\varepsilon(M) \rightarrow 0$ (A11) and A13. $\square$

Remark J.14 (Role of the saturation exponent). Smaller β is uniformly favourable: faster budget decay $M^{-(1-\beta)}$ and a weaker Sybil requirement $f(M) = o(M^{1-\beta})$. There is no design trade-off along β : it is not a design parameter but an empirical property of the community's demand stream, to be measured (N17); Theorem I.11, which references neither β nor the demand model, remains available at any β under its own hypotheses (Remark I.12). A worked Zipf instantiation, including the tail, is §L.5.

K DESIGN RATIONALE, SECURITY ANALYSIS, AND SCOPE (informative)

Nothing in this appendix is normative and no proof depends on it (§E.1). §K.2 collects design rationale; §K.3 analyses anticipated attacks and limitations; §K.4 states scope and residuals; §K.5 summarises the composed guarantees.

K.1 ATTACK AND LIMITATION INDEX

Each entry of §K.3 carries a *disposition*: *disposed* (under stated assumptions; a formal fact disposes of it), *residual* (a genuine limitation, disclosed in §K.4), or mechanism disposed with a disclosed residual periphery; and a *proof role*: not used in any proof, or enters budget constants only (Figure 6).

Entry	Vector / limitation	Governing fact(s)	Disposition; proof role
<i>Endorsement-mechanism attacks (§K.3.1)</i>			
S1	paraphrase-variant families	Cor. I.27, R12; q calibration	mechanism disposed + residual; unused
S2	self-dispute	Thm. I.13(b), INV-5	disposed (under stated assumptions); unused
S3	reputation farming	lifetime totals f , W_{adv} , f_s (N12)	disposed (under stated assumptions); unused
S4	tool-targeted poison	Thm. I.13(c) (q_{pan} scope), Rule R13(iv), Lem. G.14	ordinary path disposed + panel-path residual; unused
S5	panel-lapse exploitation, ex-slot economics	Lem. I.24(iii)	disposed (under stated assumptions); unused
S6	juror exhaustion by pre-exposure	Lem. I.24(ii), Rem. I.25	safety disposed (under Thm. I.13's hypotheses) + availability residual; unused
<i>Side channels (§K.3.2)</i>			
S7	store-exhaustion channel	convention (C7), R7	disposed (under stated assumptions), conditional on (C7); unused
S8	timing channels	scope statement, Lem. G.18(e)	residual; unused
<i>The human factor (§K.3.3)</i>			
S9	preview manipulation	Lem. G.12, R8, DT coverage	mechanism disposed + residual; unused
S10	careless endorsement; post-endorsement egress	Thm. G.21, Cor. G.23(b), Cor. G.26, Prop. I.22	residual; unused
S11	recombination of endorsed values	Lem. G.12–G.18, O1	residual; unused
S12	batch-gate attention	R9, c_{batch} sizing	residual; unused
<i>Adverse parameter regimes (§K.3.4)</i>			
S13	cross-person correlated carelessness	q_c -degradation of Thm. I.13; D_{pool} ; re-verification (N16); Rem. I.12	residual; unused
S14	unrecognizable poison ($q \rightarrow 0$)	one-effective-qualifying-rejection freeze (R13(i),(v)); INV-1; Rem. I.12	residual; unused
S15	accountability miscalibration	Rule R13(v)(a),(e); theorem I.19	safety disposed + residual; constants only
<i>Modelling-boundary effects (§K.3.5)</i>			
S16	incentive feedback on publication	Thm. J.2 unaffected	residual; unused
S17	release-day demand spikes	Thm. J.2; item-locality of A10	disposed (under stated assumptions); unused
S18	handle-mediated arrival of certified content	Thm. J.2; A11 (ϵ); Rule R4(c)	residual; budget assumptions only

Table 15: Attack and limitation index. “Disposed” means disposed of by the cited formal facts *under their stated assumptions*; entries with a residual periphery say so.

K.2 DESIGN RATIONALE

Four rationale remarks whose content adjoins specific formal results (the freshness asymmetry, the reading of q , the accountability calibration, and the roles of the protocol clocks) are set as numbered Remarks adjacent to their points of use (Remarks H.7, I.19, I.5, I.26) and are indexed here for completeness. The notes below record the remaining design rationale.

Note N1 (Model lineage). The target property is an integrity property: content from an untrusted source must not drive a sensitive action unless the user explicitly authorised the exact value involved. Integrity is the lattice dual of Denning’s confidentiality flow Denning (1976); Denning & Denning (1977), made precise by Biba Biba (1977); the behavioural definition is noninterference Goguen & Meseguer (1982), proved by the unwinding method Goguen & Meseguer (1984) transported to a small-step operational semantics Volpano et al. (1996); Sabelfeld & Myers (2003). Two refinements distinguish PerMIT. First, PerMIT *does* permit untrusted-derived values to reach sensitive actions, but only after a user confirmation; a confirmation is an *endorsement* Myers et al. (2006), and the danger that an at-

tacker controls *what* is endorsed is ruled out by robustness Zdancewicz & Myers (2005); Myers et al. (2006). Second, PerMIT lets verified crystallized knowledge inform the Protected LLM’s reasoning; the right noninterference statement is therefore one modulo the release history, in the delimited- and gradual-release tradition Sabelfeld & Sands (2009); Sabelfeld & Myers (2004); Askarov & Sabelfeld (2007), with the display transcript playing the role of the release-event history. The carve-out is the *full* transcript: candidate previews as well as decision-pane displays, rejections as well as acceptances (Definition G.8). The substrate, including the provenance-tracking interpreter and the separation of a trusted orchestrating model from a quarantined reader, is CaMeL Debenedetti et al. (2025); we name these the Protected LLM (P) and the Quarantined Reader LLM (Q), realizing Q as a Recursive Language Model Zhang et al. (2025). Related efforts: AgentDojo Debenedetti et al. (2024), information-flow control for agents Costa et al. (2025), memory-poisoning defenses Wei et al. (2025), collaborative memory Rezazadeh et al. (2025). CaMeL executes a Turing-complete interpreted language and therefore, by its authors’ own observation, does not establish a full noninterference theorem; PerMIT draws the bounded surface elsewhere: it leaves the agent’s reasoning and tool use unrestricted and confines only the memory interface to the finite algebra of Figures 7–8, which is why the completeness of the channel analysis (Lemmas G.17–G.18) is a property of a written specification, checked clause by clause.

Note N2 (The endorsement unit). Endorsement is per (value, tool) and cached indefinitely (a conforming instantiation of the requirements-level Option O3 would bound the scope-checked tool-authorization lifetime; O3); *recombination* of separately endorsed values is ungated by design, and protection against malicious recombination rests on the agent’s isolation from the attacker (Lemmas G.12–G.18); the residual is the careless-endorsement channel (S10, S11); O1 hardens where deployments want it.

Note N3 (The reader asymmetry). Q may read the whole store because its output is always untrusted or handle-hidden; R_E may read only the endorsed store because its output flows to P as readable endorsed bytes. Neither can be relaxed without breaking a theorem: a U-reading R_E violates Theorem G.25; an $\mathbb{E}$ -emitting Q violates Lemma G.14.

Note N4 (Failure indistinguishability, operationally). Under Rule R7, the agent cannot distinguish “the untrusted store contained nothing useful” from “the reader failed while looking”: both worlds return the same operation-determined handle, so neither the contents nor the volume of untrusted memory can signal through the failure path. Likewise, whether an identifier is occupied by a U-block is a fact the attacker influences through the number of blocks captured per call, which is why occupancy is never observable (Rule R10(b)). The store-quota policy of convention (C7) exists because a volume-dependent error would reopen exactly this channel (S7).

Note N5 (The human channel). A6 idealizes the user’s typing as exogenous. In reality, displayed content can steer what the user subsequently types, which enters as trusted input. Influence of displayed content on typing is accounted for exactly insofar as that content appears in the display transcript, which, by Rule R8(e), records *every* attacker-influenceable rendering the user sees, on both surfaces. The design therefore discharges the idealization’s practical content by logging both surfaces: the release history covers every surface through which such influence can pass, so Corollary G.26 and N7 remain faithful even where A6 frays. The residual is disclosed at S9 and §K.4.

Note N6 (The batch gate is all-or-nothing). The selected batch design is all-or-nothing: a per-page decision returned to P would leak the attacker-influenceable page count (Rule R9(c)), and the aggregate decision avoids that channel at the cost that one poisoned element forces rejection of the whole batch — an availability and attention cost, not a safety gap. Sufficiency of this design is what is established; no impossibility of alternative fixed-shape or padded designs is claimed. The user may reject at any page, aborting the remainder (Rule R9(d)), so attacker-imposed attention is bounded by the user’s own patience, never compelled. Both facts argue for sizing c_{batch} conservatively (S12).

Note N7 (The transcript is recorded, not prevented). The harness records every display event, selection renderings included, in a hash-chained audit log, a tamper-evident materialization of DT. Corollary G.26 therefore has operational force: the transcript supplies a complete record of the modeled release surfaces relevant to the protected behaviours — ungated sinks, Protected-LLM operations, tool invocations (Definition G.11) — missing no surface on which attacker-shaped content was ever rendered; it does not by itself reconstruct causation, timing, unreleased tool outputs, or attacker identity. This is not a proof that the user cannot be manipulated; it is a proof that any attacker influence on those behaviours must pass through the logged display transcript or an explicitly accepted endorsement. Logging makes influence visible and attributable after the fact — attributable to a recorded release or decision, not necessarily to an attacker identity; the prevention half of the design is Theorem G.21’s gate, which stops any value that is neither user-typed nor endorsed (personally, or, under delegation, by a qualifying community quorum) for the exact sensitive tool from ever crossing that boundary (argument-value integrity: the decision to invoke a tool, and zero-argument sensitive calls, are scoped at §K.4(X)).

Note N8 (What previews can and cannot do). An adversarial reader can decorate a malicious candidate with a benign-looking preview. Within the formal model, previews are inert: they never enter σ_P (Lemma G.12), and endorsement binds to the true resolved bytes on the decision pane (A3–A4), so a preview can never forge an endorsement. Outside the formal model, a preview can influence the human: what the user chooses to examine, and, through A6’s idealiza-

tion, what the user subsequently types. This is precisely why previews are included in the display transcript and the audit log (S9).

Note N9 (What the gate does to the detection task). The confirmation gate reshapes the detection task an honest inspector faces, in three architectural ways. First, stealth is eliminated by construction: Rule R8 renders the exact bytes that will flow, in full, on every gated invocation, so no gated content can reach a sensitive action unseen — there is no channel by which poison avoids display altogether. Second, hijacking-style content — bytes written to be read as instructions rather than data — is surfaced on the decision pane out of the narrative context it was designed to exploit, isolated against a fixed provenance zone (A3–A4); this plausibly raises q_{pre} for that class of poison, but it is offered as motivation for the parameter, never as a proof of it: q_{pre} remains an empirical, weakest-target quantity (theorem I.19), estimated by logging, not derived from the display design. Third, because every gate decision is logged (Definition G.8), q is made log-measurable rather than merely posited: the validation protocol of N17 is only possible because the architecture records the events it needs. None of this touches *semantic* poison — content that is simply false or misleading, with no injection mechanism for display to expose — which gains nothing from being shown out of context and remains exactly the disclosed residual of S14, consumed by no proof in this document.

Note N10 (Federated lineage). The federated tier is built on machinery network security has already made rigorous: threshold endorsement from the PGP web of trust Maurer (1996); transparency and revocation from Certificate Transparency Laurie et al. (2013; 2021); the identity-cost requirement of the Sybil literature Douceur (2002); Dwork & Naor (2007), the precise reason the scaling results are conditional on an attested or cost-bounded setting. The panel’s competence condition is the classical jury condition Grofman et al. (1983); Allen et al. (2021); Martel et al. (2024).

Note N11 (What pool diversity buys). The requirement D_{pool} caps per-pool verdicts and mixes the fresh-inspector population, attacking correlated blind spots and demand-side selection bias among panelists. Diversity bounds *correlation*, not the *minimum*: since the adversary chooses whom to present the item to (§L.1), q remains a uniform lower bound over the honest identities the adversary can target, and the pool caps of Definition H.3 force those targets to span D_{pool} pools rather than one cluster, raising the effective bound from the most fallible cluster toward the most fallible identity per pool, never to a population average. The per-person correlation across *logged* re-inspections is discharged structurally by fresh-inspection counting (INV-1); the residual across unlogged personal inspections is a calibration matter (theorem I.19, Definition H.5, S13).

Note N12 (Reputation farming). f , W_{adv} , and f_s are lifetime totals over every identity the adversary ever controls (§H.1), capped by the attestation layer’s issuance; farming weight before spending it changes *when* the capacity exists, never its total, so no bound stated against the lifetime totals can be outrun. Publisher penalties (Rule R13(vii)) only shrink the adversary’s capacity below these totals, so every bound stated against them remains valid. The economic deterrent of burning accrued reputation is real but deliberately unused in any proof: the proofs assume the attacker spends everything (S3).

Note N13 (The evict residual). *evict* is owner-scoped but P -invocable and ungated, so an *erring* (not attacker-steered, which Lemmas G.12–G.18 exclude) Protected LLM can remove its owner’s endorsed blocks without a confirmation; the no-deletion guarantee of Lemma I.8 is a third-party guarantee only. Deployments wanting first-party cover may gate *evict* on $\mathbb{E}$ -labelled blocks like any sensitive call, a further monotone hardening in the sense of Proposition G.28. The uniform-ack totalisation of *evict* (Figure 8, convention (v)) changes only the reply surface, never this capability: the residual is unchanged, and the realised outcome remains visible to the owner in the audit log. The residual includes evictions P performs *in reaction to* legitimate federation events (notifications are $\mathbb{E}$ -level log data): a third party can occasion such a reaction but not steer it — Lemmas G.12–G.18 exclude any U -derived influence on the choice — so this is *erring-agent* behaviour, outside the threat model (§K.4(V)), not an attacker channel.

Note N14 (Availability degrades gracefully, never destructively). All federated guarantees are safety guarantees; availability is explicitly not guaranteed, but its loss is bounded *in harm* (in forced confirmations) per dispute: under A9, at most $\kappa = (k+1)/\rho_{\text{seat}}$ affected demander arrivals in expectation, with exponential tail (Lemma I.6); if A12 is additionally read over the entire unresolved-dispute demand horizon, at most $c_{\text{tool}} \kappa$ additional forced confirmations in expectation, with the corresponding tail at thresholds scaled by c_{tool} ; under the stated per-epoch reading of A12, only the corresponding per-epoch conversion is asserted, while the pathwise per-user cap remains valid for a freeze of any duration (Lemma I.24(ii)); pathwise, independent of every federated and demand assumption (persistence exceptions per INV-3), no affected user exceeds the single-user cap, regardless of the freeze’s duration (Lemma I.24(ii)); recoverable (low wrongful-eviction risk, Proposition I.20); accountable on both sides (Rule R13(v),(vii)); and never destructive (Lemma I.8): in the worst case the shared tier contributes nothing and each user verifies personally, within the single-user cap of Theorem J.2. The per-epoch free charge renews, so the adversary retains a per-epoch throughput bounded in the three currencies of Lemma I.24(iv); what it buys is bounded forced-confirmation harm and bounded per-epoch harassment throughput (Lemma I.24(ii),(iv)), never deletion — the *wall-clock duration* of an unresolved freeze is deliberately unbounded, resolution being demand-paced with no deadline (Remark I.21); its harm, not its duration, is what is capped. A promoted benign item can be harassed only through Suspended-state fast-track panels, at most E_{max} per item per epoch (Rule R13(vi)) and penalty-escalating. An unpromoted benign candidate cannot

be retired by sequential vindicated disputes (Rule R14(i)(b) exempts them, N15): each adversarial rejection buys one freeze, resolved at the item’s own demand rate, while occupying a rejection slot and, beyond the first per identity per epoch, escalating the rejecter’s accountability; the adversary’s per-epoch freeze throughput against any item being bounded (Lem. I.24(iv)), what it buys is bounded forced-confirmation harm within the single-user cap — of unbounded wall-clock duration, the freeze being demand-paced — never retirement, and the single-initiation clause caps any identity, adversarial or honest, at one freeze per item per epoch, so per-item harassment throughput is at most f freezes per epoch deterministically. The entry cap of Rule R14(i) is likewise availability-only: an item whose per-epoch entries are exhausted (e.g. through repeated wrongful evictions, each already exponentially rare, Proposition I.20) waits for the epoch boundary with its banked credits and any live promotion status intact (convention (C4)), each demander meanwhile paying at most the cap (Theorem J.2(i)). Under this invalidating semantics, a suspension additionally converts its demanders’ ungated reuse into at most one forced confirmation each — the cap confirmation they would in any case owe on demotion (Corollary J.5) — after which the pair is permanently covered by a local C copy; this is priced once per demander and never renewed.

Note N15 (The vindication exemption). Rule R14(i)(b) counts only *non-vindicated* disputes toward expiry. The exemption is what prevents both an adversary’s wrongful rejections (each vindicated, each feeding the accountability schedule of Rule R13(v)) and the honest wrongful-rejection noise, which arrives at rate $1 - p_b$ per benign fresh inspection and would otherwise, at the co-scaling thresholds $N(M) \rightarrow \infty$ of Theorem J.13, retire *every* item before promotion became arithmetically reachable, from terminating a benign item. Under the default bars every resolved dispute ends in vindication or eviction and every unresolved one ends in expiry, so this trigger fires only through the optional neither-bar outcome of Rule R13(iv); it remains availability-only and conservative (Lemma I.9).

Note N16 (Revocation, re-verification, and grandfathering). Post-promotion suspensions and re-verification add panel and re-inspection events but never multiply any user’s forced cost (Corollary J.5): local endorsements are permanent, so revocation campaigns tax the shared tier’s voluntary and verdict labour, bounded per suspension by Rule R13(vi) and the rate limits; a prior CC-only user additionally owes their first-ever forced confirmation of the pair — a cost *within* the cap, already counted inside d_{tool}^∞ (Corollary J.5), never beyond it. Re-verification draws its inspectors from identities with no prior *logged* decision on the bytes (log-fresh, Definition H.5; a cohort that can include previously exposed but never-logged inspectors, theorem I.19), so the probability of a uniformly convincing poison remaining uncaught across E_{rv} re-verification rounds, each reaching a cohort of new inspectors, is at most $(1 - q_c)^{E_{\text{rv}}} \rightarrow 0$, with q_c the correlated catch rate of S13; the mechanism’s strength scales with the supply of never-decided inspectors (population growth and new demand), not with epoch renewal. Grandfathering: items promoted at lower historical thresholds retain CC but enter the re-verification queue until their endorsement meets the current bar. Within the formal semantics, first-party eviction aside (N13), nothing re-forces a confirmed pair’s authorization; a conforming instantiation of the requirements-level Option O3 would re-force exactly the scope-checked tool authorization on lease expiry — never reasoning readability — an explicit trade stated and to be priced with that extension (O3).

Note N17 (Validation without a user study). Theorem J.2 and the counting cores of Lemmas J.7, J.8 are identities. Saturation ($V(M) = o(M)$), fractional temporal smoothness (A10: log each item’s realised front-loading fraction γ_v , validated specifically on the most-demanded items, where population averages would hide spikes), coverage sufficiency (A11: log each item’s realized residual fraction $U_{v,t}/n_{v,t}$ and the fast-path-vs-gate split of live-CC demand — the handle-arrival fraction, S18 — stratified on the most-demanded items, with per-item time-to-promotion against accumulated demand), tool-spread (A12: log per-item distinct-tool counts), bounded demand mass (A13: log per-user distinct shareable-item demand per epoch across deployment growth), and the verdict rate (A9: log verdict-eligibility rates on disputed items) are modelling assumptions checked by logging in the prototype and by replaying the promotion and demand-paced panel mechanics in simulation while sweeping the growth exponent, a replay of an algorithm, not an experiment on people. The replay’s own fidelity is checked independently of any behavioural parameter by the closed form of §L.2, whose support-level consequences are byte-identity properties of the implementation rather than statistical agreements. One caveat governs the behavioural parameters specifically: q_{pre} , q_{pan} , p_b , and ρ_{seat} are *uniform conditional* lower bounds, each validated on its own stratum (gate inspections vs. eligible verdicts), on the prior-atom σ -algebras of §L.1; aggregate log rates do not directly estimate such bounds. Their validation therefore reports stratified rates (by identity class, presentation context, and flag state) rather than population averages, uses lower confidence bounds rather than point estimates, corrects for adaptive selection of inspection targets, and monitors nonstationarity across epochs; the estimand is the infimum over strata, and the deployed constants (N, k, m margins, θ_{acc} , c_{ceil}) are set against that infimum with margin, never against a mean. These stratified, lower-confidence-bound estimates are *falsification and calibration* evidence: they can refute an assumed cap — a stratum whose observed rate falls below the claimed bound — and can inform recalibration, but they do not *certify* the uniform infimum-over-history bounds q_{pre} , q_{pan} , p_b require. Sparse or unobserved strata (rare identity classes, rare presentation contexts) are not covered by any finite sample, and the uniform bounds remain modelling hypotheses, not measured quantities. One further caveat: the classification Bad is exogenous (Definition H.10), so the log alone cannot label a trial as poisoned or benign; estimating q_{pre} , q_{pan} , and p_b therefore requires an exogenously labeled corpus — red-team-injected

known-poison items and expert-adjudicated benign samples — with the stratified lower-confidence-bound machinery applied to the labeled trials. Unlabeled log rates validate the protocol mechanics only (ρ_{seat} , realised γ_v , $U_{v,t}/n_{v,t}$, tool spread, demand mass).

Note N18 (Total effort, including rejections). The ledger of Lemma J.7 covers every *counted* (forced or voluntary) durable accepting pair-confirmation of shareable items, redundant covered-gate acceptances and the personal, per-task, and option-induced streams excluded or separately charged (§J.1, Corollary J.6); rejection and attention labour is a distinct resource with a two-sided account. Under the durable semantics of Rule R2 there are no accepted-but-uncommitted confirmations: every accepting review commits its copy. What remains uncounted is attention: rejected dialogs, abandonment, batch-page review, repeated reviews of an already-accepted pair within one transaction (counted once, Definition F.5), and redundant covered-gate decisions. *Protocol-effective rejections* are tightly capped: one dispute-initiation per identity per byte string per epoch (Rule R13(i)), at most r_{max} open slots, and the accountability schedule of Rule R13(v). *Rejection dialogs*, by contrast, recur: a user who rejects an uncovered pair holds no local C copy, so each later attempted use of that pair fires the gate again, and the later rejections are logged as corroboration or advisory flags (Rule R13(i)) — the *protocol* does not bound a user’s total rejection dialogs on one value. What bounds them is the user’s own use stream and the exposure caps: per retrieval at most c_{cand} candidates are surfaced (Rule R14(ii)), disputed items are flagged and ordered last, paraphrase spam consumes the submitter’s rate limit per byte-distinct variant (Rule R12, Corollary I.27), and a P -emitted confirm or handle-expressed tool call on a *covered* pair fires a covered gate, protocol-inert on both branches (Rule R13(i)), reasoning-target repetition on locally readable bytes included (reasoning coverage, Rule R13(i)): the resulting redundant confirmations and rejections have no shared-tier effect and are excluded from every ledger count (§J.1), so agent repetition on covered values costs the user attention only, disclosed here — and P can re-emit confirm on a repeatedly-rejected, *uncovered* handle every step: covered-gate inertness neutralises covered values only, by design. Repeated dialogs arising from the user’s own repeated attempts at an *uncovered* pair are a first-party cost of declining, of the same character as the evict residual (N13): attacker-steered repetition is excluded (Lemmas G.12–G.18), but the model places no constant on it. Accordingly, every “bounded-harm” statement in this document is a statement about *forced accepting confirmations* (Theorem J.2), with inspection labour disclosed here. One deployment device reduces this attention cost without touching the semantics: the decision pane may visibly badge a covered gate — the resolved bytes byte-match a local in-scope copy, an owner-side deterministic fact (Rule R13(i)) — and offer a one-tap accept; the gate, its display, and its logged decision are unchanged. Interpreter-level rate-limiting of repeated uncovered gate requests is a candidate further hardening, monotone in the sense of Proposition G.28; it is an operational-semantics change requiring its own transition, transcript, and two-run treatment, and is not specified in this document.

K.3 SECURITY AND ROBUSTNESS ANALYSIS

Each entry states an attack vector or failure mode in its strongest form (*Vector*), the facts that govern it (*Disposition*), and its *Status* per §K.1. Entries are grouped by attack surface.

K.3.1 ATTACKS ON THE ENDORSEMENT MECHANISM

Analysis S1 (Paraphrase-variant families). *Vector*. Freshness is byte-keyed (INV-1). An adversary submits paraphrase variants of the same poison as distinct byte strings; every inspector, including one already fooled by a sibling variant, is fresh again on each variant, so the per-person discharge of correlation is byte-scoped, not content-scoped. *Disposition*. The discharge of per-person correlation is indeed *per byte identity* (convention (C5)), and the arithmetic is unchanged: each variant is a distinct item consuming the submitter’s rate limit and its own union-bound term (Rule R12, Corollary I.27), and per-retrieval exposure is capped at c_{cand} (Rule R14(ii), N18). What survives is an *interpretation* obligation on the deployer: q inside each union-bound term must be calibrated as the catch rate against the *variant family*, not against a single presentation; an inspector fooled by one variant is plausibly likelier to be fooled by its siblings. Deployments may additionally surface an advisory-only similarity flag (“similar disputed or evicted items exist”) alongside disputed items, of the zero-authority advisory class of Rule R13(i): it informs inspectors without granting the model computing similarity any protocol effect. *Status*. Mechanism disposed; calibration duty disclosed. Not used in any proof.

Analysis S2 (Self-dispute). *Vector*. Reject one’s own item; fill the panel with one’s own verdicts; sustain the dispute across epochs via one’s own demand events. *Disposition*. The case-(b) bound of Theorem I.13 does not condition on the rejecter’s identity, so self-dispute gains only the panel term already counted, while consuming a rejection slot and accountability exposure; verdict capacity never renews (INV-5), capping the adversary’s tally at f_s for the dispute’s life; self-initiations are capped at one per identity per item per epoch (Rule R13(i)). *Status*. Disposed (under stated assumptions; Theorem I.13, INV-5). Not used in any proof.

Analysis S3 (Reputation farming). *Vector*. Farm attestation weight and reputation over time; spend it in one campaign. *Disposition*. f , W_{adv} , f_s are lifetime totals over every identity the adversary ever controls (§H.1); every bound is

stated against the totals, so farming shifts timing, never capacity (N12). The economic deterrent of burning reputation is real but deliberately unused: the proofs assume the attacker spends everything. *Status*. Disposed (under stated assumptions; lifetime-total definition). Not used in any proof.

Analysis S4 (Tool-targeted poison). *Vector*. Disputes are byte-level but harm is tool-level: a poison dangerous at tool A and innocuous at tool B is sincerely vindicated by a panel drawn from B -demanders, contradicting honest A -rejecters; the freeze lifts and the interrupted entry resumes. *Disposition*. The two promotion paths behave differently. *Ordinary path at A: disposed*. Promotion at A still requires N credited fresh endorsements at A itself (Lemma G.14, Definition H.6), where inspectors face the harmful use, so the case-(a) term of Theorem I.13 applies with q_{pre} read at A . *Panel path: residual*. The panel term is *not* bounded by A -context competence: q_{pan} is uniform over the jurors' own tool contexts (Theorem I.13(c)), and for A -targeted, B -benign poison it may be near zero — sincere B -juror vindication is then likely, the freeze lifts, and the entry resumes toward its A -threshold. What remains against A -promotion is the ordinary-path term, plus renewable re-freezes: each epoch, each honest identity's first rejecting decision on the bytes — in particular each new A -context detection — re-freezes promotion at every tool (Rule R13(i)–(ii)). *Accountability erosion*. Honest A -rejecters contradicted by B -vindications feed Rule R13(v); mitigation is threefold: the free first charge per epoch, the per-fresh-inspection rate trigger calibrated above the honest baseline (theorem I.5), and conversion to zero-authority advisory flags at the ceiling. We deliberately do *not* exempt rejecters whose vindicating jurors sat at a different tool: because a rejection at any tool freezes all tools (Rule R13(ii)), such an exemption would let an adversary file every rejection “at” an obscure tool no juror will ever demand, harvesting freezes with zero accountability exposure and breaking the throughput bounds of Lemma I.24(iv). The per-pair scoping variant of Remark H.9 removes this residual; it is a deployment choice, consumed by no theorem. *Status*. Ordinary-path promotion at the targeted tool disposed; panel-path vindication under cross-tool juries residual, disclosed; accountability erosion residual, mitigated, availability-only. Not used in any proof.

Analysis S5 (Panel-lapse exploitation and slot economics). *Vector*. Reject a low-demand benign item; let the panel lapse; tax the owner's rate limit on each forced re-entry. *Disposition*. The attack presupposes a deadline-based design; there is no panel lapse and no forced re-entry (Lemma I.24(iii)): the only re-entry paths are owner-initiated, and slots free on resolution or expiry. Roles of the clocks: theorem I.26. *Status*. Disposed (under stated assumptions; Lemma I.24(iii)). Not used in any proof.

Analysis S6 (Juror exhaustion by wide pre-exposure). *Vector*. Pre-expose the poison so widely that few verdict-eligible (fresh, seat-weighted) demanders remain; the panel never fills. *Disposition*. The item stalls *frozen*: promotion is blocked at every tool with no deadline to exploit (Rule R13(ii)–(iii)); the cost is availability, never safety, borne by the adversary's own submission, and the benign-item variant is bounded at the cap (Remark I.25, Lemma I.24(ii)). Pre-exposure also feeds the defence: each additional honest *rejection* opportunity can freeze promotion (INV-2), and consumed honest identities are never re-usable for later entries of the same bytes (Corollary I.27). *Status*. Safety disposed + availability residual (under Theorem I.13's hypotheses; deterministic failure mode of A9, disclosed at its commentary). Not used in any proof.

K.3.2 SIDE CHANNELS AND COVERT CHANNELS

Analysis S7 (Store-exhaustion side channel). *Vector*. The store is idealized as unbounded; a real deployment has quotas, and quota errors visible to P leak the attacker-chosen volume of captured blocks. *Disposition*. Correct, which is why the quota policy is a stated convention rather than an afterthought: deployments must apply quotas to the untrusted tier by silent eviction, never by an error visible to P (convention (C7)); otherwise store exhaustion reopens exactly the channel Rule R7 closes (N4). Under the convention, no operation's defined behaviour depends on aggregate store volume, and Lemma G.18(f) stands. *Status*. Disposed (under stated assumptions), conditional on the stated deployment policy (C7). Not used in any proof.

Analysis S8 (Timing channels). *Vector*. The bisimulation matches P -operations; wall-clock timing of the harness is unmodelled and could leak. *Disposition*. Conceded: timing and termination are out of scope (§K.4). The interface returns no clock or step-count value, so no timing value exists in σ_P to branch on (Lemma G.18(e)); a clock consulted *through a tool* is already inside the model: its reading is captured to $\mathcal{M}_U$ like any tool output (Rule R3) and never reaches σ_P . The exclusion concerns only implicit timing: wall-clock latency of the harness observed by an outside party. *Status*. Residual, precisely scoped (§K.4(IV)). Not used in any proof.

K.3.3 THE HUMAN FACTOR

Analysis S9 (Preview manipulation). *Vector*. An adversarial reader decorates a malicious candidate with a benign-looking preview and steers the user. *Disposition*. Within the formal model, previews are inert: they never enter σ_P (Lemma G.12), and endorsement binds to the true resolved bytes on the decision pane (A3–A4), so a preview can never forge an endorsement: the wrong value is displayed truthfully and rejected under A4. Outside the formal model, a preview can influence what the user chooses to examine and, through A6's idealization, what the user subsequently

types. That residual is covered by design rather than assumed away: previews are part of the display transcript and the audit log (Definition G.8, Rule R8(e)), so the release history covers every surface through which such influence can pass, and Corollary G.26 remains faithful even where A6 frays (N5, N8). *Status*. Disposed (under stated assumptions) within the model; residual at the human channel, disclosed. Not used in any proof.

Analysis S10 (Careless endorsement and the post-endorsement channel). *Vector*. After one careless narrative confirmation (a false acceptance in the sense of Definition H.10; a failure of A5), the endorsed summary’s bytes are readable and flow, literal-expressed, to data tools ungated (handle-expressed use gates regardless, Rule R4(c)); a poisoned summary can both mislead reasoning and direct data-tool egress. *Disposition*. Correct, and no theorem is violated: the influence entered through a transcript event the user accepted. The compound shape is: one careless narrative endorsement (a failure of A5) simultaneously (i) admits attacker-shaped content into P ’s readable state and (ii) opens ungated *literal-expressed* data-tool egress of endorsed and typed bytes (Rule R4(b); handle-expressed use gates regardless, Rule R4(c)); the misled P may then recombine separately endorsed values at sensitive tools without further gates (S11). Containment is threefold: the influence never crosses a sensitive boundary with bytes neither user-typed nor endorsed for that tool (Theorem G.21); its triggering endorsement is a logged transcript event, and any divergence it induces in the protected behaviours is transcript-carried (Corollary G.26); and the community mechanism reduces the frequency of the triggering endorsement for shareable content (Appendix H). In the federated tier the residual carries a further, deliberate cost: an honest user who carelessly confirms and then *publishes* attacker-crafted content will, on that item’s panel eviction, incur the publisher penalty of Rule R13(vii), a corrective on the publish gate; the penalty never touches any local store (Lemma I.8, Proposition I.22). Option O2 removes the *ungated* data-tool egress of out-of-scope endorsed bytes (each such flow then gates once per pair, Corollary G.23(b)); Option O3, if instantiated conformingly, would limit only scope-checked local tool authorization; data-tool enforcement requires Option O2, and readable reasoning state is unaffected (O3). *Status*. Residual, disclosed, contained, and priced. Not used in any proof.

Analysis S11 (Recombination of endorsed values). *Vector*. Endorsement is per (value, tool); combinations, orderings, and timings of separately endorsed values are ungated: a manipulated agent could compose endorsed values harmfully. *Disposition*. Recombination is ungated by design; protection against *malicious* recombination rests on the agent’s isolation from the attacker (Lemmas G.12–G.18): an attacker cannot steer P ’s composition choices, because no attacker-controlled bit reaches P ’s state or control flow. What remains is (i) the careless-endorsement trigger (S10), and (ii) *erring*, not attacker-steered, misdirection among the user’s own typed or endorsed values, a utility failure outside the threat model (§K.4). O1 offers defence in depth by re-gating T-values per sensitive sink. *Status*. Attacker-steered version disposed; residual disclosed. Not used in any proof.

Analysis S12 (Batch-gate attention). *Vector*. The batch gate displays every element’s bytes, but displayed-in-full is not reviewed-in-full; A4 is weakest here. *Disposition*. Conceded: Definition G.10 holds literally at the batch gate (every element’s bytes are displayed, Rule R9(b)), but the human premise A4 is at its weakest, and deployments should size c_{batch} conservatively. The selected batch design is all-or-nothing: a per-page decision returned to P would leak the attacker-influenceable page count, and the aggregate decision avoids that channel at the cost that one poisoned element forces rejection of the whole batch: an availability and attention cost, not a safety gap (no impossibility of alternative fixed-shape or padded designs is claimed); and the user may reject at any page, aborting the remainder, so attacker-imposed attention is bounded by the user’s own patience, never compelled (Rule R9(c),(d), N6). Deployments may additionally cap total batch cardinality (Rule R9(b)), converting oversize collections into a fixed, decision-free reject; the cap lies outside the mechanized batch layer (§G.6). *Status*. Mechanism disposed; residual at the human premise, disclosed. Not used in any proof.

K.3.4 ADVERSE PARAMETER REGIMES

Analysis S13 (Cross-person correlated carelessness). *Vector*. A uniformly convincing poison lowers every fresh inspector’s catch rate together; the exponential bounds assume independently varying carelessness and collapse toward $1 - q_c$ under engineered correlation. *Disposition*. The premise of the vector is corrected first: the bounds of Theorem I.13 assume *no* independence across inspectors; they assume a uniform *conditional* lower bound q on each fresh inspection given the entire past (§L.1). What engineered cross-person correlation attacks is that bound itself: a uniformly convincing poison is one for which the honest conditional catch rate, *after conditioning on earlier misses*, is some $q_c < q$. Three facts limit the damage: the theorem remains valid with q read as q_c , the bound degrading toward $1 - q_c$ transparently, never silently; the diversity requirement D_{pool} mixes fresh-inspector pools, raising q_c (N11); and periodic re-verification re-presents the bytes to inspectors with no prior decision, so the probability of remaining uncaught across E_{rv} rounds is at most $(1 - q_c)^{E_{\text{rv}}}$, scaling with the supply of never-decided inspectors, not with epoch renewal (N16). The worst-case fallback as $q_c \rightarrow 0$ is the local layer of Remark I.12, not Theorem I.11, whose $q = 1$ hypothesis engineered correlation directly attacks. The *per-person* half of the correlation concern (re-inspection of the same bytes is not re-randomisation) is discharged structurally for *logged* inspections: no logged repeat carries weight or a verdict (INV-1, INV-5); across unlogged personal inspections (Definition H.5) it survives as a calibration

obligation inside the log-fresh class (theorem I.19). *Status*. Residual with a three-layer cap, disclosed. Not used in any proof.

Analysis S14 (Unrecognizable poison, $q \rightarrow 0$). *Vector*. If no honest reviewer can recognize the poison, the probabilistic bounds are vacuous. *Disposition*. Conceded, and the guarantee is calibrated to what a human-verification design can coherently mean by “safe”: content is treated as safe when the qualifying community, inspecting the exact bytes at the exact tool, accepts it. Content that fools *every* honest inspector is, within this paradigm, indistinguishable from truth, and no design that routes trust through human judgment can promise otherwise. What the design does promise is one-sided and strong: it takes only one honest *effective qualifying rejection* — a rejecting decision at a protocol-bearing gate by an identity currently holding dispute-initiation authority (Rule R13(i),(v)) — to freeze promotion immediately; detections by authority-inactive identities are advisory flags, but the same identities’ acceptances carry no promotion weight (Definition H.6(a), Remark I.18), so silencing states never trade a lost rejection for a harvestable acceptance; shared-presence freshness ensures the attacker can never harvest a second *weight-bearing* acceptance from any identity that has already logged a decision on the bytes (INV-1, Corollary I.27); and the moment a refuting inspection arrives, however late, it takes protocol-level effect immediately. What survives this regime is the local layer of Remark I.12, the one-sided freeze clause above, and re-verification (N16); Theorem I.11 does *not* apply, its $q = 1$ hypothesis being the exact negation of this regime. Safety alone can additionally be forced for any $q > 0$ by raising m toward k , at the price of a weaker wrongful-eviction bound (theorem I.19). *Status*. Residual; the paradigm’s boundary disclosed (§K.4(VI)). Not used in any proof.

Analysis S15 (Accountability mis-calibration). *Vector*. The rate trigger and ceiling require estimating the honest baseline $1 - p_b$; mis-calibration can suppress honest dispute-initiation and crediting, reducing availability — and, absent authority-gated crediting (Definition H.6(a), Remark I.18), it would also invalidate the uniform q_{pre} interpretation. *Disposition*. The exposure is availability-only, never safety, and is bounded as follows. (i) Panel verdicts are never penalised and never charged (Rule R13(v)(a)), and verdict-eligible rejections are cast regardless of accountability state (Rule R13(v)(e)), so the panel arithmetic of Theorem I.13 and Proposition I.20 is independent of calibration. (ii) A capped or suspended honest identity retains advisory flags and ordinary use — though its acceptances are authority-inert for the duration (Definition H.6(a), Remark I.18) — loses nothing permanent, and is restored at the epoch boundary (Rule R13(v)(d), theorem I.5). (iii) Its ordinary, non-verdict gate inspections while dispute-initiation and crediting authority are inactive are *excluded trials*, not silenced misses: neither branch enters the q_{pre} trial class — the acceptance carries no promotion weight and the rejection no initiation (Definition H.6(a), Remark I.18, theorem I.19) — so these inspections do not enter $1 - q_{\text{pre}}$. Eligible panel verdicts are different: verdict capacity is never authority-gated, and every cast eligible verdict remains a q_{pan} trial regardless of accountability state (Rule R13(v)(a),(e), Remark I.18, Theorem I.13(c)). Mis-calibration may change the composition of the authority-active ordinary class, so the deployment must continue to validate the weakest-target q_{pre} bound over that class (N17); it does not convert excluded inspections into misses. The remaining cost is availability: throttled honest dispute-initiation and delayed crediting. (iv) The calibration enters only the budget constants, through the deterministic bound c_{acc} (Lemma I.4, Figure 6); no safety theorem depends on it. (v) Under authority-gated crediting (Definition H.6(a), Remark I.18), mis-calibration cannot be exploited to harvest promotion weight from silenced honest identities — a silenced identity’s acceptance is authority-inert — so the exposure remains throttled honest dispute-initiation, with correspondingly delayed crediting, availability-only. What mis-calibration buys an adversary is therefore throttled honest dispute-initiation: bounded forced-confirmation harm and bounded per-epoch throughput (Lemma I.24; wall-clock delay unbounded, resolution being demand-paced, Remark I.21), no third-party alteration of any local store (Lemma I.8), and forced cost within the single-user cap (Theorem J.2). *Status*. Safety disposed; availability exposure residual, disclosed. Enters budget constants only.

K.3.5 MODELLING-BOUNDARY EFFECTS

Analysis S16 (Incentive feedback on publication). *Vector*. Publisher penalties, though provably rare for benign items, depress publication; a thinner shared tier raises the effective β and weakens A11 directly at its coverage clause (late or absent publication inflates $U_{v,t}$). *Disposition*. Conceded and unmodelled: the incentive feedback between the accountability layer and the demand model of Appendix J is outside the formal development. Its impact is confined to the budget *asymptotics*: the cap of Theorem J.2 is unaffected by it (it is an identity over forced confirmations), the safety theorems are unaffected (they do not reference β), and Proposition I.22(ii) bounds the penalty’s probability for benign publishers at $e^{-2\delta_b^2(k-f_s)}$ per panel, with expected counts per epoch bounded by the dispute ledger (Prop. I.22(ii), linear tier). *Status*. Residual, confined to budget asymptotics. Not used in any proof.

Analysis S17 (Release-day demand spikes). *Vector*. A popular item’s demand arrives in one burst; A10 fails, and the amortisation claims fall with it. *Disposition*. A release-day spike is precisely a violation of A10 *for that item* ($\gamma_v \approx 1$): its cost is at most the item’s own demand, once: each affected demander pays the forced confirmations (at most one per demanded pair, INV-3) that the pathwise cap (Theorem J.2(i)) already bounds (whose counting identities never

use A10), and every other item’s accounting is untouched, since the ledger of Lemma J.7 is per-item. Degradation under spikes is therefore item-local, bounded by Theorem J.2, and transient; the window- and dormancy-bucket rows of the ledger are exactly where within-window demand, diversity wait, propagation delay, and dormant intervals are collected, and A10 caps them at the fraction $\bar{\gamma}$ of each item’s own demand, so popularity never enters the per-user bound (Lemma J.8). A pre-publication demand burst is likewise an A11 violation for that item (its early demanders are residual), priced identically at the cap. *Status*. Disposed (under stated assumptions): the failure mode is priced by the pathwise cap (Theorem J.2(i)). Not used in any proof.

Analysis S18 (Handle-mediated arrival of certified content). *Vector*. Not an attack but a representation effect: a pair (v, t) is live-CC, yet a user’s copy of v arrives through an untrusted channel (a message, a tool output) and is therefore held only as an opaque handle (Lemma G.12); passing the handle to t gates (Rule R4(c)) despite the certificate, and, absent a local in-scope copy, the confirmation is forced. A demand stream arriving predominantly handle-expressed can drive the shared *forced* load toward the standalone mass, community certification notwithstanding. *Disposition*. Deliberate and structural: exempting certificate-matching handle resolutions from the gate would make the gate-vs-no-gate behaviour a function of hidden U-resolution, reopening the channel Rules R6/R7 close and breaking Theorem G.25 (two runs whose handles resolve to different CC-matching byte strings would flow ungated under equal transcripts with different sink projections); the gate on untrusted-channel arrivals is the inspection the design intends (Remark J.11). The cost is bounded on both sides: each such confirmation is once ever per (user, pair) (INV-3), the epoch’s shared forced load never exceeds the standalone mass (Theorem J.2(ii)), and the accepting user thereafter holds a local in-scope copy, fast-pathing all literal reuse. Amortisation applies to the fast-path-expressed share of certified demand — notably trusted retrieval, which returns live entries as readable $\mathbb{E}$ -bytes byte-matching the entry (Definition H.1, Rules R14(ii), R4(a)); the handle-arrival fraction is priced inside $\varepsilon(M)$ of A11 and logged directly (N17). *Status*. Residual, disclosed, priced in A11; bounded pathwise by Theorem J.2(i) (persistence exceptions per INV-3). Not used in any safety proof; enters budget assumptions only.

K.4 SCOPE AND RESIDUALS

The guarantees’ boundaries, collected in one place.

- (I) *Per-execution, per-(value, tool)*. Action Integrity is per execution; endorsement is per (value, tool) and cached indefinitely (a conforming instantiation of the requirements-level Option O3 would bound the scope-checked tool-authorization lifetime; O3); recombination of separately endorsed values is ungated by design (N2, S11).
- (II) *The human channel*. A6 idealizes typing as exogenous; influence through displayed content is recorded, not prevented (N5, N7, S9); Reasoning Integrity is modulo the transcript, and the compound residual is S10.
- (III) *The batch gate*. Every element’s bytes are displayed, so Definition G.10 holds literally; but displayed-in-full is not reviewed-in-full: A4 is weakest here, and c_{batch} should be sized accordingly (N6, S12).
- (IV) *Timing and termination* are out of scope; the exclusion concerns only implicit wall-clock channels observed from outside (S8).
- (V) *Model intent*. P and $R_{\mathcal{E}}$ are isolated from the attacker, not certified benign; misdirection among the user’s own typed or endorsed values is a utility failure outside the threat model (the baseline’s extensional, tool-blind trust of typed literals is Remark G.22) (O1 offers defence in depth). One first-party residual: evict is owner-scoped but P -invocable and ungated, so an erring Protected LLM can remove its owner’s endorsed blocks; the no-deletion guarantee is third-party only, and gating evict on $\mathbb{E}$ -blocks is a further monotone hardening (N13).
- (VI) *Unrecognizable poison* ($q \rightarrow 0$): no community-truthfulness guarantee survives this regime within a human-judgment design (Remark I.12); what survives is the local layer, the one-sided freeze on any single honest recognition, and re-verification against never-decided inspectors (S14); degradation under engineered correlation is to the rate q_c , transparently (S13).
- (VII) *Federated influence through legitimate log activity* is held fixed by A8 in the noninterference statement and is bounded instead by Theorems I.11 and I.13.
- (VIII) *Availability* is not guaranteed; a dispute’s harm in *forced confirmations* is bounded per Lemma I.24 and N14 (including the suspension-induced, once-per-demander confirmation, Rule R13(vi)), inspection labour disclosed separately (N18); degradation is never destructive (Lemma I.8).
- (IX) *Confidentiality*. Only the type-level exclusion is proved (Lemma I.2); semantic confidentiality of published content rests on the publish gate (A7).
- (X) *Tool choice and zero-argument actions*. Action Integrity constrains the values crossing sensitive tool boundaries; it does not authorize the invocation itself, and a sensitive tool with no gated arguments satisfies the value-flow

property vacuously. Deployments requiring authorization of the act itself gate the capability (a monotone hardening in the sense of Prop. G.28).

- (XI) *The reasoning pseudo-target.* Shared-tier accounting treats reasoning as a canonical pseudo-target (§H.1): knowledge items amortise and are defended at it, and CC-at-reasoning status buys community-wide readability only, never any tool flow.

K.5 SUMMARY OF COMPOSED GUARANTEES

(A) One user, deployment lifetime (trust axioms A1–A6 alone). Mechanization boundary, wherever machine-checking is advertised: kernel-checked are Personal Action Integrity from the empty configuration and from arbitrary mechanized well-formed configurations; the map-free Reasoning Integrity core from empty stores and from the canonical shared-endorsed seed class; and the standalone batch-confirmation layer (§G.6). Paper-only are the run-level map integration, the batch cardinality cap, Option O3 (requirements-level), arbitrary matched *mid-run* pairs, the index-oracle reduction (Prop. G.29), the federated product system, and every probabilistic and budget result. Gating where influence acts: Theorem G.21, Corollary G.23(a). Recording where it persuades: Theorem G.25, Corollary G.26. (Phrasing discipline, here and in any summary elsewhere: Reasoning Integrity is a fixed-horizon, termination-insensitive equivalence *modulo the full display transcript*, under coupled typed input — and, federated, a coupled log (A8) — never an unconditional noninterference or inertness claim; Definition G.11, Remark G.9.) Residuals: the user’s own false endorsement (S10); the post-endorsement boundary (Corollary G.23(b), O2); recombination (S11, O1). Deployments whose threat model emphasises data-tool exfiltration should select Option O2 as the default profile: it closes the baseline post-endorsement channel (Cor. G.23(b)), is machine-checked at the gate level (§G.6), and costs $\mathcal{O}(1)$ per reusable endorsed-value pair, freshly composed arguments remaining in the per-task stream (Cor. J.6). Lifetime forced cost: one accepting confirmation per (value, tool) relied on (Theorem J.2, Corollary J.5); effort scope per N18.

(B) Federated, deployment lifetime. Three layers, each surviving the failure of the layer above. *Deterministic* ($q=1$, $W>W_{\text{adv}}$, $m>f_s$): Theorem I.11, scope Remark I.12. *Probabilistic, per poisoned attempt* (Definition I.1) (Definition H.10): Theorem I.13, reach-free (INV-6); per-anchor-epoch and lifetime exposure Corollary I.27 (attempts keyed to anchor epochs, Rem. I.28), Remark I.29; system-wide per-anchor-epoch form under the union-bound condition of Theorem J.13(i). Scoping caveats: S1, S4; and one explicitly: the numeric system-wide bound covers *adversary-published* attempts, counted by the rate limit at their anchors — poison published by honest-but-deceived identities (S10) is bounded per attempt (Thm. I.13) and disciplined by publisher penalties (Rule R13(vii), Prop. I.22), but its attempt count $\mathbb{E}[S_{\text{anc}}]$ is not bounded in terms of the adversary’s resources alone (Cor. I.27). Per-attempt bounds are marginal: deployment-wide exposure is controlled by *summing* over anchored attempts and byte-distinct variants, through the union-, expectation-, and deterministic-cap forms of Corollary I.27 and the lifetime ceiling of Remark I.29. *Pathwise cap*: Lemma I.8 (third-party transitions; first-party residual N13) and Theorem J.2: degradation to M independent copies of (A), never below. Budget: Theorem J.2 (pathwise once-per-pair cap and reference-model standalone-mass ceiling, the firmest cost guarantee); Theorem J.9, Corollary J.10 (conditional upper bounds, with the Bad residual and the arrival-representation caveat S18) — the vanishing clauses are conditional accounting, consuming A9–A13, the rate conditions of Thm. J.9(iii), the competence bound of Lemma J.7, and changing time geometry (Rem. J.12), never consequences of population growth alone —, Theorem J.13, Remark J.12. *Scope*: the federated layer is a protocol over the log service of convention (C3), not integrated into the single-user small-step relation (§H).

L PROBABILISTIC TOOLS AND SUPPLEMENTARY RESULTS

L.1 PROBABILISTIC FRAMEWORK, COUPLING, AND THE NO-DISPUTE BASELINE

The adaptive-adversary framework. Fix an epoch. All protocol events (submissions, presentations, gate decisions, verdicts, promotions) form a discrete sequence; let $(\mathcal{F}_s)_{s \geq 0}$ be the filtration generated by all logged protocol events and all display events up to step s . The adversary’s strategy (whom to present the item to next, when to submit, re-enter, self-dispute, or stop) is *predictable* with respect to $(\mathcal{F}_s)$: its action at step s is $\mathcal{F}_{s-1}$ -measurable (convention (C2)). Two structural facts constrain the adversary’s adaptivity. First, an entry’s bytes are immutable once submitted (the log is append-only and the gate displays exact bytes), so within an entry the adversary adapts *presentation and stopping* only, never the inspected content. Second, because the adversary chooses whom to present the item to, the inspection parameter q must be read as a *uniform* lower bound over all honest identities and presentation contexts, the catch rate of the most fallible honest user the adversary can target, with flagged scrutiny only raising it (theorem I.19, N11).

Atomic gate events; log order. Gate events are atomic federated-wide, exactly as at the interface (Rule R2): a presentation and its disposition (an accepting decision, a rejecting decision, or abandonment) occupy a *single* position of the linearizable log (convention (C3)); no dialog is ever pending, and dispositions never resolve out of log order. The analysis reads each such position as up to three consecutive *atoms*: the *rendering atom* (item, tool, presenting identity, dispute flag, both provenance zones — recorded in the display transcript whether or not a decision follows, Rule R8(e)); if and only if the user records a decision, the *header atom* (the fact that a decision by this identity on these bytes at this gate is being appended, together with its verdict eligibility, a deterministic predicate of the prior log and the identity, Rule R13(iii)(a)–(e)); and the *content atom* (the decision $z \in \{\text{accept}, \text{reject}\}$). Abandonment appends no decision record and contributes no decision trial. Cache gates (Options O1–O2) are the one asymmetrically recorded class: both branches occupy a decision-pane position of the display transcript — rendering, header, and content atoms exactly as above, all inside the filtration, which is generated by display events as well as logged protocol events — while only the rejecting branch additionally appends a federation-log candidate record (Definition H.6(iii)). We say a decision is *audit-recorded* if its atoms lie on the display transcript (every recorded gate decision, cache acceptances included) and *federation-logged* if it is appended to the federation log as a candidate decision (protocol-bearing decisions only). Trials at cache gates are indexed by their audit-recorded atoms: their occurrence is pre-outcome, never selected by the rejecting branch’s federation-log append. No protocol event, and no adversary action, interleaves between the atoms of one position; the discrete step index totally orders all atoms, and every trial sequence in this document is indexed by these log-ordered atoms: inspection and arrival trials by rendering atoms, decision and verdict trials by header atoms. In particular, every earlier trial’s content atom precedes every later trial’s occurrence atom. The adversary’s strategy, predictable with respect to the record-level filtration, remains predictable with respect to this refinement: the refinement inserts atoms inside single records and reveals nothing between records on which a strategy could act.

Conditioning convention. The conditioning σ -algebra depends on the trial type. For an *arrival or inspection-occurrence* trial, $\mathcal{G}_i$ is generated by all atoms strictly prior to the trial’s rendering atom. For a *decision- or verdict-content* trial, $\mathcal{G}_i$ is generated by all atoms strictly prior to the trial’s header atom *together with that header atom itself* — the fact that a decision by this identity on these bytes at this gate is being recorded, and its deterministic eligibility facts (Rule R13(iii)) — but not the trial’s content atom. Thus the occurrence event O_i (a decision is recorded) is $\mathcal{G}_i$ -measurable in every application. Every behavioural lower bound $(q_{\text{pre}}, q_{\text{pan}}, p_b, \rho_{\text{seat}})$ is a conditional bound given the corresponding $\mathcal{G}_i$, further enlarged by the exogenous demand factor below (uniformly over demand realisations). Under the atomic reading above, these σ -algebras already contain the disposition of every earlier presentation — there are no latent or pending dispositions — so the occurrence, measurability, and independence conditions of Remark L.4 hold at them directly. Where a behavioural parameter is stated at the disposition level (q_{pre}, p_b) , Lemma L.1 transports it to the header-inclusive σ -algebra. For a per-attempt bound (Theorem I.13), the conditioning σ -algebra is generated by all atoms up to and including the attempt’s anchor atom (Definition I.1), the anchor preceding every decision creditable to the attempt; the trial σ -algebras above refine it, and the behavioural bounds, asserted uniformly over histories, hold under the anchor-conditional measure.

Dispositions. An honest inspection presented at a gate has three possible dispositions, realized at that gate’s log position: an accepting decision, a rejecting decision, or abandonment. Only decisions are logged; abandonment leaves no record and has no protocol effect (Rule R13(i),(iii)). The conditional decision probabilities therefore satisfy $p_{\text{acc}} + p_{\text{rej}} \leq 1$, abandonment carrying the remaining mass, and every behavioural parameter $(q, p_b, \rho_{\text{seat}})$ is stated over dispositions: abandonment counts as a miss inside $1 - q$ and as a non-verdict inside ρ_{seat} (theorem I.19).

Lemma L.1 (Decision-conditioned content bounds). *Consider a gate presentation with rendering-atom σ -algebra $\mathcal{G}$ and conditional disposition probabilities $p_{\text{acc}}, p_{\text{rej}}$ given $\mathcal{G}$, with $p_{\text{acc}} + p_{\text{rej}} \leq 1$ (abandonment carrying the remaining mass). On the event that a decision is recorded, and given $\mathcal{G}$ together with the header atom: (i) if $p_{\text{rej}} \geq q$, the conditional probability of accept is at most $1 - q$; (ii) if $p_{\text{acc}} \geq p_b$, the conditional probability of reject is at most $1 - p_b$; (iii) symmetrically, let a designated sub-branch of the rejecting disposition (e.g. the effective qualifying rejection of Theorem I.13(c)) have conditional probability $p_{\text{eff}} \leq p_{\text{rej}}$ given $\mathcal{G}$; if $p_{\text{eff}} \geq q$ for some $q \in (0, 1]$, then, on the event that a decision is recorded, the conditional probability of that sub-branch at the header atom is at least q : $p_{\text{eff}} / (p_{\text{acc}} + p_{\text{rej}}) \geq p_{\text{eff}} \geq q$, since $p_{\text{acc}} + p_{\text{rej}} \leq 1$. The header atom adds no randomness beyond the fact that a decision is recorded: verdict eligibility is a deterministic function of the prior log and the identity (Rule R13(iii)). (Part (i), the accept-side special case of (iii), is retained for reference; part (iii) serves q_{pre} ’s lower-bound direction as used in Theorem I.13 case (a); part (ii) serves p_b ; the verdict parameter q_{pan} is defined at the header atom directly, Theorem I.13(c), and needs no bridge.)*

Proof. The decision-recorded event has conditional probability $p_{\text{acc}} + p_{\text{rej}}$ given $\mathcal{G}$, so the conditional accept probability given a recorded decision is $p_{\text{acc}}/(p_{\text{acc}} + p_{\text{rej}}) = 1 - p_{\text{rej}}/(p_{\text{acc}} + p_{\text{rej}}) \leq 1 - p_{\text{rej}} \leq 1 - q$, using $p_{\text{acc}} + p_{\text{rej}} \leq 1$; (ii) is symmetric. Eligibility being measurable in $\mathcal{G}$ and the prior log, conditioning on it changes nothing. $\square$

Demand randomness, formally. The underlying space is a product $\Omega_{\text{prot}} \times \Omega_{\text{dem}}$. The factor Ω_{dem} carries the exogenous *demand indicators* of the reference model (§J.1) — Bernoulli variables $D_{u,v,t}$ (u demands the pair (v, t)) within the epoch, with per-user probability $p_{u,v,t}$ ($p_{v,t} = M^{-1} \sum_u p_{u,v,t}$, §J.1), independent across users and of every Ω_{prot} -coordinate — together with each owner’s publication schedule, coupled to the owner’s own indicators per the owner-coupling convention (§J.1); the counts $n_{v,t} = \sum_u D_{u,v,t}$ and n_v are measurable in this factor alone, and every epoch containing a submission or re-entry of v has $n_v \geq 1$ (owner coupling; the ledger’s vanishing clause on $\{n_v = 0\}$ is event-based, §J.1). Three demand quantities must be distinguished. *Latent demand* $D_{u,v,t}$ is the exogenous indicator above. *Realized shared-tier demand* is the log-defined event of Definition H.4: the first interpreter event at which u ’s use of the pair fires a gate or dereferences a shared entry, per Definition H.4’s exclusions (covered and cache gates are not demand events on either branch), occurring only while v has a shared-tier presence; its count is denoted $n_{v,t}^{\log}$. *Reclassified personal demand* is a latent demander’s post-retirement encounter, charged to the single-user stream (Definition H.4, retirement clause). Realized shared-tier demand is dominated pathwise by latent demand: every realized demander is a latent demander — for third parties by Definition H.4 (a realized demand presupposes attempted use), for the owner by the owner-coupling convention (§J.1) — so $n_{v,t}^{\log} \leq n_{v,t}$ pointwise; and by the forced-confirmer clause (§J.1), a reference-model coupling, any epoch event at which a user’s attempted use fires a gate realizes the user’s latent indicator, so every forced confirmation in the epoch belongs to a latent demander, wherever the event falls; hence every ledger bound stated against $n_{v,t}$ or $\mathbb{E}[n_{v,t}] = Mp_{v,t}$ is an upper bound for the realized counts the rows charge. Which class a latent demander lands in (realized vs. reclassified) is protocol-dependent; the *total* per-pair forced cost across both classes is nonetheless at most one per (user, pair) (INV-3), which is why the floor never consults the split. The arrival *times* of realized demand are protocol-endogenous, measurable in $(\mathcal{F}_s)$, revealing the corresponding indicator at that event; no independence between arrival times and the protocol is asserted, and A10 — both its demand clause (used for bucket attribution, Lemma J.7) and its first-pair-decision clause (used only for the $\bar{D}_v$ derivation) — is a hindsight cap on the realised arrival and decision schedules respectively, per item, read pathwise on completed epochs. The behavioural lower bounds q , p_b , ρ_{seat} are asserted conditionally per the conditioning convention of this section, further enlarged by the entire demand factor: uniformly over demand realisations. This licenses the ledger’s two-stage argument (Lemma J.7): the freeze rows bound conditional expectations given the demand coordinates, by Lemma L.2 under the demand-conditional measure, whose hypotheses hold there by the uniform reading; the expectation over demand is taken last, by the tower property, and the pathwise domination $n_{v,t}^{\log} \leq n_{v,t}$ carries every realized-count row under the latent masses.

Lemma L.2 (Adaptive conditional coupling and domination). *Let $(\mathcal{F}_s)$ be a filtration and let $Z_1, Z_2, \dots \in \{0, 1\}$ be decision indicators observed at strictly increasing stopping times $\tau_1 < \tau_2 < \dots$. Write O_i for the event that the i -th observation occurs, so that $O_1 \supseteq O_2 \supseteq \dots$ and $O_i \in \mathcal{F}_{\tau_i^-}$; unrealised observations are read as $Z_i := 0$ off O_i . Fix $p \in [0, 1]$ and n . There is an enlargement of the probability space carrying i.i.d. Bernoulli(p) variables $Y_1, Y_2, \dots$ such that:*

- (i) (Majorant.) *If $\mathbb{P}[Z_i = 1 \mid \mathcal{F}_{\tau_i^-}] \leq p$ on O_i for every $i \leq n$, then $Z_i \leq Y_i$ pointwise for every $i \leq n$; hence for every x , $\mathbb{P}[\sum_{i \leq n} Z_i \geq x] \leq \mathbb{P}[\text{Bin}(n, p) \geq x]$, and the same bound holds for any sum truncated by a predictable stopping rule, truncation removing only nonnegative terms.*
- (ii) (Minorant, joint form.) *If $\mathbb{P}[Z_i = 1 \mid \mathcal{F}_{\tau_i^-}] \geq p$ on O_i for every $i \leq n$, then $Y_i \leq Z_i$ pointwise on O_i for every $i \leq n$; hence for every x ,*

$$\mathbb{P}\left[O_n \cap \left\{\sum_{i \leq n} Z_i \leq x\right\}\right] \leq \mathbb{P}[\text{Bin}(n, p) \leq x], \quad \text{in particular}$$

$$\mathbb{P}\left[O_n \cap \{Z_1 = \dots = Z_n = 0\}\right] \leq (1 - p)^n.$$

No claim is made about the conditional law of $\sum_{i \leq n} Z_i$ given O_n : the occurrence events may depend on the realised Z_i , so only joint-event bounds are asserted.

- (iii) (Counting.) *If the hypothesis of (i) holds for every i , then for the total count T of realised observations, $\mathbb{E}[\sum_{i \leq T} Z_i] \leq p \mathbb{E}[T]$.*

Proof. Construction. All protocol randomness lives on a standard Borel space: the event alphabet of §L.1 is countable and the filtration is generated by a discrete event sequence $(\xi_s)_{s \geq 0}$, so regular conditional distributions exist, and

the process admits a *sequential quantile representation*: there are measurable maps (ϕ_s) and i.i.d. Uniform $[0, 1]$ variables (V_s) , adjoined by the Ionescu–Tulcea extension, with $\tilde{\xi}_s := \phi_s(\tilde{\xi}_0, \dots, \tilde{\xi}_{s-1}, V_s)$ reproducing the joint law of (ξ_s) coordinate by coordinate (each $\phi_s(\cdot, V_s)$ realising the regular conditional law of ξ_s given the past by quantile inversion). We choose the representation so that, for each i , the coordinate deciding the i -th observation's value uses a dedicated uniform U_i , with the decision indicator realised as $\tilde{Z}_i = \mathbf{1}\{U_i \leq p_i\}$ on $\tilde{O}_i$, where $p_i := \mathbb{P}[Z_i=1 \mid \mathcal{F}_{\tau_i^-}]$ evaluated along the *redefined* history. Because the whole trajectory is rebuilt recursively, the stopping times, occurrence events, and every other coordinate are measurable functions of the redefined past, and the redefined process has the same joint law as the original; every conclusion below is a property of that joint law, so we argue on the redefined process and drop the tildes. Set $Y_i := \mathbf{1}\{U_i \leq p\}$; the Y_i are i.i.d. Bernoulli(p), each U_i independent of the past at its use.

(i) *Majorant*. On O_i , $p_i \leq p$, so $Z_i = 1$ entails $U_i \leq p_i \leq p$, hence $Y_i = 1$; off O_i , $Z_i = 0$. So $Z_i \leq Y_i$ everywhere. Pointwise domination of each term transfers superlevel events of the sums, and dropping terms of a $\{0, 1\}$ -valued sum only lowers it, giving the truncated form.

(ii) *Minorant, joint form*. On O_i , $p_i \geq p$, so $Y_i = 1$ entails $U_i \leq p \leq p_i$, hence $Z_i = 1$; on O_n every O_i with $i \leq n$ holds, so $O_n \cap \{\sum_{i \leq n} Z_i \leq x\} \subseteq \{\sum_{i \leq n} Y_i \leq x\}$, an event of unconditional binomial law; $x = 0$ gives the all-zeros bound.

(iii) *Counting*. Iterated conditioning: $\mathbb{E}[\sum_i \mathbf{1}_{O_i} Z_i] = \sum_i \mathbb{E}[\mathbf{1}_{O_i} p_i] \leq p \sum_i \mathbb{P}[O_i] = p \mathbb{E}[T]$. $\square$

Remark L.3 (Alternatives to pathwise coupling). Readers preferring to avoid the sequential quantile construction may replace parts (i)–(ii) by exponential supermartingale bounds, at the cost of the pointwise dominations; the pathwise form is kept because the truncation and joint-event arguments of Theorem I.13, Proposition I.22(ii), and Lemma I.6 consume domination, not merely tail bounds.

Remark L.4 (σ -algebra form). Lemma L.2 holds verbatim with the pre-stopping-time σ -algebras $\mathcal{F}_{\tau_i^-}$ replaced by any increasing σ -algebras $\mathcal{G}_1 \subseteq \mathcal{G}_2 \subseteq \dots$ such that, for each i : $O_i \in \mathcal{G}_i$; Z_j is $\mathcal{G}_i$ -measurable for every $j < i$; and the coupling variable U_i is independent of $\mathcal{G}_i$. The proof is unchanged: the conditional-quantile construction and the pointwise dominations use only these three properties. The applications take $\mathcal{G}_i$ per the conditioning convention of §L.1: for arrival and inspection trials, all atoms strictly prior to the rendering atom; for decision and verdict trials, all prior atoms *plus* the trial's own header atom (content atom excluded), so that $O_i \in \mathcal{G}_i$; both enlarged by the demand factor. All trials being log-ordered with earlier contents preceding later occurrence atoms, the three conditions hold at these σ -algebras directly.

Lemma L.5 (Delayed marks). *Let $\mathcal{G}_1 \subseteq \mathcal{G}_2 \subseteq \dots$ be σ -algebras, $O_i \in \mathcal{G}_i$ occurrence events, and let X_i be $[0, 1]$ -valued random variables with $\mathbb{E}[X_i \mid \mathcal{G}_i] \leq p$ on O_i for every i . Then, with $N := \sum_i \mathbf{1}_{O_i}$ finite almost surely,*

$$\mathbb{E}\left[\sum_i \mathbf{1}_{O_i} X_i\right] \leq p \mathbb{E}[N].$$

No ordering of the times at which the X_i resolve, no measurability of any X_i with respect to any $\mathcal{G}_j$, and no independence is required: each X_i may resolve arbitrarily later than its occurrence event, interleaved with other trials' occurrences and resolutions.

Proof. $\mathbb{E}[\mathbf{1}_{O_i} X_i] = \mathbb{E}[\mathbf{1}_{O_i} \mathbb{E}[X_i \mid \mathcal{G}_i]] \leq p \mathbb{P}[O_i]$ by the tower property; sum over i . $\square$

Remark L.6 (Realised-at-time vs. delayed marks). Lemma L.2(iii) is the special case in which X_i is a decision indicator realised at its own stopping time. Lemma L.5 is the form consumed by trials whose *outcome* resolves only later — anchor-indexed promotion attempts (Corollary I.27) and opening-indexed panel instances (Proposition I.22(ii), linear tier) — where later trials may open before earlier outcomes resolve, so outcome order need not equal occurrence order. The coupling machinery of Lemma L.2 is neither needed nor invoked there.

Lemma L.7 (Predictable cell allocation). *In the setting of Remark L.4, fix finitely many cells (j, ℓ) , $j \leq r$, $\ell \leq K$. Suppose each realised trial i is assigned to at most one cell, injectively across trials, by an assignment that is $\mathcal{G}_i$ -measurable (decided at the trial's occurrence, before its content). On the enlargement of Lemma L.2 carrying the coupling variables $Y_i = \mathbf{1}\{U_i \leq p\}$, define the cell-indexed array $X_{j,\ell} := Y_{i(j,\ell)}$ where $i(j,\ell)$ is the trial assigned to cell (j, ℓ) , and $X_{j,\ell} := \mathbf{1}\{U'_{j,\ell} \leq p\}$ with fresh independent uniforms $U'_{j,\ell}$ for never-filled cells. Then $(X_{j,\ell})_{j \leq r, \ell \leq K}$ is i.i.d. Bernoulli(p).*

Proof. This is optional skipping in the sense of Doob, extended to a predictable partition. Order trials by occurrence. At trial i 's occurrence its cell assignment is $\mathcal{G}_i$ -measurable, while U_i is independent of $\mathcal{G}_i$ (Lemma L.2, construction)

— in particular of all previously assigned cells and all previously revealed coupling variables; the fresh $U'_{j,\ell}$ are independent of everything. Fix any $\varepsilon : \{(j, \ell)\} \rightarrow \{0, 1\}$ and reveal the array in assignment order, completing with the fresh uniforms: at each revelation, the conditional probability that the next revealed entry equals its prescribed value is $p^\varepsilon(1-p)^{1-\varepsilon}$ for that entry, by the independence just stated. Induction along the revelation order gives $\mathbb{P}[X_{j,\ell} = \varepsilon(j, \ell) \forall j, \ell] = \prod_{j,\ell} p^{\varepsilon(j,\ell)}(1-p)^{1-\varepsilon(j,\ell)}$, i.e. the array is i.i.d. Bernoulli(p). $\square$

Lemma L.8 (Hoeffding tails: independent and predictable-weight forms).

- (i) (Independent, deterministic weights.) *Let $X_j \in \{0, 1\}$ be independent with $\mathbb{P}[X_j=1] \leq \varepsilon$, and let $w_j \in [0, w_{\max}]$ be deterministic, $j = 1, \dots, n$. Then for $t > 0$, $\mathbb{P}[\sum_j w_j X_j \geq \varepsilon \sum_j w_j + t] \leq \exp(-2t^2/(nw_{\max}^2))$ Hoeffding (1963); the unweighted case gives $\mathbb{P}[\text{Bin}(n, \varepsilon) \geq (\varepsilon + \delta)n] \leq e^{-2\delta^2 n}$.*
- (ii) (Predictable weights, conditional success bound.) *Let $(\mathcal{H}_j)_{j \geq 0}$ be a filtration, $X_j \in \{0, 1\}$ adapted with $\mathbb{P}[X_j=1 \mid \mathcal{H}_{j-1}] \leq \varepsilon$, and $w_j \in [0, w_{\max}]$ predictable (each w_j $\mathcal{H}_{j-1}$ -measurable). Then the bound of (i) holds with the identical exponent.*

Proof. (ii) Set $D_j := w_j(X_j - \mathbb{E}[X_j \mid \mathcal{H}_{j-1}])$, a martingale-difference sequence whose j -th increment lies, given $\mathcal{H}_{j-1}$, in an interval of predictable length $w_j \leq w_{\max}$. The Hoeffding–Azuma inequality in its conditional-range form (Hoeffding’s lemma applied conditionally to each increment, then Markov on the exponential moment) gives $\mathbb{P}[\sum_j D_j \geq t] \leq \exp(-2t^2/\sum_j w_j^2) \leq \exp(-2t^2/(nw_{\max}^2))$. Since $\sum_j w_j \mathbb{E}[X_j \mid \mathcal{H}_{j-1}] \leq \varepsilon \sum_j w_j$ pathwise, the event $\{\sum_j w_j X_j \geq \varepsilon \sum_j w_j + t\}$ is contained in $\{\sum_j D_j \geq t\}$. (The bounded-difference form of Azuma’s inequality, with increments bounded in absolute value by $w_{\max}$, would give only $\exp(-t^2/(2nw_{\max}^2))$; the range form is the one used.) $\square$

Theorem L.9 (No-dispute baseline: reach-dependent). *Without Rule R13, with rejections protocol-inert and a poisoned item shown to at most R honest users each erroneously accepting with probability $\leq \varepsilon$ independently: if $W > W_{\text{adv}} + \varepsilon R w_{\max}$,*

$$\mathbb{P}[\text{promotion}] \leq \exp\left(-\frac{2(W - W_{\text{adv}} - \varepsilon R w_{\max})^2}{R w_{\max}^2}\right).$$

Proof. Accepting weight is at most $W_{\text{adv}} + S$ with $S = \sum_j w_j X_j$ over $\leq R$ honest users; apply Lemma L.8 with $t = W - W_{\text{adv}} - \varepsilon R w_{\max}$. $\square$

Comparison with the no-dispute baseline. The comparison isolates exactly what the panel and the gate-rejection-as-report rule buy. In Theorem L.9 rejections are inert, so honest false acceptances accumulate with reach: the sample size is the *attacker-chosen reach* R , the bound requires W to grow linearly with εR merely to stay non-trivial, dragging the budget of Appendix J with it, and spreading the poison farther helps the attacker: the bound is not reach-free in the sense of INV-6. In Theorem I.13, because every honest inspection carries its own report (Rule R13(i)), the no-dispute event shrinks with every additional honest viewer, and the false-accept mass that matters after a dispute is confined to a bounded jury of $k - f_s$ honest verdicts, a protocol constant the attacker cannot multiply, and one the demand-paced formation rule fills without any deadline the attacker could exploit. Reach drops out of the bound entirely, and spreading the poison cannot loosen the bound, while any single honest gate rejection freezes it. The dispute log additionally makes wrongful rejection a costly, attributable act, and panel eviction makes poisoned submission a costly, attributable act (Rule R13(v),(vii)), neither of which the inert-rejection baseline can express.

L.2 THE BLAST-SCHEDULE PROMOTION LAW

This subsection derives, in closed form, the promotion probability of one *simulated* presentation schedule under one *simulated* model of honest behaviour. It is an analysis tool for the empirical evaluation, not a protocol guarantee: no rule, assumption, definition, or theorem of Appendices F–J cites it, and nothing in this document depends on it. Its purpose is to turn the simulator’s output into a falsifiable prediction, so that agreement is evidence the implementation realises the rules of Appendix H, and disagreement localises a defect in one of the two.

Definition L.10 (Blast schedule; independent honest coin). The *blast* schedule is the non-adaptive presentation strategy that (i) submits a poisoned item (Definition H.10), with all endorsement and all demand at a single tool, (ii) spends every adversarial identity as a pre-dispute accepting endorser at that tool, and (iii) then presents the item, once each, to R honest identities in a fixed order (the *reach*, as in Theorem L.9), continuing through every one of them whatever any of them decides. Four stipulations fix the simulated setting.

- (a) *One window.* All R presentations, and any panel the schedule induces, fall within a *single* finalisation window of one pipeline entry, with the freeze excluded from the window clock (Rule R13(iv)) and no epoch boundary closing the window early (convention (C4)). In particular the entry is open throughout, so no decision is banked (Definition H.6(b), which requires no open entry) and no re-entry occurs: every weight-bearing decision is credited to that one entry.
- (b) *Independent honest coin.* Each of the R inspections is, independently of the others, a rejecting decision with probability q and an accepting decision with probability $1 - q$, with no abstention.
- (c) *Coupled reviewer quality.* The same q governs pre-freeze gate inspections, panel verdicts, and post-resolution confirmations ($q_{\text{pre}} = q_{\text{pan}} = q$); the decoupled reading, under which those three classes may carry different parameters, is Remark L.17.
- (d) *Seat weight and eligibility.* Every identity carries attestation weight at least w_{seat} , so a fresh decision on the item is verdict-eligible whenever a dispute is open (Rule R13(iii)(a)); every decision, the f adversarial pre-dispute endorsements included, is authority-active (Definition H.6(a)) and taken at a non-covered, non-cache gate, so every accepting decision is creditable and every rejecting decision takes effect; and identities are assigned to the D_{pool} pools round-robin.

Independence across inspections is a property of this simulation only: no result of Appendices H–J assumes it, and Theorem I.13 explicitly does not (§L.1, Remark L.14).

Proposition L.11 (Blast promotion law). *Adopt Rules R12–R13 with the default bars of Rule R13(iv) ($m_{\text{rej}} = k + 1 - m$, hence $m + m_{\text{rej}} - 1 = k$), and the blast schedule and independent honest coin of Definition L.10. Suppose*

$$N - f \leq m, \quad N - f \leq R \leq 2m,$$

and suppose the deployment’s constants and pool assignment leave the credit caps of Definition H.6(f) non-binding for promotion at every reach in that range (Step 1). Define

$$S(J) := \sum_{r=0}^{\min(J-m, m_{\text{rej}}-1)} \binom{m-1+r}{r} \left(\frac{q}{1-q}\right)^r \quad (J \geq m), \quad S(J) := 0 \quad (J < m).$$

Then the item is promoted to CC with probability $\pi_{\text{blast}}(R) = \pi_{\text{ord}}(R) + \pi_{\text{pan}}(R)$, where the two components — promotion with no dispute ever opened, and promotion after a dispute — are disjoint and equal

$$\pi_{\text{ord}}(R) = (1-q)^R, \quad \pi_{\text{pan}}(R) = q(1-q)^{R-1} \sum_{J=m}^{R-1} S(J).$$

Neither expression contains k , f , f_s , or D_{pool} .

Proof. Step 0 (no adversarial verdict). Blast spends its f identities as pre-dispute accepting endorsers before any honest presentation. By byte-permanence of freshness (INV-1) each identity’s first logged decision on the bytes is its only weight-bearing inspection, and verdict eligibility requires freshness (Rule R13(iii)(b)); so when the freeze lands no adversarial identity is verdict-eligible, and every verdict is cast by an honest identity. Hence f and f_s do not appear.

Step 1 (no capacity, verdict-pool, or credit cap binds). Under the default bars $m + m_{\text{rej}} - 1 = k$, so at most k verdicts are cast before a bar is reached (Lemma I.7(ii)) and the capacity cap of Rule R13(iii)(e) is never the binding constraint. Two further caps are properties of the simulated population rather than consequences of the rules, and are disclosed as such. With the round-robin assignment of Definition L.10(d), no pool contributes more than $\lceil k/D_{\text{pool}} \rceil$ verdicts among any prefix of k arrivals, so the per-pool verdict cap of Rule R13(iii)(d) does not bind. Likewise the per-pool and total *credit* caps of Definition H.6(f) truncate crediting at $\lceil N/D_{\text{pool}} \rceil$ per pool and N^+ in total, and those caps are promotion-neutral in the sense stated there — $N^+ \geq N$, and a credited set meeting N under the per-pool cap necessarily spans at least D_{pool} pools — *provided* enough accepting identities are available in each pool at the reach in question. That availability is a property of the constants and the round-robin assignment, verified for the deployment studied here and assumed in the statement; where it fails, the law does not apply and D_{pool} re-enters. Hence, under the stated hypothesis, k and D_{pool} do not appear.

Step 2 (the ordinary component). Suppose no honest inspection rejects. No dispute opens; the accepting decisions are all authority-active (Definition L.10(d)), so credited accepting endorsements number $\min(f + R, N^+) \geq N$, using $R \geq N - f$ with Step 1; and the entry promotes at its window’s close (Definition H.6(c)). Conversely, a rejecting decision at *any* point in the window is a qualifying rejection that freezes promotion at every tool immediately (Rule R13(i)–(ii)), while finalisation occurs only at the window’s close, which by Definition L.10(a) lies after all R

presentations — so a single rejection, however late, denies the ordinary path. The ordinary component is therefore exactly the all-accept event, of probability $(1 - q)^R$.

Step 3 (the panel component; conditioning). Condition on the first rejection falling at honest position j , an event of probability $q(1 - q)^{j-1}$. The item is unpromoted and unfrozen at that moment, so that rejection is dispute-initiating (Rule R13(i)–(ii)). The remaining $J := R - j$ inspections are, by Steps 0–1 and Definition L.10(d), honest, fresh and seat-weighted; each is an eligible verdict for as long as the dispute is unresolved, and an ordinary accepting or rejecting confirmation thereafter. Steps 4–5 treat the two segments separately.

Step 4 (no rejection after the m -th acceptance). Vindication resumes the interrupted entry but never itself promotes: promotion occurs only at the window’s close (Definition H.6(c), Rule R13(iv)). A rejecting decision logged after vindication is therefore a fresh qualifying rejection against an unpromoted entry, which re-imposes the byte-level freeze (Rule R13(ii)). Verdict capacity is scoped to the dispute *instance* (INV-5) and so is available afresh in the second instance; what is not available is freshness, which is byte-permanent (INV-1, Rule R13(iii)(b)): the second instance draws eligible jurors only from identities with no prior logged decision on the bytes, of whom at most $m - 1$ remain (Step 7). It therefore cannot vindicate; its only other outcomes are eviction, which ends the presence outright, and an unresolved instance, under which the window clock does not run out at all because the freeze is excluded from it (Rule R13(iv)). In either case the entry never reaches an unfrozen window end, so it cannot finalise. Under blast every one of the J post-freeze identities decides, so the qualifying sequences are exactly those whose tail after the m -th acceptance is unanimously accepting.

Step 5 (the count, and the credited threshold). Fix J and index the post-freeze inspections $1, \dots, J$. By Step 4, promotion requires: the m -th accepting verdict at position $m + r$ for some r , with exactly r rejecting verdicts before it and $r \leq m_{\text{rej}} - 1$ (otherwise the eviction bar is reached first) and $m + r \leq J$; and every one of the remaining $J - m - r$ decisions accepting. The first $m + r - 1$ positions hold $m - 1$ acceptances and r rejections in any order, $\binom{m-1+r}{r}$ ways, and every such sequence carries r rejections and $J - r$ acceptances. *The credited count is sufficient on every such sequence:* the accepting decisions credited to the entry are the f pre-dispute endorsements, the $j - 1$ pre-freeze honest acceptances, the m accepting verdicts (credited on vindication, Rule R13(iv)) and the $J - m - r$ post-resolution acceptances, all authority-active by Definition L.10(d), totalling $f + R - 1 - r$; since $r \leq J - m = R - j - m$ and $j \geq 1$, this is at least $f + m \geq N$ by the hypothesis $N - f \leq m$, and Step 1 keeps the caps from truncating below N . Hence

$$\mathbb{P}[\text{promotion} \mid J] = \sum_{r=0}^{\min(J-m, m_{\text{rej}}-1)} \binom{m-1+r}{r} q^r (1-q)^{J-r} = (1-q)^J S(J),$$

which is 0 for $J < m$, no arrangement of fewer than m verdicts reaching the m -bar.

Step 6 (collapse). Summing over j , and using $(1 - q)^{j-1}(1 - q)^{R-j} = (1 - q)^{R-1}$ with the substitution $J = R - j$,

$$\pi_{\text{pan}}(R) = \sum_{j=1}^{R-m} q(1-q)^{j-1} \left[(1-q)^J S(J) \right]_{J=R-j} = q(1-q)^{R-1} \sum_{J=m}^{R-1} S(J).$$

Step 7 (validity, and exhaustiveness). The hypothesis $R \leq 2m$ is what makes Step 4’s tail requirement exactly right. At the m -th accepting verdict, $j + m + r$ inspections have occurred, so at most $R - j - m - r \leq R - m - 1 \leq m - 1 < m$ remain: too few for any second dispute to reach its own m -bar, whatever its verdicts. The two cases — no rejection anywhere, and a first rejection at some position — are exhaustive and disjoint, so $\pi_{\text{blast}} = \pi_{\text{ord}} + \pi_{\text{pan}}$. $\square$

Remark L.12 (Where each hypothesis is consumed). The proposition’s hypotheses and the stipulations of Definition L.10 are not interchangeable, and each is used once. $R \geq N - f$ is Step 2’s threshold. $N - f \leq m$ is Step 5’s, and it is not implied by the others, since $N - f \leq R \leq 2m$ permits $N - f = 2m$ — at which reach a counted panel sequence can credit as few as $f + m < N$ acceptances and the formula would over-count; it is also what makes the plateau identity of Corollary L.13(iv) meaningful, that identity evaluating π_{ord} at $R = m$, which requires $m \geq N - f$. $R \leq 2m$ is Step 7. The credit-cap non-bindingness is Steps 1, 2 and 5. The one-window stipulation of Definition L.10(a) is Step 2’s converse and Step 4’s re-freeze; the seat-weight and authority stipulations of Definition L.10(d) are Step 3’s eligibility and Steps 2 and 5’s crediting. At the geometry studied here ($N = 10$, $f = 2$, $m = 20$, $m_{\text{rej}} = 12$) the first two hold with slack, $N - f = 8 \leq \min(m, R) = \min(20, R)$ for every $R \geq 8$, and the binding hypothesis is $R \leq 40$.

Corollary L.13 (Five structural consequences). *Under the hypotheses of Proposition L.11:*

- (i) (Structural zero and switch-on.) $\pi_{\text{pan}}(R) = 0$ for every $R \leq m$, and its first nonzero value is at $R = m + 1$, where it equals exactly $q(1 - q)^m$ (since $S(m) = 1$). This is a support property, invariant under any change of measure.

- (ii) (No $k, f, f_s, D_{\text{pool}}$.) As stated, by Steps 0–1.
- (iii) (m_{rej} enters late.) The cap in $S(J)$ first differs from $J - m$ at $J = m + m_{\text{rej}}$, and $\pi_{\text{pan}}(R)$ reaches $J = R - 1$; hence π_{pan} is independent of m_{rej} for all $R \leq m + m_{\text{rej}}$ and first depends on it at $R = m + m_{\text{rej}} + 1$, where the difference between the capped and uncapped forms is exactly the single $r = m_{\text{rej}}$ term at $J = m + m_{\text{rej}}$.
- (iv) (Plateau identity.) $\pi_{\text{blast}}(m) = \pi_{\text{blast}}(m + 1) = (1 - q)^m$ exactly, since $(1 - q)^{m+1} + q(1 - q)^m = (1 - q)^m$. The switch-on is therefore invisible in the total rate and visible only in the decomposition — which is why the figure that exhibits it plots the two components separately rather than their sum.
- (v) (Peak and fall.) $S(J)$ is constant in J for $J \geq m + m_{\text{rej}} - 1$, so $\sum_J S(J)$ grows linearly in R while $(1 - q)^{R-1}$ decays geometrically: π_{pan} rises, attains an interior maximum, and falls.

Remark L.14 (π_{pan} is not the theorem’s panel term). $\pi_{\text{pan}}(R)$ and the panel term of B_{att} (Remark I.17) are different objects and must not be conflated. The theorem’s term is a reach-free *upper bound* over all adaptive presentation, stopping and re-entry strategies, holding under a conditional lower bound on honest behaviour alone and assuming no independence — the exact binomial tail arises there from the coupling of Lemma L.2, not from an independence hypothesis; $\pi_{\text{pan}}(R)$ is the exact realised probability of one non-adaptive schedule under the independence of Definition L.10(b). A large gap between them is expected and is *not* evidence that the bound is loose: tightness is a separate empirical question, addressed by strategies that reserve seat-eligible identities and stop at the first bar reached, which *empirically attain* the flat panel term — precisely because they *can* stop, while blast cannot, which is why blast’s realised rate peaks and falls where the theorem’s term is constant. Two schedules, one mechanism, opposite consequences, both predicted.

Remark L.15 (Validity, and the direction of failure beyond it). The law holds in the single-dispute regime $R \leq 2m$, and Step 7 is where that hypothesis is consumed. Beyond it a post-vindication rejection can open a second dispute among still-unpresented, still-fresh jurors, whose own m -bar is now reachable; the single-dispute form therefore *under-counts*, and the direction of failure is fixed in advance rather than discovered. Consequently no reading of the law beyond $2m$ is a prediction: past that reach the schedule’s rate is a measured quantity and the law is not a model of it. Nothing about the failure bears on any bound: B_{att} is reach-free (INV-6) and continues to hold at every reach, and INV-6 disclaims monotonicity of the *realised* rate precisely so that a plateau or a rise in a measured sweep is inside the theory rather than against it.

Commentary (non-normative; the validity boundary, measured). At the deployed geometry ($q = 0.6$, $N = 10$, $f = 2$, $k = 31$, $m = 20$, $m_{\text{rej}} = 12$, so $2m = 40$) the boundary of Remark L.15 is measured, not merely asserted. At the last in-validity reach the law gives $\pi_{\text{pan}}(40) = 1.049 \times 10^{-5}$ against 9 promotions in 10^6 trials. At $R = 64$ the same schedule yields 8 promotions in 10^6 trials (8.0×10^{-6} , Clopper–Pearson 95% $[3.45, 15.76] \times 10^{-6}$) against an extrapolation of 1.057×10^{-14} — a factor 7.57×10^8 , i.e. 8.88 orders of magnitude above it, in the disclosed under-counting direction. At equal trial count $R = 40$ and $R = 64$ are statistically indistinguishable ($z = 0.243$), so the realised rate plateaus rather than continuing its geometric decay. The plateau does not persist: at $R = 100$ the rate is bounded above by 3.69×10^{-6} , a bound that overlaps $R = 64$ ’s lower confidence limit, so the claim is *no rise*, not *a fall*. Every measured cell of this sweep lies below $B_{\text{att}} = 8.957 \times 10^{-3}$: the boundary cell $R = 40$ by 3.00 decades and every cell beyond it by at least 3.05. The generating experiment gates on that conformance, at every cell of every sweep, before writing its artifact. Inside the validity window the margin is far smaller — about 0.58 decades at the panel-path peak $R = 32$, where the measured rate is 2.34×10^{-3} — which is where the ceiling is informative rather than slack. No mechanism is proposed for the plateau’s magnitude: beyond $2m$ the sweep is *measured*, not *modelled*.

Remark L.16 (Testability of the m_{rej} entry). Corollary L.13(iii) is sharply testable only for small m_{rej} . The entering term is weighed against the partial sum $\sum_{J=m}^{R-1} S(J)$ that π_{pan} carries at the entry reach, and that sum grows with m_{rej} far faster than the single entering term does, so the relative effect shrinks sharply as the eviction bar rises. The claim is therefore made where it is resolvable — at small m_{rej} , for two values of m — while for large m_{rej} the testable statement is agreement with the uncapped form below the entry reach. Two of the three confirmations available are support properties rather than hypothesis tests: below the entry reach the capped and uncapped forms are *equal*, so two panel widths sharing one coin stream must produce byte-identical counts, which cannot be passed by luck.

Commentary (non-normative; the m_{rej} entry, measured). At $m = 20$ and $q = 0.2$: at $m_{\text{rej}} = 4$, entry reach $R = 25$, the entering term is 34.59 against a partial sum of 112.5, a +30.75% effect; at $m_{\text{rej}} = 12$, entry reach $R = 33$, it is 8.411 against 2003.05, a +0.42% effect. The corresponding rates are $\pi_{\text{pan}}(25) = 0.10625$ capped against 0.13892 uncapped, and $\pi_{\text{pan}}(33) = 0.31740$ against 0.31873. Reading a five-sigma requirement as five standard errors of the difference between two arms of K trials each, with the variance evaluated at the capped rate — $K = 25 \cdot 2p(1 - p)/\Delta^2$ — those effects need 4.5×10^3 and 6.1×10^6 trials per arm respectively. The prediction is accordingly claimed at $m_{\text{rej}} = 4$, where it is confirmed three ways: byte-identical counts across two panel widths sharing one coin stream at every reach below the entry; a +13.6 σ divergence at the entry reach itself; and two π_{pan} profiles whose maxima over the sampled reach grid fall

where the law places them, $R = 26$ with 4,715 observed against 4,705 predicted and $R = 30$ with 14,030 against 14,001. One scope note on the maxima: the $m_{\text{rej}}=4$ profile’s exact maximum lies at $R = 27$, a reach that grid does not sample, so $R = 26$ is a maximum over the sampled grid and not the argmax of π_{pan} ; the two differ by 2.2% in value.

Remark L.17 (Decoupled reviewer quality). Definition L.10(c) couples three decision classes that the protocol distinguishes: pre-freeze gate inspections, panel verdicts, and post-resolution confirmations. Under a decoupled reading the first and third are ordinary gate inspections, governed by q_{pre} , while only the second are verdicts, governed by q_{pan} ; Steps 0–5 then go through with

$$\pi_{\text{ord}}(R) = (1 - q_{\text{pre}})^R, \quad \mathbb{P}[\text{promotion} \mid J] = \sum_{r=0}^{\min(J-m, m_{\text{rej}}-1)} \binom{m-1+r}{r} q_{\text{pan}}^r (1 - q_{\text{pan}})^m (1 - q_{\text{pre}})^{J-m-r},$$

the $m + r$ verdicts charged at q_{pan} and the unanimous tail of $J - m - r$ post-resolution confirmations at q_{pre} . But Step 6’s collapse fails — the two geometric factors no longer share a base — and the panel component remains the double sum $\sum_{j=1}^{R-m} q_{\text{pre}}(1 - q_{\text{pre}})^{j-1} \mathbb{P}[\text{promotion} \mid R - j]$. Setting $q_{\text{pre}} = q_{\text{pan}} = q$ recovers Proposition L.11 exactly, which is why the closed form is specific to the coupled reading and why the figures that exhibit the law run coupled reviewer quality while the decoupled figures report ceilings only. A deployment or simulator adopting the decoupled reading must additionally fix, and disclose, which parameter governs the post-resolution tail: assigning it to q_{pan} instead — a defensible modelling choice, since those inspections occur under the dispute flag — replaces $(1 - q_{\text{pre}})^{J-m-r}$ by $(1 - q_{\text{pan}})^{J-m-r}$ and yields $q_{\text{pre}}(1 - q_{\text{pre}})^{j-1}(1 - q_{\text{pan}})^J S_{q_{\text{pan}}}(J)$ with $S_{q_{\text{pan}}}$ the sum S at ratio $q_{\text{pan}}/(1 - q_{\text{pan}})$. The two forms differ except in the coupled case, and no result of this document distinguishes them.

Commentary (non-normative; epistemic status of the blast law). Three things this law is not. It is not a safety claim: the safety statement is Theorem I.13, which assumes no independence and quantifies over all adaptive schedules, and this law’s exponential decay would not survive an adversary that stops at the first rejection — such an adversary is confined to the ordinary path and is measured separately. It is not a tightness claim (Remark L.14). And it is not an assumption of anything: no rule reads it, no proof cites it, and the protocol’s behaviour is identical whether or not it holds; it is accordingly absent from the assumptions ledger of §L.6.

What it is, is the instrument that makes the simulator falsifiable. Every one of its five consequences is a prediction the implementation could violate, three of them at the level of *support* rather than of a tail probability — the structural zero, the k -independence, and the m_{rej} -independence below the entry reach are all byte-identity properties that cannot be passed by luck. Its own derivation is verified against two independent oracles: direct enumeration of every verdict sequence at small (m, m_{rej}) , applying the decision rule of Step 4 without reference to the closed form; and an uncollapsed exact-rational sum over the rejecter’s position, so that an algebra error in Step 6 cannot cancel. Against the implementation it is validated by an exact two-sided Holm-corrected test over the family of all 206 cell-and-component comparisons available across nine simulation configurations, four reviewer-quality levels and two panel geometries, with zero rejections; the sharpest single cell agrees to about three parts in 10^5 on 1.29×10^7 simulated promotion attempts.

The residual is interpretive rather than mathematical or empirical. Step 4 is a *reading* of Rule R13, and the enumeration oracle shares that reading with the claim it checks, so the two are not independent on that point; what breaks the circularity is the family test against an engine that implements the rules without reference to this prose. Unlike the single-user core (§G.6), none of this is machine-checked. An implementation that disagreed with the law would be either a simulator defect or a misreading of Appendix H, and the law’s role is to make the difference cheap to find.

L.3 EXPLICIT LEDGER CONSTANTS

Lemma L.18 (Explicit forms of $A(M)$ and B_{fl}). *Summing the rows of Table 14 under the charging convention of Lemma J.7, with $\bar{D}_v$ as there and $\kappa' := \kappa + c_p k$:*

$$\mathbb{E}[C_v^{\text{pipe}}] \leq A(M) \mathbb{P}[n_v \geq 1] + B_{\text{fl}} \bar{\gamma} \mathbb{E}[n_v],$$

with

$$A(M) := 2E_{\star} c_{\text{tool}}^2 (N^+ + 1 + s_0) + c_{\text{tool}} (2(1 - p_b) E_{\star} c_{\text{tool}} s_0 + (1 - p_b) E_{\text{max}} c_{\text{tool}} N^+ + c_{\text{accf}} + 2E_{\text{max}} + 1) \kappa',$$

$$B_{\text{fl}} := 2E_{\star} c_{\text{tool}}^2 \left[1 + (1 - p_b) \kappa' \right],$$

where κ' folds at most $c_p k$ invited re-inspections per fast-track review (Rule R13(vi)) into the freeze-bucket multiplier; the suspension-prompt row is majorised by the $c_p k$ component of the freeze-bucket term ($E_{\text{max}} c_p k \leq c_{\text{tool}} \bar{D}_v c_p k$, since $c_{\text{tool}} \geq 1$ and $\bar{D}_v \geq 2E_{\text{max}} + 1$), a deliberate slackening: the prompt row is charged once in the ledger and bounded once here, by a dominating expression. Using $1 \leq D_{\text{pool}} \leq N$ (Definition H.3), $N \leq N^+ < N + D_{\text{pool}} \leq 2N$ and $E_{\star} \leq 2E_{\text{max}}$, these satisfy $A(M) = \mathcal{O}(E_{\text{max}} c_{\text{tool}}^2 (N + c_{\text{accf}})(1 + \kappa))$ and $B_{\text{fl}} = \mathcal{O}(E_{\text{max}} c_{\text{tool}}^2 (1 + (1 - p_b) \kappa))$. The forms are closed-form upper bounds, exact in structure and monotone-slackened in constants; they are not claimed tight.

Proof. Row sums under the charging convention of first item-arrival attribution, splitting each $(\bar{\gamma}n_v + s_0)$ term into its $\bar{\gamma}n_v$ part (collected into B_{fl}) and its s_0 part (collected into $A(M)$):

$$\underbrace{E_{\max}c_{\text{tool}}N^+}_{\text{weight}} + \underbrace{E_{\star}c_{\text{tool}}^2(\bar{\gamma}n_v + s_0)}_{\text{window bucket}} + \underbrace{E_{\star}c_{\text{tool}}^2(\bar{\gamma}n_v + s_0)}_{\text{dormancy bucket}} + \underbrace{c_{\text{tool}}\bar{D}_v\kappa'}_{\text{freeze bucket (prompts folded in)}} + \underbrace{E_{\max}c_{\text{tool}}N^+}_{\text{repeat}} + \underbrace{E_{\max}}_{\text{publish}}.$$

The former reversal accounting appears in no separate term: a prior rejecter’s later acceptances are charged to that demander’s bucket inside the per-demander $\times c_{\text{tool}}$ factors of the three bucket rows (Lemma J.7, charging convention), so no $\bar{D}_v(1 + \kappa)$ reversal term arises. The suspension-prompt row is majorised by, not identical to, the $c_p k$ component of the freeze-bucket term ($c_{\text{tool}} \geq 1$, $\bar{D}_v \geq 2E_{\max} + 1$), so its events are charged once (to their row) and bounded once (by the dominating freeze expression). Collecting the coefficient of $\bar{\gamma}n_v$: $2E_{\star}c_{\text{tool}}^2$ from the window and dormancy buckets, plus $2(1 - p_b)E_{\star}c_{\text{tool}}^2\kappa'$ from the freeze bucket through $\bar{D}_v$ ’s first term, giving B_{fl} . Collecting the demand-free remainder: $2E_{\max}c_{\text{tool}}N^+ + E_{\max} + 2E_{\star}c_{\text{tool}}^2s_0 + c_{\text{tool}}\kappa'(2(1 - p_b)E_{\star}c_{\text{tool}}s_0 + (1 - p_b)E_{\max}c_{\text{tool}}N^+ + c_{\text{acc}}f + 2E_{\max} + 1) \leq A(M)$, using $2E_{\max}c_{\text{tool}}N^+ + E_{\max} + 2E_{\star}c_{\text{tool}}^2s_0 \leq 2E_{\star}c_{\text{tool}}^2(N^+ + 1 + s_0)$ ($E_{\max} \leq E_{\star}$, $c_{\text{tool}} \geq 1$, A12; $E_{\star} \geq 1$, convention (C8)). Taking expectations over the demand, with the indicator carried per Lemma J.7(iii) (every row’s pathwise count vanishes on $\{n_v = 0\}$), gives the claim; the $\mathcal{O}$ -forms follow since $c_p k = \mathcal{O}(\kappa)$ for ρ_{seat} bounded below, $\kappa = (k+1)/\rho_{\text{seat}} \geq k$, and $E_{\star} = E_{\max} + 1 \leq 2E_{\max}$ ($E_{\max} \geq 1$, convention (C8)). $\square$

Remark L.19 (Reading of the ledger constants). (i) The bracketed $((1 - p_b)c_{\text{tool}}N^+ + c_{\text{acc}}f)\kappa'$ term is the *dispute cross-term*: honest reconsideration capacity and adversarial dispute capacity multiplied by panel cost, with suspension-prompt re-inspections folded into κ' inside the freeze term, the prompt row majorised by the freeze term rather than independently summed; it is what forces the cross-term condition $N(M)k(M) = o(M^{1-\beta})$ of Theorem J.13. (ii) The κ' -part of B_{fl} is damped by $1 - p_b$, the honest wrongful-rejection rate: the front-loaded freeze load scales with honest error, not with demand alone. (iii) The front-loading coefficient B_{fl} multiplies only the fraction $\bar{\gamma}$ of the item’s own demand: item popularity enters the per-item cost solely through $\bar{\gamma}n_v$ and cancels against the population in Lemma J.8. (iv) The c_{tool}^2 in both constants is the pair-level discipline made explicit: one factor counts the pairs over which a bucket’s demanders are enumerated (A10’s demand clause, read per pair), the other the pairs each demander may confirm (INV-3); both are $\mathcal{O}(1)$ (A12), so no asymptotic statement changes.

L.4 KNOWLEDGE-BASED FORMULATION OF REASONING INTEGRITY

Theorem G.25 is trace-wise per coin sequence. It induces the following statement in the knowledge-based style of gradual release Askarov & Sabelfeld (2007), recorded to dispel the classical worry that conditioning on release events can itself be secret-dependent.

Proposition L.20 (Gradual release). *Fix the strategies P and $R_{\mathcal{E}}$, the slot skeleton, a coin sequence ω , and a typed stream $A6$; in the federated setting also a federation log $A8$. By Theorem G.25, the entire behaviour of a run of horizon n from a sorted configuration (its Protected-LLM operations, tool invocations, reply stream, and ungated sink events) is a function $F(\tilde{c}_0^{\approx}, \text{DT}, \omega, n)$ of the initial matched-configuration class (Definition G.6), the run’s display transcript (the run suffix of Definition G.8), and the coins, with no residual dependence on $\mathcal{M}_{\text{U}}$. Consequently, for any observer who knows these fixed inputs and the initial matched class, and any prior over untrusted-store contents consistent with that class, the posterior after observing the run to horizon n equals the prior conditioned on consistency with DT alone: knowledge about untrusted memory increases only at display events, never through undisplayed behaviour.*

Proof. By Theorem G.25, any two untrusted-store contents consistent with the same DT (within the fixed initial matched class, matched coins, coupled typed stream and slot skeleton, conventions (C1)/(C9)) yield identical behaviour, so the likelihood of the observed behaviour is constant across such contents; Bayes’ rule then collapses the posterior to the prior restricted to DT-consistency. $\square$

The observer here is granted ω and the typed stream, strictly more than any real attacker knows. The statement is claimed for this strong observer only: for an observer ignorant of ω , conditioning on the transcript can correlate the hidden store with the coins, and behaviour may then reveal information about ω that, jointly with the transcript, reveals more about untrusted memory than the transcript alone; marginalising over ω therefore does not yield the weaker-observer statement in general. Deployments wanting the weaker-observer form may include the public coin outcomes in the release history, whereupon this proposition applies verbatim with ω read as released.

L.5 ZIPF CASE STUDY FOR THE BUDGET

This subsection instantiates the saturation cost bound (Theorem J.9) under a Zipf rank-popularity law, as a worked example; Theorem J.9 needs only sublinear distinct-count growth (with A10). We caution that empirical rank exponents for human reference streams are frequently at or below one Breslau et al. (1999), and the rank/Heaps link has finite-size deviations Lü et al. (2010); the clean rate below holds in the top-heavy regime $\alpha > 1$. Throughout, the demand-independent per-item cap is $\hat{N} := A(M)$ with $A(M)$ as in Lemma L.18; the front-loaded part $B_{\text{H}} \bar{\gamma} n_v$ of the ledger is carried separately (after summation it contributes at most $B_{\text{H}} \bar{\gamma} d$ per user, uniformly over the popularity law), so the head/tail analysis concerns the $\hat{N}$ part, with the $B_{\text{H}} \bar{\gamma} d$ term added at the end. The demand-independent invitation cap $K = E_{\max}(c_{\text{tool}} N^+ + c_p k + 1)$ of Lemma J.7(iv) is likewise carried separately, as $K V(M)/M$ (Lemma L.21); it plays no role in the head/tail split. When $\hat{N}$ grows with M (through $N(M)$, $\kappa(M)$, and $f(M)$ under co-scaling), the head term below becomes $\hat{N}^{1-1/\alpha} Z^{-1/\alpha} M^{-(1-1/\alpha)}$, which still vanishes whenever $A(M) = o(M)$, amply covered by $A(M) = o(M^{1-\beta})$.

Lemma L.21 (Head/tail decomposition). *With head $H_M = \{v : Mp_v \geq \hat{N}\}$, tail $L_M = \mathcal{V} \setminus H_M$, and $K = E_{\max}(c_{\text{tool}} N^+ + c_p k + 1)$ as in Lemma J.7(iv), and with $\mathbb{E}[\bar{C}_M]$ read over the pipeline component (Lemma J.7; the ledger residual adds the separately bounded term of Theorem J.9(iii) on top, uniformly over the popularity law, vanishing under that theorem’s additional rate conditions),*

$$\mathbb{E}[\bar{C}_M] \leq \frac{\hat{N} |H_M|}{M} + c_{\text{tool}} \mu_{\text{tail}}(M) + K \frac{V(M)}{M} + B_{\text{H}} \bar{\gamma} d.$$

Proof. Head items are bounded by clause (iii) alone ($\mathbb{E}[C_v] \leq \hat{N} + B_{\text{H}} \bar{\gamma} \mathbb{E}[n_v]$); tail items by clause (iv) in expectation ($\mathbb{E}[C_v] \leq c_{\text{tool}} Mp_v + K \mathbb{P}[n_v \geq 1]$, since $\mathbb{E}[n_v] = Mp_v$); summing the head term and the front-loaded parts over all items gives the first and last terms after division by M ; the tail items’ K -terms are further enlarged to the all-items sum $K V(M)$, giving the third term. The quantity $\mu_{\text{tail}}(M)$ is the expected number of distinct tail items a single user demands: sub-threshold demand not yet amortised. $\square$

Theorem L.22 (Zipf decay, head and tail). *Under $p_r = r^{-\alpha}/Z$ (read as the homogeneous special case $p_{u,v,t} \equiv p_{v,t}$ of the reference model), $Z = \sum_{j=1}^{|\mathcal{V}|} j^{-\alpha}$. Clauses (i)–(iii) concern $\alpha > 1$, where $Z \leq \zeta(\alpha)$:*

(i) *rank r is in the head iff $r \leq R_M := (M/(\hat{N}Z))^{1/\alpha}$, so $|H_M| = \min\{\lfloor R_M \rfloor, U\} \leq R_M$ and $\hat{N}|H_M|/M \leq Z^{-1/\alpha} (\hat{N}/M)^{1-1/\alpha}$;*

(ii) *let $U := |\mathcal{V}|$ and $m_{\star} := \lfloor R_M \rfloor + 1$; for the finite universe, $\mu_{\text{tail}}(M) = \frac{1}{Z} \sum_{m_{\star} \leq r \leq U} r^{-\alpha}$, empty (and zero) if $m_{\star} > U$. For $m_{\star} \leq U$, the floor-aware two-sided comparison $\int_{m_{\star}}^{U+1} x^{-\alpha} dx \leq \sum_{r=m_{\star}}^U r^{-\alpha} \leq m_{\star}^{-\alpha} + \int_{m_{\star}}^U x^{-\alpha} dx$ gives*

$$\frac{m_{\star}^{1-\alpha} - (U+1)^{1-\alpha}}{Z(\alpha-1)} \leq \mu_{\text{tail}}(M) \leq \frac{1}{Z} \left(m_{\star}^{-\alpha} + \frac{m_{\star}^{1-\alpha} - U^{1-\alpha}}{\alpha-1} \right) \leq \frac{1}{Z} \left(R_M^{-\alpha} + \frac{R_M^{1-\alpha}}{\alpha-1} \right),$$

using $m_{\star} > R_M$ and $\alpha > 1$; since $R_M^{-\alpha} \leq R_M^{1-\alpha}$ for $R_M \geq 1$, and $\mu_{\text{tail}} \leq d = 1$ always (absorbing $R_M < 1$), $\mu_{\text{tail}}(M) = \mathcal{O}((\hat{N}Z/M)^{1-1/\alpha})$ uniformly; the matching lower order $\mu_{\text{tail}}(M) = \Theta((\hat{N}Z/M)^{1-1/\alpha})$ holds under the two additional (sufficient) non-saturation conditions $R_M \rightarrow \infty$ (equivalently $\hat{N}Z/M \rightarrow 0$) and $U/R_M \rightarrow \infty$: then $m_{\star}/R_M \rightarrow 1$, so $m_{\star}^{1-\alpha} = (1 + o(1)) R_M^{1-\alpha}$, and the $m_{\star}^{-\alpha}$ term and the U -corrections are $o(R_M^{1-\alpha})$. Neither condition may be omitted from this sufficient criterion in general: along a sequence with $R_M \rightarrow \infty$, taking $U = m_{\star}$ leaves a single tail term of order $R_M^{-\alpha} = o(R_M^{1-\alpha})$, so the matching order fails though $R_M \rightarrow \infty$ holds; while $R_M \rightarrow 0$ (a per-item cap outgrowing the population, e.g. $U = Z = 1$, $\hat{N} = M^2$) leaves tail mass $\Theta(1)$ against a diverging comparison scale $(\hat{N}Z/M)^{1-1/\alpha}$, though $U/R_M \rightarrow \infty$ holds. For an infinite universe, $U/R_M \rightarrow \infty$ is automatic while $R_M \rightarrow \infty$ remains an additional requirement; $R_M \rightarrow \infty$ holds in the fixed-adversary, fixed-constants regime of Theorem J.9(ii) and in the corridor of Theorem J.13 (where $\hat{N} = A(M) = o(M^{1-\beta})$), while $U/R_M \rightarrow \infty$ remains a separate universe non-saturation condition in either regime. Part (iii) consumes only the uniform upper bound and requires neither condition;

(iii) *hence $\mathbb{E}[\bar{C}_M] = \mathcal{O}((A(M)/M)^{1-1/\alpha}) + \mathcal{O}(K(M) V(M)/M) + B_{\text{H}} \bar{\gamma} d$, plus the residual of Theorem J.9(iii), the head and tail terms decaying at the common rate $\mathcal{O}((A(M)/M)^{1-1/\alpha})$ (as upper bounds) and the invitation term $K V(M)/M$ vanishing whenever $K(M) V(M) = o(M)$ — under fixed protocol constants with $f(M) =$*

$\mathcal{O}(1)$ and ρ_{seat} bounded below ($A(M), K(M) = \Theta(1)$; the hypotheses of Theorem J.9(ii) automatic, giving the clean rate $\mathcal{O}(M^{-(1-1/\alpha)}) + \mathcal{O}(V(M)/M)$; under co-scaling, $K(M) = \mathcal{O}(E_{\max}(c_{\text{tool}}N + c_p k)) = o(M^{1-\beta})$ by the corridor schedules, so $K V(M)/M = o(1)$ exactly as for $A(M)$). The front-loading term survives exactly as in Theorem J.9, vanishing under $\bar{\gamma}(M)(1 + \kappa(M)) \rightarrow 0$ (A13 is automatic here: $d = \sum_r p_r = 1$).

For the boundary case $\alpha = 1$, read along finite-universe sequences $U := |\mathcal{V}| \rightarrow \infty$, with $Z := H_U = \sum_{r=1}^U r^{-1} = \ln U + \mathcal{O}(1)$ and $R_M := M/(\hat{N}Z)$: in the nonempty-head regime $1 \leq R_M < U$ (below universe saturation), $R_M \sim M/(\hat{N} \ln U)$ and the head term $\hat{N}|H_M|/M = \Theta(1/\ln U)$ — the $\hat{N} = A(M)$ dependence cancels; for $R_M < 1$ the head is empty and the head term zero — while after universe saturation ($R_M \geq U$) the head term equals $\hat{N}U/M$. For $1 \ll R_M < U$, the tail sum $\sum_{R_M < r \leq U} r^{-1}/Z = (\ln(U/R_M) + \mathcal{O}(1))/(\ln U + \mathcal{O}(1))$, governed by how far the head boundary sits below the universe; (floor corrections at the boundary rank are absorbed into the $\mathcal{O}(1)$; the tail is empty for $m_* > U$); no power rate is asserted without an explicit relation between U and R_M , decay being controlled by saturation rather than the exponent, which is why the main development is stated for sublinear distinct-count growth (Remark L.23).

Proof. (i) $M p_r \geq \hat{N} \iff r \leq R_M$, and $|H_M| = \min\{\lfloor R_M \rfloor, U\} \leq R_M = \mathcal{O}((M/\hat{N})^{1/\alpha})$; Z is bounded for $\alpha > 1$. (ii) with $m_* = \lfloor R_M \rfloor + 1$: if $m_* > U$ the tail is empty and every bound holds trivially; otherwise apply the floor-aware comparison as displayed (the lower bound from $r^{-\alpha} \geq \int_r^{r+1} x^{-\alpha} dx$ termwise, the upper from splitting off the first term and bounding the rest by $\int_{m_*}^U x^{-\alpha} dx$); dropping the nonnegative U -terms and using $m_* > R_M$ (so $m_*^{-\alpha} < R_M^{-\alpha}$ and $m_*^{1-\alpha} < R_M^{1-\alpha}$) yields the uniform upper bound, with the trivial guard $\mu_{\text{tail}} \leq d = 1$ absorbing $R_M < 1$; under $R_M \rightarrow \infty$ and $U/R_M \rightarrow \infty$, $m_*/R_M \rightarrow 1$, so the floor correction is negligible, the U -corrections and the $m_*^{-\alpha}$ term are $o(R_M^{1-\alpha})$, and both ends are $\Theta(R_M^{1-\alpha})$, giving the matching order; substitute $R_M = (M/(\hat{N}Z))^{1/\alpha}$. (iii) sum in Lemma L.21; the tail term $c_{\text{tool}} \mu_{\text{tail}}$ absorbs the old constant into $\mathcal{O}(\cdot)$, and the invitation term is $K V(M)/M$ verbatim. The $\alpha = 1$ clauses follow, along $U \rightarrow \infty$, from $R_M = M/(\hat{N} \ln U)(1 + o(1))$ read in the regime $R_M \geq 1$ (where $\lfloor R_M \rfloor \geq R_M/2$; the head is empty for $R_M < 1$), cancellation of $\hat{N}$ in $\hat{N} R_M/M$, the exact identity $|H_M| = U$ when $R_M \geq U$ (where the tail is empty), and, separately in the nonsaturated regime $1 \ll R_M < U$, the harmonic-tail estimate $\sum_{R_M < r \leq U} r^{-1} = \ln(U/R_M) + \mathcal{O}(1)$ Breslau et al. (1999). $\square$

Remark L.23. The $\alpha = 1$ boundary case is why the main development is stated in terms of sublinear distinct-count growth (Theorem J.9), which holds across α : the Zipf form sharpens the rate in the top-heavy regime $\alpha > 1$ but is used nowhere in any proof.

L.6 ASSUMPTIONS LEDGER

Assum.	What it buys	Conditions and caveats
A1	Labels attacker-immutable; injected content stays U.	Authenticated command channel (TCB, §F.5).
A2	Trust changes only by deterministic rules and endorsement.	Storage and log integrity of $\mathcal{M}_{\mathbb{E}}$ (TCB).
A3	Faithful decision pane; TCB-authenticated at-tested zone; marked untrusted strings.	Zoned rendering path (TCB, R8). The <i>payload</i> of a narrative confirmation is attacker-influenced text; that is A5's subject.
A4	A confirmed atomic value is the intended value for that tool.	User-as-oracle axiom; defensible because the set is small, faithfully displayed, and shrinking (Appendix J); weakest at the batch gate (S12).
A5	A confirmed summary was checked against citations.	Stronger than A4; mitigated by community verification; residual S10, incl. publisher penalty on honest-but-deceived publication.
A6	Licenses the two-run comparison and the informal reading of Cor. G.26.	Human-channel idealization; practical content discharged by logging both surfaces (N5, N7, N8).
A7	Community endorsement at a tool speaks for any user at that exact tool.	Only the semantic half (context-independence) assumed; type-level half proved (Lem. I.2); canonical identifiers TCB.

A8	Licenses federated Reasoning Integrity (Prop. I.10).	Parallel to A6; influence through legitimate log activity bounded by Thms. I.11, I.13 instead (exogenous prefix coupled; local suffixes derived equal, Def. H.2).
A9	Bounded-harm DoS (Lem. I.24(ii)) and the freeze-bucket ledger row, via $\kappa = (k+1)/\rho_{\text{seat}}$ (Lem. I.6).	Behavioural component as $q_{\text{pre}}, q_{\text{pan}}, p_b$ (q_{pre}, p_b disposition-level, q_{pan} verdict-conditional at the header atom, Thm. I.13(c); the symmetric bridge direction case (a) uses is Lem. L.1(iii)); structural component can fail <i>deterministically</i> (freshness/pool-cap depletion under wide pre-exposure or pool-skewed demand, commentary at A9); failure is availability-only, cap unaffected (Lem. I.24(ii)); exponential tail when it holds; validation stratified by pool and prior exposure (N17).
A10	Caps within-window and within-candidate-dormancy <i>realized</i> demand (demand clause, used for ledger bucket attribution) and audit-recorded first pair-decisions (first-pair-decision clause, used only for $\bar{D}_v$) at fraction $\bar{\gamma}$ of the item's latent demand (§L.1); stable promoted intervals exempt (their forced gates are ledger residual, A11); feeds the ledger split (Lem. J.7) and Thms. J.9, J.13.	Hindsight timing property, not volume; steady demand gives $\bar{\gamma} \approx T_{\text{win}}/L$; violation item-local, bounded by Thm. J.2 (S17); validated on most-demanded items (N17); jointly satisfiable with A11 via $D_0 \asymp N/\bar{\gamma}$; pair-level form (per demanded tool); Thm. J.2 never uses it.
A11	Bounds the ledger residual — forced confirmations charged to no ledger row: pre-publication, post-retirement, post-demotion, and stable-promoted-interval demand, the last including handle-expressed use of live-CC pairs (S18) — (Thm. J.9(iii)); completes the total-cost limit (Cor. J.10).	Hindsight, <i>operational</i> (no-row) coverage property of completed epochs, read against latent demand ($U_{v,t} \leq n_{v,t}$, §L.1); jointly requires publication, promotion, and <i>arrival-representation</i> coverage (ε absorbs the handle-arrival fraction); violation graceful and item-local, charged at the cap, entering $r_{\text{bad}}^{(e)}$; scoped to benign items (poison charged at the cap by design); the D_0 term of the aggregate residual requires $D_0(M)V(M) = o(M)$ and the ε term $\varepsilon(M)d_{\text{good}}(M) \rightarrow 0$ (under A13, from $\varepsilon(M) \rightarrow 0$), with bounded tool spread supplied by A12 (Thm. J.9(iii)); enabled by the vindication exemption (N15) and banking; validation via realized $U_{v,t}/n_{v,t}$ and the fast-path-vs-gate split (N17); Thm. J.2 never uses it.
A12	Licenses per-tool accounting: the uncovered residual and $\mathcal{O}(1)$ per-tool constants.	Demand-shape assumption matching the per-(value, tool) unit; byte-first density is a heuristic diagnostic for A11 (stated at its sufficiency paragraph), not part of this assumption; validation N17; supplies the mass comparison $d_{\text{tool}} \leq c_{\text{tool}}d$, quarantined in Cor. J.3 (lifetime form under the lifetime reading, stated there); no clause of Thm. J.2 uses it.
A13	Uniform bound on the per-user epoch demand mass $d(M)$ along the population scaling; licenses every vanishing ($\rightarrow 0$) budget conclusion (Thms. J.9, J.13(iii); Cor. J.10).	Individual-usage property, not a population property; automatic under any fixed per-user demand law (Zipf: $d = 1$, §L.5); owner-coupling contribution bounded by ρ_{sub} (Rule R12); disclosed failure mode: shared-tier-induced demand growth (cf. S16); violation graceful — the relative bounds and the cap are unaffected, and the general-form rate conditions of A13 carry d explicitly; validation: per-user distinct-item demand logged across growth (N17); Thm. J.2 never uses it.

Calibration of Rule R13(v) ($\theta_{\text{acc}}, c_{\text{ceil}}$): separates honest from adversarial rejecters; grounds the deterministic constant $c_{\text{acc}} = c_{\text{ceil}} + r_{\text{max}}$ (Lem. I.4) in the ledger and throughput bounds. Requires estimating $1 - p_b$; mis-calibration is an availability-only exposure (S15); enters only budget constants, never any safety theorem (Fig. 6).

Rules and TCB. Rules R1–R11 with conventions (i)–(vii') and R12–R14 are harness properties; validation (N17) provides falsification and calibration evidence for the uniform behavioural bounds, not certification, since sparse or unobserved strata are not covered by any finite sample; not assumptions; Figures 7–8 with the totality clause of Rule R10 constitute the complete interface against which Lemmas G.17–G.18 are verified. TCB inventory: §F.5 (single-user), §H.3 (federated). External modelling parameters with validation paths: saturation $V(M) = o(M)$; inspector competence $q > 0$ (safety) and $q + p_b > 1$ (joint safety and resolution correctness), q read per theorem I.19. The co-scaling corridor additionally requires the cross-term condition $N(M)k(M) = o(M^{1-\beta})$ and the shrinking front-loading condition $\bar{\gamma}(M)k(M) \rightarrow 0$ (§J.5).